

Xin Wang · Tongtong Feng
Huaping Liu · Wenwu Zhu

Autonomous Embodied AI

Towards Self-evolving
Intelligence



 Springer

Autonomous Embodied AI

Xin Wang · Tongtong Feng · Huaping Liu ·
Wenwu Zhu

Autonomous Embodied AI

Towards Self-evolving Intelligence

Xin Wang 
Tsinghua University
Beijing, China

Huaping Liu
Tsinghua University
Beijing, China

Tongtong Feng
Tsinghua University
Beijing, China

Wenwu Zhu
Tsinghua University
Beijing, China

ISBN 978-981-95-7749-1 ISBN 978-981-95-7750-7 (eBook)
<https://doi.org/10.1007/978-981-95-7750-7>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

If disposing of this product, please recycle the paper.

*In grateful memory of Professor Chen-Ning
Yang (1922–2025) at Tsinghua University.*

*His spirit of exploration, transcending the
microscopic realm of particles and fields,
shall ultimately guide us to discern a new
dawn on our journey to create intelligence
with both body and mind in macroscopic
realm, and to understand intelligence not only
as an abstract code but as a phenomenon
embodied within a physical world as well.*

Foreword by Yonggang Wen

Embodied Artificial Intelligence (AI) has emerged as a cornerstone in advancing intelligent systems, highlighting that cognition and reasoning cannot be decoupled from perception and interaction with the physical world. Unlike purely digital intelligence, embodied AI enables agents to ground abstract reasoning in real-world contexts, thereby bridging the gap between virtual and physical worlds. As large-scale AI models have demonstrated unprecedented reasoning ability, this embodied paradigm provides a natural pathway for extending intelligence to dynamic, uncertain, and open environments, which is critical for realizing Artificial General Intelligence (AGI).

This book, entitled *Autonomous Embodied AI: Towards Self-evolving Intelligence*, authored by Xin Wang, Tongtong Feng, Huaping Liu, and Wenwu Zhu, provides a comprehensive and forward-looking perspective on this emerging field. Part I establishes the foundation by presenting key technologies, components, and architectures of embodied AI, ranging from large language model (LLM)-based, world model-based to joint LLM and world model-based embodied AI. It also explores autonomous embodied AI and autonomous swarm embodied AI, offering a clear progression from individual autonomy to swarm intelligence. Part II shifts the discussion to self-evolving embodied AI, identifying two central pillars: self-evolving world models that construct predictive representations of the environment, and self-evolving agents that operationalize adaptation and improvement in a real-time manner. This dual framework provides both the conceptual depth and practical guidance for advancing toward agents that can continuously learn, self-reflect, and evolve. Such a vision not only advances theoretical understanding but also provides practical guidelines for the future development of embodied AI.

The authors, renowned experts in embodied AI, autonomous learning, and swarm intelligence, have successfully combined rigorous research insights with practical applications. Their work balances conceptual clarity with technical details, making the book equally valuable for researchers seeking a broad overview and practitioners aiming for practical methodologies. By integrating foundations, state-of-the-art methods, and forward-looking visions, the book offers a unique blueprint for building the next generation of embodied AI.

This volume is a timely contribution to the AI community. It will serve as an indispensable reference for scholars, engineers, and graduate students who wish to explore embodied AI. More importantly, it marks a significant step in the journey toward self-evolving embodied AI, underscoring the importance of bridging digital intelligence with the physical world. I strongly recommend this book to those eager to shape and participate in the future of embodied AI.

October 2025

Prof. Yonggang Wen
School of Computer Science
and Engineering
Nanyang Technological University
Singapore, Singapore

Foreword by Jiangchuan Liu

Embodied Artificial Intelligence (AI) has emerged as a pivotal paradigm, positing that intelligence is not merely a computational process but is fundamentally shaped by an agent's physical interactions with its environment. As AI algorithms in the digital world begin to demonstrate thinking capabilities that approach or even surpass those of humans, embodied AI is expected to open a window from the digital world to the physical world, further extending and expanding AI in the complex physical world. In particular, embodied AI requires agents to autonomously interact and continuously learn in the open, non-deterministic physical world, overcoming their reliance on intensive human oversight and pre-programming, and ultimately achieving self-evolving intelligence, which is indeed the necessary path to Artificial General Intelligence. Embodied AI holds enormous market potential and development opportunities, and will play a significant role in a variety of fields, including smart manufacturing, smart homes, smart healthcare, and smart services.

This book, *Autonomous Embodied AI: Towards Self-evolving Intelligence*, offers a timely and comprehensive exploration of embodied AI, evolving landscape from basic to autonomous and self-evolving embodied AI paradigms. Part I covers basic concepts, key technologies, key components, key architectures, key applications, simulators, datasets, and metrics of embodied AI, autonomous embodied AI, and autonomous swarm Embodied AI, demonstrating the logical progression from individual to swarm intelligence. Part II demonstrates the future research trends of embodied AI, organized around two core components: self-evolving world models that provide accurate environmental prediction capabilities and self-evolving agents that operationalize autonomous improvement through modular architectures. This dual-component framework offers a comprehensive blueprint for building self-evolving embodied AI that learns from the real physical world. This book also emphasizes practical applications from simulators to real dynamic environments across robotics, ensuring the research has a tangible impact.

The authors are both highly respected scholars whose expertise in embodied AI, autonomous learning, and swarm intelligence makes them ideally suited to tackle this topic. They have successfully combined their deep technical knowledge with practical insights, providing a rich account of the state-of-the-art in embodied AI. This

book is structured to offer both foundational knowledge and practical methodologies, covering a range of techniques that enable agents to interact with the physical world autonomously.

This book is an essential resource for researchers, practitioners, and graduate students working in embodied AI, robotics, and agents. It offers both foundational knowledge and forward-looking perspectives, making it a valuable guide for anyone interested in the future of embodied AI. This work marks a significant milestone in the journey toward self-evolving embodied AI, and I highly recommend this book to those who wish to understand and contribute to this rapidly evolving field.

October 2025

Prof. Jiangchuan Liu
School of Computing Science
Simon Fraser University
Burnaby, Canada

Foreword by Angelo Cangelosi

The quest for Artificial General Intelligence has long been dominated by computational models that treat cognition as abstract symbol manipulation, and more recently by foundation models based on powerful data-hungry transformer models. On the other hand, decades of research in cognitive science and cognitive developmental robotics have demonstrated that true intelligence emerges from the intricate coupling of brain, body, and environment. This perspective of embodied cognition, which I have long championed in my work on developmental robotics and language grounding, provides the most viable path toward creating machines that can learn, adapt, and reason like biological systems. It is from this foundational standpoint that I find this monograph *Autonomous Embodied AI: Towards Self-evolving Intelligence* to be a remarkable and essential contribution to the field.

What distinguishes this volume is its sophisticated treatment of autonomy as a continuum, covering the development journey from basic autonomy to fully self-evolving agents. The authors masterfully bridge the gap between theoretical cognitive science and practical AI engineering, addressing the crucial challenge of how agents can ground symbols in sensory-motor experiences and continuously refine their internal models through environmental interaction. This directly resonates with the core problems of developmental robotics, particularly the symbol grounding problem and the need for cumulative, lifelong learning.

The book's organization reflects a deep understanding of this progression. Part I's comprehensive coverage of individual and swarm intelligence brilliantly captures how social interaction and collective behavior can overcome the limitations of individual agents. Part II's focus on self-evolving architectures presents a visionary framework where world models and cognitive agents co-evolve, enabling the kind of open-ended learning that characterizes human intelligence.

As someone who has researched how robots can learn embodied cognition through embodied interaction, I particularly appreciate the nuanced discussion of how Multi-modal Large Language Models can be grounded in physical reality through world models. This addresses a critical limitation of current LLMs, which lack genuine understanding rooted in physical experience, and points toward more robust, semantically grounded AI systems. The authors, all distinguished researchers in their own

right, have created a work that is both intellectually rigorous and practically valuable. They successfully integrate principles from cognitive science, robotics, machine learning, and computer vision into a coherent framework for building self-evolving embodied AI systems.

This book will be invaluable to researchers in AI and robotics. It provides the necessary theoretical foundations and practical methodologies for creating agents that can truly learn from and adapt to their environments. This book also offers both a comprehensive reference and an inspiring vision of the future. I enthusiastically recommend this book to anyone committed to understanding and building the next generation of self-evolving embodied AI.

October 2025

Prof. Angelo Cangelosi
School of Computer Science
The University of Manchester
Manchester, UK

Preface

This book aims to provide a systematic overview of embodied Artificial Intelligence (AI), with a focus on both current frontiers and future trends. While autonomous embodied AI represents the state-of-the-art in the field, summarizing the most advanced capabilities of embodied AI to date, self-evolving embodied AI points toward the next evolutionary stage. This progression from current autonomous capabilities to future self-evolving potential forms the core narrative of our work.

Our motivation for this book stems from several key considerations. First, the field urgently requires a comprehensive book that systematically organizes the rapidly evolving landscape from basic to autonomous and self-evolving embodied AI. Second, we establish a clear architectural framework where Part I builds from embodied AI through autonomous embodied AI to autonomous swarm Embodied AI, demonstrating the logical progression from individual to swarm intelligence. Third, Part II demonstrates the future research trends of embodied AI, self-evolving embodied AI, organized around two core components: self-evolving world models that provide accurate environmental prediction capabilities and self-evolving agents that operationalize autonomous improvement through modular architectures. This dual-component framework offers a comprehensive blueprint for building self-evolving embodied AI that learn from interaction. Finally, we emphasize practical applications from simulators to real dynamic environments across robotics, ensuring the research has a tangible impact.

Therefore, we deeply hope this book can benefit interested readers from both the academy and industry, covering needs from junior starters in research to senior

practitioners in IT companies, seeking to advance the frontiers of autonomous, self-evolving embodied AI.

Beijing, China
October 2025

Xin Wang
Tongtong Feng
Huaping Liu
Wenwu Zhu

Acknowledgements We thank Guangyao Li, Ren Wang, Yuwei Zhan, Jinghao Cui, Yujing Xiao, and Mingzi Wang from Tsinghua University for their contributions during the writing of this book.

We would also like to express our sincere thanks to Jiangchuan Liu from Simon Fraser University, Yonggang Wen from Nanyang Technological University, and Angelo Cangelosi from University of Manchester for writing forewords for this book. This monograph will never be finished without their valuable inputs.

Wenwu Zhu is the corresponding author.

Competing Interests The authors have no competing interests to declare that are relevant to the content of this manuscript.

Contents

Part I Autonomous Embodied Intelligence

1 Embodied AI	3
1.1 The Overview	3
1.2 Key Technologies of Embodied AI	6
1.2.1 Reinforcement Learning	6
1.2.2 Continual Learning	11
1.2.3 Imitation Learning	15
1.2.4 Multimodal Large Language Model	18
1.2.5 World Model	21
1.3 Key Components of Embodied AI	25
1.3.1 Active Perception	25
1.3.2 Embodied Cognition	35
1.3.3 Dynamic Interaction	37
1.3.4 Hardware Embodiment	39
1.4 Key Architectures of Embodied AI	40
1.4.1 LLM/MLLM for Embodied AI	41
1.4.2 World Model for Embodied AI	45
1.4.3 Joint MLLM and World Model for Embodied AI	47
1.4.4 Discussions	50
1.5 Key Applications of Embodied AI	51
1.5.1 Robots	52
1.5.2 Drones	54
1.6 Simulators, Datasets, and Metrics	55
1.6.1 Simulators for Embodied AI	56
1.6.2 Datasets and Metrics for Embodied AI	58
References	65
2 Autonomous Embodied AI	89
2.1 The Overview	89
2.2 Key Technologies of Autonomous Embodied AI	90
2.2.1 Autonomous Learning in Humans	91

2.2.2	Autonomous Machine Learning	94
2.3	Key Components of Autonomous Embodied AI	98
2.3.1	Autonomous Environment Learning	98
2.3.2	Autonomous Cognitive Learning	106
2.3.3	Autonomous Dynamic Interaction	108
2.3.4	Autonomous Hardware Embodiment	109
	References	114
3	Autonomous Swarm Embodied AI	123
3.1	The Overview	123
3.2	Key Limitations of Single Embodied Agent	125
3.2.1	Limited Environment Perception and Adaptation Capacity	125
3.2.2	Limited Decision and Reasoning Capacity	128
3.2.3	Limited Operational Robustness	130
3.3	Key Technologies of Swarm Embodied AI	131
3.3.1	Swarm Intelligence	131
3.3.2	Particle Swarm Optimization	138
3.4	Key Components of Swarm Embodied AI	141
3.4.1	Early Collaboration	143
3.4.2	Communication Mechanism	145
3.4.3	Intermediate Collaboration	146
3.4.4	Loss Function	151
3.4.5	Late Collaboration	152
3.5	Operational Robustness of Swarm Embodied AI	153
3.5.1	Localization Error	154
3.5.2	Communication Issues	154
3.5.3	Model and Task Discrepancies	155
3.5.4	Privacy and Security	156
3.6	Autonomous Swarm Embodied AI	157
3.6.1	Autonomous Collaboration	157
3.6.2	Autonomous Communication	160
3.6.3	Autonomous Robustness Optimization	163
3.7	Simulators, Datasets, and Metrics	166
3.7.1	Simulators	169
3.7.2	Datasets and Metrics	172
	References	175
Part II Self-evolving Intelligence		
4	Self-evolving Embodied Intelligence	189
4.1	The Overview	189
4.2	Key Technologies of Self-evolving Embodied Intelligence	191
4.2.1	Meta Learning	192
4.2.2	Curriculum Learning	195

4.2.3	Lifelong Learning	197
4.3	Key Components of Self-evolving Embodied Intelligence	199
4.3.1	Self-evolving World Model	201
4.3.2	Self-evolving Agent	202
	References	206
5	Self-evolving World Model	211
5.1	The Overview	211
5.2	The Foundation of Self-evolving World Model	213
5.2.1	Why Need Formalization	214
5.2.2	The Key Components	214
5.2.3	The Mathematical Formalization	217
5.2.4	The Self-evolving Mechanism	217
5.3	Self-evolution Within Components	218
5.3.1	Model Self-evolution	218
5.3.2	Memory Self-evolution	226
5.3.3	Tools Self-evolution	228
5.3.4	Dynamics Optimizer Self-evolution	230
5.4	Self-evolution Between Components	232
5.4.1	Joint Model-Memory Self-evolution	232
5.4.2	Joint Model-Tools Self-evolution	234
5.4.3	Joint Model-Dynamics Optimizer Self-evolution	235
5.4.4	Others	237
	References	237
6	Self-evolving Agent	247
6.1	The Overview	247
6.2	Key Components of Self-evolving Agent	248
6.2.1	Model	249
6.2.2	Memory	250
6.2.3	Tools	252
6.3	Key Architecture of Self-evolving Agent	252
6.3.1	Model Self-evolution	253
6.3.2	Joint Model-Memory Self-evolution	254
6.3.3	Joint Model-Tools Self-evolution	254
6.3.4	Joint Model-Memory-Tools Self-evolution	255
6.4	Key Strategy of Self-evolving Agent	256
6.4.1	Imitation-Based Self-evolution	257
6.4.2	Interaction-Based Self-evolution	258
6.5	Case Study: Self-evolving Agent	260
6.5.1	Framework Design	261
6.5.2	Learning Mechanism	262
6.5.3	Evaluation	263
	References	264
	Index	269

List of Figures

Fig. 1.1	The concept of embodied AI	4
Fig. 1.2	Key technological advances in embodied AI	6
Fig. 1.3	The development roadmap of MLLMs for embodied AI	41
Fig. 1.4	Comparison of unimodal and multimodal embodied AI	42
Fig. 1.5	The development roadmap of world models for embodied AI	46
Fig. 1.6	Embodied AI enabled by MLLMs and world models	49
Fig. 2.1	The three-phase cyclical model of autonomous learning	92
Fig. 2.2	Self-directed machine learning framework	95
Fig. 2.3	The overview of NeRF framework	100
Fig. 2.4	Foundation models for learning general-purpose representations	103
Fig. 2.5	Timeline of embodied perception, navigation, and interaction	104
Fig. 2.6	The fundamental operation of the spiking neural networks	111
Fig. 2.7	The workflow of the spiking neural networks	111
Fig. 2.8	The discrete execution pipeline of embodied AI systems	113
Fig. 2.9	The continuous execution pipeline of embodied AI systems	114
Fig. 3.1	Illustration of the limited coverage range	127
Fig. 3.2	The example of limited decision-making abilities	129
Fig. 3.3	Typical three collaboration stages	142
Fig. 3.4	Illustration of the common collaboration modules	143
Fig. 3.5	The real-world challenges in collaborative perception scenarios	153
Fig. 3.6	Sensor setup for our data collection platform	173
Fig. 3.7	Three types of swarm UAVs formation	174
Fig. 4.1	A diagram of the Model Agnostic Meta Learning (MAML) algorithm	193
Fig. 4.2	An illustration of the curriculum learning concept	196
Fig. 4.3	A conceptual comparison of three continual learning strategies	198
Fig. 4.4	The conceptual diagram of a World Model architecture	201
Fig. 5.1	The concept of world model	212

Fig. 5.2	The training paradigms evolution of world models	213
Fig. 5.3	The components of self-evolving world model	215
Fig. 5.4	The self-evolving mechanism of world model	218
Fig. 5.5	An overview of self-evolving between components	232
Fig. 6.1	The four core modules of the self-evolving agent	251
Fig. 6.2	The framework of the proposed MoDP	262

List of Tables

Table 1.1	Comparison of three categories of active perception methods: visual SLAM, 3D scene understanding, and active environment exploration (PC: Point Cloud, G: Geometric, V: Volumetric, S: Semantic, M: Multimodal, T: Topological, SC: Scene Understanding)	27
Table 1.2	Comparison of three categories of embodied cognition methods: task-driven self-planning, memory-driven self-reflection, and the embodied multimodal foundation models. I, L, and C indicate the Image, Language, and Point Cloud modalities, respectively	36
Table 1.3	Comparison of three categories of dynamic interaction methods: action control, behavioral interaction, and collaborative decision-making, compared across input modalities, interaction type, modeling paradigm, and task type. I, L, S, P, and T denote Image, Language, State, Proprioception, and Trajectory, respectively. IL denotes Imitation Learning	38
Table 1.4	Qualitative comparison of MLLM-only, world model-only, and joint architectures combining MLLMs and world models in Embodied AI	51

Table 1.5 General Simulator. HFPS: high-fidelity physical simulation; HQGR: high-quality graphics rendering; RRL: rich robot library; DLS: deep learning support; LSPC: large-scale parallel computing; ROS: tight integration with ROS; MSS: multiple sensor simulation; CP: cross-platform Nav: robot navigation AD: auto driving; RL: reinforcement learning LSPS: large-scale parallel sim MR: multi-robot systems RS: robot simulation. ✓ indicates that the simulator excels at this aspect. (PE: Physics Engine; MLA: ML-Agents; OBSD: ODE, Bullet, Simbody, DART; BOVN: Bullet, ODE, Vortex, Newton; DAR: Drone sim, AD, RL) 57

Table 1.6 Comparison of real-scene-based simulators. For sensors, S denotes Semantic, L denotes LiDAR, and A denotes Audio. (CA: Continuous Action; 3D-SS: 3D Scene Scans; MA: Multiple Agents; OS: Object States; 3D-A: 3D Assets) ... 59

Table 1.7 Comparison of VLN datasets (I: Indoor; O: Outdoor; C: Continuous; D: Discrete) 60

Table 1.8 Comparison of EQA datasets (A-EQA: Active EQA; I-EQA: Interactive EQA; EM-EQA: Episodic Memory EQA; OE: Open-Ended; MC: Multiple Choice; MINOS*: Simulator Based on MINOS) 62

Table 1.9 Embodied grasping datasets 64

Table 2.1 Embodied perception large models 105

Table 3.1 A detailed comparison between autonomous flight-related datasets—indicates that specific information is not provided. DF: Discipline formation mode, where swarm UAVs keep a consistent and relatively static array; FF: Fixed formation mode, where each UAV navigates independently with a fixed path; AF: Autonomous formation mode, where each UAV flies autonomously. D: 3D object detection; T: 3D object tracking; S: semantic segmentation 168

Table 3.2 A detailed comparison between simulators related to autonomous flight. H: High; M: Medium; L: Low. U2USim takes Yunnan Province as the prototype and uses the real meteorological data of Yunnan Province collected by the China Meteorological Center to map the simulation environment based on longitude and latitude. U2USim includes dynamic temperature, humidity, smoke, and airflow sensor information 170

Table 4.1	Comparative analysis of key enabling technologies for self-evolving embodied AI	200
Table 4.2	The key components of self-evolving embodied AI, highlighting how each work advances world modeling, embodied cognition, and dynamic interaction	206

Part I

Autonomous Embodied Intelligence

To make this book self-contained and cutting edge, **Part I** will cover basic concepts, key technologies, key components, key architectures, key applications, simulators, datasets, and metrics of embodied AI, autonomous embodied AI, and autonomous swarm embodied AI.

Chapter 1

Embodied AI



Abstract Embodied Artificial Intelligence (AI) is a paradigm toward Artificial General Intelligence (AGI), bridging cyberspace and physical systems. This chapter reviews embodied AI from foundations to advances, including history, key technologies, components, architectures, applications, simulators, datasets, and evaluation metrics. Recent progress in Large Language Models (LLMs) and world models has greatly advanced this field. LLMs enhance embodied AI through semantic reasoning and task decomposition, enabling high-level instructions and natural language actions. World models provide internal representations and predictive capacities, supporting interactions consistent with physical laws. This chapter introduces LLM/MLLM-based and world model-based approaches. It explains their roles in embodied cognition and interaction. It also advocates a joint architecture that combines MLLMs and world models to enable complex task execution in physical environments.

1.1 The Overview

Embodied Artificial Intelligence (AI) originated from the Embodied Turing Test by Alan Turing in 1950 [365], which was designed to explore whether agents can imitate human intelligence to achieve Artificial General Intelligence (AGI). Among them, agents that only solve abstract problems in the digital world (cyberspace) are generally defined as disembodied AI, while those that can also interact with the physical world are regarded as embodied AI [116]. Embodied AI builds on foundational insights from cognitive science and neuroscience [76, 284], which claim that intelligence emerges from the dynamic coupling of perception, cognition, and interaction. As shown in Fig. 1.1, embodied AI includes three key components in a closed-loop manner, i.e., (1) active perception (sensor-driven environmental observation), (2) embodied cognition (historical experience-driven cognition updating), and (3) dynamic interaction (actuator-mediated action control). Besides, hardware embodiment [66, 143, 183] is also critical due to escalating computational and energy demands, particularly under the latency and power constraints of devices in real-world deployment scenarios.

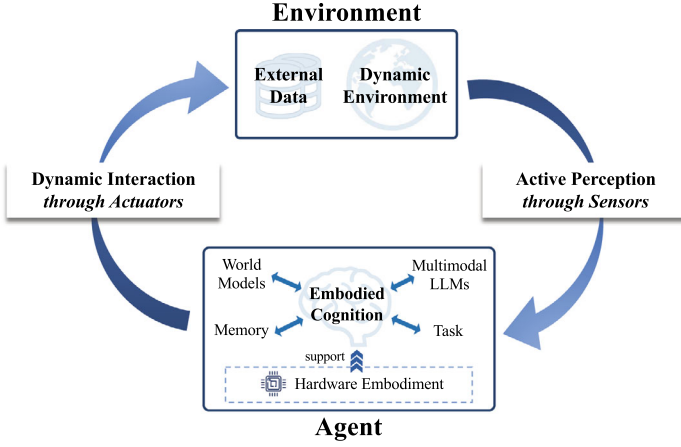


Fig. 1.1 The concept of embodied AI

The development of embodied AI has evolved from an unimodal to a multimodal paradigm. In the early stage, embodied AI was primarily studied by focusing on individual components with a single modality, such as vision, language, or action. The perception, cognition, or interaction component is driven by one sensory input [106, 265], e.g., perception tends to be dominated by the visual modality [249], cognition tends to be dominated by the language modality [389, 411], and interaction tends to be dominated by the action modality [173, 227]. Although these methods perform well within individual components, they are limited by the narrow scope of information provided by each modality and the inherent gaps between modalities across components. The continued development of embodied AI has revealed the limitations of unimodal approaches, promoting a significant shift toward integration of multiple sensory modalities [100, 176, 397]. As such, multimodal embodied AI [293, 432] naturally arises to create more adaptive, flexible, and robust agents capable of performing complex tasks in dynamic environments.

Large Language Models (LLMs) empower embodied AI via semantic reasoning [42] and task decomposition [9, 121], bringing high-level natural language instructions and low-level natural language actions into embodied cognition. Representative LLM-driven works include SayCan [6], which i) provides a real-world pretrained natural language action library to constrain LLMs from proposing infeasible and contextually inappropriate actions; ii) uses LLMs to convert natural language instructions into natural language action sequences; and iii) utilizes value functions to verify the feasibility of natural language action sequences in a particular physical environment. These works suggest that LLMs are extremely useful to robots that aim at acting upon high-level, temporally extended instructions expressed in natural language. However, LLMs are only a part of the entire embodied AI system (e.g., embodied cognition), which is limited by a fixed natural language action library and a specific physical

environment, making it difficult for LLM-driven embodied AI to achieve adaptive expansion for new robots and environments.

Recent breakthroughs in Multimodal LLMs (MLLMs) [41, 340] and world models [95, 134, 211] have opened up a new frontier in embodied AI research. MLLMs can act on the entire embodied AI system, bridging high-level multimodal input and low-level motor action sequences into end-to-end embodied applications. Semantic reasoning [44, 102, 221] leverages MLLMs’ cross-modal comprehension to interpret semantics from visual, auditory, or tactile inputs, e.g., identifying objects, inferring spatial relationships, and predicting environmental dynamics. Concurrently, task decomposition [38, 161, 471] employs MLLMs’ sequential logic to break complex objectives into sub-tasks while dynamically adapting plans based on sensor feedback. However, MLLMs often fail to ground predictions in physics-compliant dynamics [238] and exhibit poor real-time adaptation [99] to environmental feedback.

On the other hand, world models empower embodied AI by building internal representations [47, 58, 307, 394, 407] and making future predictions [140, 141, 274, 405] of the external world. Such world model-driven embodied AI facilitates interactions that comply with physical laws in dynamic environments. Internal representations compress rich sensory inputs into structured latent spaces, capturing object dynamics, physics laws, and spatial structures, as well as allowing agents to reason about “what exists” and “how things behave” in their surroundings. Simultaneously, future predictions simulate potential rewards of sequence actions across multiple time horizons aligned with physical laws, thereby preempting risky or inefficient behaviors. However, world model-driven approaches struggle with open-ended semantic reasoning [78] and lack the ability of generalizable task decomposition [134] without explicit priors.

This chapter reviews embodied AI from foundations to advances, including history, key technologies, components, architectures, applications, simulators, datasets, and evaluation metrics. Building upon the above advances, we further share our insights on the necessity of developing a joint architecture of MLLM and world model for embodied AI, shedding light on its profound significance in enabling complex tasks within physical worlds. MLLMs enable contextual task reasoning but overlook physical constraints, while world models excel at physics-aware simulation but lack high-level semantics. Jointly leveraging MLLMs and world models can bridge semantic intelligence with grounded physical interaction. For instance, EvoAgent [117] designs an autonomous-evolving agent with a joint architecture of MLLM and world model, which can autonomously complete various long-horizon tasks across environments through self-planning, self-reflection, and self-control, without human intervention. We believe that designing a joint architecture of MLLMs and world models will dominate next-generation embodied systems, bridging the gap between specialized AI agents and general physical intelligence.

1.2 Key Technologies of Embodied AI

Embodied Artificial Intelligence (embodied AI) concerns the development of agents that operate within and respond to the complexities of simulated or physical environments. These agents must continuously perceive, interpret, and act upon their surroundings while adapting to dynamic changes and uncertain feedback. Enabling such capabilities requires not only advanced perception and control mechanisms but also a deeper integration of learning, reasoning, and interaction. Over time, several foundational technologies have proven indispensable in supporting the development of embodied AI that are adaptive, robust, and capable of generalization.

This section presents five key technologies as shown in Fig. 1.2. These technologies constitute the methodological backbone of embodied intelligence: Reinforcement Learning, Continual Learning, Imitation Learning, Multi-modal Large Language Model, and World Model. Each of these areas addresses a distinct dimension of the embodied learning process. Reinforcement Learning provides a formal framework for sequential decision-making through environmental feedback. Continual Learning equips agents with the ability to accumulate and retain knowledge over time. Imitation Learning facilitates knowledge acquisition from expert behavior. Multi-modal Large Models enable integrated understanding and interaction across diverse sensory modalities. World Models support internal simulation and long-horizon reasoning. These technologies form a coherent and interdependent foundation for building capable and autonomous embodied agents.

1.2.1 Reinforcement Learning

Reinforcement learning (RL) provides a fundamental framework for autonomous agents to acquire goal-directed behavior through trial-and-error interaction with their environment. Unlike supervised learning, which relies on labeled input-output pairs,

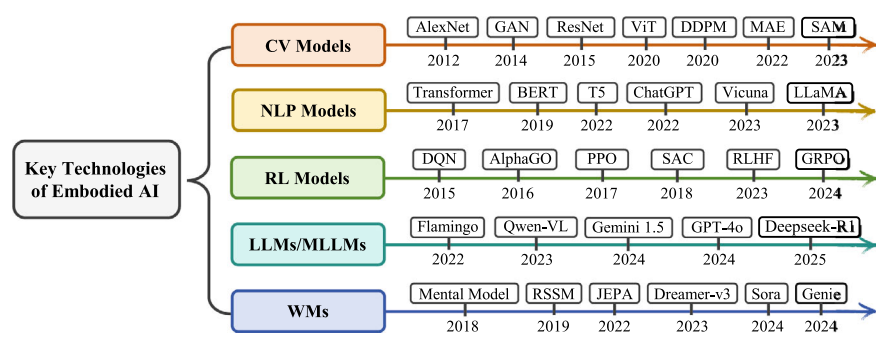


Fig. 1.2 Key technological advances in embodied AI

RL is characterized by the absence of explicit instruction, instead using scalar reward signals as the only feedback. This setup reflects the conditions under which biological organisms often learn and adapt, making RL particularly well-suited for embodied AI. Within this paradigm, an agent seeks to learn a policy that maps observations or states to actions so as to maximize the expected cumulative reward over time, even in the face of uncertainty, delayed consequences, and complex dynamics.

Formally, RL problems are often framed as Markov Decision Processes (MDPs), defined by a tuple (S, A, P, R, γ) , where S denotes the set of states, A the set of actions, $P(s' | s, a)$ the transition probability between states, $R(s, a)$ the reward function, and $\gamma \in [0, 1)$ the discount factor encoding the preference for immediate over future rewards. At each time step, the agent observes the current state s_t , selects an action a_t , receives a scalar reward r_t , and transitions to a new state s_{t+1} . The objective is to learn a policy $\pi(a | s)$ that maximizes the expected return, typically defined as the discounted sum of future rewards $\mathbb{E}_\pi[\sum_t \gamma^t r_t]$.

A. Taxonomy of RL Methods

RL algorithms are traditionally classified into three categories: value-based methods, policy-based methods, and actor-critic methods that combine elements of both. Each category reflects a distinct mechanism for how value and policy are represented or optimized. The following subsections provide an expanded account of each type.

Value-based Methods. Value-based methods aim to learn a value function that estimates the long-term utility of taking a specific action in a given state. The most commonly used value function is the action-value function $Q(s, a)$, which represents the expected cumulative reward starting from state s , taking action a , and thereafter following a policy. Once the value function is approximated, a policy can be implicitly derived by selecting actions that maximize the estimated value. A seminal contribution in this category is the Deep Q-Network (DQN) [261], which demonstrated that convolutional neural networks could be used to learn Q -functions directly from raw image observations. To stabilize training, DQN introduced two critical techniques: a target network to reduce oscillations and divergence in learning, and experience replay to break temporal correlations by uniformly sampling from a memory buffer. The learning objective in DQN involves minimizing the temporal-difference error as Eq. (1.1):

$$\mathcal{L}(\theta) = \left(Q_\theta(s_t, a_t) - \left[r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a') \right] \right)^2, \quad (1.1)$$

where θ and θ^- are parameters of the current and target network, respectively.

Building on DQN, further developments improved performance and stability. Double DQN [368] addressed the overestimation bias by decoupling the action selection and evaluation steps. Dueling DQN [398] separated the value of a state from the advantage of a particular action using Eq. (1.2).

$$Q(s, a) = V(s) + A(s, a). \quad (1.2)$$

This decomposition improves generalization by allowing the model to assess state values independently of specific actions. Value-based methods remain highly effective in discrete and low-dimensional action spaces. However, their applicability to continuous control tasks is limited, as the requirement to compute the maximum over a continuous action space is often computationally intensive or intractable.

Policy-based Methods. Policy-based methods directly optimize a parameterized policy $\pi_\theta(a | s)$ by maximizing the expected return, offering a fundamental alternative to value-based approaches. These methods are particularly effective in high-dimensional or continuous action spaces, where deriving policies from value estimates is often impractical or unstable. Their theoretical foundation lies in the policy gradient theorem [351], which provides a formal expression for computing the gradient of the expected return with respect to the policy parameters as Eq. (1.3).

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) \cdot A^\pi(s, a)], \quad (1.3)$$

where $A^\pi(s, a)$ is the advantage function estimating the relative value of an action over the expected baseline.

The REINFORCE algorithm [403] represents an early application of this idea, using Monte Carlo estimates of return and updating the policy via stochastic gradient ascent. Despite its conceptual simplicity, REINFORCE suffers from high variance and poor sample efficiency. Later improvements introduced trust-region constraints to ensure stable updates. Trust Region Policy Optimization (TRPO) [318] optimizes the policy under a constraint on the KL divergence between the new and old policies. However, its reliance on second-order optimization made it less practical. To address this, Proximal Policy Optimization (PPO) [319] proposed a clipped surrogate objective as Eq. (1.4).

$$L^{\text{clip}}(\theta) = \mathbb{E}_t [\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)], \quad (1.4)$$

where r_t is the ratio between new and old policy probabilities. This simple yet effective regularization prevents overly aggressive updates, making PPO a robust baseline for many continuous control benchmarks.

Policy-based methods enable flexible policy representations and naturally support stochastic exploration, making them particularly effective in continuous and high-dimensional control tasks. They integrate well with deep learning architectures and have demonstrated strong performance in embodied scenarios requiring end-to-end perception and action. However, their reliance on high-variance gradient estimates leads to poor sample efficiency, especially in environments with sparse rewards or long horizons. This limitation has prompted the development of hybrid approaches, such as actor-critic methods, which seek to combine the expressiveness of policy learning with the stability of value estimation.

Actor-critic Methods. Actor-critic methods integrate the strengths of both value-based and policy-based approaches by maintaining two separate components: an actor that proposes actions and a critic that evaluates them. The actor updates the

policy in the direction suggested by the critic’s estimated value, while the critic minimizes the temporal-difference error to improve value predictions. A typical objective in actor-critic methods involves estimating the policy gradient using the critic’s advantage as Eq. (1.5).

$$\nabla_{\theta} J(\theta) = \mathbb{E}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)) \right], \quad (1.5)$$

where V_{ϕ} is the learned state-value function parameterized by ϕ .

Asynchronous Advantage Actor-Critic (A3C) [260] was a milestone that introduced parallel training of multiple agents across diverse environments. This significantly improved both sample throughput and the stability of updates. In continuous control settings, Deep Deterministic Policy Gradient (DDPG) [226] applied deterministic policies with off-policy learning, using a separate target network for both actor and critic. A more recent and powerful variant, Soft Actor-Critic (SAC) [135], incorporated entropy regularization into the objective to promote exploration as Eq. (1.6).

$$J(\pi) = \mathbb{E} \left[\sum_t r_t + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right], \quad (1.6)$$

where $\mathcal{H}$ denotes the policy entropy and α is a temperature parameter balancing exploration and reward optimization.

Actor-critic methods are particularly well-suited for continuous control problems, where both stable learning and expressive policy representations are essential. By combining policy optimization (the actor) with value estimation (the critic), they leverage the advantages of both paradigms: the actor explores the action space and updates the policy, while the critic provides low-variance value estimates to guide this optimization. This synergistic structure enables more sample-efficient learning and improved stability compared to purely policy- or value-based approaches. Actor-critic frameworks have been successfully applied in a wide range of embodied tasks, including locomotion, manipulation, and real-time interaction, where agents must adapt to high-dimensional, continuous, and temporally extended dynamics.

B. Extensions and Advances in RL

While value-based, policy-based, and actor-critic methods form the core of modern RL, they often encounter limitations in terms of sample efficiency, coordination in multi-agent scenarios, and learning over long temporal horizons. To overcome these challenges, several extensions have been proposed that incorporate structured models, agent interaction dynamics, and hierarchical control. These advances significantly broaden the applicability of RL in complex embodied environments.

The core reinforcement learning methods introduced above have achieved notable successes. However, they often suffer from poor sample efficiency, especially in high-dimensional environments with sparse or delayed rewards. To address this issue, model-based reinforcement learning introduces an explicit model of environment dynamics, enabling agents to simulate future trajectories and learn from imagined

experiences rather than relying solely on real-world interactions. One of the earliest examples, PILCO [90], demonstrated remarkable data efficiency by employing probabilistic dynamics models with analytic policy gradients. Later developments, such as World Models [134], incorporated generative latent dynamics to support planning in pixel-based environments. The Dreamer family of algorithms [137–139] further advanced this direction by using recurrent state-space models and latent imagination, achieving competitive performance in both discrete and continuous control tasks with significantly reduced environment interaction.

As embodied tasks often involve multiple interacting agents, reinforcement learning has been extended to multi-agent settings. Multi-Agent Reinforcement Learning (MARL) introduces additional complexity due to the non-stationarity induced by other learning agents, as well as the need for coordination or competition. Algorithms such as Multi-Agent Deep Deterministic Policy Gradient (MADDPG) [243], QMIX [299], and Multi-Agent PPO (MAPPO) [446] address these challenges through centralized training and decentralized execution frameworks, attention mechanisms, and value decomposition strategies. These techniques have been applied in domains ranging from collaborative robotic control to strategic game playing.

Hierarchical reinforcement learning presents another line of research focused on temporal abstraction. In this setting, high-level policies select sub-goals or sub-tasks, which are then executed by lower-level controllers. This decomposition facilitates learning in long-horizon environments and promotes skill reuse. Pioneering work such as FeUdal Networks [373] and HIRO [267] has demonstrated the effectiveness of hierarchical architectures in sparse-reward and multi-stage tasks.

C. Frontiers of RL

In recent years, RL has increasingly been integrated with other learning paradigms. Representation learning methods such as CURL [207] and RAD [206] improve the efficiency of pixel-based RL by learning task-relevant embeddings. Meta-learning frameworks like RL² [103] and PEARL [296] enable agents to adapt quickly to new environments based on prior experience. Self-supervised objectives, as demonstrated in UNREAL [166] and DrQ [440], further enhance sample efficiency by augmenting RL with auxiliary tasks.

Reinforcement learning has thus evolved into a rich and diverse field, offering powerful tools for decision-making under uncertainty, long-term planning, and adaptive control. Its relevance to embodied intelligence is particularly pronounced, as it naturally captures the closed-loop interaction between perception, action, and reward that defines intelligent behavior in physical environments. From discrete action game playing to continuous control of robotic systems, from single-agent optimization to multi-agent coordination, and from model-free learning to model-based imagination, reinforcement learning continues to provide both theoretical grounding and practical mechanisms for the development of general-purpose autonomous agents.

As the challenges of real-world embodied intelligence grow in complexity, reinforcement learning will remain central to enabling agents to learn, adapt, and generalize across tasks, environments, and timescales. Its integration with advances in

world modeling, representation learning, and cognitive architectures is likely to play a key role in shaping the next generation of autonomous systems capable of operating robustly in dynamic, uncertain, and unstructured real-world settings.

1.2.2 *Continual Learning*

In real-world embodied AI systems, agents are rarely deployed in static environments. Rather, they face streams of tasks, shifting goals, changing dynamics, and extended temporal horizons. To remain robust and adaptive in such settings, agents must learn continuously, accumulating new knowledge while preserving prior competence. This continual learning (CL) paradigm has become a critical research direction situated at the intersection of machine learning, cognitive science, and robotics. It reflects the need for lifelong adaptation and resilience in environments that evolve over time and cannot be fully anticipated in advance.

CL methods aim to achieve two fundamental goals: plasticity, the ability to incorporate new knowledge efficiently, and stability, the capacity to retain previously acquired information without interference. Balancing these two properties is particularly challenging due to a phenomenon known as catastrophic forgetting, where learning new tasks degrades performance on earlier ones [196]. This issue is further amplified in reinforcement learning, where non-stationary data distributions and delayed feedback make knowledge integration more complex. In this section, we examine representative approaches to continual reinforcement learning, including regularization-based methods, memory and replay mechanisms, and task-agnostic or context-aware frameworks. These approaches provide essential tools for building agents capable of long-term adaptation in dynamic and open-ended environments.

A. Taxonomy of CL Methods

CL methods address the challenge of acquiring and retaining knowledge across sequential tasks or data distributions. As models are exposed to new information over time, they must adapt without compromising previously learned capabilities. To support this objective, a range of methods have been developed, which can be broadly classified into three categories: regularization-based methods, memory-based methods, and dynamic architectural strategies. These categories reflect different assumptions about task structure, data accessibility, and computational constraints. In the following sections, we examine representative techniques within each category, highlighting their mechanisms, strengths, and limitations in the context of long-term, adaptive learning.

Regularization-based Methods. Regularization-based methods aim to mitigate catastrophic forgetting by constraining the parameter updates that are most likely to interfere with previously acquired knowledge. These methods introduce additional terms to the loss function that penalize changes to parameters identified as important for earlier tasks, thereby promoting stability while allowing continued plasticity.

One of the most influential approaches in this category is Elastic Weight Consolidation (EWC) [196]. EWC uses the Fisher Information Matrix to estimate the importance of each parameter based on its contribution to the performance of prior tasks. A quadratic penalty term is added to the objective function, discouraging deviation from these important parameters during the learning of new tasks. EWC has been successfully applied across supervised and reinforcement learning domains, including benchmarks such as Atari and DeepMind Lab, and has served as the foundation for many subsequent developments.

Several extensions have been proposed to improve the scalability and adaptability of EWC. For example, Online EWC [320] replaces static task-specific estimation with an online update rule, making it applicable to streaming scenarios. Synaptic Intelligence (SI) [456] further enhances computational efficiency by calculating importance measures without revisiting earlier data during training. In more complex and uncertain environments, particularly within reinforcement learning, Bayesian extensions have been explored. Elastic Variational Continual Learning (EVCL) [30] combines variational inference with regularization by maintaining probabilistic representations over parameters, allowing agents to manage task transitions more effectively.

Beyond probabilistic modeling, biologically inspired mechanisms offer additional perspectives. Memory Aware Synapses (MAS) [11] proposed a data-driven regularization approach based on gradient alignment, enabling task-free estimation of parameter relevance. Similarly, Synaptic Intelligence (SI) [456] introduced a formulation motivated by synaptic consolidation in biological systems, where the stability of learned connections is modulated by learning dynamics. These approaches collectively demonstrate that integrating task importance and synaptic stability into the optimization objective provides a principled and effective way to support continual learning in both artificial and embodied settings.

Regularization-based methods provide a theoretically grounded and computationally efficient approach to continual learning. They have been shown to perform well in scenarios where task boundaries are known or can be approximated, and where parameter reuse is critical. However, they may be less effective in highly non-stationary settings or when task information is unavailable, motivating the development of complementary paradigms.

Replay-based Methods. Replay-based methods address catastrophic forgetting by retaining past data and interleaving it with new experiences during training. Inspired by complementary memory systems in the human brain, particularly the hippocampus-neocortex interaction, this approach enables models to consolidate long-term knowledge while acquiring new information. The fundamental idea is to approximate the joint distribution of all observed tasks by mixing samples from the current task with representative examples from previous ones. A common implementation involves a memory buffer $\mathcal{M}$, which stores a sub-set of previously seen data points. When the model encounters new data $\mathcal{D}_t$, it is trained on the union $\mathcal{D}_t \cup \mathcal{M}$, typically minimizing a total loss as Eq. (1.7).

$$\mathcal{L}_{\text{total}} = \mathcal{L}(\mathcal{D}_t) + \lambda \mathcal{L}(\mathcal{M}), \quad (1.7)$$

where λ controls the relative importance of preserving prior knowledge. To manage memory capacity, sampling techniques such as reservoir sampling and gradient-based selection are employed to ensure diversity and task relevance.

Experience Replay [311] demonstrated that even small buffers can effectively reduce forgetting, particularly when combined with strategic sampling. Subsequent methods like Multi-Timescale Replay (MTR) [186] enhanced retention by storing data at multiple temporal scales, allowing the model to capture both short-term and long-term dependencies. Hybrid systems have also emerged that combine fast-changing episodic memory with more stable semantic memory to maintain both flexibility and stability over extended learning periods [89].

When storing raw data is infeasible due to privacy, memory, or legal constraints, generative replay offers an alternative. In this approach, a generative model is trained to reproduce prior task data distributions. Deep Generative Replay (DGR) [331] introduced this idea in classification settings, where a generator produces synthetic samples from past tasks while the learner adapts to new data. Variational Continual Learning (VCL) [271] further integrated generative replay with Bayesian neural networks, jointly learning task-specific posteriors and using sampled data to regularize updates. While replay-based methods are among the most widely adopted and empirically successful approaches for continual learning, they are not without limitations. Key challenges include constrained memory capacity, sampling bias, and the potential degradation of generative quality over time, particularly in long training horizons or high-dimensional domains.

Dynamic Architectural Methods. Dynamic architectural methods address continual learning by modifying or extending the model’s structure to accommodate new tasks, contexts, or distributions. Unlike regularization- or replay-based approaches that operate under a fixed architecture, these methods introduce flexibility at the architectural level to separate task representations, promote knowledge retention, and facilitate adaptive learning in the absence of explicit task boundaries.

One widely studied approach involves context-dependent routing, where a gating function determines the activation of sub-modules according to the input. Let x denote the input and $\{f_1, f_2, \dots, f_M\}$ a set of functional modules. A gating function $g(x) \in [0, 1]^M$ assigns weights to each module, producing the composite output as Eq. (1.8).

$$f(x) = \sum_{i=1}^M g_i(x) \cdot f_i(x). \quad (1.8)$$

This mechanism allows the model to specialize different sub-sets of parameters for different inputs, thereby reducing interference across tasks. The Context-dependent Gating mechanism [254] utilizes binary gates pre-defined per task to enforce sparse activation, while more recent extensions [372] employ learned context embeddings to guide module selection in a task-agnostic manner. These gating strategies have demonstrated improved robustness in CL scenarios by effectively partitioning representational capacity.

Another important line of research utilizes hyper-networks to conditionally generate model parameters based on contextual embeddings. In this setting, a hypernetwork H maps a context vector $c \in \mathbb{R}^d$ to the parameters $\theta \in \mathbb{R}^n$ of a target network f_θ . This formulation allows the target network to adapt its behavior according to inferred task context without requiring separate models for each task. A representative hypernetwork-based method [375] demonstrated that such conditional parameter generation enables continual adaptation while preserving prior knowledge, even in the absence of explicit task boundaries or data replay.

In addition to selective routing and parameter conditioning, some methods focus on expanding the network structure over time. The Dynamic Expandable Network (DEN) framework [444] incrementally increases model capacity by monitoring representational drift during training. When existing parameters are no longer sufficient to represent new data, additional neurons or layers are introduced. Previously acquired knowledge is preserved by selectively freezing important parameters, determined via sparsity-based importance criteria. This controlled expansion strategy provides a mechanism to accommodate novel information while mitigating interference with previously learned representations.

Dynamic architectural methods offer a flexible and scalable foundation for continual learning, particularly in environments where task boundaries are ambiguous or entirely absent. By supporting mechanisms such as selective specialization, conditional parameter adaptation, and adaptive structural growth, these approaches enable models to incrementally incorporate new knowledge while minimizing interference with previously acquired representations. Their ability to dynamically reconfigure computational pathways in response to evolving data distributions is essential for building agents capable of sustained, long-horizon learning in complex, open-ended settings where traditional assumptions about task structure may not hold.

B. Extensions and Advances in CL

Continual learning has evolved beyond its foundational paradigms of regularization, memory replay, and architectural adaptation. As the complexity of learning environments increases, newer formulations have emerged to address the nuanced challenges posed by real-world settings, including task ambiguity, domain shifts, and multi-modal sensory integration.

One important direction involves task-agnostic continual learning [12, 457], where the learner is not provided with task boundaries during training. Instead of relying on externally defined segmentation, models must autonomously infer task structure or operate effectively without it. Notable approaches include the use of task-inference networks, Bayesian nonparametric methods, and contrastive representation learning techniques that cluster experiences based on latent structure rather than task identity. These techniques are particularly relevant in embodied contexts where task delineation may be ambiguous or inherently fluid.

Another extension centers on multi-modal CL, where agents must integrate and retain knowledge across multiple sensory modalities, such as vision, language, and proprioception. This line of work explores the fusion of heterogeneous input streams and their impact on forgetting and knowledge transfer. Strategies such as cross-modal

replay, modality-specific gating, and aligned embedding spaces have shown promise in improving retention and adaptability in multi-sensory settings [345, 447].

CL has also been integrated with meta-learning, yielding systems that not only accumulate knowledge over time but also become better learners in the process. These meta-continual learning frameworks aim to optimize not just performance, but the underlying learning dynamics, enabling faster adaptation to new tasks and improved generalization. Methods such as Meta-Experience Replay [305] and ANML [32] exemplify this trend by leveraging episodic memory in tandem with meta-learned update rules. As the field continues to mature, the frontiers of CL will require not only more powerful algorithms, but also deeper theoretical insights and more realistic evaluation frameworks. The ability to learn continually and autonomously in unstructured, multi-modal, and temporally extended environments is likely to remain a cornerstone capability for general-purpose embodied intelligence.

1.2.3 *Imitation Learning*

In many embodied AI scenarios, the explicit design of a reward function or reliance on extensive trial-and-error exploration is often infeasible, costly, or unsafe. Imitation Learning (IL), also referred to as learning from demonstration or apprenticeship learning, provides an alternative paradigm in which agents acquire behavioral policies by observing expert demonstrations. This approach is particularly valuable in contexts where human expertise is readily available, real-world interaction is expensive, or operational safety is a primary concern. In robotics and embodied systems, imitation learning helps agents learn skills quickly. It embeds structured knowledge through demonstrations and reduces the need for trial-and-error exploration.

A. Taxonomy of IL Methods

This section reviews both foundational and contemporary methods in imitation learning. We structure the field under three principal themes: behavior cloning, adversarial imitation, and hierarchical imitation methods. Each category represents a distinct modeling approach, reflecting varying assumptions about data availability, supervision, and generalization needs. Behavior cloning treats imitation as a supervised learning problem, providing a simple yet widely used baseline. Adversarial methods draw from generative modeling and reinforcement learning to match expert behavior without explicitly modeling rewards. Hierarchical and meta-imitation methods aim to enhance flexibility and data efficiency, enabling agents to acquire transferable skills and adapt to new tasks from limited demonstrations. The following sections present key ideas, representative algorithms, and insights within each of these methodological directions.

Behavior Cloning. Behavior Cloning (BC) is the most straightforward approach to imitation learning, formulating the problem as a supervised learning task that maps observations or states directly to expert actions. One of the earliest successful

implementations was the ALVINN system [286], which used a neural network to control steering based on forward-facing camera images for autonomous road following. This early work demonstrated the feasibility of learning visuomotor policies from demonstration. A recent study [39] extended this idea using deep convolutional networks to perform end-to-end driving in simulated and real-world environments, establishing a modern benchmark for imitation-based control in autonomous vehicles.

While appealing in its simplicity and ease of implementation, BC suffers from a critical limitation known as compounding errors. Since the policy is trained only on expert trajectories, it performs poorly when encountering out-of-distribution states during deployment. Small deviations can quickly accumulate, leading to performance collapse, particularly in long-horizon or safety-critical tasks. To mitigate these limitations, Inverse Reinforcement Learning (IRL) aims to infer the underlying reward function that explains expert behavior. The agent then optimizes a policy using reinforcement learning under this inferred reward. The seminal work [1] introduced this paradigm through apprenticeship learning, demonstrating improved generalization compared to BC. Subsequently, Maximum Entropy IRL [489] has advanced reward inference methods for complex, high-dimensional environments. Comprehensive surveys [162, 276] provide systematic overviews of these approaches. They highlight the strengths and limitations of BC and IRL, contextualizing them within broader probabilistic and decision-theoretic frameworks.

Adversarial IL. Adversarial IL leverages generative frameworks to bypass explicit reward modeling. A seminal contribution is Generative Adversarial Imitation Learning (GAIL) [148], which formulates imitation as a two-player game between a policy (generator) and a discriminator. The discriminator is trained to distinguish expert from agent trajectories, while the policy learns to fool it, producing behavior indistinguishable from the expert. This adversarial setup allows the agent to implicitly capture expert intent without recovering an explicit reward function and has demonstrated strong performance in continuous control benchmarks such as MuJoCo [362].

However, GAIL’s adversarial training process is known to be sensitive to hyperparameters and sample inefficient. To mitigate these drawbacks, Soft Q Imitation Learning (SQIL) [300] reframes imitation as off-policy RL by assigning a fixed positive reward to expert transitions and zero to others. This removes the need for a discriminator and allows the use of standard off-policy algorithms such as DDPG [226] or SAC [135], improving stability and training efficiency while retaining competitive performance. Recent work has also explored structuring demonstrations to enhance imitation performance. Automatic Waypoint Extraction (AWE) [328] segments expert trajectories into high-level waypoints, enabling agents to learn policies that reason over abstract sub-goals. AWE improves data efficiency and generalization, especially in long-horizon robotic tasks where low-level imitation may be brittle.

Hierarchical IL. Hierarchical IL addresses the challenge of scaling imitation to complex, long-horizon tasks by introducing temporal and structural abstraction into policy learning. Rather than learning a monolithic policy that maps observations

directly to actions, hierarchical frameworks decompose behavior into multiple levels of control. A high-level policy selects among sub-policies or behavioral primitives based on task context, while each low-level policy governs fine-grained motor execution over extended time steps. This structure enables skill reuse and improves sample efficiency, especially in environments with delayed rewards or sequential sub-goals.

A representative approach is HiLMDP [209], which jointly learns latent motor primitives and a master policy over those primitives. The method demonstrates strong generalization across locomotion tasks by modularizing policy components. Similarly, CompILE [195] learns to segment expert demonstrations into variable-length behavior segments in an unsupervised fashion, allowing the agent to imitate complex trajectories by composing interpretable sub-skills. These methods share a common principle: by disentangling what to do from how to do it, agents are better equipped to generalize across tasks, transfer skills, and reason over temporal structure. Hierarchical IL thus provides a powerful framework for building scalable and structured behavior models, especially in robotic and embodied domains.

B. Extensions and Advances in IL

Recent advances in IL have broadened its scope beyond conventional single-task, fully supervised settings. One major extension is imitation from observations (IfO), where agents learn solely from expert state trajectories without access to the corresponding actions. This setting is especially useful in real-world applications such as video demonstrations or human teleoperation logs. Methods like Time-Contrastive Networks (TCN) [322] and context translation techniques [237] address this challenge by learning reward surrogates or embedding spaces aligned with expert behavior.

Another direction involves learning from imperfect or sub-optimal demonstrations, which are common in practical scenarios. To handle noisy or inconsistent data, recent methods introduce filtering mechanisms, confidence-weighted loss functions, or robust adversarial formulations. For instance, DAC [203] modifies the GAIL framework to accommodate mixed-quality demonstrations by shaping the reward signal adaptively. These approaches improve the reliability of imitation under limited supervision or uncertain feedback.

Interactive and online imitation learning further expands the applicability of IL to embodied agents that operate in real-time environments. Algorithms like DAgger [312] and its successors enable agents to query expert input during execution, gradually improving policy robustness through iterative data aggregation. IL has also been combined with other paradigms, such as RL and meta-learning, to enhance generalization and adaptability. For example, methods like LfD+RL [268] pretrain policies with demonstrations before refining them via reinforcement signals, leveraging the strengths of both paradigms. This hybrid strategy is particularly effective in sparse-reward tasks or safety-critical settings where exploration is constrained.

C. Frontiers of IL

As IL continues to evolve, its deployment in embodied intelligence systems introduces new challenges and opportunities. Traditional IL methods often assume fixed

environments, full observability, and abundant high-quality demonstrations. In contrast, real-world embodied settings are dynamic, partially observable, and inherently multimodal. These complexities have motivated several emerging research directions that push the boundaries of IL. One key frontier concerns imitation under partial observability. Embodied agents often operate with occluded sensory inputs, delayed feedback, or incomplete state information. Addressing this requires integrating memory-based architectures, recurrent policy models, and latent inference mechanisms. For example, POMDP-aware imitation frameworks [491] combine variational Bayesian inference with recurrent policies, enabling agents to replicate expert behavior even under significant perceptual uncertainty.

Another active direction is multimodal imitation learning, where demonstrations are provided through diverse sensory channels such as vision, language, audio, and proprioception. This is particularly relevant to human-robot interaction, where instructions may be conveyed via natural language or gesture. Recent work has focused on cross-modal alignment, attention-based fusion, and modality-specific adaptation strategies [171], all of which contribute to more robust and flexible learning in real-world conditions.

Finally, recent advances seek to improve the adaptability and generalization of IL beyond static and fully supervised paradigms. Cross-embodiment imitation addresses the challenge of transferring demonstrations across differing morphologies, requiring the learning of embodiment-agnostic behavior representations [449]. Interactive imitation learning enables agents to refine their policies through limited expert feedback, rather than relying solely on pre-collected trajectories [45]. In parallel, causality-aware approaches aim to uncover the underlying causal structure of expert actions, facilitating policy transfer to novel settings by capturing intent and effect rather than superficial behavior [105]. Collectively, these directions represent important steps toward more autonomous, generalizable, and context-aware imitation systems for embodied agents.

1.2.4 Multimodal Large Language Model

Multimodal large language models (MLLMs) constitute a significant frontier in contemporary artificial intelligence. By integrating vision, language, audio, and, in some cases, action signals within a unified framework, these models extend the capabilities of conventional text-based language models to support perception, reasoning, and interaction across multiple modalities. In embodied and interactive systems, MLLMs serve as central components that bridge high-level semantic understanding with low-level sensorimotor input. Their development has enabled more fluid and naturalistic human-machine interaction and underpins the emerging class of vision-language-action (VLA) models increasingly applied in robotics and real-world settings.

A. Taxonomy of MLLMs

MLLMs extend the core capabilities of LLMs by integrating perception across modalities such as vision, audio, and action. Architecturally, MLLMs build upon the Transformer framework, augmented by modality-specific encoders, cross-modal fusion layers, or unified latent representations. In training, these models adopt strategies ranging from contrastive learning to instruction tuning and generative alignment. The field has seen a convergence toward scalable, unified architectures that support zero-shot generalization and real-world interactive deployment. Below, we review three representative paradigms that have shaped the development of MLLMs.

Contrastive Learning-based Architecture. Contrastive learning-based architecture is a foundational line to achieve modality alignment without requiring explicit supervision. CLIP [294] is a representative model in this category. It employs a dual-encoder structure that separately encodes images and texts into a shared embedding space and trains them to match across paired samples using a contrastive loss. This design allows for scalable training on web-scale noisy datasets and enables zero-shot classification and retrieval tasks without task-specific fine-tuning. Subsequent extensions include FILIP [439], which refines token-level alignment between image patches and text embeddings, and ALIGN [174], which scales CLIP-style training to billions of noisy image-text pairs, improving generalization across diverse visual domains. These contrastive approaches emphasize semantic alignment at the representation level and serve as the foundation for many retrieval and grounding systems.

Unified Multimodal Transformer Architecture. Unified multimodal transformer architecture aims to integrate multiple input modalities within a single Transformer backbone, instead of designing modality-specific encoders. A representative example is Perceiver IO [167], which introduces a latent bottleneck and cross-attention mechanism to flexibly process high-dimensional inputs across diverse domains. This design decouples input size from model complexity and enables scalable multimodal reasoning. Building on this idea, models such as PaLI [69] and Flamingo [9] further demonstrate the potential of unified backbones. PaLI employs a shared encoder-decoder Transformer for multilingual vision-language tasks, while Flamingo inserts cross-modal attention into a frozen LLM, allowing few-shot adaptation to vision-language tasks. These architectures have proven effective across tasks like image captioning, visual question answering, and instruction following, highlighting the versatility and efficiency of unified modeling strategies.

Instruction Tuning Architecture. Instruction tuning architecture provides another effective strategy to equip LLMs with multimodal capabilities. LLaVA [233] represents a representative pipeline in this category. It connects a frozen vision encoder with a pretrained LLM via a learnable projection layer, and then fine-tunes the combined model using synthetic multimodal instruction-response pairs generated by GPT-4. This approach enables vision-language alignment with minimal task-specific data and achieves strong performance in visual question answering, captioning, and dialogue. Subsequent work, such as LLaVA-1.5 [232] and MiniGPT-4 [481], further optimize data efficiency, architecture simplification, and instruction quality.

Instruction-tuned MLLMs thus enable scalable adaptation of language-centric models into multimodal agents with grounded perceptual and reasoning capabilities.

B. Extensions and Advances in MLLMs

Recent developments have demonstrated the practical integration of multimodal large language models into real-world systems. A prominent example is GPT-4V (GPT-4 with Vision), released by OpenAI in late 2023. GPT-4V extends the GPT-4 architecture by accepting visual inputs alongside textual prompts, enabling a wide range of applications, including document understanding, medical image interpretation, and diagram-based reasoning [2]. Its capabilities further converge in GPT-4o, which supports real-time multimodal interaction across text, image, and audio channels. GPT-4o integrates voice synthesis and recognition, vision-based grounding, and multimodal reasoning into a single inference loop, marking a shift toward unified, responsive embodied agents [401].

Google DeepMind’s Gemini family of models pushes the boundaries of multimodal reasoning. Gemini models are trained on large-scale, aligned vision-language-audio data, and exhibit strong performance on multilingual and long-context tasks, including visual dialogue, code interpretation, and video understanding [354]. Complementing this, the open-weight Gemma series extends the PaLI framework to support general-purpose vision-language reasoning with long context windows and efficient fine-tuning strategies [36].

In the domain of embodied AI, Vision-Language-Action (VLA) models exemplify the embodiment of MLLMs. RT-X models fine-tune vision-language foundation models on paired robot trajectories, enabling instruction-conditioned zero-shot generalization to manipulation tasks across different platforms [492]. These approaches have been extended in systems like Helix [118] and Gemini [354] Robotics, which employ modular designs that separate high-level vision-language planning from low-level visuomotor control, yielding scalable and transferable robotic policies in real-world scenarios.

C. Frontiers of MLLM

As MLLMs continue to evolve, several frontiers have emerged at the intersection of multimodal intelligence, real-world interaction, and embodied reasoning. These developments reflect a broader effort to extend MLLMs beyond static benchmarks and curated inputs, pushing toward interactive, open-world agents capable of perception, reasoning, and action in dynamic environments.

One major direction is the integration of MLLMs with embodied systems, enabling perception–language–action reasoning within robotic platforms. Models such as RT-2 [492] and ChatVLA-2 [480] demonstrate that fine-tuning pre-trained vision-language models on robot trajectory data allows agents to generalize knowledge from web-scale corpora to physical manipulation tasks. These systems bridge high-level semantic reasoning and low-level motor control, facilitating real-world zero-shot generalization. Recent advances such as Gemini Robotics and Figure’s Helix further adopt dual-system paradigms, where a high-level MLLM guides decision-making while a low-level controller handles visuomotor precision.

A second frontier centers on multi-turn, interactive reasoning across modalities. Traditional MLLMs are trained on static, paired data, but real-world applications demand multi-step dialogue grounded in perception and memory. Models like GPT-4o incorporate real-time audio and video interaction, enabling synchronous multi-modal conversation, while research in instruction-following (e.g., LLaVA-1.5 [232], PaliGemma [36]) emphasizes temporal grounding and spatial reasoning in interactive settings. Such capabilities are critical for applications like assistive robotics, tutoring systems, and autonomous agents operating in complex physical spaces.

A third active area focuses on continual adaptation in multimodal settings. As agents interact with the world, they encounter new domains, vocabularies, or visual distributions. MLLMs must thus acquire the ability to adapt without catastrophic forgetting. Methods such as memory-based finetuning [262], low-rank adaptation (LoRA) [155], and modular adapter tuning [476] offer promising solutions, but continual multimodal learning remains an open challenge. Moreover, efforts to align MLLMs with causal and embodied priors highlight the need for models that not only interpret but also act meaningfully.

Finally, interpretability and efficiency are emerging as fundamental frontiers. The growing size and complexity of MLLMs raise concerns about opacity and energy consumption. Approaches such as sparse mixture-of-experts [306], structured prompting [145], and token pruning [399] aim to improve efficiency, while saliency-based attention analysis and causal probing are being explored to uncover how MLLMs integrate multimodal information. As deployment expands into safety-critical domains, these concerns become increasingly central.

1.2.5 *World Model*

In the development of autonomous embodied intelligence, the ability to internally model, simulate, and anticipate environmental dynamics constitutes a fundamental capability. The concept of a world model refers not to a static environmental representation, but to a dynamic and structured internal mechanism that supports perception, inference, and decision-making. Through such models, agents are able to learn from imagined experience, reduce dependence on real-world interaction, and pursue long-term goals through internal deliberation. A well-formed world model allows the agent to predict the consequences of its actions, maintain temporal coherence in reasoning, and generate efficient behavioral strategies even under uncertainty.

The philosophical origins of world models can be traced to Immanuel Kant's assertion that understanding arises from internal representations rather than from the mere reception of sensory input. This idea was later expanded by cognitive theorists, including Jay Forrester, who proposed that individuals rely on internal simulations, or mental models, to guide decision-making. Contemporary neuroscience supports this perspective, revealing that the brain constructs and updates abstract representations of space and time to predict future events and organize behavior. In artificial intelligence, this theoretical foundation has motivated the design of models that learn

probabilistic approximations of environment dynamics. These models typically consist of a transition function that estimates the distribution $p(s_{t+1} | s_t, a_t)$, and in many cases also include a reward function $r(s_t, a_t)$. Such internal models provide the agent with the capacity to perform internal rollouts, enabling efficient planning, informed control, and data-efficient policy learning.

Most implementations of world models adopt a modular structure consisting of three core components: a perceptual encoder, a temporal memory unit, and a policy controller. The encoder transforms high-dimensional sensory input into compact latent representations, preserving essential information while discarding irrelevant variation. The memory unit captures temporal dependencies by updating a hidden state based on past observations and actions. This hidden state summarizes the agent’s history and supports predictions about future states. The controller then selects actions based on the current state of memory and perception, guiding either physical interaction with the environment or imagined trajectories used for planning. These components are jointly optimized using prediction-based losses for observations and rewards, often supplemented by variational objectives to regularize the latent space. Together, they form an integrated architecture that enables autonomous agents to simulate outcomes, reason over temporal sequences, and adapt to new situations without extensive real-world data collection.

A. Recurrent State-Space Models (RSSM)

The Recurrent State-Space Model provides a widely adopted architecture for modeling latent dynamics in reinforcement learning environments where observations are partial and sequential dependencies are long-range. RSSM addresses two central requirements in such settings: the need to maintain coherent temporal memory and the need to model uncertainty in perception and transition. To meet these requirements, RSSM introduces a hybrid latent structure consisting of a deterministic hidden state and a stochastic latent variable. This design allows the model to separate structured memory from uncertainty estimation, thereby improving both predictive accuracy and generalization.

At each time step, the model receives the previous hidden state, the previous stochastic state, and the previous action. These are combined to produce an updated hidden state through a recurrent function, formalized as $h_t = f(h_{t-1}, z_{t-1}, a_{t-1})$. Based on this hidden state and the current observation, the model infers a posterior distribution over the new latent variable $z_t \sim q(z_t | h_t, o_t)$, while also maintaining a learned prior distribution $p(z_t | h_t)$. This allows the model to generate predictions during imagination when no new observations are available. The pair (h_t, z_t) is used to reconstruct the observation, predict the reward, and estimate whether the current episode has terminated. The model is trained by minimizing a variational loss function that balances reconstruction accuracy and regularization, typically expressed as the sum of a log-likelihood term and a Kullback-Leibler divergence between the posterior and prior.

The deterministic hidden state acts as long-term memory, enabling the model to encode sequences with temporal consistency. The stochastic latent variable, by contrast, allows the model to represent ambiguity in observations and transitions,

which is essential when the environment is noisy or partially observable. This division of function improves stability during training and leads to more robust planning. After training, RSSM supports imagination-based planning by sampling from the prior and updating the hidden state in latent space without requiring new observations. These simulated rollouts can then be used to update policies and value functions through reinforcement learning, substantially increasing sample efficiency and reducing the burden of environment interaction.

Various extensions of RSSM have been proposed to increase its applicability. Hierarchical versions introduce multiple layers of latent variables to capture information at different abstraction levels, improving performance in multi-task or transfer learning scenarios. Other variants incorporate continuous-time dynamics using neural ordinary differential equations to handle irregularly sampled data. Regardless of specific formulation, the central insight of RSSM remains consistent: by combining structured memory and probabilistic inference in a unified framework, it enables flexible and accurate modeling of sequential experience in autonomous agents.

B. Joint Embedding Predictive Architecture (JEPA)

The Joint Embedding Predictive Architecture introduces a fundamentally different approach to world modeling by shifting the modeling objective from reconstruction of raw sensory inputs to prediction in an abstract embedding space. JEPA avoids the complexity of pixel-level or signal-level reconstruction and instead focuses on aligning latent representations of present and future states. This emphasis on representation alignment reflects the observation that high-level semantic and structural information is often more relevant for decision-making than low-level sensory details.

JEPA is composed of six functionally distinct modules. The configurator determines the prediction targets and planning horizon. The perceptor encodes raw observations into latent embeddings that serve as internal state representations. The world model predicts the future embedding given the current embedding and internal memory. The cost evaluator scores these predictions based on task objectives or learned criteria. The action planner selects control signals that minimize expected cost under the predicted dynamics. The short-term memory maintains relevant history to support sequential inference. Together, these components form an end-to-end predictive system capable of forward simulation, cost evaluation, and policy planning in the latent space.

The core principle underlying JEPA is predictive embedding alignment. Given a current embedding, the model learns a function to predict a future embedding, and this prediction is compared to the true future embedding obtained from subsequent observations. The loss function used to measure this discrepancy may take various forms, including mean squared error, cosine similarity, or contrastive divergence. This training objective encourages the model to extract temporally stable, causally relevant features, such as object permanence or motion patterns, while avoiding overfitting to irrelevant sensory variations.

By avoiding the computational overhead of high-resolution generation and focusing on task-relevant abstraction, JEPA improves training efficiency and generalization. Moreover, its latent-predictive formulation allows for seamless integration with

planning and control modules, which can operate directly on embeddings without requiring explicit decoding. JEPA also supports modular training and transfer learning, since each component can be trained independently and reused in new tasks. The architecture aligns well with predictive processing theories in neuroscience, which posit that biological systems continuously anticipate future states in internal representation space and adjust behavior accordingly. JEPA provides a formal computational structure for such mechanisms in artificial agents.

C. Generative World Models

Generative world models are designed to predict future observations conditioned on the current state and action, with the objective of producing realistic and coherent sensory outcomes. These models learn to approximate the distribution over future observations or their latent encodings, enabling the simulation of environment dynamics in a perceptually grounded manner. Such capability is particularly valuable in tasks where visual detail, multimodal integration, or human interpretability are critical for effective decision-making.

A standard generative world model consists of three components: an encoder that maps observations to latent variables, a transition model that predicts the next latent state based on the current latent state and action, and a decoder that reconstructs the predicted observation. The model is trained using a reconstruction loss combined with regularization terms that constrain the latent space, often using variational inference techniques. The result is a model capable of generating sequences of future observations that are both structurally and semantically coherent.

Recent work has extended generative models into more complex domains. Multimodal generative models such as MUVO combine visual inputs with spatial information to predict three-dimensional occupancy maps, supporting safe and goal-directed navigation. Language-conditioned models integrate textual information into the latent transition process, enabling agents to simulate outcomes in accordance with verbal instructions or dialogue context. In models like the Language World Model, latent trajectories are used to generate natural language descriptions of imagined futures, allowing agents to reflect on internal plans and communicate predictions to human users.

Generative models provide several important advantages. They offer explicit access to predicted sensory outcomes, which can be used to guide vision-based control, perform counterfactual reasoning, or generate human-aligned explanations. However, they also present challenges. High-resolution generation is computationally intensive and often unstable, and accurate synthesis of complex multimodal observations remains an open research problem. To address these issues, recent work has explored hybrid architectures that combine latent-space reasoning with generative decoding, allowing agents to plan efficiently while retaining the ability to visualize or verbalize their internal processes.

D. Frontiers of World Models

World models serve as the internal foundation for perception, imagination, and decision-making in autonomous embodied agents. They provide the structural mech-

anism through which agents can understand the consequences of their actions, learn from simulated experience, and optimize behavior with reduced reliance on external feedback. Across different modeling paradigms, from the hybrid temporal structure of RSSM, to the abstract representation learning of JEPA, and the perceptual richness of generative models, a common goal emerges: to build predictive, generalizable, and semantically meaningful internal representations of the environment.

As these models continue to evolve, their integration into unified agent architectures promises to advance the autonomy, flexibility, and interpretability of intelligent systems. By combining strengths across architecture (e.g., RSSM's temporal coherence, JEPA's latent abstraction, and the expressive power of generative models), future systems may be able to reason abstractly, act adaptively, and interact transparently with humans and the physical world. The world model, in this sense, is not merely a computational tool but a conceptual bridge connecting perception, cognition, and action within intelligent agents.

1.3 Key Components of Embodied AI

Driven by advances in these key technologies, embodied AI has experienced rapid progress. In the following, we present a structured overview of developments in three key components.

1.3.1 *Active Perception*

Active perception refers to the process of increasing information gain from environmental data through deliberate and selective actions. In essence, to understand the world more effectively, an agent actively moves through and explores its surroundings. By continuously sampling the environment via its sensory systems, the agent constructs a perceptual understanding that is fundamentally grounded in purposeful interaction and action. Within this framework, the interpretation of sensory data is inherently intertwined with the behaviors required to acquire it. Perception and action operate in close coordination to support adaptive decision-making. This concept has been most extensively explored in the visual domain, where agents such as animals, humans, robots, or camera platforms dynamically change their positions to improve views of specific objects or use motion to sense environmental structure, such as navigating around obstacles or identifying object boundaries.

From a more formal standpoint, active perception lies at the intersection of perceptual modeling and control strategy. Modeling refers to the representation of sensors, processing modules, and their interactions. These models are typically categorized as either local or global, depending on their spatial and temporal scope. Local models characterize parameters and processes such as lens distortion, focal length, spatial resolution, and band-pass filtering. In contrast, global models describe overall sys-

tem performance and predict interactions among components. The control strategy in active perception involves identifying sequences of actions that minimize a loss function while maximizing information gain. Previous studies [23] have demonstrated the viability of such formulations, including tasks such as 2D image segmentation and 3D shape parameterization. These approaches have enabled agents to estimate object depth using focus cues and stereo convergence.

A closely related but narrower definition of active perception considers perception and action within the brain to be fundamentally unified. According to this view, when a person observes an action, the brain internally simulates it as a potential motor response, interpreting the observed behavior in terms of its executable context. This perspective supports the idea that humans and animals can learn appropriate behaviors by observing others, emphasizing the deep integration between perception and action in biological systems [94]. Overall, active perception refers to the agent selectively acquiring information from environmental observations [5, 55, 113, 397]. Existing active perception methods can be roughly divided into three categories: visual SLAM, 3D scene understanding, and active environment exploration. To offer an effective perspective on active perception approaches, as summarized in Table 1.1, we analyze representative methods along three practical dimensions: sensor type, feature type, and applicable scenarios.

A. Visual SLAM

Simultaneous Localization and Mapping (SLAM) is a pivotal technology enabling agents to both localize themselves and construct environmental maps in unknown environments [249, 315]. The probabilistic SLAM problem originated at the 1986 IEEE Robotics and Automation Conference, where researchers proposed applying estimation-theoretic methods to the challenges of mapping and localization. Initially developed within the field of robotics, SLAM aims to enable an agent to build a map of its surrounding environment in real time using only sensor data, all without any prior knowledge of the environment. At the same time, the agent estimates its own position relative to the evolving map [104].

The Development Path from SLAM to VSLAM. SLAM systems can utilize a variety of sensors to collect data from the environment, including laser-based sensors, acoustic sensors, and visual sensors [454]. Among these, visual sensors encompass a wide range of imaging devices, such as monocular, stereo, event-based, omnidirectional, and RGB-D cameras. Robots equipped with visual sensors rely on the visual data captured by these cameras to estimate their position and orientation relative to the surrounding environment [127]. The process of performing SLAM using visual inputs is specifically referred to as Visual SLAM (VSLAM). Leveraging visual data in SLAM offers several advantages: lower hardware requirements, more intuitive object detection and tracking, and access to rich visual and semantic information [119]. The captured images or video frames can also be repurposed for other vision-based tasks such as semantic segmentation and object detection, as they preserve a wealth of data that can be further processed. These features have made VSLAM a prominent topic in the robotics community and have driven extensive research

Table 1.1 Comparison of three categories of active perception methods: visual SLAM, 3D scene understanding, and active environment exploration (PC: Point Cloud, G: Geometric, V: Volumetric, S: Semantic, M: Multimodal, T: Topological, SC: Scene Understanding)

Method	Year	Sensor type	Feature type	Applicable scenarios
<i>Visual SLAM</i>				
CoSLAM [493]	2012	RGB-D	G + V	Dynamic SLAM
SLAM++ [314]	2013	RGB-D	S	Object-level mapping
ORB-SLAM [265]	2015	RGB-D + Stereo	G	Dynamic SLAM
DS-SLAM [445]	2018	RGB-D	G + S	Dynamic SLAM
TwistSLAM [131]	2022	RGB-D + Stereo	G + S	Dynamic SLAM
GS-SLAM [425]	2024	RGB-D	V	Object-level mapping
<i>3D scene understanding</i>				
Gaudi [31]	2022	RGB	V	General SC
Clip2Scene [63]	2023	RGB + PC	M	Language-guided SC
OpenScene [282]	2023	RGB + PC	M	General SC
Lexicon3D [252]	2024	RGB-D	S	Language-guided SC
GraphDreamer [125]	2024	RGB	T + S	Structured SC
HUGS [475]	2024	RGB-D	M	General SC
RegionPLC [428]	2024	RGB + PC	M	Language-guided SC
<i>Active environment exploration</i>				
MAX [336]	2019	RGB	S	Semantic-guided exploration
Active Neural SLAM [56]	2020	RGB-D	V	Geometry-based exploration
APT [230]	2021	RGB	S	Semantic-guided exploration
Conan [422]	2023	RGB	T	Geometry-based exploration
DBMF-BPI [313]	2023	RGB-D	V	Geometry-based exploration
ActiveRIR [341]	2024	RGB + Audio	M	Cross-modal active perception

and development by experts in both robotics and computer vision over the past few decades. As a result, VSLAM is applicable to a wide range of scenarios requiring 3D environment reconstruction, such as autonomous robotics, augmented reality (AR), and service robotics.

Fundamental Components of VSLAM. Following the standard architecture introduced by [198], most SLAM methods consist of two parallel processing threads: tracking and mapping. One fundamental way to categorize VSLAM algorithms is based on the different approaches and strategies adopted in these two threads. Based on the type of data used, VSLAM methods are typically divided into two main categories: direct methods and indirect (feature-based) methods. Indirect methods extract feature points (or keypoints) from the texture of the scene and track them across frames by matching their descriptors. Although these methods are computationally efficient in the feature extraction and matching stages, they demonstrate strong robustness and accuracy under photometric variations in frame intensity. On the other hand, direct methods estimate camera motion directly from pixel intensity values, constructing an optimization problem to minimize the photometric error. These methods rely on photogrammetry principles, using all output pixels from the camera and tracking their correspondences across frames by modeling constraints such as brightness constancy and color consistency. Such characteristics allow direct methods to capture more detailed information from images, enabling higher-precision 3D reconstructions. However, while direct methods perform better in textureless environments and avoid the computational overhead of feature extraction, they often face large-scale optimization challenges [277]. The respective strengths and weaknesses of each approach have prompted researchers to explore hybrid methods that integrate both. These approaches typically combine the detection stages of direct and indirect methods, where one phase initializes or corrects the other to achieve better overall performance.

Moreover, since a typical VSLAM system consists of a visual odometry (VO) front-end for estimating the local motion of the camera and a SLAM back-end for optimizing the constructed map, there exists a wide diversity in how these components are implemented across different approaches. VO estimates the robot's pose based on local frame-to-frame consistency and passes the result to the back-end for global optimization. Thus, the primary distinction between VSLAM and VO lies in whether global consistency of both the map and trajectory is considered. State-of-the-art VSLAM systems often include two additional modules: loop closure detection and map reconstruction. These components identify previously visited locations, allowing for more accurate trajectory correction and map refinement based on updated camera poses.

As a foundational technology of active perception, visual SLAM has been extensively studied [22, 104]. According to Wang et al. [395], existing methods fall into geometric-based and semantic-based categories. Geometric methods exploit spatial or temporal cues [265], such as dense scene flow [10, 193], triangulation consistency [493], and graph structure [85, 101], performing well in static settings but

struggling with dynamic scenes. In contrast, semantic methods improve localization and mapping in dynamic environments by leveraging high-level information.

Semantic VSLAM. VSLAM has experienced rapid progress due to its advantages, such as low sensor cost, ease of integration with other sensing modalities, and the ability to capture rich environmental information. Traditional vision-based SLAM methods have achieved remarkable results; however, their performance may decline in complex or challenging environments. The development of deep learning has significantly advanced computer vision and has attracted growing attention in its application to SLAM. In this context, the integration of semantic information, which provides a high-level understanding of the environment, has become increasingly important. By leveraging semantic cues, robots can more effectively interpret their surroundings and enhance the robustness and intelligence of SLAM systems. Representative early methods include SLAM++ [314], integrating object-level semantics, and DS-SLAM [445], applying deep learning to dynamic scene understanding. Recent models such as TwistSLAM [131] and GS-SLAM [425] further enhance robustness by combining geometric optimization with semantic or generative modeling.

In general, semantic VSLAM methods consist of the following four main components: **Tracking Module:** This module estimates the camera pose and constructs 3D map points using 2D feature points extracted from consecutive video frames. Through pose estimation and 3D point construction, it establishes the foundations for localization and mapping, respectively. **Local Mapping Module:** By processing two consecutive video frames, this module generates new 3D map points. These points, together with the bundle adjustment (BA) module, are used to refine the estimated camera pose. **Loop Closure Module:** This module compares keyframes based on their extracted visual features and evaluates their similarity to detect previously visited locations. Once a loop is detected, it adjusts the camera trajectory and optimizes the constructed map for global consistency. **Non-Rigid Context Culling (NRCC):** The primary goal of NRCC is to filter out temporally dynamic objects from video frames, thereby reducing their negative impact on the localization and mapping stages. It mainly involves a masking or segmentation process to isolate unstable instances in the scene, such as humans. By reducing the number of feature points involved in the optimization process, NRCC simplifies computation and improves system robustness. By incorporating semantic-level information into VSLAM, these methods can reduce uncertainty in pose estimation and map reconstruction. However, a key ongoing challenge lies in how to effectively leverage the extracted semantic information without introducing significant computational overhead.

B. 3D Scene Understanding

Scene understanding focuses on enabling agents to perceive, segment, and reason about complex environments in a structured and semantically meaningful way. Recent works have advanced this field by integrating vision-language models and generative priors. Early efforts like Gaudi [31] introduced generative models for 3D-aware scene synthesis. Clip2Scene [63] and OpenScene [282] leveraged vision-

language embeddings to facilitate label-efficient and open-vocabulary 3D understanding. Structured scene understanding is further enhanced by Lexicon3D [252] and GraphDreamer [125], which model object-level relations in 3D space through structured representations such as scene graphs or semantic lexicons. Meanwhile, region-level multimodal grounding techniques, exemplified by HUGS [475] and RegionPLC [428], incorporate prompts and spatial grounding to achieve fine-grained, goal-conditioned 3D perception. These methods advance holistic, language-aligned 3D understanding.

3D Object Detection. Recent research on 3D object detection can be broadly categorized into three main approaches: camera-LiDAR projection-based methods, attention-based methods, and cross-modal Transformer-based methods.

Since LiDAR sensors and RGB cameras operate in different coordinate systems, camera-LiDAR calibration requires enforcing geometric consistency between 3D point clouds and 2D multi-view images. To align 3D point features with 2D pixel features, projection-based methods have been proposed. These approaches use camera intrinsics (e.g., focal length, principal point) and extrinsics (e.g., position, orientation) to project 2D image data into 3D space, enabling one-to-one correspondence between 3D points and their projected pixel features. Early frameworks like PointPainting [376] generate 2D semantic segmentation scores and project them onto 3D points to enhance point cloud representations for LiDAR-only detection. Recent methods such as EPNet [160] and EPNet++ [242] introduce new loss functions to enhance consistency in classification, localization, and cross-modal semantics. PointAugmenting [378] further improves point features using CNN-derived 2D cues, which are more robust than segmentation masks. MSMDFusion [179] introduces virtual points by lifting 2D image pixels with semantic information into 3D voxel space via depth completion. These virtual points are adaptively selected and grouped for enhanced multimodal fusion. While these methods excel at detecting large objects near the LiDAR sensor, identifying small or distant objects remains challenging due to sparse point density. To address this, methods like Frustum-PointPillars [280] and Faraway Frustum [460] use 2D instance masks or bounding boxes to construct 3D frustums that localize distant objects. MVP [442] augments the point cloud by generating dense virtual points around foreground frustums to reduce the disparity between nearby and distant object representations, improving detection performance across varying ranges.

Traditional camera-LiDAR projection-based methods often use simple feature fusion techniques like concatenation or averaging, assuming equal importance across modalities. However, RGB images can contain noise due to occlusion or cluttered backgrounds, and naive fusion may impair 3D detection accuracy. To address this, attention-based methods have emerged, adaptively weighting features across modalities [202, 240]. Global fusion methods align multi-modal features in a shared space and apply channel attention. BEVFusion [225] and PA3DNet [386] use Squeeze-and-Excitation (SE) attention [156] to recalibrate channel importance. However, channel attention lacks spatial resolution. DeepFusion [222] addresses this using cross-attention, treating LiDAR features as queries and image features as keys and

values, enabling each 3D point to focus on relevant pixels. Yet, global methods may fail on small or distant objects due to redundant scene-wide information. Local fusion methods like SparseFusion [417] and SFD [408] isolate foreground instances and apply attention at the instance level, reducing background interference. However, they may miss fine 2D cues during instance proposal. Hybrid approaches combine both global and local fusion. 3D-CVF [443] uses channel attention in both stages. AutoAlign [73] and AutoAlignV2 [72] enhance 2D-3D consistency using cross-attention and similarity loss. LoGoNet [220] performs parallel fusion of scene and instance features, followed by self-attention for interaction between them.

Transformer-based architectures have shown strong capabilities in multi-modal 3D object detection due to their strengths in feature alignment and fusion [327, 335, 484]. Typical frameworks use modality-specific Transformers for intra-modal encoding and query-based Transformers for cross-modal interactions. Learnable object queries align object localization across modalities and encode instance-level information. CAT-Det [467] pioneers this direction by introducing Two-stream and Cross-Modal Transformers to capture long-range intra- and inter-modal dependencies. TransFusion [21] applies a two-stage Transformer decoder, initializing bounding boxes from LiDAR and fusing with image features in the second stage. CMT [426] enhances spatial awareness by injecting 3D position information into multi-modal tokens and initializing position-guided queries based on PETR [239]. To reduce false negatives from occlusion and clutter, FocalFormer3D [70] introduces Hard Instance Probing, which focuses training on hard examples by progressively refining object candidates.

3D Semantic Segmentation. Recent multi-modal 3D semantic segmentation methods can be categorized into interactive fusion-based, knowledge distillation-based, and unsupervised domain adaptation (UDA)-based approaches. (1) Interactive Fusion-Based Methods. These approaches integrate features from 2D images and 3D point clouds via cross-modal interaction, often following a U-Net structure. Meyer et al. [256] proposed the first such method by aligning LiDAR points with camera pixels and feeding the fused features into LaserNet [257]. Later works, such as FuseSeg [205], improved feature extraction, while PMF [488] projected features across modalities to handle modal discrepancies. RPNNet [420] adaptively fuses range, point, and voxel views. Lif-Seq [470] introduced coarse-to-fine fusion with offset rectification, and MSeg3D [217] combined geometry- and semantic-based fusion with cross-attention for long-range interactions. (2) Knowledge Distillation-Based Methods. Due to limited camera-LiDAR field-of-view (FOV) overlap [33], these methods transfer 2D knowledge to the 3D encoder during training and discard 2D input at inference. 2DPASS [427] aligns fused and pure 3D features via one-way distillation. CMDFusion [51] further enhances this by storing 2D priors in a memory branch to generate pseudo-2D features, enabling bi-directional fusion during both training and inference. (3) UDA-Based Methods. Labeling 3D point clouds is expensive, making UDA a practical alternative. Cross-domain learning aligns source and target domain predictions, while cross-modal learning aligns 2D and 3D predictions using KL divergence. xMUDA [172] combines cross-modal learning and

self-training, using pseudo-labels to exploit unlabeled data. MM2D3D [50] leverages robust depth and semantic features across domains. However, previous methods often align features one-to-one, neglecting unmatched features. DsCML [281] addresses this via deformable convolutions [84], allowing each 3D point to gather information from multiple relevant 2D pixels through sparse-to-dense learning.

3D Visual Grounding. 3D visual grounding (VG) methods can be categorized into: two-stage detect-then-match and one-stage language-guided approaches. Two-stage methods first generate proposals via a pretrained 3D detector or segmenter [107, 146, 479], then match proposals with language features. Early methods like ScanRefer [59] concatenate features for matching, while ReferIt3D [3] and TGNN [157] introduce relational modeling via graph networks. FFL-3DOG [112] incorporates scene graphs and multi-level relations, and InstanceRefer [452] models instance-level contexts and holistic feature matching. Transformer-based methods further improve feature fusion. LanguageRefer [309] formulates 3D VG as a language modeling task using BERT [93]. 3DVG-Transformer [469] integrates coordinate guided contextual aggregation and multiplex attention. TransRefer3D [147] proposes entity- and relation-aware attention for fine-grained grounding, while MVT [158] uses a multi-view Transformer to learn view-invariant representations via projected multi-view fusion. Some methods incorporate 2D visual cues to overcome the limitations of sparse and noisy point clouds. SAT [438] aligns 2D semantics (labels, features, geometry) with 3D proposals using auxiliary object correspondence loss. LAR [25] generates synthetic 2D views of 3D scenes and fuses them with language and point cloud features via a multi-modal Transformer. However, two-stage methods are limited by the language-irrelevant keypoint sampling of 3D detectors, leading to either missed targets (with sparse proposals) or confusion due to redundant objects (with dense proposals), as noted in [245]. One-stage methods integrate object detection and language-guided feature extraction. 3D-SPS [245] uses language-aware keypoint selection and progressive mining to focus on relevant regions. BUTD-DETR [168], inspired by MDETR [185], employs a Transformer combining proposal and language cues. To achieve finer granularity, EDA [409] decouples textual elements (object, attributes, relations) and aligns them individually with 3D candidates, enabling dense, interpretable matching and reducing ambiguity from irrelevant sentence parts.

3D Dense Captioning. Unlike 2D image captioning that describes global visual content [68, 374, 406], 3D dense captioning demands a finer understanding of object attributes and spatial locations. Existing methods fall into two-stage detect-then-describe and one-stage parallel detect-describe approaches. Two-stage methods detect objects first and then generate captions per proposal. For example, Scan2Cap [71] constructs a relational graph over object proposals and uses context-aware attention for captioning. MORE [178] models complex relations by encoding first-order spatial relations and composing them into triplet graphs to infer multi-order semantics. SpaCap3D [380] explicitly supervises relational learning among proposals to enhance spatial modeling. To enrich semantic representation, some methods integrate 2D image features. While effective, this increases inference cost. X-Trans2Cap [451] addresses this with cross-modal distillation: a teacher network learns from both

2D and 3D data, guiding a student network that uses only 3D input during inference. This maintains high-quality captioning while improving efficiency. One-stage methods overcome two-stage limitations such as dependence on detector quality and lack of joint optimization. Inspired by 3DETR [259], these approaches treat dense captioning as a set-to-set prediction problem. Vote2Cap-DETR [65] introduces learnable vote queries and dual-clued captioning. Each object query encodes spatial bias and semantic context. The decoder jointly produces object boxes and captions, where caption generation is guided by both object-level and surrounding context features. Together, these approaches reflect a shift toward unified, efficient, and context-aware 3D scene understanding for dense captioning.

3D Question Answering. 3D Visual Question Answering (VQA) requires strong spatial understanding and reasoning over complex scenes. ScanQA [19] is a representative 3D VQA approach that predicts answers along with bounding boxes for referred objects in the question. It first encodes object proposals and question words using visual and language encoders, then fuses them via Transformer layers. The fused features are used to estimate object locations, labels, and answers, with auxiliary detection loss enhancing spatial understanding. 3D Situated Question Answering (SQA) extends 3D VQA by incorporating first-person agent information. Given a 3D scene, a question, and a situation description (e.g., agent position and orientation), the model must answer from the agent’s perspective. SQA3D [247] introduces two baselines: a modified ScanQA model with an added location prediction head, and a zero-shot model that leverages large language models (LLMs). The latter uses a 3D captioning system to convert the scene into textual descriptions, which, combined with situation context, are input into LLMs such as GPT-3 [46] or Unified QA [192] to generate answers without task-specific training.

Text-driven 3D Scene Generation. Text-driven 3D scene generation aims to synthesize realistic 3D environments from natural language descriptions. Since collecting large-scale scene-text pairs is impractical [54, 62], many methods leverage pretrained 2D text-to-image diffusion models for supervision. Mesh-based methods, such as SceneScape [123] and Text2Room [150], generate 3D scenes by iteratively updating mesh representations [187]. They combine content priors from diffusion models with geometric priors from monocular depth estimators to construct and refine meshes from multiple views. However, their applicability is mostly limited to indoor scenes due to mesh limitations and the geometric complexity of outdoor environments. Neural radiance field (NeRF)-based methods offer finer detail and fewer artifacts. DreamFusion [287] optimizes NeRFs using Score Distillation Sampling to align 2D renderings with a diffusion model. GAUDI [31] adopts a two-stage framework: disentangling camera pose and scene representation before generative modeling via DDPM [149]. Compositional generation is tackled by Comd3D [285], which edits sub-regions of a VoxNeRF via region-specific diffusion models guided by bounding boxes and text prompts. To enhance modularity and reusability, Set-the-Scene [79] introduces a collection of independent NeRFs as object proxies. It alternates between local optimization of individual objects and global scene opti-

mization. Applying rigid transformations to align each object within the full scene enables compositional and editable scene construction with stylistic consistency.

Open-vocabulary 3D Recognition. Current 3D recognition benchmarks rely on human annotations with closed-set semantics, such as the 20 classes in ScanNet [83], limiting the ability of supervised models to handle open-vocabulary categories. Vision-language foundation models like CLIP [294], trained on large-scale image-text pairs, exhibit strong open-vocabulary reasoning and have inspired new 3D recognition paradigms. CLIP2Scene [64] transfers CLIP knowledge to 3D networks through semantic regularization. It first generates dense pixel-text pairs using MaskCLIP [96], maps them to 3D point clouds via multi-view calibration, and selects contrastive point samples based on text semantics for training. RegionPLC [429] further enhances 3D understanding through regional point-language contrastive learning. It generates dense 2D prompts and captions, aligns them with 3D regions using geometry, and learns discriminative point-level embeddings with reduced noise from irrelevant neighbors. To bridge 2D and 3D modalities, methods such as OpenScene [282] align RGB-D images with point clouds using CLIP-guided contrastive learning. Rather than relying on global alignment, CLIP-FO3D [461] introduces local classification tokens in CLIP to extract regional image features, which are then projected into 3D space. The 3D model is trained to minimize the distance between these projected features and learned point features, enabling region-aware semantic grounding. These approaches leverage CLIP’s knowledge to enable open-vocabulary 3D recognition by aligning point cloud features with language semantics through 2D visual intermediaries.

3D Vision-Language Pretraining. Current 3D vision-language approaches are often task-specific and lack a unified framework for general-purpose cross-modal representation. To address this, 3D-VisTA [486] proposes a pretrained Transformer for 3D scene-text alignment, adaptable to tasks like visual grounding, dense captioning, and question answering. It uses self-attention for both single-modal encoding and multimodal fusion, without task-specific customization. To support pretraining, 3D-VisTA introduces ScanScribe, the first large-scale dataset of 3D scene-text pairs. Using masked language/object modeling and scene-text matching, it learns effective 3D-text alignment. Similarly, 3D-VLP [182] enhances cross-modal understanding through context-aware spatial-semantic alignment and mutual 3D-language masked modeling. It aligns 3D objects with both class nouns and referring expressions, capturing spatial relations. Jointly predicting masked object labels and masked text enables semantic exchange between 3D geometry and language.

C. Active Environment Exploration

Active exploration refers to an agent’s ability to acquire information, build internal representations, and accomplish task objectives through autonomous decision-making and adaptive behavior in unknown or partially known environments. Unlike traditional passive perception, active exploration emphasizes that AI should not only perceive its surroundings but also understand its own state and use this understanding to guide future actions. This capability is fundamental to achieving human-like

intelligence, as humans adapt to new environments, solve problems, and accumulate experience through continual exploration. In the evolution of artificial intelligence, the significance of active exploration goes beyond technical innovation. It marks a shift from static recognition to dynamic cognition [231, 326], enabling machines to operate with greater autonomy and adaptability in complex, ever-changing real-world scenarios. For example, in domains such as robotic navigation, virtual assistant interaction, and autonomous driving, the capacity for active exploration directly impacts a system’s efficiency and reliability.

Active exploration focuses on enabling agents to autonomously acquire informative observations through interaction with the environment. Early approaches relied on building explicit or implicit environmental models. Representative model-based methods include MAX [336] and Active Neural SLAM [56], which leverage predictive modeling and mapping to support efficient navigation in unseen spaces. In contrast, APT [230] and DBMF-BPI [313] focus on model-free exploration through direct environmental interaction to reduce reliance on explicit modeling. Recent efforts further enhance exploration capabilities by incorporating multimodal perception [341] and semantic reasoning [422].

1.3.2 Embodied Cognition

Embodied cognition refers to the emergence of internal representations and reasoning capabilities during the interaction, driven by the agent’s self-reflection on its perception and accumulated experience [77, 130, 490]. This component forms the core of embodied AI, enabling agents to perform task planning [236], causal inference [346], and long-horizon reasoning [321, 462]. Recent studies of embodied cognition primarily focus on three aspects: task-driven self-planning, memory-driven self-reflection, and embodied multimodal foundation models. Table 1.2 presents representative methods analyzed from four perspectives: input modalities, cognition type, reasoning mode, and output type. These dimensions reflect how embodied agents perceive information, form internal models, and conduct reasoning.

Task-driven Self-planning. In task-driven self-planning, agents autonomously generate structured plans based on task goals, environmental context, and internal knowledge, without explicit human instructions [269, 279, 466]. Structured learning is a classical solution that develops latent planning spaces or direct policy mappings, achieving high efficiency within training distributions but lacking robustness to out-of-distribution scenarios. Representative approaches include L3P [463], Egoplaner [477], and ETPNav [14]. Recent advances incorporate LLMs or generative models into self-planning. LLM-Planner [343] and AutoAct [291] integrate LLMs into planning by grounding language-guided reasoning into various tasks, while RPG [431] offers a generative perspective, aiming to unify planning and content creation through multimodal reasoning.

Table 1.2 Comparison of three categories of embodied cognition methods: task-driven self-planning, memory-driven self-reflection, and the embodied multimodal foundation models. I, L, and C indicate the Image, Language, and Point Cloud modalities, respectively

Method	Year	Modalities	Type	Reasoning mode	Output
<i>Task-driven self-planning</i>					
L3P [463]	2021	I + L	Planner	Neural + Symbolic	Action
LLM-Planner [343]	2023	I + L	Planner	Neural + Symbolic	Action
Egoplaner [477]	2023	I	Planner	Symbolic	Action
AutoAct [291]	2024	L	Planner	Neural	Action
RPG [431]	2024	I + L	Planner	Neural	Policy
ETPNav [14]	2024	I + L	Planner	Neural + Symbolic	Policy
<i>Memory-driven self-reflection</i>					
Reflexion [332]	2023	L	Memory	Beam + Replay	Policy
Reflect [241]	2023	I + L	Memory	Neural + Symbolic	Policy
RILA [437]	2024	L	Memory	Neural	Policy
Optimus-1 [223]	2024	I + L	Memory	Neural	Policy
EvoAgent [117]	2025	I + L	Memory	Neural	Policy
REMAC [450]	2025	L	Memory	Neural + Symbolic	Policy
<i>Embodied multimodal foundation models</i>					
SayCan [43]	2022	I + L	Planner + Aligner	Neural	Answer + Action
GATO [301]	2022	I + L + P	Aligner	Neural	Action
EmbodiedGPT [264]	2023	I + L	Aligner	Neural	Answer + Action
Kosmos-2 [283]	2023	I + L	Aligner	Neural	Answer
MultiPLY [154]	2024	I + L	Aligner	Neural	Answer
ManipLLM [221]	2024	I + L	Aligner	Neural	Answer + Action

Memory-driven Self-reflection. Memory-driven self-reflection enables agents to leverage past experiences for long-horizon reasoning, error correction, and self-improvement [117, 468]. Early studies focus on memory processing, including fixed-size replay buffers [24, 298, 483] and differentiable memory architectures [191, 288]. Recent advances introduce reflective mechanisms, where agents summarize or verbalize past experiences to guide future decisions. Reflexion [332] and Reflect [241] enable agents to iteratively self-correct by integrating verbalized feedback into action planning, while RILA [437] extends reflective reasoning to multimodal semantic navigation. Beyond individual reflection, Optimus-1 [223] and REMAC [450] integrate multimodal or multi-agent memory to support long-horizon collaboration. EvoAgent [117] further advances this direction by coupling continual world modeling with a memory-driven planner, enabling fully autonomous evolution across sequential tasks.

Embodied Multimodal Foundation Models. In the era of MLLMs, embodied multimodal foundation models [120, 304, 423] have emerged as one of the most promising solutions for unifying planning, reasoning, and other embodied cognitive capabilities. Recent progress is driven by both data construction and model development. Data efforts focus on constructing high-quality benchmarks to support scalable and cognitively meaningful evaluation, such as MuEP [219], ECBench [86], MFE-ETP [464], and EmbodiedBench [432]. On the model side, recent advances include affordance-grounded agents (e.g., SayCan [43] and GATO [301]) that align language understanding with embodied action spaces, vision-language pretraining approaches (like EmbodiedGPT [264] and Kosmos-2 [283]) that promote scalable embodied reasoning, and object-centric designs (such as MultiPLY [154] and ManipLLM [221]) that enhance manipulation and interaction capabilities. These models collectively aim to build transferable and generalizable embodied AI.

1.3.3 *Dynamic Interaction*

Dynamic interaction refers to the process in which an agent influences the environment through actions or behaviors grounded in its perception and cognition [180, 347]. Existing research highlights the significance of this capability in enabling agents not only to respond but also to change their surroundings [8, 37]. Studies on dynamic interaction encompass action control, behavioral interaction, and collaborative decision-making. To better understand existing methods, we analyze representative approaches from four perspectives, including input modalities, interaction type, learning paradigm, and task type, as shown in Table 1.3. These dimensions reflect how agents sense the environment, determine the level and structure of interaction, and generate appropriate behaviors in dynamic multi-agent or human-in-the-loop scenarios.

Action Control. Action control generates motor commands for embodied interaction. Early methods were based on control theory with dynamic system modeling [189, 308] or RL via trial and error [197, 339]. The former is effective for structured or repetitive tasks, while the latter is adaptable to high-dimensional, nonlinear problems. Recent advances mainly follow three directions. Vision-language-action (VLA) models, such as PaLM-E [100], RT-2 [41], OpenVLA [194], and CogAgent [153], integrate language-guided reasoning for flexible control and have been comprehensively reviewed by Ma et al. [248]. Open-ended frameworks like MineDojo [109] promote continual skill acquisition from open-world knowledge. In addition, Cross-embodiment learning, including CrossFormer [98], HPT [384], and Octo [355], aims to unify policy learning across diverse robots and modalities.

Behavioral Interaction. The behavior of an agent is composed of a sequence of actions. Compared to action control, it emphasizes high-level control through meaningful action patterns, enabling agents to interact in a flexible and goal-directed manner. Recent advances mainly fall into two directions. Imitation learning, includ-

Table 1.3 Comparison of three categories of dynamic interaction methods: action control, behavioral interaction, and collaborative decision-making, compared across input modalities, interaction type, modeling paradigm, and task type. I, L, S, P, and T denote Image, Language, State, Proprioception, and Trajectory, respectively. IL denotes Imitation Learning

Method	Year	Input modalities	Interaction type	Learning paradigm	Task type
<i>Action control</i>					
MineDojo [109]	2022	I + L	High-level planning	LLM	Instruction following
PaLM-E [100]	2023	I + L + P	Low-level control	MLLM	Embodied manipulation
RT-2 [41]	2023	I + L	Low-level control	VLA	Embodied manipulation
OpenVLA [194]	2024	I + L	Low-level control	VLA	Embodied manipulation
Cogagent [153]	2024	I + L	Low-level control	MLLM	Instruction following
Octo [355]	2024	I + L + P	Low-level control	VLA	Embodied manipulation
CrossFormer [98]	2024	I + L	Low-level control	VLA	Embodied manipulation
HPT [384]	2024	I + L	Low-level control	VLA	Embodied manipulation
<i>Behavioral interaction</i>					
GAIL [148]	2016	T	Behavioral	IL	Trajectory learning
MGAIL [27]	2017	T	Behavioral	IL	Trajectory learning
TrafficSim [350]	2021	T	Behavioral	RL	Trajectory learning
TrajGen [465]	2022	I + T	Behavioral	RL	Trajectory learning
Behavior-1K [213]	2023	I	Trajectory	IL	Behavior understanding
AgentLens [244]	2024	I + S	Trajectory	IL	Behavior understanding
ECL [57]	2024	I + L	High-level Planning	IL	Embodied manipulation
<i>Collaborative decision</i>					
QMIX [299]	2018	S	Behavioral	RL	Cooperative decision
Qtran [342]	2019	S	Behavioral	RL	Cooperative decision
QPLEX [382]	2019	S	Behavioral	RL	Cooperative decision
MAT [400]	2022	S	Behavioral	RL	Cooperative decision
CoELA [458]	2024	I + L	Low-level control	LLM	Cooperative manipulation
AgentVerse [67]	2024	L	High-level planning	LLM	Agent society simulation
MetaGPT [152]	2024	L	High-level planning	LLM	Agent society simulation
Combo [459]	2024	L	High-level planning	LLM	Cooperative planning

ing GAIL [148], MGAIL [27], TrafficSim [350], and TrajGen [465], enables efficient acquisition and simulation of complex behaviors. BEHAVIOR-1K [213] provides a large-scale benchmark for evaluating behavior generalization across 1,000 embodied tasks. Behavior-aware enhancement methods, such as AgentLens [244] and ECL [57], improve policy robustness and interpretability. Despite these advances, achieving reliable long-horizon behavioral interaction under sparse feedback remains challenging.

Collaborative Decision. Collaborative decision focuses on coordinating multiple agents to achieve shared goals, which is essential for multi-agent systems and human-robot collaboration [175, 246, 485]. Multi-agent RL is a classical solution, with methods like QTRAN [342], QPLEX [382], and Qatten [299] addressing cooperation via centralized training with decentralized execution. MAT [400] reframes MARL as a sequence modeling problem to mitigate scalability limitations in multi-agent RL. Recent advances integrate LLMs and world models to enhance multi-agent collaboration. MetaGPT [152], CoELA [458], and AgentVerse [67] leverage LLMs for task reasoning and coordination, while COMBO [459] composes modular world models to support scalable collaborative embodied decision.

1.3.4 *Hardware Embodiment*

As embodied AI evolves, model complexity and size have grown, increasing computational and energy demands. Embodied systems, often operating in dynamic, real-world environments, face strict latency and power constraints, especially at the edge. Thus, developing hardware-friendly directions that maintain performance while optimizing efficiency is crucial for enabling responsive, energy-aware embodied agents. Hardware optimization in embodied AI typically includes four components: hardware-aware model compression, compiler-level optimization, domain-specific accelerators, and hardware-software co-design.

Hardware-aware Model Compression. Quantization and pruning [143] are key techniques for reducing model size and computational cost. In embodied agents, which frequently run on low-power embedded hardware, such techniques are vital for enabling fast and efficient inference. Quantization [416] maps weights and activations to lower bit-widths, while pruning [381] removes redundant parameters. To support real-world embodied tasks, such as robotic control or visual navigation, hardware efficiency metrics like power, performance, and area (PPA) can guide bit-width allocation or pruning ratios [383], enabling task-specific trade-offs between accuracy and deployability on physical platforms.

Compiler-level Optimization. Compilers bridge high-level embodied AI models and hardware execution. In real-time embodied systems, compiler toolchains are essential for efficient processing of sensor data and decision-making. TVM [66], built on LLVM [208] and CUDA, generates optimized code across platforms. These com-

plers transform computational graphs through operator fusion and redundant computation elimination [349], enabling responsive behavior. Mapping strategies like loop reordering and tiling enhance data locality, parallelism, and memory access [48], all of which are critical to maintaining low-latency inference in embodied agents.

Domain-specific Accelerators. With growing computational demands, domain-specific accelerators (DSAs) are a promising solution for embodied AI. These systems, from robots to AR/VR agents, benefit from fast, energy-efficient hardware tailored for frequent operations. Google’s TPU [183], typically integrated with CPUs and GPUs via PCIe, accelerates key operations like matrix multiplication. FPGA-based accelerators [234] allow reconfigurability for adapting to new tasks or changing workloads; CGRA accelerators [292] improve structured, dataflow-heavy computations common in perception or control. Meanwhile, ASIC-based accelerators [129] offer high throughput and energy efficiency, ideal for deploying high-performance embodied models in real-world environments.

Hardware-software Co-design. Separating the algorithm and hardware design can lower runtime efficiency. Hardware-software co-design addresses this through algorithm-system and algorithm-hardware co-optimization. Algorithm-system co-optimization focuses on how to take full advantage of GPU resources like tensor cores and CUDA cores to better support the algorithm [229]. Algorithm-hardware co-optimization aims to improve deployment efficiency by tuning both the model and the hardware architecture. For example, we can perform multi-objective optimization based on the types of operators in the network and the configuration parameters of the hardware [385]. We can also design different numerical quantization schemes along with matching hardware accelerators to better support embodied AI tasks [494].

1.4 Key Architectures of Embodied AI

Embodied AI architectures reflect different strategies for integrating perception, cognition, and action within a unified framework. While traditional designs often emphasize modular separation, recent research highlights the necessity of end-to-end architectures capable of bridging high-level semantics with grounded physical interaction. This section reviews three representative directions: LLM/MLLM for embodied AI, world model for embodied AI, and the integration of MLLM and world models for embodied AI. By examining these approaches, we illustrate how architectural design shapes the balance between semantic reasoning and physics-compliant dynamics, offering insights into the construction of scalable and adaptive embodied agents.

1.4.1 LLM/MLLM for Embodied AI

This section provides a comprehensive overview of embodied AI with LLMs/MLLMs. We first elaborate in detail how LLMs boost embodied AI in Sect. 1.4.1A and how MLLMs boost embodied AI in Sect. 1.4.1B. Then we discuss the classification of MLLMs for embodied AI in Sect. 1.4.1C (Fig. 1.3).

A. LLMs Boost Embodied AI

LLMs empower embodied AI via semantic reasoning and task decomposition, bringing high-level natural language instructions and low-level natural language actions into embodied cognition.

Semantic Reasoning. Semantic reasoning [42] relies on LLMs to extract and understand meaning from natural language instructions by examining patterns in text, connecting words to their contexts, and drawing on prior knowledge. Using transformer structures [371], these models translate input words into internal representations that capture multiple layers of meaning, from syntax to real-world context. The attention modules help focus on useful information while filtering out irrelevant noise, supporting both inference and analogy-making. With the knowledge they gain during large-scale pretraining, LLMs combine general world information with specific task inputs to piece together conceptual structures that link what is written with what needs to be done. This mechanism makes it possible for LLMs to carry out step-by-step reasoning by predicting possible next tokens in a way that checks for consistency and makes sure the intended meaning stays clear.

Task Decomposition. Task decomposition [9, 121] employs LLMs’ sequential logic to break complex objectives into sub-tasks by hierarchically analyzing contextual dependencies and goal alignment. Leveraging chain-of-thought prompting, LLMs iteratively parse instructions into actionable steps, prioritizing interdependencies while resolving ambiguities through semantic coherence checks. Representative works like SayCan [6] first provide a real-world pretrained natural language actions library, which is used to constrain LLMs to propose both feasible and contextually

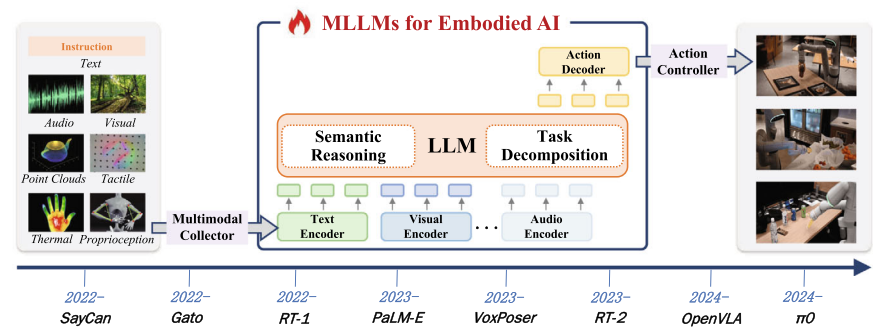


Fig. 1.3 The development roadmap of MLLMs for embodied AI

appropriate actions; then uses LLMs to convert natural language instructions into natural language action sequences; finally uses value functions to verify the feasibility of natural language action sequences in a particular physical environment. These works suggest that LLMs are extremely useful to robots aiming to act upon high-level, temporally extended instructions expressed in natural language. However, LLMs are only a part of the entire embodied AI system, which is limited by a fixed natural language actions library and a specific physical environment, and it is difficult to achieve adaptive expansion in new robots and environments.

B. From Unimodal to Multimodal

The development of embodied AI has evolved from unimodal to multimodal systems, as shown in Fig. 1.4. Initially, embodied AI was primarily concerned with individual modalities, such as vision, language, or action, where the perception, cognition, and interaction were driven by one sensory input [106, 265]. As the field matured, the limitations of unimodal embodied AI became apparent, and there has been a significant shift toward integrating multiple sensory modalities [100, 176, 397]. Multimodal embodied AI is now seen as crucial for creating more adaptive, flexible, and robust agents capable of performing complex tasks in dynamic environments [293, 432].

Unimodal embodied AI has benefited from rapid developments in fundamental areas such as computer vision, natural language processing, and reinforcement learning [173, 227]. These unimodal methods excel in dealing with a specific module in embodied AI. For example, computer vision techniques have driven advances in visual SLAM and 3D scene understanding in the active perception module [249]. Natural language processing techniques, especially LLM, have become popular solutions to address task planning and long-horizon reasoning in the embodied cognition module [389, 411]. Although unimodal embodied AI performs well in independent

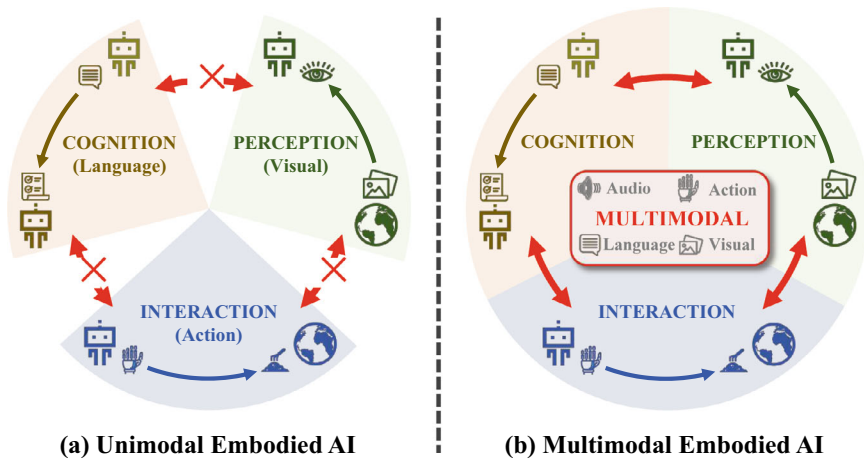


Fig. 1.4 Comparison of unimodal and multimodal embodied AI

modules, it always faces two inherent limitations. On the one hand, the information contained in a single modality is limited, hindering the performance of perception, cognition, and interaction. For example, visual-only systems struggle to understand environments in dynamic or ambiguous settings, while auditory-based systems face challenges in real-world noise and signal processing [293, 390]. On the other hand, diverse and heterogeneous modalities hinder information transfer and sharing among modules. The agent’s perception of the environment fails to facilitate the formation of its cognition, while the evolution of cognition fails to facilitate interaction with the environment.

In contrast, multimodal embodied AI has emerged as a more promising paradigm [432]. By integrating data from multiple sensing modalities, such as visual, auditory, and olfactory feedback, these methods can provide a more holistic and precise understanding of the environment. More importantly, multimodal embodied AI can facilitate deeper integration among perception, cognition, and interaction. Recent advances in MLLMs and world models enable agents to more effectively handle multiple modalities, promising to improve the capabilities of embodied AI [393, 405, 487]. The integration of these models is considered a key step toward enabling multimodal embodied AI in dynamic, uncertain environments.

C. MLLMs Boost Embodied AI

MLLMs can act on the entire embodied AI system and can solve LLMs’ problems well by bridging high-level multimodal input and low-level motor action sequences into end-to-end embodied applications (as shown in Fig. 1.3). Compared with LLMs, semantic reasoning [44, 102, 221] leverages MLLMs’ cross-modal comprehension to interpret semantics from visual, auditory, or tactile inputs, e.g., identifying objects, inferring spatial relationships, or predicting environmental dynamics. Concurrently, task decomposition [38, 161, 471] employs MLLMs’ sequential logic to break complex objectives into sub-tasks while dynamically adapting plans based on sensor feedback. MLLMs mainly include Vision-Language Models (VLMs) and Vision-Language-Action models (VLAs).

VLMs for Embodied AI. VLMs for embodied AI integrate visual and language instruction understanding to enable physical or virtual agents to perceive their environments in goal-driven tasks. Representative works like PaLM-E [100] first train visual and language encodings end-to-end, in conjunction with a pretrained large language model; then incorporate the results of real-world continuous sensor modalities encodings into VLMs and establish the link between words and percepts; finally, achieve multi-task completion through fixed action space mapping. For navigation, ShapeNet [358], which fine-tunes contrastive embeddings for 3D spatial reasoning, greatly reduces path planning errors. These works suggest that VLMs can combine perception and reasoning in embodied AI to solve a large number of tasks with fixed action spaces.

VLAs for Embodied AI. VLAs integrate multimodal inputs with low-level action control through differentiable pipelines. Representative works like RT-2 [41] first encode the robot’s current image, language instructions, and robot actions at a specific

timestep and convert them into text tokens; then use LLMs for semantic reasoning and task decomposition; finally, de-tokenize generated tokens into the final action. Octo [355] pretrains on 100K robot demonstrations with language annotations, achieving cross-embodiment tool use. For dexterous manipulation, PerAct [387] utilizes 3D voxel representations to reach millimeter-level grasp accuracy. These works suggest that VLAs can act on the entire embodied AI system and achieve adaptive expansion in new robots and environments.

D. Representative Works

MLLMs can empower active perception, embodied cognition, and dynamic interaction of embodied AI.

MLLMs for Active Perception. First, MLLMs can enhance 3D SLAM. By grounding visual observations into semantic representations, MLLMs augment traditional SLAM pipelines with high-level contextual information such as object categories, spatial relations, and scene semantics. Representative works like SEO-SLAM [151] utilize MLLMs to generate more specific and descriptive labels for objects, while dynamically updating a multiclass confusion matrix to mitigate biases in object detection. Second, MLLMs can enhance 3D scene understanding. Camera-based perception [44] remains the dominant setup in MLLM-driven embodied AI, as RGB inputs align naturally with the visual-language pretraining of many foundation models [430]. Representative works like EmbodiedGPT [264] leverage this synergy to map 2D visual inputs into semantically rich features aligned with language-based goals. Finally, MLLMs can enhance active environment exploration. MLLMs have also revolutionized how robots interact with their environments, particularly in feedback-driven closed-loop interactions. Representative works like LLM³ [388] focus on structured motion-level feedback, which incorporates signals such as collision detections into the planning loop, allowing the model to iteratively revise symbolic action sequences. MART [453], on the other hand, leverages interaction feedback to improve retrieval quality.

MLLMs for Embodied Cognition. First, MLLMs can enhance task-driven self-planning. Embodied agents with MLLMs can either directly map high-level goals to structured action sequences [38], or adopt an intermediate planning strategy that continually interacts with the environment to refine their plans [161]. Representative works like CoT-VLA [471] predict intermediate sub-goal images that depict the desired outcomes of sub-tasks, helping the agent visualize and reason through each step of a complex task. Second, MLLMs can enhance memory-driven self-reflecting. MLLMs allow agents to learn from experience using this inherent memory module [355]. Representative works like Reflexion [332] enhance agent performance through self-generated linguistic feedback, which is stored in an episodic memory buffer and leveraged to guide future planning. Finally, MLLMs can enhance embodied multimodal foundation models. MLLMs can be adapted to the physical world through continued pretraining or fine-tuning in embodied settings. Representative works include Qwen-VL [20] and InternVL [74], along with models supporting broader modality alignment, such as Qwen2.5-Omni [418].

MLLMs for Dynamic Interaction. First, MLLMs can enhance action control. MLLMs have the ability to decompose complex tasks into actionable sub-tasks [161]. To further produce continuous control signals for each sub-task, MLLMs either generate actions autoregressively in a sequential manner [194, 492] or employ auxiliary policy heads to further process their internal representations [355]. Recent advances also explore generating executable code with MLLMs [391], enabling robots to follow interpretable and adaptable control policies. Second, MLLMs can enhance behavioral interaction. Through interaction with the environment, MLLMs are also capable of generating sequences of behavioral actions in a single step. Representative works like π -0 [38] combine a vision-language backbone with a flow-matching decoder to produce smooth, temporally extended behavioral trajectories. Finally, MLLMs can enhance collaborative decision-making. One line of research focuses on multi-agent systems that aim to achieve human-level coordination and adapt rapidly to unforeseen challenges [404]. For instance, Combo [459] introduces a novel framework that enhances cooperation among decentralized agents operating solely with egocentric visual observations. Other efforts investigate human-agent collaboration. VLAS [472] exemplifies this by aligning human verbal commands with visual context via a speech encoder and a LLaVA-style MLLM [233], enabling fluid and conversational human-agent interaction.

1.4.2 World Model for Embodied AI

This section provides a comprehensive overview of embodied AI with world models. We first elaborate in detail how world models boost embodied AI in Sect. 1.4.2A. Then we discuss the classification of world models for embodied AI in Sect. 1.4.2B.

A. World Models Boost Embodied AI

World models empower embodied AI by building internal representations and future predictions of the external world (as shown in Fig. 1.5), facilitating physical law-compliant embodied interactions in dynamic environments.

Internal Representations of the External World. Internal representations help compress raw sensory data into organized latent spaces. These spaces store object motion, physical laws, and spatial layouts, which help agents figure out what is present and how it behaves. Such embeddings keep clear layers of relationships [122] between objects and surroundings, matching how real-world structures build up. This design makes it easier to apply the same ideas, like gravity or object permanence, in different settings. They also enable agents to ask “what if” questions [407] by keeping separate variables for what an object is [307] and how it connects to other things [394]. This separation lets parts be adjusted flexibly in the mind. It also improves learning, since agents can reuse shared features when switching tasks. World models with good internal spaces can check their own uncertainty and look for missing information when needed. By storing time flow and spatial structure [47], they keep

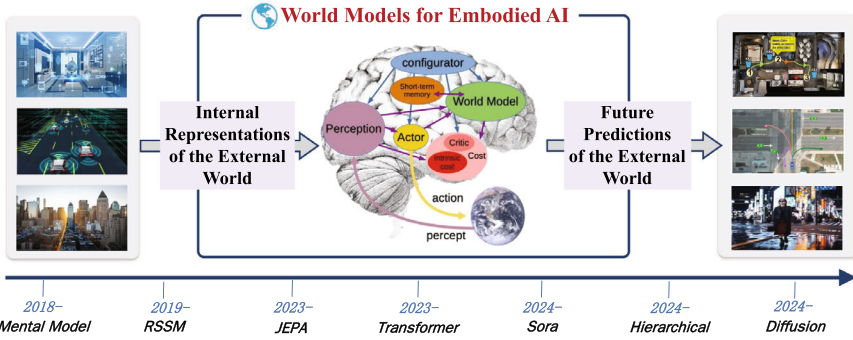


Fig. 1.5 The development roadmap of world models for embodied AI

plans realistic and filter out actions that break physical rules before trying them. In the end, such spaces give a basis for causal thinking [58], similar to how humans build simple models from what they sense.

Future Predictions of the External World. Future predictions let agents estimate possible rewards for action sequences over different time spans, while following physical rules [140, 141]. This foresight connects immediate steps to longer-term aims [274]. Implausible paths, like walking through walls, or wasteful choices, like using up resources too fast, can be avoided early. Long-horizon prediction [405] helps agents adjust between trying new options and sticking to safe plans. By looking ahead, they can skip bad local choices but still act on near-term goals. These forecasts also measure uncertainty [18, 140]. They tell apart stable patterns, like daily routines, from sudden changes. This makes planning under risk more reliable. Simulated predictions save samples [75, 216, 394, 433] because mental rehearsal costs less than trial and error in the real world. This is crucial for tasks where errors are costly, like self-driving cars or medical robots. Continuous checks on prediction mistakes [133, 180, 255, 414] push the model to refine itself over time. These self-corrections keep their inner physics closer to reality. In the end, this ability to look ahead gives artificial agents a kind of practical foresight. They move from just reacting to acting with clear goals in mind.

B. Representative Works

Embodied AI with world models can mainly be divided into three critical structures: the Recurrent State Space Model-based (RSSM-based) world models for embodied AI, the Joint-Embedding Predictive Architecture-based (JEPA-based) world models for embodied AI, and the Transformer-based world models for embodied AI. Hierarchical-based world models [144] and diffusion-based world models [13] are similar to other structures and are shown in Fig. 1.5.

RSSM-based World Models for Embodied AI. RSSM constitutes the fundamental architecture underpinning the Dreamer algorithm family [140, 141, 274, 405].

This framework enhances predictive capabilities in latent representations by acquiring temporal environment dynamics through visual inputs, subsequently enabling action selection via latent trajectory optimization. Through orthogonal decomposition of hidden states into probabilistic and deterministic components, the architecture explicitly accounts for both systematic patterns and environmental uncertainties. Its demonstrated effectiveness in robotic motion control applications has inspired numerous derivative studies building upon its theoretical framework.

JEPA-based World Models for Embodied AI. JEPA [211] provides a structure for developing autonomous machine intelligence systems. This architecture establishes mapping relationships between input data and anticipated outcomes through representation learning. Diverging from conventional generative approaches, JEPA operates in abstract latent spaces rather than producing pixel-wise reconstructions, thereby prioritizing semantic feature extraction over low-level signal synthesis. A key methodological foundation of JEPA [18] involves self-supervised training paradigms where neural networks learn to infer occluded or unobserved data segments. Such pre-training on extensive unlabeled datasets enables transfer learning across downstream applications, demonstrating enhanced generalization capabilities for both visual [28, 29] and non-visual domains [111].

Transformer-based World Models for Embodied AI. Originating in natural language processing research, the Transformer structure [371] fundamentally relies on attention mechanisms to process input sequences through parallelized context weighting. This design allows simultaneous computation of inter-element dependencies, overcoming the sequential processing constraints inherent in Recurrent Neural Networks (RNNs). Empirical evidence demonstrates superior performance in domains requiring persistent memory retention and explicit memory addressing for cognitive reasoning [26], which has propelled its adoption in reinforcement learning research since 2020. Existing advancements have successfully implemented world models using Transformer variants [258, 307, 407], outperforming RSSM architectures in memory-intensive interactive scenarios [58]. Notably, Google’s Genie framework [47] employs the Spatial-Temporal Transformer (ST-Transformer) [421] to create synthetic interactive environments through large-scale self-supervised video pretraining. This breakthrough establishes novel paradigms for actionable world modeling, revealing transformative potential for world models development trajectories.

1.4.3 *Joint MLLM and World Model for Embodied AI*

In this subsection, we focus on the integration of MLLMs and world models for embodied AI. In Section 1.4.3.A, we first elaborate in detail on the limitations of MLLMs and world models for embodied AI and explain how MLLMs boost world models’ reasoning, and how world models boost MLLMs’ interaction. Then we

design a joint embodied AI architecture driven by both MLLMs and world models in Sect. 1.4.3B.

A. The Relationship Between MLLMs and World Models

MLLMs enable contextual task reasoning but overlook physical constraints, while world models excel at physics-aware simulation but lack high-level semantics. Their joint bridges semantic intelligence with grounded physical interaction.

The Limitations of MLLMs for Embodied AI (without world models).

MLLMs still have two main shortcomings in embodied AI. First, they often miss grounding their predictions [238] in actual physics. This can lead to plans that make sense in text but fail in the real world. For instance, ignoring friction or the texture of a surface can make a robot drop an object or slip. Second, MLLMs react poorly when the environment shifts quickly [99]. They handle task breakdowns well in language but struggle to tweak actions on the fly. The core reason is that they rely on fixed training data. They lack the constant feedback that comes from real physical interaction.

The Limitations of World Models for Embodied AI (without LLMs/MLLMs).

World models also come with their own limits. They are weak at handling abstract or open-ended reasoning [78]. Their main strength is simulating the physical world, not grasping the broader context. They also fall short when tasks need flexible steps [134]. Without built-in knowledge, it is hard for them to break a big goal into workable parts. For example, a world model that knows only rigid shapes may fail if it has to deal with soft or flexible objects unless retrained from scratch. Their predictions depend heavily on data from specific tasks. This makes it tough to scale them to new domains without more training.

MLLMs Boosting World Models’ Reasoning. By leveraging cross-modal alignment and semantic grounding, MLLMs enable world models to process complex environments dynamically, improving semantic reasoning, task decomposition, and human-robot interaction. (1) MLLMs can enrich world models by fusing visual, auditory, and textual data into unified semantic representations. For instance, CLIP-based architectures [294] enable agents to align visual scenes with linguistic cues, reducing ambiguity in object recognition [159]. (2) MLLMs can augment world model’s task decomposition capacity by decomposing high-level goals into executable sub-tasks. Models like GPT-4V [275] generate step-by-step plans using environmental context stored in the world model. For robotic manipulation, Code-as-Policies [224] translates natural language instructions into code snippets, leveraging the world model to track intermediate states. (3) MLLMs enable world models to refine internal representations through human feedback. Techniques like Reinforcement Learning with Human Feedback (RLHF) [278] allow agents to update the world model priors based on corrective inputs [332]. Those works in this subsection are all possible ways for MLLMs to boost world models’ reasoning, which is not achieved in existing works.

World Models Boosting MLLMs’ Interaction. World models can play a pivotal role in refining MLLMs by providing physical laws, spatio-temporal relationships,

and closed-loop interaction experiences. World models can mitigate MLLMs’ inherent limitations in temporal coherence and environmental grounding, enabling more robust decision-making in dynamic embodied tasks. (1) World models can provide MLLMs with explicit representations of physical laws (e.g., gravity, friction) and commonsense rules to constrain action proposals. For example, Physion++ [364], integrating world model-stored biomechanical models, can be used to filter MLLM-generated robotic motions violating torque limits; RoboGuide [329] injects spatial occupancy maps into MLLM planners, preventing collisions during navigation. (2) World models can stabilize MLLMs’ reasoning by maintaining spatio-temporal context during multimodal processing. For instance, MemPrompt [377] can use world model buffers to align visual object trajectories with linguistic descriptions, resolving ambiguities in cluttered environments; RoboMem [348] can leverage world model-prioritized attention to filter irrelevant sensory noise, improving MLLM-based scene understanding. (3) World models can enable iterative refinement of MLLM outputs through closed-loop interaction. Reflexion [332] can store task-execution histories in world model, allowing MLLMs to correct kinematic errors using failure patterns [224]. Those works in this subsection are all possible ways for world models to boost MLLMs’ decisions, which have not been achieved in existing works.

B. Joint Architecture of MLLM and World Model for Embodied AI

We propose a joint architecture of MLLM and world model for embodied AI (as shown in Fig. 1.6), shedding light on their profound significance in enabling complex tasks within physical worlds. The specific workflow is as follows, with arrows highlighting the data exchange process.

Robots → Self-State Inputting → MLLMs/world models → Hardware Embodiment → Robots. The process initiates with self-state inputting tracking proprioceptive metrics, such as degrees of freedom, number of sensors, etc. These metrics feed into both world models and MLLMs: world models use them to build internal representations of the agent’s physical state, while MLLMs contextualize these states for task alignment. Hardware embodiment is focused on implementing world models and MLLMs into physical devices to solve sim-to-real problems. This bidirectional flow ensures actions respect both mechanical limits and high-level goals.

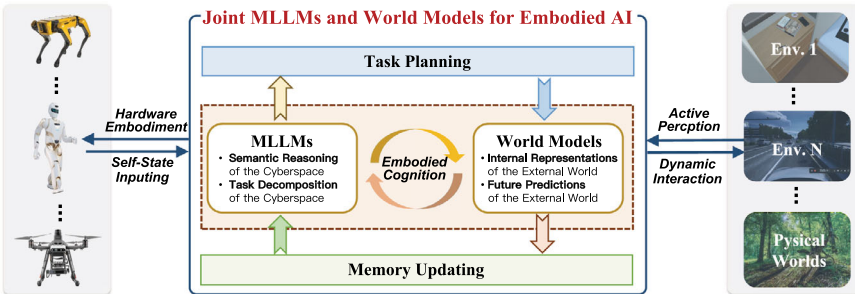


Fig. 1.6 Embodied AI enabled by MLLMs and world models

MLLMs $\rightarrow$ *Task Planning* $\rightarrow$ *world models* $\rightarrow$ *Memory Updating* $\rightarrow$ *MLLMs*. MLLMs decompose abstract instructions into sub-tasks. A forward arrow delivers this plan to world models, which predict outcomes based on existing environmental modeling. During execution, world models log outcomes into memory. A vertical arrow transmits these logs to memory updating modules, which structure memory into experiences, represent the forgetting of past task memories, the renewal of current task memories, and the prediction of future task memories. These are then fed back to MLLMs via an arrow, enriching their knowledge base. This enables lifelong learning, where past failures directly inform future planning.

Environments $\rightarrow$ *Active Perception* $\rightarrow$ *MLLMs/world models* $\rightarrow$ *Dynamic Interaction* $\rightarrow$ *Environments*. World models first drive active perception by predicting key environmental changes. Multimodal inputs are then used to construct an internal representation of the external world through world models and semantic reasoning through MLLMs. Then, the task decomposition of MLLMs and future prediction of world models enable action selection and environmental interaction. Adaptive perception and interaction of dynamic environments are achieved through continuous iteration.

1.4.4 Discussions

A joint MLLM-world model setup offers a strong framework for embodied AI. As shown in Table 1.4, MLLMs handle semantic reasoning well. They break down tasks, follow context, and adjust plans by using multiple input types. In parallel, world models anchor these plans in physical reality. They simulate real-world rules so actions stay feasible. Combining both lets an agent link abstract thinking with actions that obey physics. This balance improves choices in changing conditions. For example, an MLLM can draft a plan. The world model then checks if it works in practice. This loop refines the plan step by step. Such a joint design also helps generalize across different data types. It makes the system stronger when some information is missing or the situation is new. It does so by connecting symbols with sensor signals.

However, this combined setup brings new challenges. First, MLLMs often run slowly when reasoning, while world models must track physics in real time. Keeping them in sync is hard and can slow down reactions. Second, plans made by MLLMs might break physical limits that the world model has not yet modeled. Third, world models update memory all the time. If not handled well, this can overload MLLMs with too much detail. On top of that, training these systems needs huge mixed datasets. These must include rare edge cases to ensure safety. It is still tough to make them robust to noisy sensors or missing data. Solving these issues calls for faster MLLMs, tighter feedback between parts, and smarter ways to filter context on the fly.

Table 1.4 Qualitative comparison of MLLM-only, world model-only, and joint architectures combining MLLMs and world models in Embodied AI

Performance	LLM/MLLM-only	world model-only	Joint MLLM-world model
Semantic Understanding	Advantages in contextual task reasoning and natural language understanding	Limited in open-ended semantic understanding	Combines high-level semantic abstraction with grounded contextual alignment
Task Decomposition	Sequential logic enables sub-task planning via language prompts	Lacks generalizable task decomposition mechanisms	Semantic plans refined through physical feasibility via joint planning-execution loop
Physics Compliance	Ignores physical constraints and dynamics in real-world interaction	Physics-aware simulation with temporal consistency	Enforces semantic-physical alignment for safe and executable plans
Future Prediction	Lacks imagination-based reasoning	Long-horizon multi-step prediction with uncertainty modeling	Combines symbolic foresight and physically grounded imagination
Real-time Interaction	Poor responsiveness to environmental feedback and significant reasoning latency	Supports real-time predictive control via future state simulation	Enables online adaptation through iterative plan refinement and memory updating
Memory Structure	Sparse and unstructured memory	Structured latent space encodes object dynamics and causal relationships	Integrates semantic memory and world modeling for lifelong learning and reflection
Scalability	Limited to pre-trained task space	Poor transfer to unseen tasks without retraining	Cross-task, cross-domain generalization through symbolic and sensorimotor synergy

1.5 Key Applications of Embodied AI

The advancement of embodied AI technologies has stimulated their deployment in diverse application domains where physical interaction is essential. From autonomous robots navigating unstructured environments to drones operating in dynamic aerial settings, embodied AI demonstrates its capacity to handle perception, reasoning, and control in real-world contexts. This section highlights representative applications, focusing on how embodied AI enables agents to achieve robust autonomy, task generalization, and safe execution. By analyzing these cases, we show how embodied intelligence is being translated from conceptual models into practical systems with tangible societal and industrial impact.

1.5.1 Robots

Embodied agents actively interact with the physical world and encompass a wide range of embodied forms, including robots, smart appliances, augmented reality glasses, and autonomous vehicles. Among these, robots stand out as one of the most prominent and widely studied embodiments. Depending on specific application scenarios, robots are designed in various forms to leverage their hardware capabilities for targeted tasks. Embodied robots can generally be categorized into the following types: (1) Fixed-base robots, such as robotic arms, are commonly used in laboratory automation, education, and industrial manufacturing. (2) Wheeled robots are known for their high mobility and are widely deployed in logistics, warehousing, and security inspection. (3) Tracked robots offer excellent off-road capability and maneuverability, showing strong potential in agriculture, construction, and disaster response. (4) Quadruped robots are valued for their stability and adaptability, making them ideal for exploration in complex terrains, rescue operations, and military applications. (5) Humanoid robots, equipped with dexterous hands, are increasingly applied in service industries, healthcare, and human-robot collaborative environments. (6) Bio-inspired robots mimic the efficient movement and functionality of natural organisms, enabling them to operate effectively in complex and dynamic environments.

Fixed-Base Robots. Fixed-base robots are widely used in laboratory automation, educational training, and industrial manufacturing due to their high accuracy, compact structure, and operational stability. These robots are anchored to a base, enabling them to achieve micron-level precision for tasks that require exact repeatability [337]. Equipped with precise actuators and sensors, fixed-base robots offer excellent control, making them suitable for delicate operations such as automated chemical synthesis, circuit assembly, and surgical assistance. They are also highly programmable, allowing customization for various applications. Notable examples include the Franka Emika Panda [136], KUKA iiwa [215], and Sawyer by Rethink Robotics [190]. However, the immobility of fixed-base robots limits their working range and adaptability. Their inability to relocate autonomously restricts them to static environments and requires additional systems for multi-location tasks. This limitation also reduces their collaborative flexibility when integrated into dynamic human-robot teams. Despite their precision and reliability, these robots are best suited for environments where spatial coverage is not critical, emphasizing task-specific automation over spatial versatility.

Wheeled Robot. Wheeled robots are widely adopted in logistics, warehousing, and security due to their efficient mobility and structural simplicity. With high energy efficiency, low cost, and fast navigation on flat surfaces, they are ideal for structured environments [337]. These robots typically integrate advanced sensors such as LiDAR and cameras to enable autonomous path planning and obstacle avoidance. Prominent examples include Kiva robots used in Amazon warehouses [412] and the Jackal robot by Clearpath Robotics [302]. Their ability to rapidly transport goods and patrol defined spaces has revolutionized inventory management and

security inspection. However, their utility is limited in complex terrains due to their reliance on smooth surfaces for optimal performance. Wheeled robots have constrained load capacity and struggle with tasks requiring dynamic reorientation or maneuvering across uneven ground. Although capable of limited indoor collaboration with humans, their adaptability in unstructured or outdoor environments is significantly lower than that of legged or tracked platforms. As a result, wheeled robots are best suited for repetitive, high-throughput applications where consistent terrain and reliable navigation infrastructure are available.

Tracked Robots. Tracked robots offer superior off-road capabilities and are well-suited for challenging environments such as agriculture, construction, and disaster recovery scenarios [367]. Their continuous track design increases ground contact, reducing pressure and enhancing stability on soft or uneven terrain. This enables effective traversal through mud, sand, rubble, and inclines where wheeled robots fail. Tracked systems often integrate strong suspension and drive mechanisms, allowing them to maintain traction while supporting heavy payloads. These capabilities make them favorable for high-risk operations, including military reconnaissance and explosive ordnance disposal, as exemplified by iRobot's PackBot [424]. Despite these advantages, tracked robots tend to suffer from low energy efficiency due to high surface friction. Their top speed on flat surfaces is typically lower than that of wheeled robots, and their maneuverability in confined spaces is limited. Moreover, complex track systems are mechanically intensive, increasing maintenance costs. Therefore, while tracked robots excel in demanding field conditions, their practicality diminishes in scenarios prioritizing speed, efficiency, or modular adaptability. They remain indispensable in roles that prioritize robustness and terrain independence over precision or agility.

Quadruped Robots. Quadruped robots, modeled after four-legged animals, provide excellent mobility and stability across irregular terrain. Their multi-jointed limb structures allow them to traverse uneven environments, climb obstacles, and maintain balance under dynamic conditions [295]. This makes them ideal for rescue missions, exploration, and military applications, especially in areas inaccessible to wheels or tracks. Leading platforms include Unitree Robotics' A1 and Go1 [34, 210], Boston Dynamics' Spot [40], and ANYbotics' ANYmal C [17]. These robots combine sensory systems such as LiDAR and stereo vision with reinforcement learning or model-based control to achieve adaptive behavior. The Spot robot, for instance, is deployed for industrial inspections and autonomous missions in hazardous sites. ANYmal C's modular design and autonomous navigation are suitable for prolonged outdoor operations and space exploration. However, quadruped robots are typically expensive due to their complex mechanical design and perception systems. They also face limitations in battery endurance, often requiring frequent recharging for extended deployment [273]. Despite these challenges, their ability to navigate cluttered and unstructured environments makes them invaluable in scenarios demanding terrain flexibility and semi-autonomous decision-making.

Humanoid Robots. Humanoid robots replicate human form and behavior, allowing them to perform complex tasks in collaborative, healthcare, and service-oriented

environments. Their anthropomorphic design includes dexterous hands with multiple degrees of freedom and sensor integration, enabling fine manipulation and human-like interaction [370]. Atlas by Boston Dynamics demonstrates highly dynamic movements such as running and flipping [253], while Honda's ASIMO can walk, climb stairs, and interact socially [330]. The HRP series by AIST focuses on human-robot collaboration, particularly in industrial settings [4]. Additionally, Pepper from Softbank Robotics offers emotional recognition and natural language processing, widely used in customer service and education [353]. Despite their versatility, humanoid robots face challenges in maintaining stability and reliability in real-world environments. Precise bipedal locomotion and fine hand control require complex control algorithms and real-time feedback loops [363]. Moreover, traditional hydraulic systems are bulky and costly to maintain. Recent efforts, such as Tesla's and Unitree's motor-based humanoids, aim to enhance performance and scalability. With the integration of large language models, humanoid robots are expected to perform increasingly diverse tasks autonomously, filling labor gaps in demanding sectors [415].

Biomimetic Robots. Biomimetic robots aim to replicate the movement and functional traits of natural organisms to perform tasks in complex and dynamic environments [337]. These robots often utilize soft, flexible materials and energy-efficient designs to emulate biological locomotion. Applications span environmental monitoring, biomedicine, and ecological research. Fish-inspired robots [35, 188] exhibit silent and agile underwater locomotion, ideal for marine exploration. Insect-like robots [81, 478] excel in swarm behavior and can access confined spaces for structural inspections. Soft-bodied robots [338] can safely interact with delicate structures, making them suitable for minimally invasive medical procedures. These designs reduce mechanical complexity and enable operation in environments too sensitive for rigid systems. However, biomimetic robots face notable limitations. Their design and fabrication processes are technically demanding and costly, hindering large-scale production. Moreover, the use of soft materials often compromises durability, particularly in harsh or extreme conditions. While promising in niche applications, broader deployment requires advancements in material science, energy storage, and control precision [163, 434]. Despite these challenges, biomimetic robots offer a compelling direction for robotics that prioritizes adaptability, safety, and environmental harmony.

1.5.2 Drones

In dynamic environments, unmanned aerial vehicles (UAVs) commonly rely on onboard cameras and various sensors to capture real-time imagery and video streams, which support high-level decision-making. To perform semantic understanding tasks, UAVs must go beyond acting as mere data collection platforms. They are increasingly expected to exhibit embodied intelligence, enabling them to model the environment semantically and interact through natural language. This allows UAVs to generate

meaningful semantic responses that align with human operational commands based on their perceptual input [410].

Recent advances in Foundation Models (FMs), particularly Large Language Models (LLMs), have shown strong autonomy and adaptability across different domains, including embodied intelligence [360, 361, 474]. A growing body of research has explored integrating LLMs to enhance UAV perception and reasoning in dynamic settings. For instance, de Zarzà et al. [455] and de Curtò et al. [88] developed UAV video understanding systems by combining multiple foundation models. These methods have made notable strides in semantic modeling. However, they often lack explicit task planning capabilities, resulting in limited control over response consistency. Additionally, due to the absence of mechanisms for invoking external tools and orchestrating modular components, these systems struggle to generalize effectively in complex, open-ended tasks [473]. To overcome these limitations, the emerging paradigm of agentic artificial intelligence offers a more flexible and generalizable framework. Built around LLMs, this paradigm supports essential functions such as task decomposition, tool utilization, and coordination among multiple agents [324]. It provides a promising direction for enabling UAVs to perform semantic understanding tasks in dynamic environments in a unified and scalable manner [359]. AirVista-II [228] exemplifies this approach by constructing an agentic system for embodied UAVs. It organizes dynamic scenes into three task categories: images, short videos, and long videos, and designs tailored agent workflows and model invocation strategies for each. This design enables a complete autonomous pipeline encompassing task recognition, data processing, semantic interpretation, and high-level reasoning.

In addition, research in embodied artificial intelligence has been gaining increasing attention. One of the core tasks in this area is Vision-and-Language Navigation (VLN), which focuses on guiding agents to target locations based on natural language instructions and visual observations. Recently, AerialVLN [235] and Open-UAV [392] have made significant progress by leveraging UAV simulators to fill the gap in aerial VLN datasets. These efforts have greatly advanced the field, but still face challenges such as limited data diversity and insufficient scale. To address these issues, OpenFly [128] has been introduced as a dedicated platform for large-scale data collection in aerial vision-and-language navigation. OpenFly integrates multiple rendering engines and provides an automated data generation toolchain, enabling the efficient collection of diverse and high-quality aerial VLN data.

1.6 Simulators, Datasets, and Metrics

Developing and evaluating embodied AI systems requires reliable tools for experimentation, benchmarking, and validation. Since real-world deployment is often costly and risky, simulators play a pivotal role by providing safe and scalable testbeds. Complementary to this, curated datasets and well-designed metrics ensure reproducibility and enable fair comparison across methods. This section reviews commonly used simulators, datasets, and evaluation measures in embodied AI research,

emphasizing their significance in bridging algorithmic innovation with practical performance assessment. Together, these resources form the methodological foundation for advancing the field in a systematic and rigorous manner.

1.6.1 Simulators for Embodied AI

Embodied simulators play a central role in the advancement of embodied AI by providing a safe, efficient, and scalable framework for developing and evaluating intelligent agents. These simulation platforms eliminate the risks and costs associated with physical testing, particularly in scenarios that are hazardous or difficult to reproduce in the real world. They enable researchers to experiment with complex behaviors in a controlled and repeatable manner, making them ideal for tasks such as robotic manipulation, autonomous navigation, and human-agent interaction.

In addition to safety and flexibility, simulators facilitate large-scale data generation for training and evaluation. They support high-throughput experimentation, allowing researchers to explore a wide range of environmental conditions and agent configurations. This accelerates the development of learning algorithms, supports detailed ablation studies, and ensures reproducibility through standardized benchmarks and shared virtual environments, which are essential for meaningful comparisons across different methods and systems. The effectiveness of embodied simulators depends heavily on their ability to model realistic environments. This includes accurate simulation of physical properties, such as gravity, friction, and object dynamics, as well as sensory modalities like vision, audio, and touch. A high degree of realism is critical for bridging the gap between simulation and real-world deployment, ensuring that agents trained in virtual settings can successfully transfer their capabilities to physical environments.

This section introduces commonly used embodied AI simulation platforms in two categories. The first consists of general simulators built on physics engines that allow extensive customization of tasks and environments. The second includes simulators based on real-world scene reconstructions, which prioritize photorealism and structural accuracy to support evaluations that closely mirror real-world conditions.

A. General Simulator

While the richness of physical interaction and dynamic changes in real-world environments is irreplaceable, deploying embodied models in the physical world remains costly and presents numerous practical challenges. These include hardware expenses, maintenance, safety concerns, and limited scalability. As a result, general simulators have become indispensable tools in embodied AI.

General simulators offer several core benefits. They support repeatable experimentation, allow rapid prototyping, and generate synthetic data for perception, control, and decision-making tasks. By offering full observability and control over environmental parameters, they facilitate detailed analyses and algorithm benchmarking under consistent conditions. Among the widely used platforms, Isaac Sim [272] is a

state-of-the-art simulator developed by NVIDIA, designed to support robotics and AI research at scale. It features high-fidelity physical simulation, real-time ray tracing for photorealistic rendering, and seamless integration with deep learning frameworks. Isaac Sim supports various applications, including autonomous vehicle simulation, industrial robotics, and human-robot collaboration. Gazebo [199] is another prominent open-source simulator, especially valued for its compatibility with the Robot Operating System (ROS). It offers a rich set of tools for modeling robot dynamics, simulating sensors such as cameras and lidars, and testing multi-robot coordination strategies. Gazebo is commonly used in mobile robot navigation, control systems development, and academic research due to its extensibility and large community support. PyBullet [80], the Python interface for the Bullet physics engine, is known for its simplicity and accessibility. It supports real-time physical simulation with rigid body dynamics, collision detection, and constraints. PyBullet is especially suitable for tasks requiring rapid iteration, such as reinforcement learning and robotic manipulation. It also integrates well with deep learning frameworks, allowing for the simulation of perception-based control policies.

Table 1.5 summarizes the core features and application domains of ten general-purpose simulators frequently used in embodied AI. Each offers distinct strengths in modeling physical interactions, sensory feedback, or integration with AI frameworks. By selecting the most appropriate simulator based on their research goals, developers can accelerate the prototyping, testing, and deployment of embodied intelligence systems across diverse real-world scenarios.

Table 1.5 General Simulator. HFPS: high-fidelity physical simulation; HQGR: high-quality graphics rendering; RRL: rich robot library; DLS: deep learning support; LSPC: large-scale parallel computing; ROS: tight integration with ROS; MSS: multiple sensor simulation; CP: cross-platform Nav: robot navigation AD: auto driving; RL: reinforcement learning LSPS: large-scale parallel sim MR: multi-robot systems RS: robot simulation. ✓ indicates that the simulator excels at this aspect. (PE: Physics Engine; MLA: ML-Agents; OBSD: ODE, Bullet, Simbody, DART; BOVN: Bullet, ODE, Vortex, Newton; DAR: Drone sim, AD, RL)

Simulator	Year	HFPS	HQGR	RRL	DLS	LSPC	ROS	MSS	CP	PE	Applications
Isaac Sim [272]	2023	✓	✓	✓	✓	✓	✓	✓	✓	PhysX	Nav, AD
Isaac Gym [251]	2019		✓		✓	✓				PhysX	RL, LSPS
Gazebo [200]	2004		✓	✓			✓	✓	✓	OBSD	Nav, MR
PyBullet [80]	2017				✓				✓	Bullet	RL, RS
Webots [82]	1996		✓	✓				✓	✓	ODE	RS
MuJoCo [362]	2012	✓			✓				✓	Custom	RL, RS
Unity MLA [184]	2017		✓		✓				✓	Custom	RL, RS
AirSim [323]	2017		✓						✓	Custom	DAR
MORSE [165]	2015							✓	✓	Bullet	Nav, MR
V-REP [310]	2013		✓	✓				✓	✓	BOVN	MR, RS

B. Real-Scene Based Simulators

To advance the development of embodied AI agents capable of performing complex household tasks, high-fidelity simulators based on real-world environments have become essential. These agents must understand the nuances of human daily life and execute embodied tasks such as navigation, manipulation, and interaction in intricate indoor settings. Consequently, the demand for simulators that offer photorealistic scenes, realistic object interactions, and diverse sensory modalities has grown significantly. Many of these simulators rely on 3D scanning, photogrammetry, and game engines like Unity or Unreal Engine to recreate real-world conditions [142], making them ideal platforms for training and evaluating embodied agents.

AI2-THOR [201], developed by the Allen Institute for AI, is a Unity-based simulator offering over 2000 interactive objects and multiple types of household scenes. Its environments are divided into iTHOR, which supports rich object interactions, and RoboTHOR, which aligns virtual layouts with real-world spaces. SAPIEN [413] emphasizes fine-grained physical interactions. Built on the PhysX engine and PartNet-Mobility dataset, it allows embodied agents to manipulate object joints using realistic force-based control, which is essential for precise manipulation tasks. iGibson [212, 325], developed by Stanford, focuses on diverse physical properties, including temperature, humidity, and cleanliness. It provides rich sensory input (including LiDAR) and fine-grained motor control. TDW [124] from MIT integrates multi-modal rendering (visual and auditory) and supports interaction with rigid, soft, and fluid materials, offering flexibility for diverse simulation needs. Other notable platforms include VirtualHome [289], which introduces an environment graph for high-level planning based on language descriptions; Habitat [317], known for its speed, open extensibility, and strong support for navigation; and Matterport3D [16, 52], a large-scale dataset that enables panoramic embodied navigation. Recent developments in automated simulation construction include RoboGen [396], which uses LLMs to generate scenes from assets, HOLODECK [436], which builds scenes from instructions in AI2-THOR, PhyScene [435] for diffusion-based scene generation, and ProcTHOR [91] for scalable, diverse scene creation. Table 1.6 compares these simulators, highlighting their core features and primary use cases across embodied navigation, interaction, and planning.

1.6.2 Datasets and Metrics for Embodied AI

Datasets and evaluation metrics form the empirical foundation for embodied AI research. While simulators provide controlled environments for testing, datasets supply real-world or synthetic sensory streams that benchmark learning and generalization. Metrics further ensure fair comparison by quantifying aspects such as accuracy, robustness, efficiency, and adaptability. This subsection surveys widely used datasets and evaluation protocols, emphasizing their importance in validating

Table 1.6 Comparison of real-scene-based simulators. For sensors, S denotes Semantic, L denotes LiDAR, and A denotes Audio. (CA: Continuous Action; 3D-SS: 3D Scene Scans; MA: Multiple Agents; OS: Object States; 3D-A: 3D Assets)

Simulation Platform	Year	Num of Scenes	CA	3D SS	Sensors	Physics	MA	OS	3D-A
A12-THOR [201]	2017	120	✓	×	RGB-D, S	✓	✓	✓	✓
Matterport 3D [52]	2018	90	×	✓	RGB-D, S	×	×	×	×
Habitat [317]	2019	1000+	✓	✓	RGB-D, S	✓	✓	×	✓
Virtual Home [289]	2018	50	×	×	RGB-D, S	✓	✓	✓	✓
SAPIEN [413]	2020	46	✓	×	RGB-D, S	✓	×	✓	✓
iGibson [325][212]	2021	15	✓	✓	RGB-D, S, L	✓	×	✓	✓
TDW [124]	2021	-	✓	✓	RGB-D, S, A	✓	✓	✓	✓

embodied AI systems and guiding progress toward standardized and reproducible research practices.

A. Vision-and-Language Navigation

Vision-and-Language Navigation (VLN) presents a diverse range of task settings depending on the nature of the language instructions and the complexity of agent behavior. Instructions may vary from fine-grained step-by-step directions to high-level goals or even vague human demands. Likewise, task complexity ranges from a single navigation goal to multi-step plans involving interaction, exploration, and sequential sub-tasks. These differences result in varied research challenges, leading to the development of multiple benchmark datasets [114, 115] designed to evaluate embodied agents under different conditions, as shown in Table 1.7.

Dataset. The Room-to-Room (R2R) dataset [16], built on Matterport3D, is a foundational benchmark where agents follow detailed instructions through discrete navigation graphs. Agents must align language and visual observations to select the next node and reach the goal location. Room-for-Room (R4R) [169] extends this setting by increasing path lengths, demanding more effective long-term memory and language grounding. VLN-CE [204] introduces continuous control in photorealistic environments, increasing the difficulty of action decision-making. Beyond indoor scenes, TOUCHDOWN [60] places agents in urban outdoor environments using Google Street View. Agents must follow complex instructions to locate target objects in realistic city-scale environments, requiring spatial reasoning at scale. REVERIE [290] also uses Matterport3D but focuses on high-level goal identification. Agents are required to search for and interact with unseen target objects based on concise natural language descriptions. SOON [482] emphasizes coarse-to-fine instruction following, guiding agents to narrow their search area progressively. DDN [379] further abstracts the instruction form by replacing explicit targets with general human

Table 1.7 Comparison of VLN datasets (I: Indoor; O: Outdoor; C: Continuous; D: Discrete)

Dataset	Year	Simulator	Env.	Feature	Size
R2R [16]	2018	Matterport3D	I, D	Step-by-step instructions	21,567
R4R [169]	2019	Matterport3D	I, D	Step-by-step instructions	200,000+
VLN-CE [204]	2020	Habitat	I, C	Step-by-step instructions	–
TOUCHDOWN [60]	2019	–	O, D	Step-by-step instructions	9,326
REVERIE [290]	2020	Matterport3D	I, D	Described goal navigation	21,702
SOON [482]	2021	Matterport3D	I, D	Described goal navigation	3,848
DDN [379]	2023	AI2-THOR	I, C	Demand-driven navigation	30,000+
ALFRED [334]	2020	AI2-THOR	I, C	Navigation w/ interaction	25,743
OVMM [441]	2023	Habitat	I, C	Navigation w/ interaction	7,892
BEHAVIOR-1K [214]	2023	OmniGibson	I, C	Long-span navigation w/ interaction	1,000
CVDN [357]	2020	Matterport3D	I, D	Dialog, oracle	2,050
DialFRED [126]	2022	AI2-THOR	I, C	Dialog, oracle	53,000

needs, challenging agents to infer suitable objects and behaviors from contextual cues. In task-oriented settings, ALFRED [334] combines navigation and manipulation using hierarchical instructions in AI2-THOR. OVMM [441] evaluates object transport tasks across unseen scenes, with both simulation and real-world applicability. BEHAVIOR-1K [214] pushes the limits with long-horizon, skill-rich tasks rooted in daily human activities using the OmniGibson platform. Dialogue-based navigation is also explored in CVDN [357] and DialFRED [126], where agents can ask questions to resolve uncertainty or request help during navigation or manipulation. These datasets introduce interactive learning and decision-making, promoting deeper agent-environment-human interaction.

Metrics. Depending on the nature of the task, different metrics emphasize either goal achievement or path adherence. Broadly, they can be categorized into goal-oriented metrics, which focus on the agent’s ability to reach the target, and path-fidelity metrics, which evaluate how well the agent follows a desired path.

Goal-oriented Metrics. Focus primarily on assessing how effectively an agent reaches or approaches the designated goal. The most basic metric is the Success Rate (SR), which calculates the percentage of episodes in which the agent ends up within a pre-defined threshold distance of the target. While SR provides a binary measure of task completion, Goal Progress [356] offers a continuous evaluation by measuring

the reduction in distance to the goal over the course of an episode, capturing partial success even when the task is not fully completed. Path Length (PL) represents the total distance traveled by the agent, serving as a measure of efficiency. In practice, shorter paths are often preferable, especially in real-world robotic applications where longer paths incur greater time and energy costs. To balance task completion with efficiency, Success weighted by Path Length (SPL) [15] adjusts the success rate based on the ratio of the shortest path to the actual path taken. Similarly, Success weighted by Edit Distance (SED) [61] compares the sequence of agent actions to that of an expert, incorporating both task success and action-level fidelity. To account for near-success scenarios, Oracle Navigation Error (ONE) measures the shortest distance between the goal and any point along the agent’s trajectory, rather than only considering the final position. Building upon this, Oracle Success Rate (OSR) determines whether any point on the trajectory falls within a pre-defined success threshold, offering a more relaxed yet informative success criterion.

Path-fidelity Metrics. Evaluate how accurately an agent follows a given or expert path, which is crucial in scenarios where the route itself is important—due to safety, instruction following, or interpretability. These metrics typically involve sequence-level comparisons between the agent’s trajectory and a reference path. Coverage weighted by Length Score (CLS) [170] combines Path Coverage (PC), which measures how much of the reference path is visited, with a Length Score (LS) that rewards trajectories similar in length to the expert’s. Normalized Dynamic Time Warping (nDTW) [164] softly penalizes deviations from the reference trajectory while allowing flexibility in temporal alignment. Success weighted by normalized Dynamic Time Warping (SDTW) [164] restricts nDTW to only successful episodes, jointly reflecting both task completion and fidelity to the expected path.

B. Embodied Question Answering

In Embodied Question Answering (EQA) tasks, the agent must actively navigate and observe its environment from a first-person perspective to collect information required for answering a given question. This process demands not only effective exploration strategies but also the ability to make informed decisions about when sufficient information has been gathered. An intelligent agent should be capable of planning appropriate actions to maximize information gain while also recognizing the optimal moment to stop exploring and provide an accurate response.

Dataset. Performing embodied question answering (EQA) experiments in the physical world is often limited by environmental constraints, hardware requirements, and high operational costs. To address these challenges, simulators have become essential tools for developing and evaluating embodied AI systems. They provide controlled and reproducible environments, facilitate the creation of large-scale datasets, and allow for safe and cost-effective model training. Simulated environments also significantly increase the feasibility of deploying trained models on real-world agents by enabling rapid experimentation and refinement. Several benchmark EQA datasets have been developed using simulation platforms, each with dis-

Table 1.8 Comparison of EQA datasets (A-EQA: Active EQA; I-EQA: Interactive EQA; EM-EQA: Episodic Memory EQA; OE: Open-Ended; MC: Multiple Choice; MINOS*: Simulator Based on MINOS)

Dataset	Year	Type	Data sources	Simulator	Query creation	Answer	Size
EQA v1 [87]	2018	A-EQA	SUNCG	House3D	Rule-Based	OE	5,000+
MT-EQA [448]	2019	A-EQA	SUNCG	House3D	Rule-Based	OE	19,000+
MP3D-EQA [402]	2019	A-EQA	MP3D	MINOS*	Rule-Based	OE	1,136
IQUAD V1 [132]	2018	I-EQA	–	AI2THOR	Rule-Based	MC	75,000+
VideoNavQA [49]	2019	EM-EQA	SUNCG	House3D	Rule-Based	OE	101,000
SQA3D [247]	2022	QA only	ScanNet	–	Manual	MC	33,400
K-EQA [352]	2023	A-EQA	–	AI2THOR	Rule-Based	OE	60,000
OpenEQA [250]	2024	A-EQA, EM-EQA	ScanNet, HM3D	Habitat	Manual	OE	1,600+
HM-EQA [303]	2024	A-EQA	HM3D	Habitat	VLM	MC	500
S-EQA [97]	2024	A-EQA	–	VirtualHome	LLM	Binary	–

tinct characteristics and goals. Table 1.8 provides an overview of representative datasets.

The earliest EQA dataset, EQA v1 [87], was created in the House3D simulator using synthetic SUNCG scenes [344]. It features over 5,000 questions across more than 750 environments, with four question types formulated through functional programs. MT-EQA [448] extends EQA v1 to multi-object scenarios with over 19,000 questions covering comparisons of color, distance, and size. MP3D-EQA [402] incorporates realistic indoor Matterport3D scenes [53] into the MINOS simulator [316], enabling more lifelike navigation. IQUAD V1 [132], developed in AI2-THOR, emphasizes interaction and affordance reasoning, offering over 75,000 questions in dynamically configured scenes. VideoNavQA [49] decouples navigation and reasoning by presenting agents with video trajectories. It uses House3D to generate over 100,000 question-video pairs, spanning eight categories. SQA3D [247], based on ScanNet [83], introduces more complex reasoning over 6,800 scenes with 33,400 questions. K-EQA [352] enhances EQA with structured knowledge. Built in AI2-THOR, it links entities to knowledge graphs to support 60,000 questions involving logical and comparative reasoning. OpenEQA[250] focuses on open-vocabulary, real-world scenarios from ScanNet and HM3D [297], supporting both episodic memory and active exploration. Recent datasets such as HM-EQA[303] and S-EQA[97] leverage large language models like GPT-4 for scalable data generation, improving diversity and quality. HM-EQA uses HM3D scenes to generate 500 multiple-choice questions, while S-EQA applies cosine similarity filtering in VirtualHome to ensure relevance and variation. These diverse datasets provide valuable

resources for advancing embodied AI, supporting research on perception, reasoning, memory, and exploration in simulated 3D environments.

Metrics. Performance in embodied question answering is typically evaluated across two key aspects: navigation and question answering. For navigation, many studies follow the methodology proposed by Das et al. [87], using indicators such as the final distance to the target object (d_T), the change in distance from the initial to final position (d_Δ), and the minimum distance to the target achieved during the episode (d_{min}). These metrics are commonly measured after a fixed number of actions, such as 10, 30, or 50 steps. Other approaches evaluate navigation using indicators like trajectory length and intersection-over-union score (IoU) for the target object. In terms of question answering, performance is typically assessed using metrics such as the mean rank (MR) of the correct answer within a ranked list and the overall accuracy. To evaluate open-vocabulary answers, Majumdar et al. [250] introduced the LLM-Match metric, which leverages a large language model to assess the semantic correctness of the answers. They also proposed incorporating the normalized trajectory length as a weighting factor, enabling a combined evaluation of both answer correctness and agent efficiency. These methods together provide a comprehensive assessment of the model’s embodied reasoning capabilities.

C. Embodied Grasping

Embodied interaction extends beyond question answering to include executing physical actions based on human instructions, such as grasping and placing objects. These tasks involve interaction among robots, humans, and objects, requiring integrated capabilities in semantic understanding, scene perception, decision-making, and control planning. Recent approaches combine traditional robotic grasping with LLMs [46] and vision-language foundation models [294], enabling robots to interpret and perform tasks under multi-modal sensory inputs, including vision-based active perception and language-guided reasoning. Current research in robotic grasping primarily focuses on two types of end-effectors: two-finger parallel grippers and five-finger dexterous hands. For parallel grippers, grasp postures are typically represented in either 4-DOF or 6-DOF formats [270]. The 4-DOF configuration defines a grasp using a 3D position and a yaw angle, often referred to as top-down grasping [7, 92, 177]. In contrast, the 6-DOF configuration specifies full 3D position and orientation for more flexible grasping [108, 110, 369]. For dexterous hands, such as the ShadowHand with 26 degrees of freedom, the increased dimensionality poses significant challenges for both grasp synthesis and trajectory planning.

Dataset. In recent years, numerous grasping datasets have been introduced to advance research in robotic manipulation [92, 108, 177, 263, 266]. These datasets typically provide annotated grasp configurations based on various input modalities, such as RGB images, depth maps, point clouds, or full 3D scene reconstructions. With the integration of multi-modal learning models and the application of foundational language models in robotic grasping, there is a growing need for datasets that incorporate linguistic information. In response, several existing datasets have been expanded or restructured into semantic grasping datasets [181, 218, 366, 419], sup-

porting the development of models that can understand and execute grasps guided by language. Traditional grasping datasets include both single-object and cluttered scene scenarios. For example, datasets like [177] provide 4-DOF or 6-DOF stable grasp annotations in real desktop environments, along with RGB, depth, and point cloud data. Others, such as [108], are collected in simulated environments and offer large-scale, diverse scene models and grasp annotations. While these datasets are well-suited for training grasping algorithms, they often lack semantic grounding, limiting their utility for language-guided tasks. To address this limitation, several efforts have been made to augment grasping datasets with textual annotations or instruction-based labels [333, 366]. These enriched datasets enable agents to align visual and physical interaction with language understanding, thereby fostering more intelligent and context-aware grasping behaviors. Table 1.9 summarizes both conventional and language-augmented grasping datasets, highlighting their key features and contributions to the field.

Metrics. In embodied grasping, evaluation metrics span physical execution, semantic alignment, and task-level performance. Physical execution metrics primarily assess whether a grasp is mechanically successful. The most common is the grasp success rate, defined as the ratio of successful grasps to total attempts. For methods predicting grasp poses, precision and recall are used to measure detection accuracy, while grasp pose error quantifies deviation between predicted and ground-truth poses using positional and rotational differences. Semantic alignment metrics evaluate whether the grasp behavior aligns with linguistic intent. Semantic grasp accuracy measures the proportion of grasps that correctly fulfill the language instruction (e.g., grasping the “red cup”). In addition, language grounding score quantifies the alignment between language and action embeddings using similarity functions such as cosine similarity, reflecting the degree to which the grasp matches the intended semantics. Task-level metrics consider whether the grasp enables the successful com-

Table 1.9 Embodied grasping datasets

Dataset	Year	Type	Modality	Grasp label	Gripper finger	Objects	Grasps	Scenes	Language
Cornell [177]	2011	Real	RGB-D	Rect.	2	240	8K	Single	×
Jacquard [92]	2018	Sim	RGB-D	Rect.	2	11K	1.1M	Single	×
GraspNet [263]	2019	Sim	3D	6D	2	206	7.07M	Single	×
ACRONYM [108]	2021	Sim	3D	6D	2	8872	17.7M	Multi	×
MultiGripperGrasp [266]	2024	Sim	3D	-	2-5	345	30.4M	Single	×
OCID-Grasp [7]	2021	Real	RGB-D	Rect.	2	89	75K	Multi	×
OCID-VLG [366]	2023	Real	RGB-D, 3D	Rect.	2	89	75K	Multi	✓
ReasoningGrasp [181]	2024	Real	RGB-D	6D	2	64	99.3M	Multi	✓
CapGrasp [218]	2024	Sim	3D	-	5	1.8K	50K	Single	✓

pletion of downstream tasks. Task completion rate calculates the ratio of completed tasks to attempted ones, relevant for multi-step interactions such as pick-and-place or sorting. Grasp efficiency measures the temporal or behavioral cost per successful grasp, expressed as the inverse of completion time or number of action steps. Together, these metrics provide a holistic view of embodied grasping performance, capturing not only mechanical feasibility but also the agent's semantic understanding and task effectiveness. Incorporating all three types of metrics is essential for evaluating modern grasping systems that integrate vision, language, and decision-making capabilities in complex, real-world scenarios.

References

1. Abbeel, P., Ng, A.Y.: Apprenticeship learning via inverse reinforcement learning. In: Proceedings of the Twenty-first International Conference on Machine Learning, p. 1 (2004)
2. Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al.: Gpt-4 technical report (2023). [arXiv:2303.08774](https://arxiv.org/abs/2303.08774)
3. Achlioptas, P., Abdelreheem, A., Xia, F., Elhoseiny, M., Guibas, L.: Referit3d: neural listeners for fine-grained 3d object identification in real-world scenes. In: Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part I, vol. 16, pp. 422–440. Springer (2020)
4. Agravante, D.J., Cherubini, A., Sherikov, A., Wieber, P.B., Kheddar, A.: Human-humanoid collaborative carrying. *IEEE Trans. Rob.* **35**(4), 833–846 (2019)
5. Ahmed, M.F., Masood, K., Fremont, V., Fantoni, I.: Active slam: a review on last decade. *Sensors* **23**(19), 8097 (2023)
6. Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., et al.: Do as i can, not as i say: Grounding language in robotic affordances (2022). [arXiv:2204.01691](https://arxiv.org/abs/2204.01691)
7. Ainetter, S., Fraundorfer, F.: End-to-end trainable deep neural network for robotic grasp detection and semantic segmentation from rgb. In: IEEE International Conference on Robotics and Automation, pp. 13452–13458. IEEE (2021)
8. Ajoudani, A., Zanchettin, A.M., Ivaldi, S., Albu-Schäffer, A., Kosuge, K., Khatib, O.: Progress and prospects of the human-robot collaboration. *Auton. Robot.* **42**, 957–975 (2018)
9. Alayrac, J.B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., et al.: Flamingo: a visual language model for few-shot learning. *Adv. Neural. Inf. Process. Syst.* **35**, 23716–23736 (2022)
10. Alcantarilla, P.F., Yebes, J.J., Almazán, J., Bergasa, L.M.: On combining visual slam and dense scene flow to increase the robustness of localization and mapping in dynamic environments. In: 2012 IEEE International Conference on Robotics and Automation, pp. 1290–1297. IEEE (2012)
11. Aljundi, R., Babiloni, F., Elhoseiny, M., Rohrbach, M., Tuytelaars, T.: Memory aware synapses: learning what (not) to forget. In: Proceedings of the European Conference on Computer Vision (ECCV), pp. 139–154 (2018)
12. Aljundi, R., Kelchtermans, K., Tuytelaars, T.: Task-free continual learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11254–11263 (2019)
13. Alonso, E., Jelley, A., Micheli, V., Kanervisto, A., Storkey, A.J., Pearce, T., Fleuret, F.: Diffusion for world modeling: visual details matter in atari. *Adv. Neural. Inf. Process. Syst.* **37**, 58757–58791 (2024)

14. An, D., Wang, H., Wang, W., Wang, Z., Huang, Y., He, K., Wang, L.: Etpnav: evolving topological planning for vision-language navigation in continuous environments. *IEEE Trans. Pattern Anal. Mach. Intell.* (2024)
15. Anderson, P., Chang, A., Chaplot, D.S., Dosovitskiy, A., Gupta, S., Koltun, V., Kosecka, J., Malik, J., Mottaghi, R., Savva, M., et al.: On evaluation of embodied navigation agents (2018). [arXiv:1807.06757](https://arxiv.org/abs/1807.06757)
16. Anderson, P., Wu, Q., Teney, D., Bruce, J., Johnson, M., Sunderhauf, N., Reid, I., Gould, S., van den Hengel, A.: Vision-and-language navigation: interpreting visually-grounded navigation instructions in real environments. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2018). <https://doi.org/10.1109/cvpr.2018.00387>
17. Arm, P., Waibel, G., Preisig, J., Tuna, T., Zhou, R., Bickel, V., Ligeza, G., Miki, T., Kehl, F., Kolvenbach, H., et al.: Scientific exploration of challenging planetary analog environments with a team of legged robots. *Sci. Robot.* **8**(80), eade9548 (2023)
18. Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., Ballas, N.: Self-supervised learning from images with a joint-embedding predictive architecture. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15619–15629 (2023)
19. Azuma, D., Miyanishi, T., Kurita, S., Kawanabe, M.: ScanQA: 3D question answering for spatial scene understanding. In: *CVPR*, pp. 19129–19139 (2022)
20. Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., et al.: Qwen technical report (2023). [arXiv:2309.16609](https://arxiv.org/abs/2309.16609)
21. Bai, X., Hu, Z., Zhu, X., Huang, Q., Chen, Y., Fu, H., Tai, C.L.: TransFusion: robust LiDAR-camera fusion for 3D object detection with transformers. In: *CVPR*, pp. 1090–1099 (2022)
22. Bailey, T., Durrant-Whyte, H.: Simultaneous localization and mapping (slam): Part ii. *IEEE Robot. Autom. Mag.* **13**(3), 108–117 (2006)
23. Bajcsy, R.: Active perception. *Proc. IEEE* **76**(8), 966–1005 (1988)
24. Bakker, B.: Reinforcement learning with long short-term memory. In: *Advances in Neural Information Processing Systems*, vol. 14 (2001)
25. Bakr, E., Alsaedy, Y., Elhoseiny, M.: Look around and refer: 2d synthetic semantics knowledge distillation for 3d visual grounding. *NeurIPS* **35**, 37146–37158 (2022)
26. Banino, A., Badia, A.P., Koster, R., Chadwick, M.J., Zambaldi, V.F., Hassabis, D., Barry, C., Botvinick, M.M., Kumaran, D., Blundell, C.: Memo: a deep network for flexible combination of episodic models. In: *ICLR* (2020)
27. Baram, N., Anschel, O., Caspi, I., Mannor, S.: End-to-end differentiable adversarial imitation learning. In: *International Conference on Machine Learning*, pp. 390–399. PMLR (2017)
28. Bardes, A., Garrido, Q., Ponce, J., Chen, X., Rabbat, M., LeCun, Y., Assran, M., Ballas, N.: Revisiting feature prediction for learning visual representations from video (2024). [arXiv:2404.08471](https://arxiv.org/abs/2404.08471)
29. Bardes, A., Ponce, J., LeCun, Y.: Mc-jepa: a joint-embedding predictive architecture for self-supervised learning of motion and content features (2023). [arXiv:2307.12698](https://arxiv.org/abs/2307.12698)
30. Batra, H., Clark, R.: Evcl: elastic variational continual learning with weight consolidation (2024). [arXiv:2406.15972](https://arxiv.org/abs/2406.15972)
31. Bautista, M.A., Guo, P., Abnar, S., Talbott, W., Toshev, A., Chen, Z., Dinh, L., Zhai, S., Goh, H., Ulbricht, D., et al.: Gaudi: a neural architect for immersive 3d scene generation. *Adv. Neural. Inf. Process. Syst.* **35**, 25102–25116 (2022)
32. Beaulieu, S., Frati, L., Miconi, T., Lehman, J., Stanley, K.O., Clune, J., Cheney, N.: Learning to continually learn. In: *ECAI 2020*, pp. 992–1001. IOS Press (2020)
33. Behley, J., Garbade, M., Milioto, A., Quenzel, J., Behnke, S., Stachniss, C., Gall, J.: SemanticKITTI: a dataset for semantic scene understanding of LiDAR sequences. In: *ICCV*, pp. 9297–9307 (2019)
34. Bellegarda, G., Chen, Y., Liu, Z., Nguyen, Q.: Robust high-speed running for quadruped robots via deep reinforcement learning. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 10364–10370 (2022)

35. Berlinger, F., Gauci, M., Nagpal, R.: Implicit coordination for 3d underwater collective behaviors in a fish-inspired robot swarm. *Sci. Robot.* **6**(50), eabd8668 (2021)
36. Beyer, L., Steiner, A., Pinto, A.S., Kolesnikov, A., Wang, X., Salz, D., Neumann, M., Alabdulmohsin, I., Tschannen, M., Bugliarello, E., et al.: Paligemma: a versatile 3b vlm for transfer (2024). [arXiv:2407.07726](https://arxiv.org/abs/2407.07726)
37. Billard, A., Kragic, D.: Trends and challenges in robot manipulation. *Science* **364**(6446), eaat8414 (2019)
38. Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al.: $\pi 0$: a vision-language-action flow model for general robot control (2024). <https://arxiv.org/abs/2410.24164>
39. Bojarski, M., Del Testa, D., Dworakowski, D., Firner, B., Flepp, B., Goyal, P., Jackel, L.D., Monfort, M., Muller, U., Zhang, J., et al.: End to end learning for self-driving cars (2016). [arXiv:1604.07316](https://arxiv.org/abs/1604.07316)
40. Bouman, A., Ginting, M.F., Alatur, N., Palieri, M., Fan, D.D., Touma, T., Pailevanian, T., Kim, S.K., Otsu, K., Burdick, J., et al.: Autonomous spot: long-range autonomous exploration of extreme environments with legged locomotion. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 2518–2525 (2020)
41. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Chormanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al.: Rt-2: vision-language-action models transfer web knowledge to robotic control (2023). [arXiv:2307.15818](https://arxiv.org/abs/2307.15818)
42. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al.: Rt-1: robotics transformer for real-world control at scale (2022). [arXiv:2212.06817](https://arxiv.org/abs/2212.06817)
43. Brohan, A., Chebotar, Y., Finn, C., Hausman, K., Herzog, A., Ho, D., Ibarz, J., Irpan, A., Jang, E., Julian, R., et al.: Do as i can, not as i say: grounding language in robotic affordances. In: *Conference on Robot Learning*, pp. 287–318. PMLR (2023)
44. Brophy, T., Mullins, D., Parsi, A., Horgan, J., Ward, E., Denny, P., Eising, C., Deegan, B., Glavin, M., Jones, E.: A review of the impact of rain on camera-based perception in automated driving systems. *IEEE Access* (2023)
45. Brown, D., Goo, W., Nagarajan, P., Niekum, S.: Extrapolating beyond suboptimal demonstrations via inverse reinforcement learning from observations. In: *International Conference on Machine Learning*, pp. 783–792. PMLR (2019)
46. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al.: Language models are few-shot learners. *Adv. Neural. Inf. Process. Syst.* **33**, 1877–1901 (2020)
47. Bruce, J., Dennis, M.D., Edwards, A., Parker-Holder, J., Shi, Y., Hughes, E., Lai, M., Mavalankar, A., Steigerwald, R., Apps, C., et al.: Genie: Generative interactive environments. In: *Forty-First International Conference on Machine Learning* (2024)
48. Cai, J., Wu, Z., Peng, S., Wei, Y., Tan, Z., Shi, G., Gao, M., Ma, K.: Gemini: mapping and architecture co-exploration for large-scale dnn chiplet accelerators. In: *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 156–171. IEEE (2024)
49. Cangea, C., Belilovsky, E., Liò, P., Courville, A.: Videonavqa: bridging the gap between visual and embodied question answering (2019). [arXiv:1908.04950](https://arxiv.org/abs/1908.04950)
50. Cardace, A., Ramirez, P.Z., Salti, S., Di Stefano, L.: Exploiting the complementarity of 2D and 3D networks to address domain-shift in 3D semantic segmentation. In: *CVPR*, pp. 98–109 (2023)
51. Cen, J., Zhang, S., Pei, Y., Li, K., Zheng, H., Luo, M., Zhang, Y., Chen, Q.: CMDFusion: bidirectional fusion network with cross-modality knowledge distillation for LIDAR semantic segmentation (2023). [arXiv:2307.04091](https://arxiv.org/abs/2307.04091)
52. Chang, A., Dai, A., Funkhouser, T., Halber, M., Niebner, M., Savva, M., Song, S., Zeng, A., Zhang, Y.: Matterport3d: learning from rgb-d data in indoor environments. In: *2017 International Conference on 3D Vision (3DV)* (2017)

53. Chang, A., Dai, A., Funkhouser, T., Halber, M., Niessner, M., Savva, M., Song, S., Zeng, A., Zhang, Y.: Matterport3d: learning from rgb-d data in indoor environments (2017). [arXiv:1709.06158](https://arxiv.org/abs/1709.06158)
54. Chang, A.X., Funkhouser, T., Guibas, L., Hanrahan, P., Huang, Q., Li, Z., Savarese, S., Savva, M., Song, S., Su, H., et al.: ShapeNet: an information-rich 3D model repository (2015). [arXiv:1512.03012](https://arxiv.org/abs/1512.03012)
55. Chaplot, D.S., Dalal, M., Gupta, S., Malik, J., Salakhutdinov, R.R.: Seal: self-supervised embodied active learning using exploration and 3d consistency. *Adv. Neural. Inf. Process. Syst.* **34**, 13086–13098 (2021)
56. Chaplot, D.S., Gandhi, D., Gupta, S., Gupta, A., Salakhutdinov, R.: Learning to explore using active neural slam. In: 8th International Conference on Learning Representations, ICLR 2020 (2020)
57. Chen, B., Kang, J., Zhong, P., Liang, Y., Sheng, Y., Wang, J.: Embodied contrastive learning with geometric consistency and behavioral awareness for object navigation. In: Proceedings of the 32nd ACM International Conference on Multimedia, pp. 4776–4785 (2024)
58. Chen, C., Wu, Y.F., Yoon, J., Ahn, S.: Transdreamer: reinforcement learning with transformer world models (2022). [arXiv:2202.09481](https://arxiv.org/abs/2202.09481)
59. Chen, D.Z., Chang, A.X., Nießner, M.: ScanRefer: 3D object localization in RGB-D scans using natural language. In: ECCV, pp. 202–221 (2020)
60. Chen, H., Suhr, A., Misra, D., Snavely, N., Artzi, Y.: Touchdown: natural language navigation and spatial reasoning in visual street environments. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (2019). <https://doi.org/10.1109/cvpr.2019.01282>
61. Chen, H., Suhr, A., Misra, D., Snavely, N., Artzi, Y.: Touchdown: natural language navigation and spatial reasoning in visual street environments. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 12538–12547 (2019)
62. Chen, K., Choy, C.B., Savva, M., Chang, A.X., Funkhouser, T., Savarese, S.: Text2Shape: generating shapes from natural language by learning joint embeddings. In: ACCV, pp. 100–116 (2019)
63. Chen, R., Liu, Y., Kong, L., Chen, N., Zhu, X., Ma, Y., Liu, T., Wang, W.: Towards label-free scene understanding by vision foundation models. *Adv. Neural. Inf. Process. Syst.* **36**, 75896–75910 (2023)
64. Chen, R., Liu, Y., Kong, L., Zhu, X., Ma, Y., Li, Y., Hou, Y., Qiao, Y., Wang, W.: CLIP2Scene: towards label-efficient 3D scene understanding by CLIP. In: CVPR, pp. 7020–7030 (2023)
65. Chen, S., Zhu, H., Chen, X., Lei, Y., Yu, G., Chen, T.: End-to-end 3D dense captioning with Vote2Cap-DETR. In: CVPR, pp. 11124–11133 (2023)
66. Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Shen, H., Cowan, M., Wang, L., Hu, Y., Ceze, L., et al.: {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In: 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pp. 578–594 (2018)
67. Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Qian, C., Chan, C.M., Qin, Y., Lu, Y., Xie, R., et al.: Agentverse: facilitating multi-agent collaboration and exploring emergent behaviors in agents. **2**(4), 6 (2023). [arXiv:2308.10848](https://arxiv.org/abs/2308.10848)
68. Chen, X., Lawrence Zitnick, C.: Mind’s eye: a recurrent visual representation for image caption generation. In: CVPR, pp. 2422–2431 (2015)
69. Chen, X., Wang, X., Changpinyo, S., Piergiovanni, A.J., Padlewski, P., Salz, D., Goodman, S., Grycner, A., Mustafa, B., Beyer, L., et al.: Pali: a jointly-scaled multilingual language-image model (2022). [arXiv:2209.06794](https://arxiv.org/abs/2209.06794)
70. Chen, Y., Yu, Z., Chen, Y., Lan, S., Anandkumar, A., Jia, J., Alvarez, J.M.: FocalFormer3D: focusing on hard instance for 3D object detection. In: ICCV, pp. 8394–8405 (2023)
71. Chen, Z., Gholami, A., Nießner, M., Chang, A.X.: Scan2cap: context-aware dense captioning in RGB-D scans. In: CVPR, pp. 3193–3203 (2021)
72. Chen, Z., Li, Z., Zhang, S., Fang, L., Jiang, Q., Zhao, F.: AutoAlignV2: deformable feature aggregation for dynamic multi-modal 3D object detection (2022). [arXiv:2207.10316](https://arxiv.org/abs/2207.10316)

73. Chen, Z., Li, Z., Zhang, S., Fang, L., Jiang, Q., Zhao, F., Zhou, B., Zhao, H.: AutoAlign: pixel-instance feature aggregation for multi-modal 3D object detection (2022). [arXiv:2201.06493](https://arxiv.org/abs/2201.06493)
74. Chen, Z., Wu, J., Wang, W., Su, W., Chen, G., Xing, S., Zhong, M., Zhang, Q., Zhu, X., Lu, L., et al.: Intervl: scaling up vision foundation models and aligning for generic visual-linguistic tasks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (2024)
75. Cho, J., Puspitasari, F.D., Zheng, S., Zheng, J., Lee, L.H., Kim, T.H., Hong, C.S., Zhang, C.: Sora as an agi world model? A complete survey on text-to-video generation (2024). [arXiv:2403.05131](https://arxiv.org/abs/2403.05131)
76. Clark, A.: Being There: Putting Brain, Body, and World Together Again. MIT press (1998)
77. Clark, A.: An embodied cognitive science? Trends Cogn. Sci. **3**(9), 345–351 (1999)
78. Clark, A.: Whatever Next? Predictive Brains and Embodied Cognition. MIT Press (2013)
79. Cohen-Bar, D., Richardson, E., Metzger, G., Giryas, R., Cohen-Or, D.: Set-the-scene: global-local training for generating controllable nerf scenes (2023). [arXiv:2303.13450](https://arxiv.org/abs/2303.13450)
80. Coumans, E., Bai, Y.: Pybullet, A Python Module for Physics Simulation for Games, Robotics and Machine Learning (2016)
81. de Croon, G.C., Dupeyroux, J., Fuller, S.B., Marshall, J.A.: Insect-inspired ai for autonomous robots. Sci. Robot. **7**(67), eabl6334 (2022)
82. Cyberbotics: Webots: open-source robot simulator. <https://github.com/cyberbotics/webots>
83. Dai, A., Chang, A.X., Savva, M., Halber, M., Funkhouser, T., Nießner, M.: ScanNet: richly-annotated 3D reconstructions of indoor scenes. In: CVPR, pp. 5828–5839 (2017)
84. Dai, J., Qi, H., Xiong, Y., Li, Y., Zhang, G., Hu, H., Wei, Y.: Deformable convolutional networks. In: ICCV, pp. 764–773 (2017)
85. Dai, W., Zhang, Y., Li, P., Fang, Z., Scherer, S.: Rgb-d slam in dynamic environments using point correlations. IEEE Trans. Pattern Anal. Mach. Intell. **44**(1), 373–389 (2020)
86. Dang, R., Yuan, Y., Zhang, W., Xin, Y., Zhang, B., Li, L., Wang, L., Zeng, Q., Li, X., Bing, L.: Ecbench: can multi-modal foundation models understand the egocentric world? A holistic embodied cognition benchmark (2023). [arXiv:2501.05031](https://arxiv.org/abs/2501.05031)
87. Das, A., Datta, S., Gkioxari, G., Lee, S., Parikh, D., Batra, D.: Embodied question answering. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1–10 (2018)
88. De Curtò, J., De Zarza, I., Calafate, C.T.: Semantic scene understanding with large language models on unmanned aerial vehicles. Drones **7**(2), 114 (2023)
89. De Lange, M., Tuytelaars, T.: Continual prototype evolution: Learning online from non-stationary data streams. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 8250–8259 (2021)
90. Deisenroth, M., Rasmussen, C.E.: Pilco: A model-based and data-efficient approach to policy search. In: Proceedings of the 28th International Conference on Machine Learning (ICML-11), pp. 465–472 (2011)
91. Deitke, M., VanderBilt, E., Herrasti, A., Weihs, L., Salvador, J., Ehsani, K., Han, W., Kolve, E., Farhadi, A., Kembhavi, A., Mottaghi, R.: ProcTHOR: large-scale embodied AI using procedural generation. In: NeurIPS (2022). Outstanding Paper Award
92. Depierre, A., Dellandréa, E., Chen, L.: Jacquard: a large scale dataset for robotic grasp detection. In: IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 3511–3516 (2018)
93. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding (2018). [arXiv:1810.04805](https://arxiv.org/abs/1810.04805)
94. Dijksterhuis, A., Van Knippenberg, A.: The relation between perception and behavior, or how to win a game of trivial pursuit. J. Pers. Soc. Psychol. **74**(4), 865 (1998)
95. Ding, J., Zhang, Y., Shang, Y., Zhang, Y., Zong, Z., Feng, J., Yuan, Y., Su, H., Li, N., Sukiennik, N., et al.: Understanding world or predicting future? A comprehensive survey of world models (2024). [arXiv:2411.14499](https://arxiv.org/abs/2411.14499)
96. Dong, X., Bao, J., Zheng, Y., Zhang, T., Chen, D., Yang, H., Zeng, M., Zhang, W., Yuan, L., Chen, D., et al.: MaskCLIP: masked self-distillation advances contrastive language-image pretraining. In: CVPR, pp. 10995–11005 (2023)

97. Dorbala, V.S., Goyal, P., Piramuthu, R., Johnston, M., Manocha, D., Ghanadhan, R.: S-eqa: tackling situational queries in embodied question answering (2024). [arXiv:2405.04732](#)
98. Doshi, R., Walke, H., Mees, O., Dasari, S., Levine, S.: Scaling cross-embodied learning: one policy for manipulation, navigation, locomotion and aviation (2024). [arXiv:2408.11812](#)
99. Driess, D., Huang, Y., Zeng, A.e.a.: Vision-language-action models: unified frameworks for embodied execution. In: Conference on Robot Learning (CoRL), pp. 1234–1245 (2023)
100. Driess, D., Xia, F., Sajjadi, M.S., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., et al.: Palm-e: an embodied multimodal language model. In: International Conference on Machine Learning, pp. 8469–8488. PMLR (2023)
101. Du, Z.J., Huang, S.S., Mu, T.J., Zhao, Q., Martin, R.R., Xu, K.: Accurate dynamic slam using crf-based long-term consistency. *IEEE Trans. Visual Comput. Graph.* **28**(4), 1745–1757 (2020)
102. Duan, J., Yu, S., Tan, H.L., Zhu, H., Tan, C.: A survey of embodied ai: From simulators to research tasks. *IEEE Trans. Emerg. Top. Comput. Intell.* (2022)
103. Duan, Y., Schulman, J., Chen, X., Bartlett, P.L., Sutskever, I., Abbeel, P.: RI^2 : fast reinforcement learning via slow reinforcement learning (2016). [arXiv:1611.02779](#)
104. Durrant-Whyte, H., Bailey, T.: Simultaneous localization and mapping: part i. *IEEE Robot. Autom. Mag.* **13**(2), 99–110 (2006)
105. Edwards, A., Sahni, H., Schroecker, Y., Isbell, C.: Imitating latent policies from observation. In: International Conference on Machine Learning, pp. 1755–1763. PMLR (2019)
106. Engel, J., Schöps, T., Cremers, D.: Lsd-slam: large-scale direct monocular slam. In: European Conference on Computer Vision, pp. 834–849. Springer (2014)
107. Engelmann, F., Kontogianni, T., Hermans, A., Leibe, B.: Exploring spatial context for 3D semantic segmentation of point clouds. In: ICCVW, pp. 716–724 (2017)
108. Eppner, C., Mousavian, A., Fox, D.: Acronym: A large-scale grasp dataset based on simulation. In: IEEE International Conference on Robotics and Automation, pp. 6222–6227. IEEE (2021)
109. Fan, L., Wang, G., Jiang, Y., Mandlekar, A., Yang, Y., Zhu, H., Tang, A., Huang, D.A., Zhu, Y., Anandkumar, A.: Minedojo: building open-ended embodied agents with internet-scale knowledge. *Adv. Neural. Inf. Process. Syst.* **35**, 18343–18362 (2022)
110. Fang, H.S., Wang, C., Gou, M., Lu, C.: Graspnet-1billion: a large-scale benchmark for general object grasping. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11444–11453 (2020)
111. Fei, Z., Fan, M., Huang, J.: A-jepa: joint-embedding predictive architecture can listen (2023). [arXiv:2311.15830](#)
112. Feng, M., Li, Z., Li, Q., Zhang, L., Zhang, X., Zhu, G., Zhang, H., Wang, Y., Mian, A.: Free-form description guided 3D visual graph network for object grounding in point cloud. In: CVPR, pp. 3722–3731 (2021)
113. Feng, T., Li, Q., Wang, X., Wang, M., Li, G., Zhu, W.: Multi-weather cross-view geo-localization using denoising diffusion models. In: Proceedings of the 2nd Workshop on UAVs in Multimedia: Capturing the World from a New Perspective, pp. 35–39 (2024)
114. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata: a large-scale cooperative perception dataset for swarm uavs autonomous flight. In: Proceedings of the 32nd ACM International Conference on Multimedia, pp. 7600–7608 (2024)
115. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata-2: a scalable swarm uavs autonomous flight dataset for long-horizon tasks (2025). [arXiv:2509.00055](#)
116. Feng, T., Wang, X., Jiang, Y.G., Zhu, W.: Embodied ai: from llms to world models. In: *IEEE Circuits Syst. Mag.* (2025)
117. Feng, T., Wang, X., Zhou, Z., Wang, R., Zhan, Y., Li, G., Li, Q., Zhu, W.: Evoagent: Agent autonomous evolution with continual world model for long-horizon tasks (2025). [arXiv:2502.05907](#)
118. Figure, A.: Helix: A vision-language-action model for generalist humanoid control. *Figure AI News* (2024)
119. Filipenko, M., Afanasyev, I.: Comparison of various slam systems for mobile robot in an indoor environment. In: 2018 International Conference on Intelligent Systems (IS), pp. 400–407. IEEE (2018)

120. Firoozi, R., Tucker, J., Tian, S., Majumdar, A., Sun, J., Liu, W., Zhu, Y., Song, S., Kapoor, A., Hausman, K., et al.: Foundation models in robotics: applications, challenges, and the future. *Int. J. Robot. Res.* 02783649241281508 (2023)
121. Floridi, L., Chiriatti, M.: Gpt-3: Its nature, scope, limits, and consequences. *Minds Mach.* (2020)
122. Forrester, J.W.: Counterintuitive behavior of social systems. *Theor. Decis.* **2**(2), 109–140 (1971)
123. Fridman, R., Abecasis, A., Kasten, Y., Dekel, T.: SceneScape: text-driven consistent scene generation (2023). [arXiv:2302.01133](https://arxiv.org/abs/2302.01133)
124. Gan, C., Schwartz, J., Alter, S., Schrimpf, M., Traer, J., Freitas, J., Kubilius, J., Bhandwaldar, A., Haber, N., Sano, M., Kim, K., Wang, E., Mrowca, D., Lingelbach, M., Curtis, A., Feigelis, K., Bear, D., Gutfreund, D., Cox, D., DiCarlo, J., McDermott, J., Tenenbaum, J., Yamins, D.: Threedworld: a platform for interactive multi-modal physical simulation. *Neural Inf. Process. Syst.* 2021 (2021)
125. Gao, G., Liu, W., Chen, A., Geiger, A., Schölkopf, B.: Graphdreamer: compositional 3d scene synthesis from scene graphs. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 21295–21304 (2024)
126. Gao, X., Gao, Q., Gong, R., Lin, K., Thattai, G., Sukhatme, G.: Dialfred: dialogue-enabled agents for embodied instruction following (2022). [arXiv:2202.13330](https://arxiv.org/abs/2202.13330)
127. Gao, X., Zhang, T.: *Introduction to Visual SLAM: from Theory to Practice*. Springer Nature (2021)
128. Gao, Y., Li, C., You, Z., Liu, J., Li, Z., Chen, P., Chen, Q., Tang, Z., Wang, L., Yang, P., et al.: Openfly: a comprehensive platform for aerial vision-language navigation (2025). [arXiv:2502.18041](https://arxiv.org/abs/2502.18041)
129. Genc, H., Kim, S., Amid, A., Haj-Ali, A., Iyer, V., Prakash, P., Zhao, J., Grubb, D., Liew, H., Mao, H., et al.: Gemini: enabling systematic deep-learning architecture evaluation via full-stack integration. In: *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pp. 769–774. IEEE (2021)
130. Gibbs Jr, R.W.: *Embodiment and Cognitive Science*. Cambridge University Press (2005)
131. Gonzalez, M., Marchand, E., Kacete, A., Royan, J.: Twistslam: constrained slam in dynamic environment. *IEEE Robot. Autom. Lett.* **7**(3), 6846–6853 (2022)
132. Gordon, D., Kembhavi, A., Rastegari, M., Redmon, J., Fox, D., Farhadi, A.: Iqa: visual question answering in interactive environments. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4089–4098 (2018)
133. Gupta, T., Gong, W., Ma, C., Pawlowski, N., Hilmkil, A., Scetbon, M., Rigter, M., Famoti, A., Llorens, A.J., Gao, J., et al.: The essential role of causality in foundation world models for embodied ai (2024). [arXiv:2402.06665](https://arxiv.org/abs/2402.06665)
134. Ha, D., Schmidhuber, J.: World models (2018). [arXiv:1803.10122](https://arxiv.org/abs/1803.10122)
135. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *International Conference on Machine Learning*, pp. 1861–1870. Pmlr (2018)
136. Haddadin, S., Parusel, S., Johannsmeier, L., Golz, S., Gabl, S., Walch, F., Sabaghian, M., Jähne, C., Hausperger, L., Haddadin, S.: The franka emika robot: a reference platform for robotics research and education. *IEEE Robot. Autom. Mag.* **29**(2), 46–64 (2022)
137. Hafner, D., Lillicrap, T., Ba, J., Norouzi, M.: Dream to control: learning behaviors by latent imagination (2019). [arXiv:1912.01603](https://arxiv.org/abs/1912.01603)
138. Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., Davidson, J.: Learning latent dynamics for planning from pixels. In: *International Conference on Machine Learning*, pp. 2555–2565. PMLR (2019)
139. Hafner, D., Lillicrap, T., Norouzi, M., Ba, J.: Mastering atari with discrete world models (2020). [arXiv:2010.02193](https://arxiv.org/abs/2010.02193)
140. Hafner, D., Lillicrap, T.P., Ba, J., Norouzi, M.: Dream to control: learning behaviors by latent imagination. In: *ICLR* (2020)

141. Hafner, D., Lillicrap, T.P., Norouzi, M., Ba, J.: Mastering atari with discrete world models. In: ICLR (2021)
142. Han, F., Zhang, L., Wang, X., Zhao, K.A., Zhong, Y., Su, Z., Feng, T., Zhu, W.: U2usim-a uav telepresence simulation platform with multi-agent sensing and dynamic environment. In: Proceedings of the 32nd ACM International Conference on Multimedia, pp. 11258–11260 (2024)
143. Han, S., Mao, H., Dally, W.J.: Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding. In: International Conference on Learning Representations (2015)
144. Hansen, N., SV, J., Sobal, V., LeCun, Y., Wang, X., Su, H.: Hierarchical world models as visual whole-body humanoid controllers (2024). [arXiv:2405.18418](https://arxiv.org/abs/2405.18418)
145. Hao, Y., Sun, Y., Dong, L., Han, Z., Gu, Y., Wei, F.: Structured prompting: scaling in-context learning to 1,000 examples (2022). [arXiv:2212.06713](https://arxiv.org/abs/2212.06713)
146. He, C., Zeng, H., Huang, J., Hua, X.S., Zhang, L.: Structure aware single-stage 3D object detection from point cloud. In: CVPR, pp. 11873–11882 (2020)
147. He, D., Zhao, Y., Luo, J., Hui, T., Huang, S., Zhang, A., Liu, S.: TransRefer3D: entity-and-relation aware transformer for fine-grained 3D visual grounding. In: ACM MM, pp. 2344–2352 (2021)
148. Ho, J., Ermon, S.: Generative adversarial imitation learning. In: Advances in Neural Information Processing Systems, vol. 29 (2016)
149. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In: NeurIPS, vol. 33, pp. 6840–6851 (2020)
150. Höllein, L., Cao, A., Owens, A., Johnson, J., Nießner, M.: Text2Room: extracting textured 3D meshes from 2D text-to-image models (2023). [arXiv:2303.11989](https://arxiv.org/abs/2303.11989)
151. Hong, J., Choi, R., Leonard, J.J.: Learning from feedback: semantic enhancement for object slam using foundation models (2024). [arXiv:2411.06752](https://arxiv.org/abs/2411.06752)
152. Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., et al.: Metagtpt: meta programming for multi-agent collaborative framework. **3**(4), 6 (2023). [arXiv:2308.00352](https://arxiv.org/abs/2308.00352)
153. Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Dong, Y., Ding, M., et al.: Cogagent: a visual language model for gui agents. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 14281–14290 (2024)
154. Hong, Y., Zheng, Z., Chen, P., Wang, Y., Li, J., Gan, C.: Multiply: a multisensory object-centric embodied large language model in 3d world. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (2024)
155. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al.: Lora: low-rank adaptation of large language models. ICLR **1**(2), 3 (2022)
156. Hu, J., Shen, L., Sun, G.: Squeeze-and-excitation networks. In: CVPR, pp. 7132–7141 (2018)
157. Huang, P.H., Lee, H.H., Chen, H.T., Liu, T.L.: Text-guided graph neural networks for referring 3D instance segmentation. In: AAAI, vol. 35, pp. 1610–1618 (2021)
158. Huang, S., Chen, Y., Jia, J., Wang, L.: Multi-view transformer for 3D visual grounding. In: CVPR, pp. 15524–15533 (2022)
159. Huang, S., Dong, L., Wang, W., Hao, Y., Singhal, S., Ma, S., Lv, T., Cui, L., Mohammed, O.K., Patra, B., et al.: Language is not all you need: aligning perception with language models. In: Advances in Neural Information Processing Systems (2023)
160. Huang, T., Liu, Z., Chen, X., Bai, X.: EPNet: enhancing point features with image semantics for 3D object detection. In: ECCV, pp. 35–52 (2020)
161. Huang, W., Wang, C., Li, Y., Zhang, R., Fei-Fei, L.: Rekep: spatio-temporal reasoning of relational keypoint constraints for robotic manipulation (2024). [arXiv:2409.01652](https://arxiv.org/abs/2409.01652)
162. Hussein, A., Gaber, M.M., Elyan, E., Jayne, C.: Imitation learning: a survey of learning methods. ACM Comput. Surv. (CSUR) **50**(2), 1–35 (2017)
163. Ilami, M., Bagheri, H., Ahmed, R., Skowronek, E.O., Marvi, H.: Materials, actuators, and sensors for soft bioinspired robots. Adv. Mater. **33**(19), 2003139 (2021)

164. Ilharco, G., Jain, V., Ku, A., Ie, E., Baldridge, J.: General evaluation for instruction conditioned navigation using dynamic time warping (2019). [arXiv:1907.05446](https://arxiv.org/abs/1907.05446)
165. ISAE-SUPAERO: Morse: the modular open robots simulator engine (2015). <https://morse-simulator.github.io/>
166. Jaderberg, M., Mnih, V., Czarnecki, W.M., Schaul, T., Leibo, J.Z., Silver, D., Kavukcuoglu, K.: Reinforcement learning with unsupervised auxiliary tasks (2016). [arXiv:1611.05397](https://arxiv.org/abs/1611.05397)
167. Jaegle, A., Gimeno, F., Brock, A., Vinyals, O., Zisserman, A., Carreira, J.: Perceiver: general perception with iterative attention. In: International Conference on Machine Learning, pp. 4651–4664. PMLR (2021)
168. Jain, A., Gkanatsios, N., Mediratta, I., Fragkiadaki, K.: Bottom up top down detection transformers for language grounding in images and point clouds. In: ECCV, pp. 417–433 (2022)
169. Jain, V., Magalhaes, G., Ku, A., Vaswani, A., Ie, E., Baldridge, J.: Stay on the path: instruction fidelity in vision-and-language navigation. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (2019)
170. Jain, V., Magalhaes, G., Ku, A., Vaswani, A., Ie, E., Baldridge, J.: Stay on the path: instruction fidelity in vision-and-language navigation. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 1862–1872. Association for Computational Linguistics, Florence, Italy (2019). <https://doi.org/10.18653/v1/P19-1181>. <https://aclanthology.org/P19-1181>
171. Jang, E., Irpan, A., Khansari, M., Kappler, D., Ebert, F., Lynch, C., Levine, S., Finn, C.: Bc-z: zero-shot task generalization with robotic imitation learning. In: Conference on Robot Learning, pp. 991–1002 (2022)
172. Jaritz, M., Vu, T.H., Charette, R.d., Wirbel, E., Pérez, P.: xMUDA: cross-modal unsupervised domain adaptation for 3D semantic segmentation. In: CVPR, pp. 12605–12614 (2020)
173. Jayaraman, D., Grauman, K.: Learning to look around: intelligently exploring unseen environments for unknown tasks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1238–1247 (2018)
174. Jia, C., Yang, Y., Xia, Y., Chen, Y.T., Parekh, Z., Pham, H., Le, Q., Sung, Y.H., Li, Z., Duerig, T.: Scaling up visual and vision-language representation learning with noisy text supervision. In: International Conference on Machine Learning, pp. 4904–4916. PMLR (2021)
175. Jiang, B., Xie, Y., Wang, X., Su, W.J., Taylor, C.J., Mallick, T.: Multi-modal and multi-agent systems meet rationality: a survey. ICML Workshop on LLMs and Cognition (2024)
176. Jiang, W., Lei, B., Ashton, K., Daniilidis, K.: Multimodal llm guided exploration and active mapping using fisher information (2024). [arXiv:2410.17422](https://arxiv.org/abs/2410.17422)
177. Jiang, Y., Moseson, S., Saxena, A.: Efficient grasping from rgb-d images: learning using a new rectangle representation. In: IEEE International Conference on Robotics and Automation, pp. 3304–3311. IEEE (2011)
178. Jiao, Y., Chen, S., Jie, Z., Chen, J., Ma, L., Jiang, Y.G.: MORE: multi-order relation mining for dense captioning in 3D scenes. In: ECCV, pp. 528–545 (2022)
179. Jiao, Y., Jie, Z., Chen, S., Chen, J., Ma, L., Jiang, Y.G.: MSMDfusion: fusing lidar and camera at multiple scales with multi-depth seeds for 3D object detection. In: CVPR, pp. 21643–21652 (2023)
180. Jin, L., Jia, L.: Embodied world models emerge from navigational task in open-ended environments (2025). [arXiv:2504.11419](https://arxiv.org/abs/2504.11419)
181. Jin, S., Xu, J., Lei, Y., Zhang, L.: Reasoning grasping via multimodal large language model (2024). [arXiv:2402.06798](https://arxiv.org/abs/2402.06798)
182. Jin, Z., Hayat, M., Yang, Y., Guo, Y., Lei, Y.: Context-aware alignment and mutual masking for 3D-language pre-training. In: CVPR, pp. 10984–10994 (2023)
183. Jouppi, N.P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A., et al.: In-datacenter performance analysis of a tensor processing unit. In: Proceedings of the 44th Annual International Symposium on Computer Architecture, pp. 1–12 (2017)
184. Juliani, A., Berges, V.P., Teng, E., Cohen, A., Harper, J., Elion, C., Goy, C., Gao, Y., Henry, H., Mattar, M., Lange, D.: Unity: a general platform for intelligent agents (2020). [arXiv:1809.02627](https://arxiv.org/abs/1809.02627)

185. Kamath, A., Singh, M., LeCun, Y., Synnaeve, G., Misra, I., Carion, N.: MDETR-modulated detection for end-to-end multi-modal understanding. In: CVPR, pp. 1780–1790 (2021)
186. Kaplanis, C., Shanahan, M., Clopath, C.: Continual reinforcement learning with complex synapses. In: International Conference on Machine Learning, pp. 2497–2506. PMLR (2018)
187. Kato, H., Ushiku, Y., Harada, T.: Neural 3d mesh renderer. In: CVPR, pp. 3907–3916 (2018)
188. Katzschmann, R.K., DelPreto, J., MacCurdy, R., Rus, D.: Exploration of underwater life with an acoustically controlled soft robotic fish. *Sci. Robot.* **3**(16), eaar3449 (2018)
189. Kelly, R.: A tuning procedure for stable pid control of robot manipulators. *Robotica* **13**(2), 141–148 (1995)
190. Kennedy, M., Schmeckpeper, K., Thakur, D., Jiang, C., Kumar, V., Daniilidis, K.: Autonomous precision pouring from unknown containers. *IEEE Robot. Autom. Lett.* **4**(3), 2317–2324 (2019)
191. Khan, A., Zhang, C., Atanasov, N., Karydis, K., Kumar, V., Lee, D.D.: Memory augmented control networks (2017). [arXiv:1709.05706](https://arxiv.org/abs/1709.05706)
192. Khashabi, D., Min, S., Khot, T., Sabharwal, A., Tafjord, O., Clark, P., Hajishirzi, H.: UnifiedQA: crossing format boundaries with a single QA system (2020). [arXiv:2005.00700](https://arxiv.org/abs/2005.00700)
193. Kim, D.H., Kim, J.H.: Effective background model-based rgb-d dense visual odometry in a dynamic environment. *IEEE Trans. Rob.* **32**(6), 1565–1573 (2016)
194. Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al.: Openvla: an open-source vision-language-action model (2024). [arXiv:2406.09246](https://arxiv.org/abs/2406.09246)
195. Kipf, T., Li, Y., Dai, H., Zambaldi, V., Sanchez-Gonzalez, A., Grefenstette, E., Kohli, P., Battaglia, P.: Compile: compositional imitation learning and execution. In: International Conference on Machine Learning, pp. 3418–3428. PMLR (2019)
196. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al.: Overcoming catastrophic forgetting in neural networks. *Proc. Natl. Acad. Sci.* **114**(13), 3521–3526 (2017)
197. Kiumarsi, B., Vamvoudakis, K.G., Modares, H., Lewis, F.L.: Optimal and autonomous control using reinforcement learning: a survey. *IEEE Trans. Neural Netw. Learn. Syst.* **29**(6), 2042–2062 (2017)
198. Klein, G., Murray, D.: Parallel tracking and mapping for small ar workspaces. In: 2007 6th IEEE and ACM International Symposium on Mixed and Augmented Reality, pp. 225–234. IEEE (2007)
199. Koenig, N., Howard, A.: Design and use paradigms for gazebo, an open-source multi-robot simulator. In: IEEE/RSJ International Conference on Intelligent Robots and Systems, vol. 3, pp. 2149–2154 (2004)
200. Koenig, N., Howard, A.: Design and use paradigms for gazebo, an open-source multi-robot simulator. In: IEEE/RSJ International Conference on Intelligent Robots and Systems, vol. 3, pp. 2149–2154 (2004)
201. Kolve, E., Mottaghi, R., Gordon, D., Zhu, Y., Gupta, A., Farhadi, A.: Ai2-thor: an interactive 3d environment for visual ai. *Computer Vision and Pattern Recognition* (2017)
202. Korban, M., Youngs, P., Acton, S.T.: A multi-modal transformer network for action detection. *PR* **142**, 109713 (2023)
203. Kostrikov, I., Agrawal, K.K., Dwibedi, D., Levine, S., Tompson, J.: Discriminator-actor-critic: addressing sample inefficiency and reward bias in adversarial imitation learning (2018). [arXiv:1809.02925](https://arxiv.org/abs/1809.02925)
204. Krantz, J., Wijmans, E., Majumdar, A., Batra, D., Lee, S.: Beyond the nav-graph: vision-and-language navigation in continuous environments. In: European Conference on Computer Vision, pp. 104–120. Springer (2020)
205. Krispel, G., Opitz, M., Waltner, G., Possegger, H., Bischof, H.: FuseSeg: LiDAR point cloud segmentation fusing multi-modal data. In: WACV, pp. 1874–1883 (2020)
206. Laskin, M., Lee, K., Stooke, A., Pinto, L., Abbeel, P., Srinivas, A.: Reinforcement learning with augmented data. *Adv. Neural. Inf. Process. Syst.* **33**, 19884–19895 (2020)

207. Laskin, M., Srinivas, A., Abbeel, P.: Curl: Contrastive unsupervised representations for reinforcement learning. In: International Conference on Machine Learning, pp. 5639–5650. PMLR (2020)
208. Lattner, C., Adve, V.: Llvm: a compilation framework for lifelong program analysis & transformation. In: International Symposium on Code Generation and Optimization, 2004. CGO 2004., pp. 75–86. IEEE (2004)
209. Le, H., Jiang, N., Agarwal, A., Dudík, M., Yue, Y., Daumé III, H.: Hierarchical imitation and reinforcement learning. In: International conference on machine learning, pp. 2917–2926. PMLR (2018)
210. Le Cleac'h, S., Howell, T.A., Yang, S., Lee, C.Y., Zhang, J., Bishop, A., Schwager, M., Manchester, Z.: Fast contact-implicit model predictive control. *IEEE Trans. Robot.* (2024)
211. LeCun, Y.: A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Rev.* **62**(1), 1–62 (2022)
212. Li, C., Xia, F., Martín-Martín, R., Lingelbach, M., Srivastava, S., Shen, B., Vainio, K.E., Gokmen, C., Dharan, G., Jain, T., Kurenkov, A., Liu, K., Gweon, H., Wu, J., Fei-Fei, L., Savarese, S.: igibson 2.0: Object-centric simulation for robot learning of everyday household tasks. In: A. Faust, D. Hsu, G. Neumann (eds.) *Proceedings of the 5th Conference on Robot Learning, Proceedings of Machine Learning Research*, vol. 164, pp. 455–465 (2022)
213. Li, C., Zhang, R., Wong, J., Gokmen, C., Srivastava, S., Martín-Martín, R., Wang, C., Levine, G., Lingelbach, M., Sun, J., et al.: Behavior-1k: a benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In: *Conference on Robot Learning*, pp. 80–93 (2023)
214. Li, C., Zhang, R., Wong, J., Gokmen, C., Srivastava, S., Martín-Martín, R., Wang, C., Levine, G., Lingelbach, M., Sun, J., et al.: Behavior-1k: a benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In: *Conference on Robot Learning*, pp. 80–93. PMLR (2023)
215. Li, C., Zhu, S., Sun, Z., Rogers, J.: Bas optimized elm for kuka iiwa robot learning. *IEEE Trans. Circuits Syst. II Express Briefs* **68**(6), 1987–1991 (2020)
216. Li, D., Fang, Y., Chen, Y., Yang, S., Cao, S., Wong, J., Luo, M., Wang, X., Yin, H., Gonzalez, J.E., et al.: Worldmodelbench: judging video generation models as world models (2025). [arXiv:2502.20694](https://arxiv.org/abs/2502.20694)
217. Li, J., Dai, H., Han, H., Ding, Y.: Mseg3d: multi-modal 3d semantic segmentation for autonomous driving. In: *CVPR*, pp. 21694–21704 (2023)
218. Li, K., Wang, J., Yang, L., Lu, C., Dai, B.: Semgrasp: semantic grasp generation via language aligned discretization (2024). [arXiv:2404.03590](https://arxiv.org/abs/2404.03590)
219. Li, K., Yu, B., Zheng, Q., Zhan, Y., Zhang, Y., Zhang, T., Yang, Y., Chen, Y., Sun, L., Cao, Q., et al.: Muep: a multimodal benchmark for embodied planning with foundation models. In: *International Joint Conferences on Artificial Intelligence. IJCAI*, pp. 129–138 (2024)
220. Li, X., Ma, T., Hou, Y., Shi, B., Yang, Y., Liu, Y., Wu, X., Chen, Q., Li, Y., Qiao, Y., et al.: LoGoNet: towards accurate 3D object detection with local-to-global cross-modal fusion. In: *CVPR*, pp. 17524–17534 (2023)
221. Li, X., Zhang, M., Geng, Y., Geng, H., Long, Y., Shen, Y., Zhang, R., Liu, J., Dong, H.: Manipllm: embodied multimodal large language model for object-centric robotic manipulation. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024)
222. Li, Y., Yu, A.W., Meng, T., Caine, B., Ngiam, J., Peng, D., Shen, J., Lu, Y., Zhou, D., Le, Q.V., et al.: DeepFusion: LiDAR-camera deep fusion for multi-modal 3D object detection. In: *CVPR*, pp. 17182–17191 (2022)
223. Li, Z., Xie, Y., Shao, R., Chen, G., Jiang, D., Nie, L.: Optimus-1: hybrid multimodal memory empowered agents excel in long-horizon tasks (2024). [arXiv:2408.03615](https://arxiv.org/abs/2408.03615)
224. Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., Zeng, A.: Code as policies: language model programs for embodied control. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500. IEEE (2023)

225. Liang, T., Xie, H., Yu, K., Xia, Z., Lin, Z., Wang, Y., Tang, T., Wang, B., Tang, Z.: BEVFusion: a simple and robust LiDAR-camera fusion framework. In: *NeruIPS*, vol. 35, pp. 10421–10434 (2022)
226. Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning (2015). [arXiv:1509.02971](https://arxiv.org/abs/1509.02971)
227. Lin, B.Y., Huang, C., Liu, Q., Gu, W., Sommerer, S., Ren, X.: On grounded planning for embodied tasks with language models. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 13192–13200 (2023)
228. Lin, F., Tian, Y., Zhang, T., Huang, J., Guan, S., Wang, F.Y.: Airvista-ii: an agentic system for embodied uavs toward dynamic scene semantic understanding (2025). <https://arxiv.org/abs/2504.09583>
229. Lin, Y., Tang, H., Yang, S., Zhang, Z., Xiao, G., Gan, C., Han, S.: Qserve: W4a8kv4 quantization and system co-design for efficient llm serving (2024). [arXiv:2405.04532](https://arxiv.org/abs/2405.04532)
230. Liu, H., Abbeel, P.: Behavior from the void: unsupervised active pre-training. *Adv. Neural. Inf. Process. Syst.* **34**, 18459–18473 (2021)
231. Liu, H., Huang, X., Gu, J., Shi, J., He, N., Feng, T.: Tcdformer-based momentum transfer model for long-term sports prediction. *Exp. Syst. Appl.* 128310 (2025)
232. Liu, H., Li, C., Li, Y., Lee, Y.J.: Improved baselines with visual instruction tuning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 26296–26306 (2024)
233. Liu, H., Li, C., Wu, Q., Lee, Y.J.: Visual instruction tuning. In: *Advances in Neural Information Processing Systems* (2023)
234. Liu, J., Zeng, S., Ding, L., Soedarmadji, W., Zhou, H., Wang, Z., Li, J., Li, J., Dai, Y., Wen, K., et al.: Flightvgm: efficient video generation model inference with online sparsification and hybrid precision on fpgas. In: *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pp. 2–13 (2025)
235. Liu, S., Zhang, H., Qi, Y., Wang, P., Zhang, Y., Wu, Q.: Aerialvln: vision-and-language navigation for uavs. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 15384–15394 (2023)
236. Liu, X., Palacios, H., Muise, C.: Egocentric planning for scalable embodied task achievement. *Adv. Neural. Inf. Process. Syst.* **36**, 54586–54613 (2023)
237. Liu, Y., Gupta, A., Abbeel, P., Levine, S.: Imitation from observation: Learning to imitate behaviors from raw video via context translation. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1118–1125. IEEE (2018)
238. Liu, Y., Smith, J., Chen, Q.: Task alignment in embodied ai: bridging semantics and physical constraints. *Trans. Robot.* **40**, 1–15 (2024)
239. Liu, Y., Wang, T., Zhang, X., Sun, J.: PETR: position embedding transformation for multi-view 3D object detection. In: *ECCV*, pp. 531–548 (2022)
240. Liu, Y., Yoshie, O., Watanabe, H.: Application of multi-modal fusion attention mechanism in semantic segmentation. In: *ACCV*, pp. 1245–1264 (2022)
241. Liu, Z., Bahety, A., Song, S.: Reflect: summarizing robot experiences for failure explanation and correction (2023). [arXiv:2306.15724](https://arxiv.org/abs/2306.15724)
242. Liu, Z., Huang, T., Li, B., Chen, X., Wang, X., Bai, X.: EPNet++: cascade bi-directional fusion for multi-modal 3D object detection. *IEEE TPAMI* (2022)
243. Lowe, R., Wu, Y.I., Tamar, A., Harb, J., Pieter Abbeel, O., Mordatch, I.: Multi-agent actor-critic for mixed cooperative-competitive environments. In: *Advances in Neural Information Processing Systems*, vol. 30 (2017)
244. Lu, J., Pan, B., Chen, J., Feng, Y., Hu, J., Peng, Y., Chen, W.: Agentlens: visual analysis for agent behaviors in llm-based autonomous systems. *IEEE Trans. Vis. Comput. Graph.* (2024)
245. Luo, J., Fu, J., Kong, X., Gao, C., Ren, H., Shen, H., Xia, H., Liu, S.: 3d-sps: single-stage 3d visual grounding via referred point progressive selection. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16454–16463 (2022)
246. Ma, M., Cheng, L.: A human-robot collaboration controller utilizing confidence for disagreement adjustment. *IEEE Trans. Robot.* (2024)

247. Ma, X., Yong, S., Zheng, Z., Li, Q., Liang, Y., Zhu, S.C., Huang, S.: SQA3D: situated question answering in 3D scenes (2022). [arXiv:2210.07474](https://arxiv.org/abs/2210.07474)
248. Ma, Y., Song, Z., Zhuang, Y., Hao, J., King, I.: A survey on vision-language-action models for embodied ai (2024). [arXiv:2405.14093](https://arxiv.org/abs/2405.14093)
249. Macario Barros, A., Michel, M., Moline, Y., Corre, G., Carrel, F.: A comprehensive survey of visual slam algorithms. *Robotics* **11**(1), 24 (2022)
250. Majumdar, A., Ajay, A., Zhang, X., Putta, P., Yenamandra, S., Henaff, M., Silwal, S., Mcvay, P., Maksymets, O., Arnaud, S., et al.: Openeqa: embodied question answering in the era of foundation models. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16488–16498 (2024)
251. Makoviychuk, V., Wawrzyniak, L., Guo, Y., Lu, M., Storey, K., Macklin, M., Hoeller, D., Rudin, N., Allshire, A., Handa, A., et al.: Isaac gym: High performance gpu-based physics simulation for robot learning (2021). [arXiv:2108.10470](https://arxiv.org/abs/2108.10470)
252. Man, Y., Zheng, S., Bao, Z., Hebert, M., Gui, L., Wang, Y.X.: Lexicon3d: Probing visual foundation models for complex 3d scene understanding. *Adv. Neural. Inf. Process. Syst.* **37**, 76819–76847 (2024)
253. Maniatopoulos, S., Schillinger, P., Pong, V., Conner, D.C., Kress-Gazit, H.: Reactive high-level behavior synthesis for an atlas humanoid robot. In: *IEEE International Conference on Robotics and Automation*, pp. 4192–4199. IEEE (2016)
254. Masse, N.Y., Grant, G.D., Freedman, D.J.: Alleviating catastrophic forgetting using context-dependent gating and synaptic stabilization. *Proc. Natl. Acad. Sci.* **115**(44), E10467–E10475 (2018)
255. Mazzaglia, P., Verbelen, T., Dhoedt, B., Courville, A., Rajeswar, S.: Genrl: multimodal-foundation world models for generalization in embodied agents. *Adv. Neural. Inf. Process. Syst.* **37**, 27529–27555 (2024)
256. Meyer, G.P., Charland, J., Hegde, D., Laddha, A., Vallespi-Gonzalez, C.: Sensor fusion for joint 3D object detection and semantic segmentation. In: *CVPRW*, pp. 1–8 (2019)
257. Meyer, G.P., Laddha, A., Kee, E., Vallespi-Gonzalez, C., Wellington, C.K.: LaserNet: An efficient probabilistic 3D object detector for autonomous driving. In: *CVPR*, pp. 12677–12686 (2019)
258. Micheli, V., Alonso, E., Fleuret, F.: Transformers are sample-efficient world models. In: *ICLR* (2023)
259. Misra, I., Girdhar, R., Joulin, A.: An end-to-end transformer model for 3D object detection. In: *CVPR*, pp. 2906–2917 (2021)
260. Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., Kavukcuoglu, K.: Asynchronous methods for deep reinforcement learning. In: *International conference on machine learning*, pp. 1928–1937. PmLR (2016)
261. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al.: Human-level control through deep reinforcement learning. *Nature* **518**(7540), 529–533 (2015)
262. Modarressi, A., Köksal, A., Imani, A., Fayyaz, M., Schütze, H.: Memllm: Finetuning llms to use an explicit read-write memory (2024). [arXiv:2404.11672](https://arxiv.org/abs/2404.11672)
263. Mousavian, A., Eppner, C., Fox, D.: 6-dof graspnet: variational grasp generation for object manipulation. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 2901–2910 (2019)
264. Mu, Y., Zhang, Q., Hu, M., Wang, W., Ding, M., Jin, J., Wang, B., Dai, J., Qiao, Y., Luo, P.: Embodiedgpt: vision-language pre-training via embodied chain of thought. In: *Advances in Neural Information Processing Systems* (2023)
265. Mur-Artal, R., Montiel, J.M.M., Tardos, J.D.: Orb-slam: a versatile and accurate monocular slam system. *IEEE Trans. Rob.* **31**(5), 1147–1163 (2015)
266. Murilo, L.F.C., Khargonkar, N., Prabhakaran, B., Xiang, Y.: Multigrippergrasp: a dataset for robotic grasping from parallel jaw grippers to dexterous hands (2024). [arXiv:2403.09841](https://arxiv.org/abs/2403.09841)
267. Nachum, O., Gu, S.S., Lee, H., Levine, S.: Data-efficient hierarchical reinforcement learning. In: *Advances in Neural Information Processing Systems*, vol. 31 (2018)

268. Nair, A., McGrew, B., Andrychowicz, M., Zaremba, W., Abbeel, P.: Overcoming exploration in reinforcement learning with demonstrations. In: 2018 IEEE International Conference on Robotics and Automation (ICRA), pp. 6292–6299. IEEE (2018)
269. Nayak, S., Morrison Orozco, A., Have, M., Zhang, J., Thirumalai, V., Chen, D., Kapoor, A., Robinson, E., Gopalakrishnan, K., Harrison, J., et al.: Long-horizon planning for multi-agent robots in partially observable environments. *Adv. Neural. Inf. Process. Syst.* **37**, 67929–67967 (2024)
270. Newbury, R., Gu, M., Chumbley, L., Mousavian, A., Eppner, C., Leitner, J., Bohg, J., Morales, A., Asfour, T., Kragic, D., et al.: Deep learning approaches to grasp synthesis: a review. *IEEE Trans. Robot.* (2023)
271. Nguyen, C.V., Li, Y., Bui, T.D., Turner, R.E.: Variational continual learning (2017). [arXiv:1710.10628](https://arxiv.org/abs/1710.10628)
272. NVIDIA: Nvidia isaac sim: robotics simulation and synthetic data (2023). <https://developer.nvidia.com/isaac-sim>
273. Nygaard, T.F., Martin, C.P., Torresen, J., Glette, K., Howard, D.: Real-world embodied ai through a morphologically adaptive quadruped robot. *Nat. Mach. Intell.* **3**(5), 410–419 (2021)
274. Okada, M., Taniguchi, T.: Dreamingv2: reinforcement learning with discrete world models without reconstruction. In: IROS (2022)
275. OpenAI: Gpt-4 technical report (2023). [arXiv:2303.08774](https://arxiv.org/abs/2303.08774)
276. Osa, T., Pajarinen, J., Neumann, G., Bagnell, J.A., Abbeel, P., Peters, J., et al.: An algorithmic perspective on imitation learning. *Found. Trends® in Robot.* **7**(1–2), 1–179 (2018)
277. Outahar, M., Moreau, G., Normand, J.M.: Direct and indirect vslam fusion for augmented reality. *J. Imag.* **7**(8), 141 (2021)
278. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al.: Training language models to follow instructions with human feedback. *Adv. Neural. Inf. Process. Syst.* **35**, 27730–27744 (2022)
279. Padmakumar, A., Thomason, J., Shrivastava, A., Lange, P., Narayan-Chen, A., Gella, S., Piramuthu, R., Tur, G., Hakkani-Tur, D.: Teach: task-driven embodied agents that chat. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, pp. 2017–2025 (2022)
280. Paigwar, A., Sierra-Gonzalez, D., Erkent, Ö., Laugier, C.: Frustum-PointPillars: a multi-stage approach for 3D object detection using RGB camera and LiDAR. In: *ICCV*, pp. 2926–2933 (2021)
281. Peng, D., Lei, Y., Li, W., Zhang, P., Guo, Y.: Sparse-to-dense feature matching: intra and inter domain cross-modal learning in domain adaptation for 3D semantic segmentation. In: *ICCV*, pp. 7108–7117 (2021)
282. Peng, S., Genova, K., Jiang, C., Tagliasacchi, A., Pollefeys, M., Funkhouser, T., et al.: Open-scene: 3d scene understanding with open vocabularies. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 815–824 (2023)
283. Peng, Z., Wang, W., Dong, L., Hao, Y., Huang, S., Ma, S., Wei, F.: Kosmos-2: grounding multimodal large language models to the world (2023). [arXiv:2306.14824](https://arxiv.org/abs/2306.14824)
284. Pfeifer, R., Bongard, J.: *How the Body Shapes the Way We Think: A New View of Intelligence*. MIT Press (2006)
285. Po, R., Wetzstein, G.: Compositional 3D scene generation using locally conditioned diffusion (2023). [arXiv:2303.12218](https://arxiv.org/abs/2303.12218)
286. Pomerleau, D.A.: Alvin: an autonomous land vehicle in a neural network. In: *Advances in Neural Information Processing Systems*, vol. 1 (1988)
287. Poole, B., Jain, A., Barron, J.T., Mildenhall, B.: DreamFusion: text-to-3D using 2D diffusion (2022). [arXiv:2209.14988](https://arxiv.org/abs/2209.14988)
288. Pritzel, A., Uria, B., Srinivasan, S., Badia, A.P., Vinyals, O., Hassabis, D., Wierstra, D., Blundell, C.: Neural episodic control. In: *International Conference on Machine Learning*, pp. 2827–2836. PMLR (2017)
289. Puig, X., Ra, K., Boben, M., Li, J., Wang, T., Fidler, S., Torralba, A.: Virtualhome: simulating household activities via programs. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2018)

290. Qi, Y., Wu, Q., Anderson, P., Wang, X., Wang, W.Y., Shen, C., van den Hengel, A.: Reverie: Remote embodied visual referring expression in real indoor environments. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (2020). <https://doi.org/10.1109/cvpr42600.2020.01000>
291. Qiao, S., Zhang, N., Fang, R., Luo, Y., Zhou, W., Jiang, Y.E., Lv, C., Chen, H.: Autoact: automatic agent learning from scratch for qa via self-planning (2024). [arXiv:2401.05268](https://arxiv.org/abs/2401.05268)
292. Qin, J., Xia, T., Tan, C., Zhang, J., Zhang, S.Q.: Picachu: plug-in cgra handling upcoming nonlinear operations in llms. In: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 2, pp. 845–861 (2025)
293. Qin, Y., Zhou, E., Liu, Q., Yin, Z., Sheng, L., Zhang, R., Qiao, Y., Shao, J.: Mp5: a multi-modal open-ended embodied system in minecraft via active perception. In: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16307–16316. IEEE (2024)
294. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al.: Learning transferable visual models from natural language supervision. In: International Conference on Machine Learning, pp. 8748–8763. PMLR (2021)
295. Raibert, M., Blankespoor, K., Nelson, G., Playter, R.: Bigdog, the rough-terrain quadruped robot. IFAC Proc. Vols. **41**(2), 10822–10825 (2008)
296. Rakelly, K., Zhou, A., Finn, C., Levine, S., Quillen, D.: Efficient off-policy meta-reinforcement learning via probabilistic context variables. In: International Conference on Machine Learning, pp. 5331–5340. PMLR (2019)
297. Ramakrishnan, S.K., Gokaslan, A., Wijmans, E., Maksymets, O., Clegg, A., Turner, J., Under-sander, E., Galuba, W., Westbury, A., Chang, A.X., et al.: Habitat-matterport 3d dataset (hm3d): 1000 large-scale 3d environments for embodied ai (2021). [arXiv:2109.08238](https://arxiv.org/abs/2109.08238)
298. Ramani, D.: A short survey on memory based reinforcement learning (2019). [arXiv:1904.06736](https://arxiv.org/abs/1904.06736)
299. Rashid, T., Samvelyan, M., De Witt, C.S., Farquhar, G., Foerster, J., Whiteson, S.: Monotonic value function factorisation for deep multi-agent reinforcement learning. *J. Mach. Learn. Res.* **21**(178), 1–51 (2020)
300. Reddy, S., Dragan, A.D., Levine, S.: Sqil: imitation learning via reinforcement learning with sparse rewards (2019). [arXiv:1905.11108](https://arxiv.org/abs/1905.11108)
301. Reed, S., Zolna, K., Parisotto, E., Colmenarejo, S.G., Novikov, A., Barth-Maron, G., Gimenez, M., Sulsky, Y., Kay, J., Springenberg, J.T., et al.: A generalist agent (2022). [arXiv:2205.06175](https://arxiv.org/abs/2205.06175)
302. Reily, B., Gao, P., Han, F., Wang, H., Zhang, H.: Real-time recognition of team behaviors by multisensory graph-embedded robot learning. *Int. J. Robot. Res.* **41**(8), 798–811 (2022)
303. Ren, A.Z., Clark, J., Dixit, A., Itkina, M., Majumdar, A., Sadigh, D.: Explore until confident: efficient exploration for embodied question answering (2024). [arXiv:2403.15941](https://arxiv.org/abs/2403.15941)
304. Ren, L., Dong, J., Liu, S., Zhang, L., Wang, L.: Embodied intelligence toward future smart manufacturing in the era of ai foundation model. *IEEE/ASME Trans. Mechatron.* (2024)
305. Riemer, M., Cases, I., Ajemian, R., Liu, M., Rish, I., Tu, Y., Tesauro, G.: Learning to learn without forgetting by maximizing transfer and minimizing interference (2018). [arXiv:1810.11910](https://arxiv.org/abs/1810.11910)
306. Riquelme, C., Puigcerver, J., Mustafa, B., Neumann, M., Jenatton, R., Susano Pinto, A., Keysers, D., Houlsby, N.: Scaling vision with sparse mixture of experts. *Adv. Neural. Inf. Process. Syst.* **34**, 8583–8595 (2021)
307. Robine, J., Hoffmann, M., Uelwer, T., Harmeling, S.: Transformer-based world models are happy with 100k interactions. In: ICLR (2023)
308. Rocco, P.: Stability of pid control for industrial robot arms. *IEEE Trans. Robot. Autom.* **12**(4), 606–614 (2002)
309. Roh, J., Desingh, K., Farhadi, A., Fox, D.: LanguageRefer: spatial-language model for 3D visual grounding. In: CoRL, pp. 1046–1056 (2022)
310. Rohmer, E., Singh, S.P., Freese, M.: V-rep: a versatile and scalable robot simulation framework. In: IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 1321–1326 (2013)

311. Rolnick, D., Ahuja, A., Schwarz, J., Lillicrap, T., Wayne, G.: Experience replay for continual learning. In: *Advances in Neural Information Processing Systems*, vol. 32 (2019)
312. Ross, S., Gordon, G., Bagnell, D.: A reduction of imitation learning and structured prediction to no-regret online learning. In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics. JMLR Workshop and Conference Proceedings*, pp. 627–635 (2011)
313. Russo, A., Proutiere, A.: Model-free active exploration in reinforcement learning. *Adv. Neural. Inf. Process. Syst.* **36**, 54740–54753 (2023)
314. Salas-Moreno, R.F., Newcombe, R.A., Strasdat, H., Kelly, P.H., Davison, A.J.: Slam++: Simultaneous localisation and mapping at the level of objects. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1352–1359 (2013)
315. Saputra, M.R.U., Markham, A., Trigoni, N.: Visual slam and structure from motion in dynamic environments: a survey. *ACM Comput. Surv. (CSUR)* **51**(2), 1–36 (2018)
316. Savva, M., Chang, A.X., Dosovitskiy, A., Funkhouser, T., Koltun, V.: Minos: multimodal indoor simulator for navigation in complex environments (2017). [arXiv:1712.03931](https://arxiv.org/abs/1712.03931)
317. Savva, M., Kadian, A., Maksymets, O., Zhao, Y., Wijmans, E., Jain, B., Straub, J., Liu, J., Koltun, V., Malik, J., Parikh, D., Batra, D.: Habitat: A platform for embodied ai research. In: *IEEE/CVF International Conference on Computer Vision* (2019)
318. Schulman, J., Levine, S., Abbeel, P., Jordan, M., Moritz, P.: Trust region policy optimization. In: *International Conference on Machine Learning*, pp. 1889–1897. PMLR (2015)
319. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms (2017). [arXiv:1707.06347](https://arxiv.org/abs/1707.06347)
320. Schwarz, J., Czarniecki, W., Luketina, J., Grabska-Barwinska, A., Teh, Y.W., Pascanu, R., Hadsell, R.: Progress & compress: a scalable framework for continual learning. In: *International Conference on Machine Learning*, pp. 4528–4537. PMLR (2018)
321. Sermanet, P., Ding, T., Zhao, J., Xia, F., Dwibedi, D., Gopalakrishnan, K., Chan, C., Dulac-Arnold, G., Maddineni, S., Joshi, N.J., et al.: Robovqa: multimodal long-horizon reasoning for robotics. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 645–652. IEEE (2024)
322. Sermanet, P., Lynch, C., Chebotar, Y., Hsu, J., Jang, E., Schaal, S., Levine, S., Brain, G.: Time-contrastive networks: Self-supervised learning from video. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1134–1141. IEEE (2018)
323. Shah, S., Dey, D., Lovett, C., Kapoor, A.: Airsim: High-fidelity visual and physical simulation for autonomous vehicles. In: *Field and Service Robotics* (2017)
324. Shavit, Y., Agarwal, S., Brundage, M., Adler, S., O’Keefe, C., Campbell, R., Lee, T., Mishkin, P., Eloundou, T., Hickey, A., et al.: Practices for governing agentic ai systems. Research Paper, OpenAI (2023)
325. Shen, B., Xia, F., Li, C., Martín-Martín, R., Fan, L., Wang, G., Pérez-D’Arpino, C., Buch, S., Srivastava, S., Tchapmi, L., Tchapmi, M., Vainio, K., Wong, J., Fei-Fei, L., Savarese, S.: igibson 1.0: A simulation environment for interactive tasks in large realistic scenes. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 7520–7527 (2021). <https://doi.org/10.1109/IROS51168.2021.9636667>
326. Shen, Y., Liu, H., Liu, P., Xia, R., Yao, T., Sun, Y., Feng, T.: Detach: cross-domain learning for long-horizon tasks via mixture of disentangled experts (2025). [arXiv:2508.07842](https://arxiv.org/abs/2508.07842)
327. Shen, Y., Wang, L., Jin, Y.: AAFormer: a multi-modal transformer network for aerial agricultural images. In: *CVPR*, pp. 1705–1711 (2022)
328. Shi, L.X., Sharma, A., Zhao, T.Z., Finn, C.: Waypoint-based imitation learning for robotic manipulation (2023). [arXiv:2307.14326](https://arxiv.org/abs/2307.14326)
329. Shi, X., Li, D., Zhao, P., Tian, Q., Tian, Y., Long, Q., Zhu, C., Song, J., Qiao, F., Song, L., et al.: Are we ready for service robots? The openloris-scene datasets for lifelong slam. In: *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3139–3145. IEEE (2020)
330. Shigemi, S., Goswami, A., Vadakkepat, P.: Asimo and humanoid robot research at honda. *Hum. Robot. Ref.* **55**, 90 (2018)

331. Shin, H., Lee, J.K., Kim, J., Kim, J.: Continual learning with deep generative replay. In: *Advances in Neural Information Processing Systems*, vol. 30 (2017)
332. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: language agents with verbal reinforcement learning. In: *Advances in Neural Information Processing Systems* (2023)
333. Shridhar, M., Manuelli, L., Fox, D.: Cliport: What and where pathways for robotic manipulation. In: *Conference on Robot Learning*, pp. 894–906. PMLR (2022)
334. Shridhar, M., Thomason, J., Gordon, D., Bisk, Y., Han, W., Mottaghi, R., Zettlemoyer, L., Fox, D.: Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2020). <https://doi.org/10.1109/cvpr42600.2020.01075>
335. Shvetsova, N., Chen, B., Rouditchenko, A., Thomas, S., Kingsbury, B., Feris, R.S., Harwath, D., Glass, J., Kuehne, H.: Everything at once-multi-modal fusion transformer for video retrieval. In: *CVPR*, pp. 20020–20029 (2022)
336. Shyam, P., Jaśkowski, W., Gomez, F.: Model-based active exploration. In: *International Conference on Machine Learning*, pp. 5779–5788. PMLR (2019)
337. Siciliano, B., Khatib, O., Kröger, T.: *Springer Handbook of Robotics*, vol. 200. Springer (2008)
338. Sinatra, N.R., Teeple, C.B., Vogt, D.M., Parker, K.K., Gruber, D.F., Wood, R.J.: Ultragentle manipulation of delicate structures using a soft robotic gripper. *Sci. Robot.* **4**(33), eaax5425 (2019)
339. Singh, B., Kumar, R., Singh, V.P.: Reinforcement learning in robotic applications: a comprehensive survey. *Artif. Intell. Rev.* **55**(2), 945–990 (2022)
340. Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., Garg, A.: Progprompt: program generation for situated robot task planning using large language models. *Auton. Robot.* **47**(8), 999–1012 (2023)
341. Somayazulu, A., Majumder, S., Chen, C., Grauman, K.: Activerir: active audio-visual exploration for acoustic environment modeling. In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 13830–13836. IEEE (2024)
342. Son, K., Kim, D., Kang, W.J., Hostallero, D.E., Yi, Y.: Qtran: learning to factorize with transformation for cooperative multi-agent reinforcement learning. In: *International conference on machine learning*, pp. 5887–5896. PMLR (2019)
343. Song, C.H., Wu, J., Washington, C., Sadler, B.M., Chao, W.L., Su, Y.: Llm-planner: few-shot grounded planning for embodied agents with large language models. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 2998–3009 (2023)
344. Song, S., Yu, F., Zeng, A., Chang, A.X., Savva, M., Funkhouser, T.: Semantic scene completion from a single depth image. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1746–1754 (2017)
345. Srinivasan, T., Chang, T.Y., Pinto Alva, L., Chochlakis, G., Rostami, M., Thomason, J.: Climb: A continual learning benchmark for vision-and-language tasks. *Adv. Neural. Inf. Process. Syst.* **35**, 29440–29453 (2022)
346. Stocking, K.C., Gopnik, A., Tomlin, C.: From robot learning to robot understanding: leveraging causal graphical models for robotics. In: *Conference on Robot Learning*, pp. 1776–1781. PMLR (2022)
347. Sun, F., Chen, R., Ji, T., Luo, Y., Zhou, H., Liu, H.: A comprehensive survey on embodied intelligence: advancements, challenges, and future perspectives. *CAAI Artif. Intell. Res.* **3** (2024)
348. Sun, H., Zhuang, Y., Kong, L., Dai, B., Zhang, C.: Adaplaner: adaptive planning from feedback with language models. *Adv. Neural. Inf. Process. Syst.* **36**, 58202–58245 (2023)
349. Sun, Q., Zhang, X., Geng, H., Zhao, Y., Bai, Y., Zheng, H., Yu, B.: Gtuner: tuning dnn computations on gpu via graph attention network. In: *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pp. 1045–1050 (2022)
350. Suo, S., Regalado, S., Casas, S., Urtasun, R.: Trafficsim: learning to simulate realistic multi-agent behaviors. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10400–10409 (2021)

351. Sutton, R.S., McAllester, D., Singh, S., Mansour, Y.: Policy gradient methods for reinforcement learning with function approximation. In: *Advances in Neural Information Processing Systems*, vol. 12 (1999)
352. Tan, S., Ge, M., Guo, D., Liu, H., Sun, F.: Knowledge-based embodied question answering. *IEEE Trans. Pattern Anal. Mach. Intell.* (2023)
353. Tanaka, F., Isshiki, K., Takahashi, F., Uekusa, M., Sei, R., Hayashi, K.: Pepper learns together with children: development of an educational application. In: *IEEE-RAS 15th International Conference on Humanoid Robots*, pp. 270–275 (2015)
354. Team, G., Anil, R., Borgeaud, S., Alayrac, J.B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A.M., Hauth, A., Millican, K., et al.: Gemini: a family of highly capable multimodal models (2023). [arXiv:2312.11805](https://arxiv.org/abs/2312.11805)
355. Team, O.M., Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., et al.: Octo: an open-source generalist robot policy (2023). [arXiv:2405.12213](https://arxiv.org/abs/2405.12213)
356. Thomason, J., Murray, M., Cakmak, M., Zettlemoyer, L.: Vision-and-dialog navigation. In: *Conference on Robot Learning (CoRL)* (2019). <https://arxiv.org/abs/1907.04957>
357. Thomason, J., Murray, M., Cakmak, M., Zettlemoyer, L.: Vision-and-dialog navigation. In: *Conference on Robot Learning*, pp. 394–406. PMLR (2020)
358. Thomason, J., Shridhar, M., Bisk, Y., Paxton, C., Zettlemoyer, L.: Language grounding with 3d objects. In: *Conference on Robot Learning*, pp. 1691–1701. PMLR (2022)
359. Tian, Y., Lin, F., Li, Y., Zhang, T., Zhang, Q., Fu, X., Huang, J., Dai, X., Wang, Y., Tian, C., Li, B., Lv, Y., Kovács, L., Wang, F.Y.: Uavs meet llms: overviews and perspectives towards agentic low-altitude mobility. *Inf. Fusion* **122**, 103158 (2025). <https://doi.org/10.1016/j.inffus.2025.103158>
360. Tian, Y., Lin, F., Zhang, X., Ge, J., Wang, Y., Dai, X., Lv, Y., Wang, F.Y.: Logisticsvista: 3d terminal delivery services with uavs, ugvs and usvs based on foundation models and scenarios engineering. In: *2024 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI)*. IEEE (2024)
361. Tian, Y., Zhang, Y., Wang, F.Y.: *Algorithmic Foundations of Large Models: Principles and Applications of Transformers*. Tsinghua University Press, Beijing (2025)
362. Todorov, E., Erez, T., Tassa, Y.: Mujoco: a physics engine for model-based control. 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 5026–5033 (2012)
363. Tong, Y., Liu, H., Zhang, Z.: Advancements in humanoid robots: a comprehensive review and future prospects. *IEEE/CAA J. Autom. Sinica* **11**(2), 301–328 (2024)
364. Tung, H.Y., Ding, M., Chen, Z., Bear, D., Gan, C., Tenenbaum, J., Yamins, D., Fan, J., Smith, K.: Physion++: Evaluating physical scene understanding that requires online inference of different physical properties. *Adv. Neural. Inf. Process. Syst.* **36**, 67048–67068 (2023)
365. Turing, A.M.: Computing machinery and intelligence. *Mind* **59**(236), 433–460 (1950)
366. Tzafas, G., Xu, Y., Goel, A., Kasaei, M., Li, Z., Kasaei, H.: Language-guided robot grasping: clip-based referring grasp synthesis in clutter (2023). [arXiv:2311.05779](https://arxiv.org/abs/2311.05779)
367. Ugenti, A., Galati, R., Mantriota, G., Reina, G.: Analysis of an all-terrain tracked robot with innovative suspension system. *Mech. Mach. Theory* **182**, 105237 (2023)
368. Van Hasselt, H., Guez, A., Silver, D.: Deep reinforcement learning with double q-learning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30 (2016)
369. Varley, J., DeChant, C., Richardson, A., Ruales, J., Allen, P.: Shape completion enabled robotic grasping. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 2442–2447 (2017)
370. Vaskov, A.K., Vu, P.P., North, N., Davis, A.J., Kung, T.A., Gates, D.H., Cederna, P.S., Chestek, C.A.: Surgically implanted electrodes enable real-time finger and grasp pattern recognition for prosthetic hands. *IEEE Trans. Rob.* **38**(5), 2841–2857 (2022)
371. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *NeurIPS* (2017)
372. Veniat, T., Denoyer, L., Ranzato, M.: Efficient continual learning with modular networks and task-driven priors (2020). [arXiv:2012.12631](https://arxiv.org/abs/2012.12631)

373. Vezhnevets, A.S., Osindero, S., Schaul, T., Heess, N., Jaderberg, M., Silver, D., Kavukcuoglu, K.: Feudal networks for hierarchical reinforcement learning. In: International Conference on Machine Learning, pp. 3540–3549. PMLR (2017)
374. Vinyals, O., Toshev, A., Bengio, S., Erhan, D.: Show and tell: a neural image caption generator. In: CVPR, pp. 3156–3164 (2015)
375. Von Oswald, J., Henning, C., Grewe, B.F., Sacramento, J.: Continual learning with hypernetworks (2019). [arXiv:1906.00695](https://arxiv.org/abs/1906.00695)
376. Vora, S., Lang, A.H., Helou, B., Beijbom, O.: PointPainting: sequential fusion for 3D object detection. In: CVPR, pp. 4604–4612 (2020)
377. Wadhawan, R., Bansal, H., Chang, K.W., Peng, N.: Contextual: evaluating context-sensitive text-rich visual reasoning in large multimodal models (2024). [arXiv:2401.13311](https://arxiv.org/abs/2401.13311)
378. Wang, C., Ma, C., Zhu, M., Yang, X.: PointAugmenting: cross-modal augmentation for 3D object detection. In: CVPR, pp. 11794–11803 (2021)
379. Wang, H., Chen, A.G.H., Li, X., Wu, M., Dong, H.: Find what you want: learning demand-conditioned object attribute space for demand-driven navigation. In: Advances in Neural Information Processing Systems (2023)
380. Wang, H., Zhang, C., Yu, J., Cai, W.: Spatiality-guided transformer for 3D dense captioning on point clouds (2022). [arXiv:2204.10688](https://arxiv.org/abs/2204.10688)
381. Wang, H., Zhang, Z., Han, S.: Spatten: efficient sparse attention architecture with cascade token and head pruning. In: 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), pp. 97–110. IEEE (2021)
382. Wang, J., Ren, Z., Liu, T., Yu, Y., Zhang, C.: Qplex: duplex dueling multi-agent q-learning (2020). [arXiv:2008.01062](https://arxiv.org/abs/2008.01062)
383. Wang, K., Liu, Z., Lin, Y., Lin, J., Han, S.: Haq: hardware-aware automated quantization with mixed precision. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 8612–8620 (2019)
384. Wang, L., Chen, X., Zhao, J., He, K.: Scaling proprioceptive-visual learning with heterogeneous pre-trained transformers. *Adv. Neural. Inf. Process. Syst.* **37**, 124420–124450 (2024)
385. Wang, M., Meng, Y., Tang, C., Zhang, W., Qin, Y., Yao, Y., Li, Y., Feng, T., Wang, X., Guan, X., et al.: Jq: joint efficient architecture design and low-bit quantization with hardware-software co-exploration (2025). [arXiv:2501.05339](https://arxiv.org/abs/2501.05339)
386. Wang, M., Zhao, L., Yue, Y.: PA3DNet: 3D vehicle detection with pseudo shape segmentation and adaptive camera-LiDAR fusion. *TII* (2023)
387. Wang, S., Han, M., Jiao, Z., Zhang, Z., Wu, Y.N., Zhu, S.C., Liu, H.: Llm³: large language model-based task and motion planning with motion failure reasoning. In: 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 12086–12092. IEEE (2024)
388. Wang, S., Han, M., Jiao, Z., Zhang, Z., Wu, Y.N., Zhu, S.C., Liu, H.: Llm³: large language model-based task and motion planning with motion failure reasoning. In: 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (2024)
389. Wang, S., Yu, Z., Jiang, X., Lan, S., Shi, M., Chang, N., Kautz, J., Li, Y., Alvarez, J.M.: Omnidrive: A holistic llm-agent framework for autonomous driving with 3d perception, reasoning and planning (2024). [arXiv:2405.01533](https://arxiv.org/abs/2405.01533)
390. Wang, T., Mao, X., Zhu, C., Xu, R., Lyu, R., Li, P., Chen, X., Zhang, W., Chen, K., Xue, T., et al.: Embodiedscan: a holistic multi-modal 3d perception suite towards embodied ai. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (2024)
391. Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., Ji, H.: Executable code actions elicit better llm agents. In: Forty-first International Conference on Machine Learning (2024)
392. Wang, X., Yang, D., Wang, Z., Kwan, H., Chen, J., Wu, W., Li, H., Liao, Y., Liu, S.: Towards realistic uav vision-language navigation: Platform, benchmark, and methodology (2024). [arXiv:2410.07087](https://arxiv.org/abs/2410.07087)
393. Wang, X., Zhu, Z., Huang, G., Chen, X., Zhu, J., Lu, J.: Drivedreamer: towards real-world-drive world models for autonomous driving. In: European Conference on Computer Vision, pp. 55–72. Springer (2024)

394. Wang, X., Zhu, Z., Huang, G., Wang, B., Chen, X., Lu, J.: Worlddreamer: towards general world models for video generation via predicting masked tokens (2024). [arXiv:2401.09985](#)
395. Wang, Y., Tian, Y., Chen, J., Xu, K., Ding, X.: A survey of visual slam in dynamic environment: the evolution from geometric to semantic approaches. *IEEE Transactions on Instrumentation and Measurement* (2024)
396. Wang, Y., Xian, Z., Chen, F., Wang, T.H., Wang, Y., Fragkiadaki, K., Erickson, Z., Held, D., Gan, C.: Robogen: Towards unleashing infinite data for automated robot learning via generative simulation (2023). [arXiv:2311.01455](#)
397. Wang, Z., Chen, C., Luo, F., Dong, Y., Zhang, Y., Xu, Y., Wang, X., Li, P., Liu, Y.: Actiview: evaluating active perception ability for multimodal large language models (2024). [arXiv:2410.04659](#)
398. Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., Freitas, N.: Dueling network architectures for deep reinforcement learning. In: *International Conference on Machine Learning*, pp. 1995–2003. PMLR (2016)
399. Wei, S., Ye, T., Zhang, S., Tang, Y., Liang, J.: Joint token pruning and squeezing towards more aggressive compression of vision transformers. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2092–2101 (2023)
400. Wen, M., Kuba, J., Lin, R., Zhang, W., Wen, Y., Wang, J., Yang, Y.: Multi-agent reinforcement learning is a sequence modeling problem. *Adv. Neural. Inf. Process. Syst.* **35**, 16509–16521 (2022)
401. Wiggers, K.: Openai debuts gpt-4o’omni’model now powering chatgpt. *TechCrunch*. Retrieved May 16, 2024 (2024)
402. Wijmans, E., Datta, S., Maksymets, O., Das, A., Gkioxari, G., Lee, S., Essa, I., Parikh, D., Batra, D.: Embodied question answering in photorealistic environments with point cloud perception. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6659–6668 (2019)
403. Williams, R.J.: Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.* **8**(3), 229–256 (1992)
404. Wu, D., Wei, X., Chen, G., Shen, H., Wang, X., Li, W., Jin, B.: Generative multi-agent collaboration in embodied ai: a systematic review (2025). [arXiv:2502.11518](#)
405. Wu, P., Escontrela, A., Hafner, D., Abbeel, P., Goldberg, K.: Daydreamer: world models for physical robot learning. In: *Conference on Robot Learning*, pp. 2226–2240. PMLR (2023)
406. Wu, Q., Shen, C., Liu, L., Dick, A., Van Den Hengel, A.: What value do explicit high level concepts have in vision to language problems? In: *CVPR*, pp. 203–212 (2016)
407. Wu, W., Li, Z., He, Y., Shou, M.Z., Shen, C., Cheng, L., Li, Y., Gao, T., Zhang, D., Wang, Z.: Paragraph-to-image generation with information-enriched diffusion model (2023). [arXiv:2311.14284](#)
408. Wu, X., Peng, L., Yang, H., Xie, L., Huang, C., Deng, C., Liu, H., Cai, D.: Sparse fuse dense: towards high quality 3D detection with depth completion. In: *CVPR*, pp. 5418–5427 (2022)
409. Wu, Y., Cheng, X., Zhang, R., Cheng, Z., Zhang, J.: EDA: Explicit text-decoupling and dense alignment for 3D visual grounding. In: *CVPR*, pp. 19231–19242 (2023)
410. Wu, Y., Zhang, P., Gu, M., Zheng, J., Bai, X.: Embodied navigation with multi-modal information: a survey from tasks to methodology. *Inf. Fusion* 102532 (2024)
411. Wu, Z., Wang, Z., Xu, X., Lu, J., Yan, H.: Embodied task planning with large language models (2023). [arXiv:2307.01848](#)
412. Wurman, P.R., D’Andrea, R., Mountz, M.: Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI Mag.* **29**(1), 9–9 (2008)
413. Xiang, F., Qin, Y., Mo, K., Xia, Y., Zhu, H., Liu, F., Liu, M., Jiang, H., Yuan, Y., Wang, H., Yi, L., Chang, A.X., Guibas, L.J., Su, H.: Sapien: a simulated part-based interactive environment. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020)
414. Xiang, J., Tao, T., Gu, Y., Shu, T., Wang, Z., Yang, Z., Hu, Z.: Language models meet world models: embodied experiences enhance language models. *Adv. Neural. Inf. Process. Syst.* **36**, 75392–75412 (2023)

415. Xiang, J., Tao, T., Gu, Y., Shu, T., Wang, Z., Yang, Z., Hu, Z.: Language models meet world models: embodied experiences enhance language models. In: *Advances in Neural Information Processing Systems*, vol. 36 (2024)
416. Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., Han, S.: Smoothquant: accurate and efficient post-training quantization for large language models. In: *International Conference on Machine Learning*, pp. 38087–38099. PMLR (2023)
417. Xie, Y., Xu, C., Rakotosaona, M.J., Rim, P., Tombari, F., Keutzer, K., Tomizuka, M., Zhan, W.: SparseFusion: fusing multi-modal sparse representations for multi-sensor 3D object detection (2023). [arXiv:2304.14340](#)
418. Xu, J., Guo, Z., He, J., Hu, H., He, T., Bai, S., Chen, K., Wang, J., Fan, Y., Dang, K., et al.: Qwen2. 5-omni technical report (2025). [arXiv:2503.20215](#)
419. Xu, J., Jin, S., Lei, Y., Zhang, Y., Zhang, L.: Reasoning tuning grasp: adapting multi-modal large language models for robotic grasping. In: *2nd Workshop on Language and Robot Learning: Language as Grounding* (2023)
420. Xu, J., Zhang, R., Dou, J., Zhu, Y., Sun, J., Pu, S.: RPNNet: a deep and efficient range-point-voxel fusion network for LiDAR point cloud segmentation. In: *ICCV*, pp. 16024–16033 (2021)
421. Xu, M., Dai, W., Liu, C., Gao, X., Lin, W., Qi, G.J., Xiong, H.: Spatial-temporal transformer networks for traffic flow forecasting (2020). [arXiv:2001.02908](#)
422. Xu, M., Jiang, G., Liang, W., Zhang, C., Zhu, Y.: Active reasoning in an open-world environment. *Adv. Neural. Inf. Process. Syst.* **36**, 11716–11736 (2023)
423. Xu, Z., Wu, K., Wen, J., Li, J., Liu, N., Che, Z., Tang, J.: A survey on robotics with foundation models: toward embodied ai (2024). [arXiv:2402.02385](#)
424. Yamauchi, B.M.: Packbot: a versatile platform for military robotics. In: *Unmanned ground vehicle technology VI*, vol. 5422, pp. 228–237. SPIE (2004)
425. Yan, C., Qu, D., Xu, D., Zhao, B., Wang, Z., Wang, D., Li, X.: Gs-slam: dense visual slam with 3d gaussian splatting. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 19595–19604 (2024)
426. Yan, J., Liu, Y., Sun, J., Jia, F., Li, S., Wang, T., Zhang, X.: Cross modal transformer: towards fast and robust 3D object detection. In: *ICCV*, pp. 18268–18278 (2023)
427. Yan, X., Gao, J., Zheng, C., Zheng, C., Zhang, R., Cui, S., Li, Z.: 2DPASS: 2D priors assisted semantic segmentation on LiDAR point clouds. In: *ECCV*, pp. 677–695 (2022)
428. Yang, J., Ding, R., Deng, W., Wang, Z., Qi, X.: Regionplc: regional point-language contrastive learning for open-world 3d scene understanding. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 19823–19832 (2024)
429. Yang, J., Ding, R., Wang, Z., Qi, X.: RegionPLC: regional point-language contrastive learning for open-world 3D scene understanding (2023). [arXiv:2304.00962](#)
430. Yang, J., Tan, R., Wu, Q., Zheng, R., Peng, B., Liang, Y., Gu, Y., Cai, M., Ye, S., Jang, J., et al.: Magma: a foundation model for multimodal ai agents (2025). [arXiv:2502.13130](#)
431. Yang, L., Yu, Z., Meng, C., Xu, M., Ermon, S., Cui, B.: Mastering text-to-image diffusion: recaptioning, planning, and generating with multimodal llms. In: *Forty-First International Conference on Machine Learning* (2024)
432. Yang, R., Chen, H., Zhang, J., Zhao, M., Qian, C., Wang, K., Wang, Q., Koripella, T.V., Movahedi, M., Li, M., et al.: Embodiedbench: comprehensive benchmarking multi-modal large language models for vision-driven embodied agents (2025). [arXiv:2502.09560](#)
433. Yang, R., Srivastava, P., Mandt, S.: Diffusion probabilistic modeling for video generation. *Entropy* **25**(10), 1469 (2023)
434. Yang, Y., He, Z., Jiao, P., Ren, H.: Bioinspired soft robotics: How do we learn from creatures? *IEEE Rev. Biomed. Eng.* (2022)
435. Yang, Y., Jia, B., Zhi, P., Huang, S.: Physcene: physically interactable 3d scene synthesis for embodied ai. In: *Proceedings of Conference on Computer Vision and Pattern Recognition (CVPR)* (2024)
436. Yang, Y., Sun, F.Y., Weihs, L., VanderBilt, E., Herrasti, A., Han, W., Wu, J., Haber, N., Krishna, R., Liu, L., Callison-Burch, C., Yatskar, M., Kembhavi, A., Clark, C.: Holodeck: language guided generation of 3d embodied ai environments. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024)

437. Yang, Z., Liu, J., Chen, P., Cherian, A., Marks, T.K., Le Roux, J., Gan, C.: Rila: reflective and imaginative language agent for zero-shot semantic audio-visual navigation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 16251–16261 (2024)
438. Yang, Z., Zhang, S., Wang, L., Luo, J.: SAT: 2D semantics assisted training for 3D visual grounding. In: ICCV, pp. 1856–1866 (2021)
439. Yao, L., Huang, R., Hou, L., Lu, G., Niu, M., Xu, H., Liang, X., Li, Z., Jiang, X., Xu, C.: Filip: fine-grained interactive language-image pre-training (2021). [arXiv:2111.07783](https://arxiv.org/abs/2111.07783)
440. Yarats, D., Kostrikov, I., Fergus, R.: Image augmentation is all you need: regularizing deep reinforcement learning from pixels. In: International Conference on Learning Representations (2021)
441. Yenamandra, S., Ramachandran, A., Yadav, K., Wang, A., Khanna, M., Gervet, T., Yang, T.Y., Jain, V., Clegg, A., Turner, J., Kira, Z., Savva, M., Chang, A., Chaplot, D., Batra, D., Mottaghi, R., Bisk, Y., Paxton, C.: Homerobot: Open-vocabulary mobile manipulation. In: NeurIPS (2023)
442. Yin, T., Zhou, X., Krähenbühl, P.: Multimodal virtual point 3D detection. In: NeurIPS, vol. 34, pp. 16494–16507 (2021)
443. Yoo, J.H., Kim, Y., Kim, J., Choi, J.W.: 3D-CVF: Generating joint camera and LiDAR features using cross-view spatial feature fusion for 3D object detection. In: ECCV, pp. 720–736 (2020)
444. Yoon, J., Yang, E., Lee, J., Hwang, S.J.: Lifelong learning with dynamically expandable networks (2017). [arXiv:1708.01547](https://arxiv.org/abs/1708.01547)
445. Yu, C., Liu, Z., Liu, X.J., Xie, F., Yang, Y., Wei, Q., Fei, Q.: Ds-slam: a semantic visual slam towards dynamic environments. In: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 1168–1174. IEEE (2018)
446. Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A., Wu, Y.: The surprising effectiveness of ppo in cooperative multi-agent games. *Adv. Neural. Inf. Process. Syst.* **35**, 24611–24624 (2022)
447. Yu, J., Zhuge, Y., Zhang, L., Hu, P., Wang, D., Lu, H., He, Y.: Boosting continual learning of vision-language models via mixture-of-experts adapters. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 23219–23230 (2024)
448. Yu, L., Chen, X., Gkioxari, G., Bansal, M., Berg, T.L., Batra, D.: Multi-target embodied question answering. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 6309–6318 (2019)
449. Yu, T., Finn, C., Xie, A., Dasari, S., Zhang, T., Abbeel, P., Levine, S.: One-shot imitation from observing humans via domain-adaptive meta-learning (2018). [arXiv:1802.01557](https://arxiv.org/abs/1802.01557)
450. Yuan, P., Ma, A., Yao, Y., Yao, H., Tomizuka, M., Ding, M.: Remac: self-reflective and self-evolving multi-agent collaboration for long-horizon robot manipulation (2025). [arXiv:2503.22122](https://arxiv.org/abs/2503.22122)
451. Yuan, Z., Yan, X., Liao, Y., Guo, Y., Li, G., Cui, S., Li, Z.: X-Trans2Cap: cross-modal knowledge transfer using transformer for 3D dense captioning. In: CVPR, pp. 8563–8573 (2022)
452. Yuan, Z., Yan, X., Liao, Y., Zhang, R., Wang, S., Li, Z., Cui, S.: InstanceRefer: cooperative holistic understanding for visual grounding on point clouds through instance multi-level contextual referring. In: CVPR, pp. 1791–1800 (2021)
453. Yue, J., Xu, X., Karlsson, B.F., Lu, Z.: Mllm as retriever: interactively learning multimodal retrieval for embodied agents (2024). [arXiv:2410.03450](https://arxiv.org/abs/2410.03450)
454. Zaffar, M., Ehsan, S., Stolkin, R., Maier, K.M.: Sensors, slam and long-term autonomy: a review. In: 2018 NASA/ESA Conference on Adaptive Hardware and Systems (AHS), pp. 285–290. IEEE (2018)
455. de Zarza, I., de Curto, J., Calafate, C.T.: Socratic video understanding on unmanned aerial vehicles. *Proc. Comput. Sci.* **225**, 144–154 (2023)
456. Zenke, F., Poole, B., Ganguli, S.: Continual learning through synaptic intelligence. In: International Conference on Machine Learning, pp. 3987–3995. PMLR (2017)
457. Zeno, C., Golan, I., Hoffer, E., Soudry, D.: Task-agnostic continual learning using online variational bayes with fixed-point updates. *Neural Comput.* **33**(11), 3139–3177 (2021)

458. Zhang, H., Du, W., Shan, J., Zhou, Q., Du, Y., Tenenbaum, J.B., Shu, T., Gan, C.: Building cooperative embodied agents modularly with large language models (2023). [arXiv:2307.02485](https://arxiv.org/abs/2307.02485)
459. Zhang, H., Wang, Z., Lyu, Q., Zhang, Z., Chen, S., Shu, T., Dariush, B., Lee, K., Du, Y., Gan, C.: Combo: compositional world models for embodied multi-agent cooperation (2024). [arXiv:2404.10775](https://arxiv.org/abs/2404.10775)
460. Zhang, H., Yang, D., Yurtsever, E., Redmill, K.A., Özgüner, Ü.: Faraway-Frustum: dealing with LiDAR sparsity for 3D object detection using fusion. In: ITSC, pp. 2646–2652 (2021)
461. Zhang, J., Dong, R., Ma, K.: CLIP-FO3D: learning free open-world 3D scene representations from 2D dense CLIP (2023). [arXiv:2303.04748](https://arxiv.org/abs/2303.04748)
462. Zhang, J., Tang, L., Song, Y., Meng, Q., Qian, H., Shao, J., Song, W., Zhu, S., Gu, J.: Fltrnn: faithful long-horizon task planning for robotics with large language models. In: 2024 IEEE International Conference on Robotics and Automation (ICRA), pp. 6680–6686. IEEE (2024)
463. Zhang, L., Yang, G., Stadie, B.C.: World model as a graph: learning latent landmarks for planning. In: International Conference on Machine Learning, pp. 12611–12620. PMLR (2021)
464. Zhang, M., Fu, X., Hao, J., Han, P., Zhang, H., Shi, L., Tang, H., Zheng, Y.: Mfe-etp: a comprehensive evaluation benchmark for multi-modal foundation models on embodied task planning (2024). [arXiv:2407.05047](https://arxiv.org/abs/2407.05047)
465. Zhang, Q., Gao, Y., Zhang, Y., Guo, Y., Ding, D., Wang, Y., Sun, P., Zhao, D.: Trajgen: generating realistic and diverse trajectories with reactive and feasible agent behaviors for autonomous driving. *IEEE Trans. Intell. Transp. Syst.* **23**(12), 24474–24487 (2022)
466. Zhang, X., Qin, H., Wang, F., Dong, Y., Li, J.: Lamma-p: generalizable multi-agent long-horizon task allocation and planning with lm-driven pddl planner (2024). [arXiv:2409.20560](https://arxiv.org/abs/2409.20560)
467. Zhang, Y., Chen, J., Huang, D.: CAT-Det: contrastively augmented transformer for multi-modal 3D object detection. In: CVPR, pp. 908–917 (2022)
468. Zhang, Z., Bo, X., Ma, C., Li, R., Chen, X., Dai, Q., Zhu, J., Dong, Z., Wen, J.R.: A survey on the memory mechanism of large language model based agents (2024). [arXiv:2404.13501](https://arxiv.org/abs/2404.13501)
469. Zhao, L., Cai, D., Sheng, L., Xu, D.: 3dvg-transformer: relation modeling for visual grounding on point clouds. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 2928–2937 (2021)
470. Zhao, L., Zhou, H., Zhu, X., Song, X., Li, H., Tao, W.: LIF-Seg: LiDAR and camera image fusion for 3D lidar semantic segmentation. *IEEE TMM* (2023)
471. Zhao, Q., Lu, Y., Kim, M.J., Fu, Z., Zhang, Z., Wu, Y., Li, Z., Ma, Q., Han, S., Finn, C., et al.: Cot-vla: visual chain-of-thought reasoning for vision-language-action models (2025). [arXiv:2503.22020](https://arxiv.org/abs/2503.22020)
472. Zhao, W., Ding, P., Zhang, M., Gong, Z., Bai, S., Zhao, H., Wang, D.: Vlas: vision-language-action model with speech instructions for customized robot manipulation (2025). [arXiv:2502.13508](https://arxiv.org/abs/2502.13508)
473. Zhao, Y., Xu, K., Zhu, Z., Hu, Y., Zheng, Z., Chen, Y., Ji, Y., Gao, C., Li, Y., Huang, J.: Cityeqa: a hierarchical llm agent on embodied question answering benchmark in city space (2025). <https://arxiv.org/abs/2502.12532>
474. Zheng, W., Wang, F.Y.: Computational Knowledge Vision: The First Footprints. Elsevier (2024)
475. Zhou, H., Shao, J., Xu, L., Bai, D., Qiu, W., Liu, B., Wang, Y., Geiger, A., Liao, Y.: Hugs: Holistic urban 3d scene understanding via gaussian splatting. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 21336–21345 (2024)
476. Zhou, Q., Wang, Z., Chu, W., Xu, Y., Li, H., Qi, Y.: Infmlm: a unified framework for visual-language tasks (2023). [arXiv:2311.06791](https://arxiv.org/abs/2311.06791)
477. Zhou, X., Wang, Z., Ye, H., Xu, C., Gao, F.: Ego-planner: an esdf-free gradient-based local planner for quadrotors. *IEEE Robot. Autom. Lett.* **6**(2), 478–485 (2020)
478. Zhou, X., Wen, X., Wang, Z., Gao, Y., Li, H., Wang, Q., Yang, T., Lu, H., Cao, Y., Xu, C., et al.: Swarm of micro flying robots in the wild. *Sci. Robot.* **7**(66), eabm5954 (2022)
479. Zhou, Y., Tuzel, O.: VoxelNet: End-to-end learning for point cloud based 3D object detection. In: CVPR, pp. 4490–4499 (2018)

480. Zhou, Z., Zhu, Y., Wen, J., Shen, C., Xu, Y.: Vision-language-action model with open-world embodied reasoning from pretrained knowledge (2025). [arXiv:2505.21906](https://arxiv.org/abs/2505.21906)
481. Zhu, D., Chen, J., Shen, X., Li, X., Elhoseiny, M.: Minigpt-4: enhancing vision-language understanding with advanced large language models (2023). [arXiv:2304.10592](https://arxiv.org/abs/2304.10592)
482. Zhu, F., Liang, X., Zhu, Y., Yu, Q., Chang, X., Liang, X.: Soon: scenario oriented object navigation with graph-based exploration. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (2021). <https://doi.org/10.1109/cvpr46437.2021.01250>
483. Zhu, G., Lin, Z., Yang, G., Zhang, C.: Episodic reinforcement learning with associative memory. In: International Conference on Learning Representations (2020)
484. Zhu, X., Zhu, J., Li, H., Wu, X., Li, H., Wang, X., Dai, J.: Uni-Perceiver: pre-training unified architecture for generic perception for zero-shot and few-shot tasks. In: CVPR, pp. 16804–16815 (2022)
485. Zhu, Z., Liu, M., Mao, L., Kang, B., Xu, M., Yu, Y., Ermon, S., Zhang, W.: Madiff: offline multi-agent learning with diffusion models. *Adv. Neural. Inf. Process. Syst.* **37**, 4177–4206 (2024)
486. Zhu, Z., Ma, X., Chen, Y., Deng, Z., Huang, S., Li, Q.: 3D-VisTA: pre-trained transformer for 3D vision and text alignment. In: ICCV, pp. 2911–2921 (2023)
487. Zhu, Z., Wang, X., Zhao, W., Min, C., Deng, N., Dou, M., Wang, Y., Shi, B., Wang, K., Zhang, C., et al.: Is sora a world simulator? A comprehensive survey on general world models and beyond (2024). [arXiv:2405.03520](https://arxiv.org/abs/2405.03520)
488. Zhuang, Z., Li, R., Jia, K., Wang, Q., Li, Y., Tan, M.: Perception-aware multi-sensor fusion for 3D LiDAR semantic segmentation. In: ICCV, pp. 16280–16290 (2021)
489. Ziebart, B.D., Maas, A.L., Bagnell, J.A., Dey, A.K., et al.: Maximum entropy inverse reinforcement learning. In: *Aaai*, vol. 8, pp. 1433–1438. Chicago, IL, USA (2008)
490. Ziemke, T.: What’s that thing called embodiment? In: *Proceedings of the 25th Annual Cognitive Science Society*, pp. 1305–1310. Psychology Press (2013)
491. Zintgraf, L., Shiarlis, K., Igl, M., Schulze, S., Gal, Y., Hofmann, K., Whiteson, S.: Varibad: A very good method for bayes-adaptive deep rl via meta-learning (2019). [arXiv:1910.08348](https://arxiv.org/abs/1910.08348)
492. Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohlhart, P., Welker, S., Wahid, A., et al.: Rt-2: Vision-language-action models transfer web knowledge to robotic control. In: *Conference on Robot Learning*, pp. 2165–2183. PMLR (2023)
493. Zou, D., Tan, P.: Coslam: collaborative visual slam in dynamic environments. *IEEE Trans. Pattern Anal. Mach. Intell.* **35**(2), 354–366 (2012)
494. Zou, L., Zhao, W., Yin, S., Bai, C., Sun, Q., Yu, B.: Bie: bi-exponent block floating-point for large language models quantization. In: *Forty-First International Conference on Machine Learning* (2024)

Chapter 2

Autonomous Embodied AI



Abstract Embodied agents operating in complex, open-ended environments require autonomy beyond predefined tasks and human guidance. Traditional systems, though effective under controlled conditions, often fail when facing uncertainty, ambiguity, dynamic environments, or evolving goals. To achieve autonomous learning like a baby, autonomous embodied Artificial Intelligence (AI) integrates three core capacities: Autonomous Environment Learning, which enables agents to autonomously explore, perceive, and interpret the dynamic and open environment; Autonomous Cognitive Learning, which focuses on the autonomous development of reasoning and decision-making skills; and Autonomous Dynamic Interaction, which facilitates agents grounded in environmental learning and cognitive models to engage in autonomous dynamic interaction with entities (who/what) and at the right time (when). These capacities are not independent modules but interconnected processes forming a continuous loop of perception, cognition, and interaction. Together, they allow agents to reduce uncertainty, expand action possibilities, and generate meaningful behavior without external instruction. This framework provides a foundation for advancing embodied AI in domains such as robotics, autonomous driving, and scientific discovery, where long-term adaptability and independent decision-making are essential.

2.1 The Overview

As embodied agents are increasingly deployed in complex, open-ended environments, the demand for autonomy has become critical. Traditional systems often rely on externally defined tasks, handcrafted policies, and human-curated learning signals. While effective in controlled settings, such systems falter when confronted with unfamiliar situations, ambiguous feedback, or evolving goals. In practical domains such as domestic robotics, autonomous driving, and scientific discovery, agents must perceive, interact with, and act upon their environments without constant human oversight. This necessity underscores the importance of embodied intelligence systems capable of adapting, deciding, and learning independently.

Autonomous embodied AI rests on three fundamental capacities: Autonomous Environment Learning, Autonomous Cognitive Learning, and Autonomous Dynamic Interaction. Together, these capacities enable agents to learn and act meaningfully in the real world without external instruction.

- **Autonomous Environment Learning** is the capacity to determine what, when, and how to perceive based on internal goals and contextual uncertainty. Unlike passive sensing, it allows the agent to direct attention, modulate sensors, and selectively acquire information most relevant to its state or task. Through this, the agent reduces uncertainty, resolves ambiguity, and gathers evidence in a goal-driven manner.
- **Autonomous Cognitive Learning** concerns how the agent engages with its environment to influence both physical and informational states. This includes strategic exploration, tool use, communication with humans or other agents, and environmental manipulation. Instead of relying on pre-specified rules, the agent learns when and how to interact in ways that enhance understanding and expand its repertoire of possible actions.
- **Autonomous Dynamic Interaction** involves selecting and executing actions based on internal objectives, learned models, and environmental dynamics. This enables behavior that is not only reactive but also anticipatory and intentional. By integrating perception, memory, and interaction history, the agent generates coherent, context-sensitive plans and actions.

These three capacities are deeply interconnected rather than isolated. Active perception shapes what information is acquired, guiding subsequent interactions; interaction transforms both the environment and the agent's internal state, influencing future decisions; and action both expresses prior learning and generates new feedback. Together, they form a continuous loop of perception, interaction, and adaptation that underpins embodied autonomy.

The remainder of this chapter reviews the foundations of autonomy in human and machine learning, highlighting mechanisms of goal setting, self-regulation, and adaptive evaluation. Building on this foundation, it introduces the core components of autonomous embodied intelligence in detail, examining their internal mechanisms, interdependencies, and roles in supporting long-term adaptability [30].

2.2 Key Technologies of Autonomous Embodied AI

Human autonomous learning involves setting goals, selecting strategies, regulating behavior, and reflecting on outcomes. It is a dynamic, context-sensitive process in which learners proactively plan, execute, and evaluate their learning, adjusting strategies based on feedback and experience. This cyclical process enhances problem-solving, lifelong learning, and adaptability in complex environments. These insights inform autonomous machine learning, where artificial agents replicate self-directed

learning. Modern frameworks implement interdependent modules for task selection, model choice, data prioritization, optimization, and performance evaluation. Task selection establishes goals, model and data management guide effective learning execution, and evaluation supports adaptive refinement. By emulating human autonomy, autonomous systems achieve iterative, goal-driven, and self-correcting learning, enabling efficient knowledge acquisition, robust adaptation, and effective problem-solving in dynamic, open-ended domains.

2.2.1 Autonomous Learning in Humans

The study of autonomous learning in humans has long been a cornerstone of research in cognitive psychology, educational theory, and developmental research. Rather than relying solely on external instruction, autonomous learners are able to set their own goals, select appropriate strategies, regulate their behavior, and reflect on outcomes. This capacity is fundamental not only to effective learning within structured settings, but also to adaptive problem-solving, lifelong learning, and creative exploration in open-ended environments [32, 106].

In the field of adult education, Knowles [62] was among the earliest scholars to conceptualize self-directed learning. He emphasized that mature learners are capable of independently identifying their learning needs, planning strategies to address them, and critically evaluating progress. This perspective, which highlights the learner's active agency, has since exerted wide-reaching influence across both formal education and informal lifelong learning contexts [17]. Cognitive developmental theory likewise provided essential foundations for understanding the emergence of autonomy in learning. Piaget [95] posited that individuals construct knowledge through active interaction with their environment, continuously reshaping cognitive structures via processes of assimilation and accommodation. In contrast, Vygotsky [1] emphasized the sociocultural dimensions of learning, arguing that autonomy develops through guided participation and gradually transitions toward independent self-regulation within the learner's zone of proximal development.

The field of metacognition has further advanced this understanding by revealing that autonomy in learning is not merely behavioral, but also fundamentally reflective. Flavell [32] introduced the notion of metacognition as the capacity to monitor and control one's own thinking processes. Building on this, Schraw and Moshman [106] distinguished between declarative knowledge about strategies and procedural knowledge for deploying them. These insights laid the groundwork for contemporary models of strategic, self-regulated learning [39].

Among these models, one of the most comprehensive and empirically grounded frameworks is Barry Zimmerman's influential framework of self-regulated learning. As shown in Fig. 2.1, Zimmerman's model views learning as a dynamic, cyclical process comprising three interdependent phases: forethought, performance, and self-reflection [135]. Each phase is governed by distinct but complementary cognitive and motivational mechanisms that enable learners to plan, actively execute, and evaluate

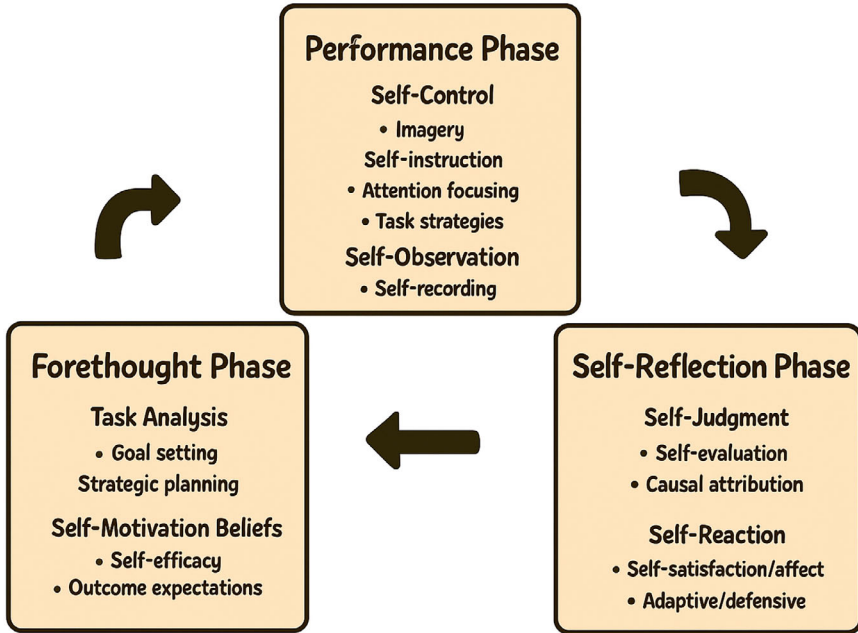


Fig. 2.1 The three-phase cyclical model of autonomous learning

their own learning behaviors. This framework has been validated across various age groups, subject domains, and instructional settings, and offers a robust lens through which to examine human autonomy in diverse educational contexts [27]. Moreover, it serves as a conceptual bridge for analyzing the structure and function of autonomy in machine learning and embodied intelligence.

A. Forethought Phase

The forethought phase precedes observable learning behavior and involves cognitive and motivational preparation for successful task engagement. Zimmerman highlights two core components in this phase: task analysis and self-motivational beliefs. Task analysis encompasses the learner’s ability to critically evaluate the demands of the task and formulate a clear plan of action. This includes setting specific goals, devising effective strategies, and anticipating potential challenges. Effective task analysis is proactive and strategic, where learners draw upon prior knowledge, align available resources with task requirements, and tailor their plan to contextual constraints. It provides the essential structural scaffolding for autonomous learning.

Equally crucial are self-motivational beliefs, which influence the learner’s willingness to initiate and sustain effort. These include self-efficacy, perceived task value, anticipated outcomes, and goal orientation. Learners with strong efficacy beliefs and

intrinsic valuation of the task are more likely to engage deeply and persist through difficulties. Notably, such beliefs are dynamic and context-sensitive, evolving with experience and reflection. The interplay between task analysis and motivation forms the foundation of autonomy, enabling learners to pursue tasks that align with their interests and capabilities while maintaining a sense of agency.

B. Performance Phase

In the performance phase, learners engage in the active execution and regulation of learning strategies. This phase is driven by two interrelated processes: self-control and self-observation. Self-control involves the deliberate application of cognitive strategies (e.g., elaboration, rehearsal, imagery) and the regulation of attention, emotions, and behaviors to optimize learning conditions. Environmental management is also integral, such as minimizing distractions or allocating time effectively. Emotional self-regulation plays a critical role, as unmanaged anxiety or overconfidence can impair cognitive functioning.

Self-observation complements self-control by enabling learners to monitor their own progress and adjust behaviors in real time. This process involves tracking performance indicators, recognizing moments of confusion or error, and assessing the efficacy of chosen strategies. Learners might use checklists, think-aloud protocols, or performance logs to maintain an ongoing awareness of their cognitive state. Effective self-observation allows for mid-course corrections, such as slowing down when facing conceptual difficulty or revisiting earlier content to reinforce understanding. These processes enable a form of situated autonomy, in which learners dynamically adapt their behaviors in response to internal feedback and external demands.

C. Self-Reflection Phase

The self-reflection phase follows task completion and provides an opportunity for learners to evaluate their performance and derive insights for future learning. This phase consists of two processes: self-judgment and self-reaction. Self-judgment involves critical evaluation of one's performance, including whether goals were met and strategies were effective. Learners consider both the outcomes and the underlying causes of success or failure, drawing on attributions related to effort, strategy use, task difficulty, or external conditions. These evaluations are not merely retrospective but play a central role in shaping motivation and behavior in future tasks.

Self-reaction refers to the learner's emotional and strategic response to the judgments formed. Positive evaluations may lead to satisfaction, increased self-efficacy, and the reinforcement of effective strategies. Negative evaluations, if processed constructively, can motivate revision of goals, the abandonment of ineffective habits, or the adoption of new strategies. Importantly, reflection also facilitates abstraction: learners begin to generalize from specific experiences and develop meta-level under-

standings about how they learn best. This culminates in a more resilient and flexible learning identity, one capable of adapting across contexts and over time.

These three phases constitute a coherent and nuanced model of autonomous learning. They highlight how learners proactively initiate, strategically manage, and reflectively refine their engagement with learning tasks. Rather than being a static trait, autonomy is revealed here as a dynamic, context-sensitive capacity grounded in awareness, adaptability, and agency. This framework not only deepens our understanding of human learning but also informs the design of autonomous learning systems in artificial agents.

2.2.2 *Autonomous Machine Learning*

The study of autonomous learning in humans provides foundational insights for the development of autonomous machine learning (AML). Rooted in cognitive psychology, metacognition, and educational theory, human autonomy is defined as the capacity to set goals, devise strategies, monitor progress, and adapt behavior based on reflective evaluation. These cognitive functions not only characterize effective human learning but also offer a conceptual scaffold for enabling analogous forms of autonomy in intelligent systems.

The impetus for AML arises from the increasing complexity and dynamism of real-world learning environments. Traditional machine learning pipelines require extensive human supervision for task definition, data curation, model design, evaluation, and deployment. Such dependency imposes limitations on scalability, adaptability, and generalization. Autonomous machine learning aims to overcome these limitations by endowing artificial agents with the capacity to guide and optimize their learning behavior with minimal human intervention.

Recent research [38, 132] has formalized autonomous machine learning as a coherent framework aimed at self-driven learning across all stages of the pipeline. Zhu et al. [132] propose a self-directed machine learning framework, emphasizing that AML should not be limited to automating isolated components such as model training or hyperparameter tuning, but rather should support a fully closed-loop system capable of discovering tasks, selecting models, managing data, evaluating performance, and refining its own learning behavior. Their formulation frames AML as a multi-level nested optimization problem, wherein the task space, model space, data space, and objective space are all subject to adaptive search and refinement. This approach enables the learner to determine not only optimal solutions, but also what problems to solve and how to approach them.

More concretely, as shown in Fig. 2.2, the AML process is structured into six interdependent modules: self-task construction, self-model selection, self-data curriculum, self-evaluation metric selection, self-optimization strategy selection, and self-deployment. These modules are recursively interlinked: for example, task construction shapes model selection; optimization influences data prioritization; eval-

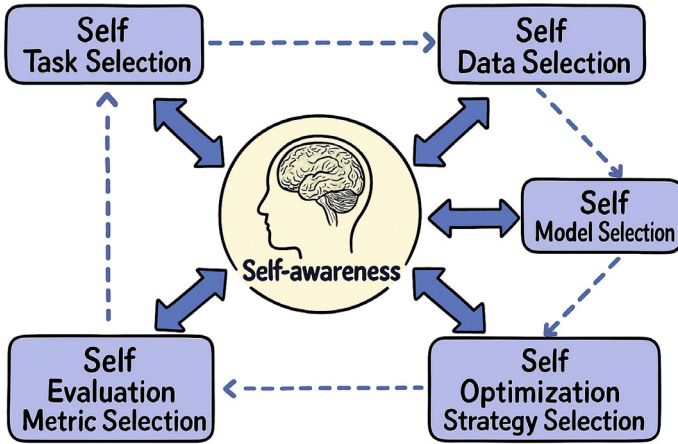


Fig. 2.2 Self-directed machine learning framework

uation drives strategy revision. This layered architecture mirrors the recursive and adaptive nature of human learning.

A. Self-Task Selection

Self-task selection refers to the system's ability to autonomously determine the most beneficial task to pursue from a candidate task space. This process is guided by a utility function $\mathcal{V}(\mathcal{T})$ that quantifies the expected value of each task, incorporating considerations such as learning potential, novelty, difficulty, or long-term utility. The learner selects the task $\mathcal{T}^*$ that maximizes this utility:

$$\mathcal{T}^* = \arg \max_{\mathcal{T} \in \mathcal{T}} \mathcal{V}(\mathcal{T}). \quad (2.1)$$

This capability allows the AML system to act beyond reactive learning and engage in proactive decision-making, similar to how a human learner chooses which skills to develop or which knowledge gaps to address. In practice, task selection may involve learning task embeddings, estimating future reward from task mastery, or using exploration strategies to balance known and unknown goals. Importantly, this module lays the foundation for a goal-oriented learning loop, ensuring that the system's learning trajectory aligns with internal or external objectives.

B. Self-Model Selection

Once a task has been clearly identified, the learner must carefully select an appropriate model architecture or hypothesis representation to successfully carry out the task

effectively. Self-model selection involves systematically evaluating multiple candidate models $m \in \mathcal{M}$ and choosing the one that offers the best expected generalization performance. Formally, this is expressed as:

$$m^* = \arg \min_{m \in \mathcal{M}} \mathbb{E}_{(x,y) \sim \mathcal{D}} [\ell(m(x), y)]. \quad (2.2)$$

This selection may be realized through neural architecture search (NAS), hyper-parameter optimization, or meta-modeling based on prior task-model performance. From a functional perspective, this module enables the learner to adaptively construct internal representations and inductive biases that are task specific, thereby increasing both accuracy and sample efficiency. It also mirrors human behavior in selecting the right mental model or conceptual toolset to approach a learning challenge.

C. Self-Data Selection

Data selection is a critical part of learning efficiency. Rather than training on a static or randomly sampled dataset, an autonomous learner continuously prioritizes or filters samples based on their learning value. In AML, this is operationalized through assigning weights w_i to data instances (x_i, y_i) and optimizing a weighted loss:

$$\min_{\theta} \sum_i w_i \cdot \ell(f_{\theta}(x_i), y_i). \quad (2.3)$$

The weights w_i may be computed based on uncertainty estimation, curriculum learning objectives, or sample diversity measures. This mechanism allows the learner to construct its own curriculum by focusing on difficult examples, identifying under-represented patterns, or skipping redundant samples, paralleling how humans choose to study more challenging material or review previous errors. Effective self-data selection leads to faster convergence and more robust generalization, particularly in non-stationary or open-ended environments.

D. Self-Optimization Strategy Selection

Beyond the selection of model and data, the learner must also determine how to train, i.e., what optimization strategy to apply. This includes decisions about gradient update rules, learning rate schedules, regularization techniques, and optimizer configurations. Formally, the learner selects the optimal strategy $s \in \mathcal{S}$ by minimizing a meta-objective over multiple tasks:

$$s^* = \arg \min_{s \in \mathcal{S}} \mathbb{E}_{\mathcal{T} \sim p(\mathcal{T})} [\mathcal{L}^{\text{val}}(\theta_s^*)]. \quad (2.4)$$

This module captures the capacity for learning to learn, where the system identifies training dynamics that work best across contexts. Techniques such as reinforcement learning for optimizer design, population-based training, and meta-gradient methods fall into this category. Conceptually, it resembles how human learners experiment with different study habits or problem-solving routines and gradually internalize the ones that lead to improved performance.

E. Self-Evaluation Metric Selection

The final module in the AML loop concerns how the learner evaluates its progress. Instead of relying on a fixed metric chosen by a human designer, an autonomous system must determine which evaluation criterion best captures its success for the given task. Given a space of potential metrics $\mathcal{E}$, the learner selects the one that maximizes task-relevance:

$$e^* = \arg \max_{e \in \mathcal{E}} \mathcal{R}(e, \mathcal{T}). \quad (2.5)$$

This selection may depend on statistical sensitivity, alignment with downstream objectives, or historical task-metric correlations. For example, in language tasks the learner may choose BLEU, ROUGE, or BERTScore based on how well they reflect semantic fidelity for the given use case. This module plays a key role in enabling meta-cognitive awareness: it allows the system to judge performance in context, adapt evaluation strategies over time, and close the feedback loop necessary for sustained improvement.

The five-layered framework of autonomous machine learning closely parallels the three-phase process of human self-regulated learning. In the forethought phase, human learners engage in goal setting, task analysis, and the formation of motivational beliefs that determine their willingness and readiness to learn. This corresponds directly to the self-task selection module in autonomous machine learning, where the system identifies the most beneficial task to pursue from a given task space. Just as human learners rely on internal evaluations of relevance, challenge, or interest, machine learners estimate utility functions over candidate tasks and select objectives that maximize long-term learning value. This phase lays the cognitive foundation for all subsequent behaviors in both humans and machines.

The performance phase in human learning involves the implementation of strategies, real-time monitoring, and adaptive control over both cognitive and behavioral processes. In autonomous machine learning, this stage is reflected in three interrelated modules: self-model selection, self-data selection, and self-optimization strategy selection. These modules govern how the system chooses the most suitable model architecture, selects and weights informative training data, and adapts its learning strategy to ensure efficient convergence. Together, they enable the system to dynamically regulate its learning process, much like human learners who refine their approaches in response to progress and difficulty. Finally, the self-reflection

phase, in which human learners evaluate their performance and revise their future plans, corresponds to the self-evaluation metric selection module. Here, the machine selects appropriate evaluation criteria that best reflect success in the current task, allowing it to refine its internal assessment mechanisms and guide future improvements. This structural alignment between human and machine autonomy illustrates a shared logic of iterative, goal-driven, and self-correcting learning.

2.3 Key Components of Autonomous Embodied AI

Autonomous embodied systems integrate perception, cognition, and action to operate independently in complex, dynamic environments. Central to this capability is the development of rich scene representations, enabling robots to understand spatial structures, object properties, and inter-object relationships. Techniques such as Neural Radiance Fields (NeRF) facilitate high-fidelity 3D reconstruction from sparse visual data, enhancing perception, localization, and decision-making. Complementing these advances, Large Language Models (LLMs) and vision-language models provide high-level reasoning, planning, and multimodal grounding, allowing robots to interpret instructions, generate actionable plans, and adapt to novel scenarios. Embodied multimodal models extend these capabilities by integrating spatial reasoning, sequential decision-making, and real-time control, bridging perception and action. Coupled with autonomous hardware embodiment and neuromorphic computing, these systems achieve low-latency, energy-efficient interaction with the physical world. Together, these developments form a foundation for robust, flexible, and cognitively capable robots, advancing the frontiers of autonomous learning, dynamic interaction, and intelligent embodied behavior.

2.3.1 *Autonomous Environment Learning*

Autonomous environmental learning enables the agent to actively explore and perceive the surrounding environment and autonomously update environmental cognition. Autonomous environmental learning approaches can be divided into three categories: NeRF-based, foundation model-based, LLM-based approaches.

A. NeRF Based

The primary objective of scene representation is to encode task-relevant environments into structured formats that can be efficiently stored and processed by computational systems. A well-formed scene representation should capture not only the attributes of individual objects, such as shape, size, position, and interactivity, but also the spatial relationships between objects, including distance and occlusion. Such repre-

representations enable autonomous systems to accurately perceive the spatial structure of their surroundings, track dynamic changes in the environment, and support effective reasoning and decision-making. Three-dimensional scene reconstruction refers to the process of building a digital 3D model of a scene based on available data, such as images, videos, or point clouds. Compared with traditional 2D imagery or video, 3D reconstruction provides a more comprehensive and geometry-aware description of the environment, offering a richer foundation for scene representation.

The fusion of artificial intelligence and autonomous robotics, particularly through deep learning techniques, has brought about transformative progress in how robots perceive and interact with their surroundings. Among the various technologies contributing to this advancement, NeRF has emerged as a particularly promising and influential method. Originally proposed by Mildenhall et al. [84], NeRF introduces a new paradigm for novel view synthesis by using neural networks to model the 3D structure of a scene. Specifically, NeRF employs MLPs to map five-dimensional inputs, consisting of three-dimensional spatial coordinates and two-dimensional viewing directions, to volume density and RGB color. Trained on a sparse set of images, this mapping enables the generation of photorealistic novel views with remarkable fidelity. Over time, the utility of NeRF has expanded across various research domains. In the field of autonomous robotics, it has become especially valuable for enhancing perception capabilities and enabling more sophisticated interactions between robots and their environments. Conventional robotic perception techniques often rely on depth maps, point clouds, or similar sensor data to reconstruct the surrounding environment and support decision-making. While effective in controlled scenarios, these approaches tend to degrade significantly when the input data is sparse or noisy, or when robots operate in dynamic and unpredictable environments. NeRF offers an alternative by representing 3D scenes through neural networks, which allows the reconstruction of high-quality 3D models even from limited data.

Autonomous robots are highly complex systems composed of multiple interacting components and subsystems. Over the past decades, a wide range of robotic architectures have developed [6, 63, 92]. For clarity, these systems can be abstracted into six fundamental modules: perception, localization and navigation, decision-making, execution, communication, and state monitoring with fault detection and handling. NeRF has demonstrated particular effectiveness in the first three modules, consistently providing notable improvements over conventional techniques in robotics.

In the area of perception, NeRF enables highly accurate reconstruction of rigid objects and large-scale environments, which is critical for robotic tasks such as manipulation, assembly, and inspection. It also supports the modeling of deformable objects and dynamic scenes, thereby significantly increasing adaptability and overall robustness. Furthermore, NeRF contributes to advanced semantic scene understanding by facilitating fine-grained segmentation, allowing robots to reliably distinguish between multiple objects and surfaces with high precision.

In localization and navigation, NeRF plays a significant role in precise camera pose estimation and simultaneous localization and mapping (SLAM). Its unique ability to synthesize high-fidelity 3D geometry from sparse views helps address challenges in pose estimation, especially when available training data is limited. NeRF also

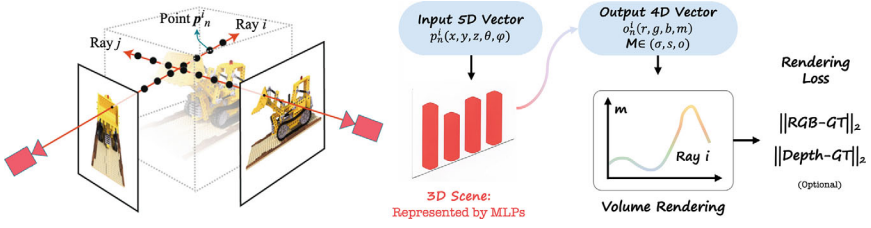


Fig. 2.3 The overview of NeRF framework

enhances SLAM systems by providing differentiable rendering of RGB and depth information, which supports iterative optimization of camera pose and detailed scene structure. These advanced capabilities enable robots to build more accurate maps and plan optimal paths more effectively in complex, dynamic environments.

In decision-making, NeRF supports active SLAM, enabling robots to explore and map unknown environments in real time. This function is especially important for mobile robots such as drones and autonomous vehicles operating in dynamic unstructured settings. Additionally, rich scene representations provided by NeRF facilitate learning-based decision processes, including object grasping and policy optimization, allowing robots to better understand and interact with physical surroundings.

Scene representation. NeRF represents 3D scenes implicitly through the parameters of a neural network. Specifically, it utilizes multi-layer perceptrons (MLPs) to map spatial coordinates to the scene's underlying structure. As shown in Fig. 2.3, the original NeRF framework [84, 85] takes a 5D input $(\mathbf{x}, \mathbf{d})$, where $\mathbf{x} = (x, y, z)^T$ denotes the 3D spatial position and $\mathbf{d} = (\theta, \phi)^T$ indicates the 2D viewing direction. The network outputs a volumetric density value σ_i to encode surface geometry implicitly. In addition to volume density, subsequent works have explored other implicit representations such as the (truncated) signed distance function (SDF) s_i and occupancy probability o_i [10, 133, 134]. Along with RGB color $\mathbf{c} = (r, g, b)^T$, the final MLP output is a 4D vector $(\mathbf{c}, m)$, where m represents one of $[\sigma, s, o]$. Letting Θ denote the network parameters, the mapping function of NeRF is formulated as:

$$F_{\Theta}(\mathbf{x}, \mathbf{d}) = (\mathbf{c}, m). \quad (2.6)$$

Positional encoding. The NeRF model [84] observes that directly feeding raw 5D inputs $(\mathbf{x}, \mathbf{d})$ into the MLP leads to poor reconstruction of high-frequency details, resulting in blurred renderings. To address this issue, positional encoding is applied to lift the input into a higher-dimensional space. The original NeRF adopts a sinusoidal encoding scheme defined as:

$$\gamma(\mathbf{p}) = (\sin(2^0 \pi \mathbf{p}), \cos(2^0 \pi \mathbf{p}), \dots, \sin(2^{L-1} \pi \mathbf{p}), \cos(2^{L-1} \pi \mathbf{p})), \quad (2.7)$$

where $\mathbf{p}$ corresponds to either $\mathbf{x}$ or $\mathbf{d}$. A follow-up study [112] introduces a Fourier feature mapping:

$$\gamma(\mathbf{p}) = (\sin(2\pi \mathbf{B} \mathbf{p}), \cos(2\pi \mathbf{B} \mathbf{p})). \quad (2.8)$$

Here, each entry of the matrix $\mathbf{B} \in \mathbb{R}^{L \times d}$ is drawn from a zero-mean Gaussian distribution, with the covariance serving as a tunable hyperparameter carefully chosen based on the specific task.

Volume rendering. To recover both geometry and appearance along a camera ray, classical volume rendering techniques [54] are employed. Given a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, where $\mathbf{o}$ is the camera origin and $t \in [t_n, t_f]$ represents the near and far bounds, the original NeRF [84] samples N points t_i uniformly along the ray segment. The estimated color of the ray $\hat{\mathbf{C}}(\mathbf{r})$ is computed by:

$$\hat{\mathbf{C}}(\mathbf{r}) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) \mathbf{c}_i, \quad \text{where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right). \quad (2.9)$$

In this equation, $\delta_i = t_{i+1} - t_i$ denotes the interval between consecutive samples. When employing the signed distance function as in [10], the color estimation:

$$\hat{\mathbf{C}}(\mathbf{r}) = \frac{1}{\sum_{i=1}^N w_i} \sum_{i=1}^N w_i \mathbf{c}_i, \quad \text{where } w_i = \sigma\left(\frac{s_i}{tr}\right) \cdot \sigma\left(-\frac{s_i}{tr}\right). \quad (2.10)$$

Here, $\sigma(\cdot)$ denotes the sigmoid function, and tr is a task-specific truncation threshold. Lastly, when using occupancy as the implicit representation [133], the ray color is approximated using:

$$\hat{\mathbf{C}}(\mathbf{r}) = \sum_{i=1}^N w_i \mathbf{c}_i, \quad \text{where } w_i = o_i \prod_{j=1}^{i-1} (1 - o_j). \quad (2.11)$$

This formulation models the transmittance along the ray as a product of the probabilities of not being occupied up to each point.

B. Foundation Model-Based

Robotic systems increasingly require high-level cognitive capabilities, such as interpreting user intent, generating executable plans, and adapting to dynamic environments. These demands have positioned large language models (LLMs) as key enablers of advanced robot intelligence [45, 48, 110]. Their ability to support general, interpretable, and context-aware reasoning has grown alongside the evolution of their architectures and training strategies. Scaling transformer-based designs and training on massive, diverse datasets have yielded emergent abilities including instruction following [57, 121], commonsense inference [19], and code generation [37]. Such

capabilities open new opportunities in robotics, ranging from natural language planning [35] to multimodal grounding [5] and interactive behavior control [59].

Recent advancements further expand LLM utility for robots. Enhanced reasoning, improved dialogue management, and support for visual inputs broaden their applicability in perception-driven decision-making. In parallel, open-source and fine-tunable models [56, 78] offer deployment on local hardware, a critical factor for privacy-sensitive, real-time systems [69]. Tracing the progression from early milestones to current state-of-the-art models provides a foundation for understanding how LLMs are adapted and embedded into embodied architectures (see Fig. 2.4).

While LLMs excel at processing and generating text, robotic systems often rely on multimodal perception to act effectively in the physical world. Vision-language models (VLMs) extend LLMs with visual understanding capabilities, enabling joint reasoning over language and imagery. These models support tasks such as image captioning [77], visual question answering (VQA) [75], and referring expression comprehension [41], all of which are essential for grounding linguistic commands in real-world context.

The development of VLMs has progressed from early joint architectures aligning visual and textual features [34, 68, 129] to large-scale contrastive learning approaches, such as CLIP [98], which enable open-vocabulary recognition through shared image-text embeddings. This allows robots to identify and locate objects without explicit category training, facilitating tasks like semantic mapping and retrieval from natural language prompts. Scaling multimodal transformer architectures, as demonstrated by SimVLM [120] and Flamingo [4], has further shown that few-shot visual reasoning, scene captioning, and dialogue can be achieved without task-specific fine-tuning, providing versatile perception interfaces for embodied agents.

Recent modular approaches, exemplified by BLIP-2 [65], connect frozen image encoders with LLMs via lightweight adapters, achieving strong visual performance at reduced training cost. Such designs enable real-time scene understanding, responsive instruction following, and contextually grounded decision-making, making them practical for deployment in robotic systems. Overall, VLMs represent a critical step toward scalable, sample-efficient multimodal intelligence, bridging the gap between raw sensory inputs and language-based cognition in embodied AI.

C. LLM-Based

Unlike traditional neural networks or large-scale models that focus primarily on object recognition, Embodied Multi-modal Language Models (EMLMs) are designed in accordance with the concept of embodied intelligence, where agents can perceive, interact with, and navigate within the physical world. Achieving this capability requires EMLMs to possess advanced perceptual and reasoning abilities that extend beyond recognizing objects to understanding their spatial relationships and the logical dependencies among them. In this context, embodied perception encompasses not only visual recognition but also spatial reasoning, comprehension of three-dimensional scene structures, and the capacity to plan and execute complex

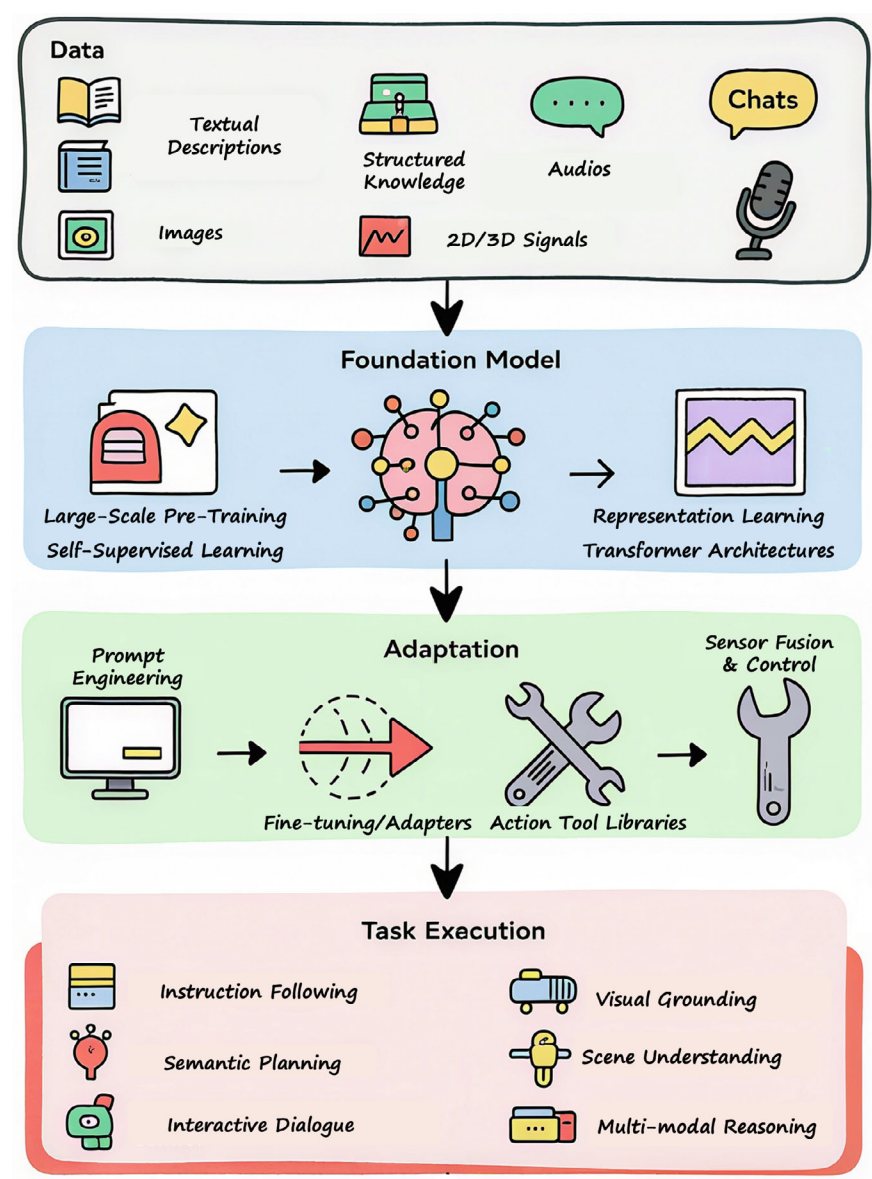


Fig. 2.4 Foundation models for learning general-purpose representations

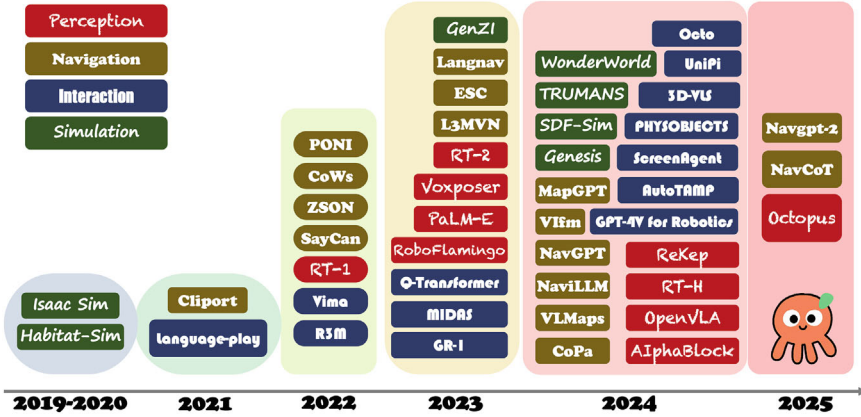


Fig. 2.5 Timeline of embodied perception, navigation, and interaction

tasks directly from visual input. The development trajectory of embodied perception models is illustrated in Fig. 2.5.

Current research on large models for embodied perception can be broadly categorized into two main types: models built upon the GPT architecture and models developed using alternative large-scale architectures, as summarized in Table 2.1.

GPT-based models are capable of interpreting task-specific natural language instructions and producing scene understanding outputs in textual form, enabling them to support a wide variety of perception tasks. Representative examples include GPT-1 [99], GPT-2 [100], GPT-3 [19], and GPT-4 [2]. For instance, Octopus [124] utilizes GPT-4V [125] to dynamically describe and analyze observed images at different task stages, identifying interactable objects and their spatial positions, which are subsequently used as inputs for decision-making. Similarly, CoPa [43] applies GPT-4V to detect and highlight specific grasping regions and potential grasp poses, thereby enabling fine-grained object understanding, while GraspNet [7] selects the most confident grasp pose for execution.

Although GPT-4V demonstrates strong capability in multimodal reasoning, it often encounters limitations in handling coordinate-based inputs due to restricted spatial perception. To mitigate this challenge, AlphaBlock [51], drawing inspiration from chain-of-thought reasoning [122], requires GPT-4V to first reason about spatial layouts before producing coordinate predictions. This generates both descriptive and positional information, which are then leveraged to provide real-time execution feedback. Other approaches integrate visual feature extraction with language models, such as MiniGPT-4 [131], which employs visual adapters to capture multi-stage visual features from ViT [22], and BLIP’s visual Q-former [65], which aligns image features with the LLM embedding space through a language-specific projector.

Some GPT-based solutions directly generate executable code from visual inputs. Voxposer [47] employs a multimodal large model to extract geometric object information, determine 3D coordinates, and embed them into executable code for gen-

Table 2.1 Embodied perception large models

Model	Architecture	Platform	Dataset	Year
Octopus [124]	MPT-7B	OctoGibson, OctoGTA	–	2025
AlphaBlock [51]	Vicuna [21], ViT-G/14 [26]	Franka Research 3 robot arm	AlphaBlock [51]	2023
CoPa [43]	GraspNet [25]	Franka Emika Panda	–	2024
ReKep [46]	DINOv2 [90], GPT-4o [50]	Wheeled Single-Arm, Stationary Dual-Arm	–	2024
RobotGPT [52]	–	PyBullet, MoveIt, ROS Franka	–	2024
Voxposer [47]	–	Franka Emika Panda	SAPIEN [123]	2023
PaLM-E [23]	ViT, OSRT [104]	TAMP, Language-Table, Mobile Manipulation	–	2023
RT-1 [16]	EfficientNet-B3 [111]	Everyday Robots	RT-1 [16]	2022
RT-2 [136]	PaLI-X, PaLM-E	7DoF Mobile Manipulator	–	2023
RT-H [11]	ViT [22], PaLI-X [20]	–	–	2024
RoboFlamingo [67]	OpenFlamingo [9]	Franka Emika Panda	CALVIN [82]	2023
Openvla [60]	Prismatic-7B [55]	WidowX Robot	BridgeData V2 [119]	2024
RoboMamba [74]	Mamba [36]	Franka Emika Panda, SAPIEN	LLaVA-LCS [73], LLaVA-v1.5 [73], LRV-INSTRUCT [71], RoboVQA [107]	2024

erating functional and constraint diagrams. ReKep [46] enhances adaptability by integrating DINOv2 [90] for patch-level feature extraction, SAM [61] for keypoint detection, and GPT-4o [50] for multi-stage task planning with sub-goals and path constraints. RobotGPT [52] adopts a structured five-component prompting framework encompassing background, objects, environment, task, and examples to guide GPT-based code generation for precise robotic control.

Non-GPT large models for embodied perception include PaLM-E [23], which directly integrates multimodal sensory inputs into a shared latent representation processed via Transformer self-attention [114], enabling grounded sequential decision-making. The RT-series models, including RT-1 [16], RT-2 [136], and RT-H [11], progressively advance visual-language-action learning by encoding visual inputs with ViT [22] or EfficientNet-B3 [111] and training on large-scale vision-language datasets to directly produce low-level robotic actions.

Other notable contributions include RoboFlamingo [67], which fine-tunes OpenFlamingo [9] for manipulation tasks using limited robot demonstration data; OpenVLA [60], which augments Llama 2 [113] with DINOv2 [90] and SigLIP [128] visual encoders to enhance spatial reasoning and semantic understanding; and RoboMamba

[74], which integrates the Mamba model [36] with a visual encoder to jointly train visual-linguistic reasoning for robotic applications.

2.3.2 *Autonomous Cognitive Learning*

Autonomous embodied cognitive learning is an interdisciplinary domain that draws from robotics, cognitive science, artificial intelligence, neuroscience, and developmental psychology. It addresses the fundamental challenge of creating robotic systems that not only perform actions but also learn and adapt through physical interaction with their environment. Embodied cognition, the idea that cognitive abilities arise not solely from internal symbolic computation but from the interaction between body and world, is central to this field [127]. This chapter explores the theoretical foundations, architectural principles, learning mechanisms, practical implementations, and future directions of autonomous embodied cognitive learning. We begin with the principles of embodied cognition, then examine cognitive robotics and developmental learning systems, continue with modern frameworks coupling learning and action in physical systems, and conclude with emerging trends and challenges.

Foundations of Embodied Cognition. Embodied cognition challenges traditional views that locate cognition solely within the brain. It emphasizes that perception, memory, reasoning, and language are grounded in sensorimotor interactions. Proponents argue that cognition emerges from the interplay among an agent's body, sensory-motor systems, and environment. Several theoretical viewpoints illustrate this shift: (1) Ecological Psychology highlights that perception emerges through direct interaction with environmental affordances. The agent perceives action opportunities directly, such as recognizing that a chair affords sitting. (2) Embodied Simulation [40] proposes that cognitive processes involve simulating sensorimotor experiences to understand and predict outcomes in the environment. (3) Enactivism frames cognition as active, situated, and emergent, stressing that cognition arises from real-time dynamic engagement with the world. These perspectives challenge abstract symbolic and disembodied conceptualizations of thinking and support the view that intelligent behavior cannot be separated from bodily experience.

Cognitive Robotics and Autonomous Learning. Cognitive robotics builds on embodied cognition by designing robotic systems that learn and reason through interaction rather than relying on pre-specified rules. Key principles include embodiment, sensorimotor integration, online learning, and adaptability. Human-like learning is incremental, cumulative, and driven by curiosity and intrinsic motivation. Robots designed with these principles explore their environment autonomously and choose their own goals, gradually building capability over time [93]. This approach contrasts with deep learning systems that typically require large labeled datasets [28, 29] and static objectives. Developmental robotics embodies this vision by modeling learning trajectories similar to those observed in child development. For instance, layered learning mechanisms replicate how a child acquires sensorimotor skills first,

then builds upon them. An important development is the integration of world models and predictive coding into robotic cognition. Through continual interaction, a robot builds an internal model of its environment and itself to predict future sensory inputs and plan actions accordingly. This predictive mechanism parallels how human brains anticipate sensory consequences of actions and adapt to minimize prediction errors. Sophisticated cognitive architectures replicate multiple cognitive functions, perception, planning, memory, and language, in an embodied framework. For example, the DAC-h3 [87] architecture enables a humanoid robot to proactively explore, learn from its interactions, and narrate its experience.

Representations in Embodied Learning. Representing knowledge in a way that is both grounded and generalizable remains a core challenge. The Object-Action Complex (OAC) framework proposes that cognition can be built on representational primitives combining affordance (what actions objects allow) with logical planning structures. OACs facilitate hierarchical learning of actions by grounding them in perceptual and action contexts. Neurorobotics embeds neurobiologically inspired models into robot controllers, enabling them to learn through mechanisms like synaptic plasticity, hierarchical motor control, and memory systems. Such systems simulate brain-like functions such as central pattern generators for locomotion, hippocampal maps for navigation, and sensory perception modules. These systems illustrate how embodied cognition benefits from biologically inspired frameworks.

Embodied Learning in Practice. Recent work has convincingly demonstrated embodied cognitive learning in robots. The ELLMER system leverages GPT-4 and a retrieval-augmented generation infrastructure to produce context-aware plans that incorporate feedback during task execution. Robots equipped with ELLMER successfully completed long-horizon tasks such as coffee making and plate decoration by adapting based on force and vision feedback [86]. In surgical robotics, vision-based embodied learning enables the automation of generalized sub-tasks across diverse procedures. Models trained in embodied learning paradigms have demonstrated the ability to generalize beyond specific trained tasks [76]. Embodied AI systems are increasingly deployed in real-world contexts. For example, robots in China including humanoids and delivery drones operate autonomously in public spaces, learning from sensory inputs and adjusting behaviors accordingly. The integration of embodied AI into daily life underscores the societal relevance and impact of embodied cognitive learning.

Autonomous embodied cognitive learning presents a holistic and powerful framework for building intelligent systems that learn through situated, physical experience. Combining embodiment, intrinsically motivated developmental learning, predictive modeling, adaptive representation, and social interaction yields robust and flexible intelligence in robots. As we continue advancing in computational models, sensorimotor integration, language understanding, and human-robot interaction, autonomous embodied cognitive learning will remain a vital paradigm guiding the development of future intelligent machines.

2.3.3 Autonomous Dynamic Interaction

The notion of an embodied robotic agent emerged in the late 1980s, rooted in the understanding that intelligent behavior arises from a system’s physical embodiment and its continuous sensorimotor interaction with the surrounding environment [18]. This perspective inspired early end-to-end neural architectures that directly mapped sensory inputs to motor outputs, enabling autonomous control without explicit symbolic reasoning [14, 64, 108]. The introduction of large language models (LLMs) has substantially broadened the functional capabilities of autonomous agents, shifting them from narrowly scoped, rule-based systems toward flexible entities with the ability to generalize, reason in context, and adapt to novel situations [126]. Prior to the ChatGPT era, Code as Policies (CaP) [70] pioneered the concept of robot-executable language model programs (LMPs), enabling reactive and vision-based control by generating code that indirectly invoked robotic tools.

The release of OpenAI’s ChatGPT in November 2022 marked a turning point in embodied agent research, accelerating the adoption of LLMs as core components of robotic control frameworks. While the Robot Operating System (ROS) had long served as the de facto middleware for robotic integration, early LLM-robotics prototypes primarily targeted ROS-compatible platforms. In 2023, initial implementations focused on establishing protocol integration and bidirectional communication channels between LLMs and ROS. Examples include natural language extensions for ROS 2 such as *ros2ai* [33], which augments the ROS 2 command-line interface with LLM capabilities, and ROScribe [102], which generates ROS codebases from high-level descriptions of desired system architectures. The ROS-LLM framework [8] provided a configurable template for defining robot APIs, automatically handling prompt construction and API calls to facilitate natural language interaction. Concurrently, Microsoft’s ChatGPT for Robotics [115] introduced a modular design for LLM-based robot programming, defining high-level APIs per robot and leveraging prompt engineering with predefined function libraries to enable natural language control across diverse tasks and environments. Most of these systems adopted a chatbot-style interface, interpreting free-form user input and producing corresponding robotic commands.

Advances in 2024 introduced specialized LLM-robotics interfaces and vision-language-action (VLA) systems that achieved unprecedented levels of generalization and semantic reasoning. These could process open-ended language instructions, ground them in visual perception, and execute relevant actions [12]. RT-2 [136] demonstrated a co-finetuning strategy that combined pretrained models with robotic trajectory data, enabling integrated perception, reasoning, and control. The $\pi 0$ system [13] further unified perception, reasoning, and motion generation through real-time continuous control based on flow-matched diffusion policies.

In parallel, new middleware frameworks emerged to bridge LLM reasoning and traditional robotic software stacks. ROSA (Robot Operating System Agent) [103] implemented an LLM-based control layer using the LangChain and the ReAct paradigm, abstracting ROS operations into Python-based tools for tasks such as

sensor querying and actuator control, while embedding safety mechanisms like parameter validation and human approval. RAI (Robotic AI Agent)[97] introduced a distributed multi-agent architecture, where specialized LLM agents, including perception, planning, control, and safety modules, collaborated in real time. BUMBLE (BUilding-wide MoBiLE Manipulation) [109] applied a unified VLM-based framework for large-scale mobile manipulation, integrating open-world perception, memory systems, and diverse motor skills to operate over extended spatial and temporal scales.

Another line of development focused on Model Context Protocol (MCP) servers, enabling AI assistants to integrate seamlessly with robotics tools. The ROS-MCP library [53], for instance, connected assistants like Claude to ROS 2 systems, allowing resource management, simulator control, and debugging through natural language interfaces. This plugin-based approach contrasts with monolithic agent frameworks, offering more modular extensibility.

More recently, OpenMind’s OM1 [89] introduced a hardware-agnostic AI runtime built on the decentralized coordination protocol, supporting secure identity management and interoperability among heterogeneous robotic platforms. While adopting perception-action loops akin to VLA systems, OM1 emphasized distributed coordination rather than centralized orchestration. Over the past three years, these developments have established a clear trend toward integrating an “AI agent” layer into robotic systems, one that interprets human intent, reasons over multimodal inputs, and autonomously manages a robot’s software capabilities.

2.3.4 Autonomous Hardware Embodiment

The concept of Autonomous Hardware Embodiment refers to the integration of intelligent control systems with physical robotic bodies, enabling agents to perceive, reason, and act within the real world without direct human intervention. Unlike purely software-based AI models, autonomous embodiments must address the complexities of sensorimotor coordination, hardware reliability, and real-time environmental adaptation. This paradigm lies at the intersection of robotics, artificial intelligence, mechatronics, and control theory, and plays a crucial role in fields ranging from industrial automation and autonomous vehicles to service robotics and embodied AI research. The notion of embodiment emphasizes that intelligence is not solely a function of abstract computation but emerges from the tight coupling between a physical body, sensory perception, and environmental interaction. In autonomous hardware systems, the hardware form factor, actuation capabilities, and sensor placement are not just engineering constraints, they shape the very nature of the agent’s cognition and performance.

A. Neuromorphic Computing-Based

The demand for intelligent, adaptive, and energy-efficient autonomous systems in domains such as robotics, mobile agents (e.g., UAVs), and self-driving vehicles has significantly increased in recent years, fueling growing interest in neuromorphic computing. Drawing inspiration from the computational principles of biological neural systems, neuromorphic approaches provide promising pathways to enhance the perception, decision-making, and responsiveness of autonomous platforms. In practical applications, systems such as autonomous vehicles, drones, and wearable devices must make real-time decisions under stringent constraints on energy consumption and computational resources. Conventional computing architectures often face limitations in meeting these requirements due to high power demands and processing latency. Neuromorphic computing addresses these challenges through the use of Spiking Neural Networks (SNNs), which process information as discrete events, enabling both low-power and low-latency inference [116, 117]. This event-driven paradigm is naturally suited for processing temporally sparse data and integrates effectively with event-based sensing modalities. Positioned at the intersection of robotics, neuroscience, machine learning, and hardware design, neuromorphic computing offers a multidisciplinary framework for advancing the capabilities of next-generation autonomous systems [81, 96].

Spiking Neural Networks. Spiking Neural Networks (SNNs), recognized as the third generation of neural network models, are designed to more faithfully capture the dynamics of biological neurons [80]. In SNNs, information is transmitted via discrete spikes rather than continuous activation values. Each neuron accumulates incoming spikes into a membrane potential and emits a spike when a defined threshold is surpassed, as illustrated in Fig. 2.6. This spike-driven computation naturally encodes temporal dependencies, enabling efficient recognition of time-varying patterns. Such characteristics make SNNs particularly suitable for applications that demand low-latency responses and temporal sensitivity, including motion detection and real-time decision-making in autonomous systems [96].

Neuromorphic Hardware. The efficient deployment of SNNs relies on neuromorphic hardware, which implements network computation by mimicking both the structural connectivity and functional dynamics of biological neural systems. Architectures such as IBM TrueNorth [3], Intel Loihi [91], SpiNNaker, and BrainScaleS [94], as well as commercial solutions like Brainchip Akida and Synsense DYNAP-SE [101], exemplify this approach. These platforms are optimized for high-throughput, low-power operation, making it feasible to deploy large-scale SNN models in resource-constrained environments. By integrating algorithmic efficiency with specialized hardware design, neuromorphic computing creates a scalable foundation for energy-efficient autonomous perception and control.

As shown in Fig. 2.7, the proposed workflow begins with the acquisition of raw data through traditional or event-based sensors, followed by a pre-processing stage that includes event discretization and encoding. Yielding temporally structured event sequences suitable for spiking neural processing. Subsequently, a reliable SNN model

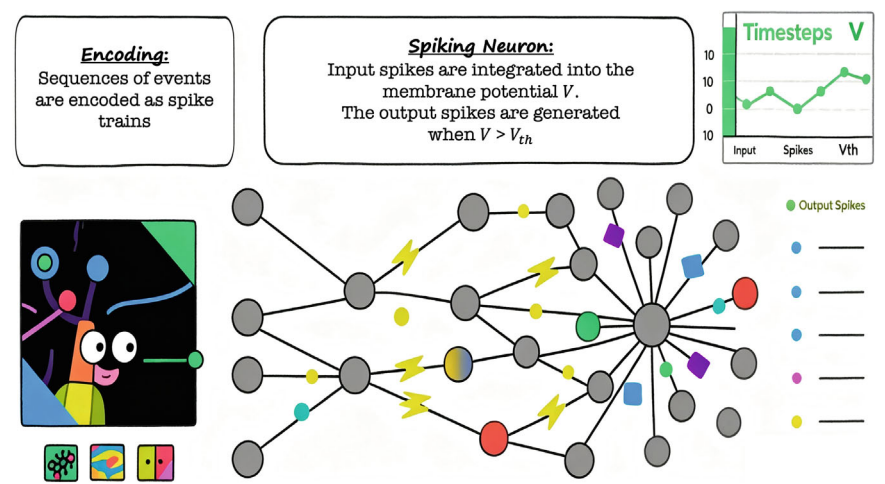


Fig. 2.6 The fundamental operation of the spiking neural networks

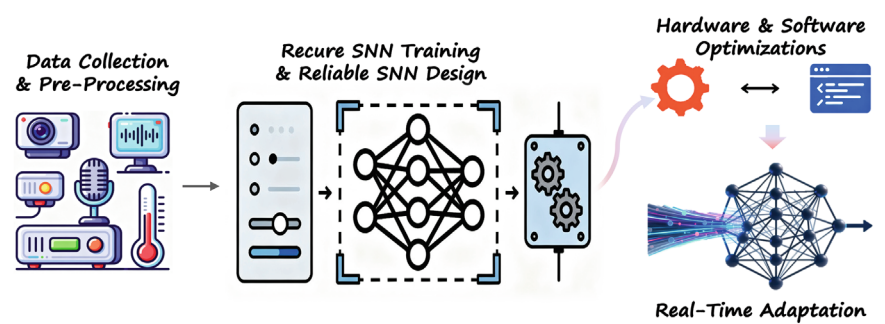


Fig. 2.7 The workflow of the spiking neural networks

is designed, incorporating hyperparameter tuning and fault tolerance for robustness against variability in inputs and system behavior. Afterward, a secure training pipeline combines adversarial training with encryption mechanisms to produce a resilient SNN. Next, hardware- and software-level optimizations, such as pruning, quantization, and hardware-aware NAS, are applied, guided by design constraints of the neuromorphic chip. The final step introduces real-time adaptation capabilities like continual learning and noise filtering, so the model can remain efficient under dynamic conditions. These stages result in a secure, efficient, and reliable SNN system that supports embodied intelligence in autonomous systems.

A. LLM-Based

Traditional robots have typically relied on rule-based algorithms for decision-making and task planning, which limits their applicability to relatively simple and predetermined scenarios. The rapid progress of Large Language Models (LLMs) has inspired new approaches aimed at endowing robots with advanced reasoning and long-term planning abilities [72]. This progress has driven the development of applications that integrate LLMs into robotic control, yielding notable improvements in performance, particularly in increasing success rates for robots performing complex tasks in dynamic environments [31, 42, 67, 118].

Embodied AI (EAI) systems form a category of intelligent systems that leverage the reasoning capabilities of LLMs to guide robots in carrying out complex real-world tasks such as housekeeping and industrial manufacturing, thereby reducing the need for human intervention. These systems generally consist of two key components: a high-performance server with GPUs for LLM inference and a robotic platform that executes commands and interacts with the physical environment. Most EAI systems adopt a multimodal LLM [44, 79, 88, 130] as the planning module. This model integrates natural language instructions, for example “put the blue mug on the table and bring me the red one,” with conventional sensor data streams in the robotic pipeline, including continuous video, IMU signals, and point clouds [24, 66, 105]. The LLM processes these multimodal inputs and generates a sequence of actions for the robot to execute, based on both the provided instructions and recent sensory observations. Recent advances in EAI have demonstrated significant potential for replacing humans in a variety of tasks. For example, Google’s Robotic Transformer [15, 16] has achieved a success rate exceeding 75.0% in household tasks such as “pick up objects,” “open drawers,” and “place objects into designated locations.” RoboFlamingo [67], a recently introduced EAI framework, has further improved single-task success rates to over 89.5%. These achievements indicate that the integration of LLMs into robotic systems for real-world physical interaction represents a promising step toward the realization of Artificial General Intelligence (AGI).

A key distinction between applying LLMs for text or image generation and integrating them as decision-making and planning modules in robotic systems lies in the strict real-time constraints that robots must satisfy [58, 83]. Without guaranteed real-time performance, the applicability of EAI systems remains limited to theoretical exploration rather than practical deployment. However, meeting real-time constraints remains a significant challenge for current EAI systems. The root cause lies in their execution paradigm. Existing EAI systems predominantly adopt a sequential execution model, where video inputs are processed, and robot actions are generated on a frame-by-frame basis. Specifically, after initialization, the robot begins with a video sequence containing N frames and a language instruction i . The LLM predicts an action tuple $(\Delta x, \Delta y, \Delta z, \dots)$ based on the current input tuple $(Frame_{t-N}, Frame_{t-N+1}, \dots, Frame_t, i)$, where Δ represents proposed movements and $Frame_t$ denotes the most recent image in the sequence. After executing the predicted action, the robot captures the subsequent frame $Frame_{t+1}$ at the

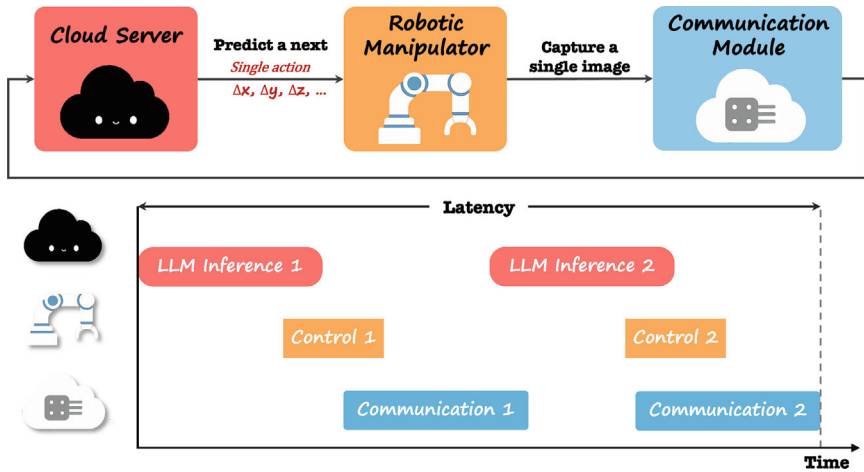


Fig. 2.8 The discrete execution pipeline of embodied AI systems

updated position. The next LLM inference then processes the updated input tuple ($Frame_{t-N+1}, Frame_{t-N+2}, \dots, Frame_{t+1}, i$). This sequential execution model is inherently time-consuming for two main reasons. As shown in Fig. 2.8, first, the dependence of each stage on the previous one leads to substantial cumulative latency. Since most robots rely on high-performance remote servers for LLM inference, the total delay for each frame results from the combined effects of LLM inference latency, robot action execution latency, and data transmission latency, which can sum to several hundred milliseconds. Second, because all three stages must be completed for every single frame, the system's responsiveness is further constrained, making it difficult to achieve true real-time operation.

To reduce the end-to-end latency of existing embodied AI systems, CORKI [49] redesigns the execution pipeline. As illustrated in Fig. 2.9, the first improvement occurs at the algorithmic level. CORKI departs from the conventional paradigm of discretely predicting the robot's movement for the next frame and instead proposes a novel embodied AI framework capable of forecasting the robot's trajectory over the near future. Unlike methods that predict only the immediate subsequent step, this framework accurately generates action sequences spanning multiple future steps. By doing so, it substantially lowers the frequency of LLM inferences, thereby reducing both latency and energy consumption. The second improvement targets the control process. CORKI develops a dedicated accelerator that converts the predicted trajectories into continuous, real-time control signals. This accelerator is specifically optimized for task-space computed torque control and incorporates a customized data-flow architecture, specialized circuits, and a dedicated on-chip buffer design. Finally, at the system level, CORKI optimizes the transmission of newly captured frames by overlapping the communication process with the robot's execution pro-

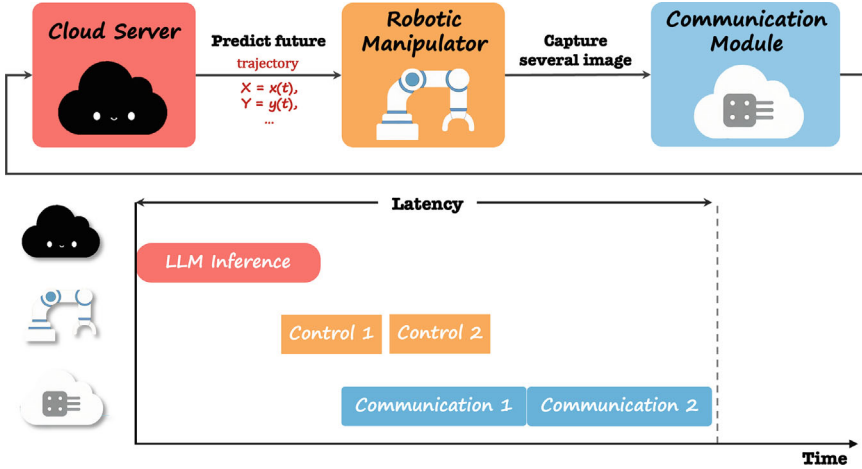


Fig. 2.9 The continuous execution pipeline of embodied AI systems

cess. This design effectively conceals communication latency within the execution latency, further minimizing the overall end-to-end delay.

Autonomous Hardware Embodiment represents a convergence of mechanical design, sensing, computation, and AI-driven control into a cohesive whole capable of operating independently in complex environments. The hardware body shapes the agent's perceptual and action capabilities, while embedded intelligence allows adaptation, resilience, and purposeful behavior. As enabling technologies mature, we can expect embodiments that are more adaptive, energy-efficient, and socially aware, accelerating the integration of autonomous systems into every facet of human life.

References

1. Forme avant 2007 Vygotskij, L.S., John-Steiner, V.: *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press (1979)
2. Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altschmidt, J., Altman, S., Anadkat, S., et al.: Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023)
3. Akopyan, F., et al.: Truenorth: design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip. *IEEE TCAD* (2015)
4. Alayrac, J.B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., et al.: Flamingo: a visual language model for few-shot learning. *Adv. Neural. Inf. Process. Syst.* **35**, 23716–23736 (2022)
5. Allgeuer, P., Ali, H., Wermter, S.: When robots get chatty: grounding multimodal human-robot conversation and collaboration. In: *International Conference on Artificial Neural Networks*, pp. 306–321. Springer (2024)
6. Arkin, R.C.: *Behavior-Based Robotics*. MIT Press (1998)

7. Asif, U., Tang, J., Harrer, S.: Graspnet: an efficient convolutional neural network for real-time grasp detection for low-powered devices. In: IJCAI, vol. 7, pp. 4875–4882 (2018)
8. Auromix: ROS-LLM: <https://github.com/Auromix/ROS-LLM> (2024)
9. Awadalla, A., Gao, L., Gardner, J., Hessel, J., Hanafy, Y., Zhu, W., Marathe, K., Bitton, Y., Gadre, S., Sagawa, S., et al.: Openflamingo: an open-source framework for training large autoregressive vision-language models. arXiv preprint [arXiv:2308.01390](https://arxiv.org/abs/2308.01390) (2023)
10. Azinović, D., Martin-Brualla, R., Goldman, D.B., Nießner, M., Thies, J.: Neural rgb-d surface reconstruction. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6280–6291 (2022). <https://doi.org/10.1109/CVPR52688.2022.00619>
11. Belkhale, S., Ding, T., Xiao, T., Sermanet, P., Vuong, Q., Tompson, J., Chebotar, Y., Dwibedi, D., Sadigh, D.: Rt-h: action hierarchies using language. arXiv preprint [arXiv:2403.01823](https://arxiv.org/abs/2403.01823) (2024)
12. Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., Hu, F., Huang, S., et al.: Gr00t n1: an open foundation model for generalist humanoid robots. arXiv preprint [arXiv:2503.14734](https://arxiv.org/abs/2503.14734) (2025)
13. Black, K., Brown, N., Driess, D., Esmail, A., et al.: $\pi 0$: a vision-language-action flow model for general robot control. corr, abs/2410.24164, 2024. <https://doi.org/10.48550/arXiv.2410.24164> (2024)
14. Bojarski, M., Del Testa, D., Dworakowski, D., Firner, B., Flepp, B., Goyal, P., Jackel, L.D., Monfort, M., Muller, U., Zhang, J., et al.: End to end learning for self-driving cars. arXiv preprint [arXiv:1604.07316](https://arxiv.org/abs/1604.07316) (2016)
15. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Chormanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al.: Rt-2: vision-language-action models transfer web knowledge to robotic control. arXiv preprint [arXiv:2307.15818](https://arxiv.org/abs/2307.15818) (2023)
16. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al.: Rt-1: robotics transformer for real-world control at scale. arXiv preprint [arXiv:2212.06817](https://arxiv.org/abs/2212.06817) (2022)
17. Brookfield, S.: Understanding and facilitating adult learning: a comprehensive analysis of principles and effective practices. McGraw-Hill Education (UK) (1986)
18. Brooks, R.: A robust layered control system for a mobile robot. IEEE J. Robot. Autom. **2**(1), 14–23 (2003)
19. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al.: Language models are few-shot learners. Adv. Neural. Inf. Process. Syst. **33**, 1877–1901 (2020)
20. Chen, X., Djolonga, J., Padlewski, P., Mustafa, B., Changpinyo, S., Wu, J., Ruiz, C.R., Goodman, S., Wang, X., Tay, Y., et al.: Pali-x: on scaling up a multilingual vision and language model. arXiv preprint [arXiv:2305.18565](https://arxiv.org/abs/2305.18565) (2023)
21. Chiang, W.L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J.E., et al.: Vicuna: an open-source chatbot impressing gpt-4 with 90
22. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al.: An image is worth 16x16 words: transformers for image recognition at scale. arXiv preprint [arXiv:2010.11929](https://arxiv.org/abs/2010.11929) (2020)
23. Driess, D., Xia, F., Sajjadi, M.S.M., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., Huang, W., Chebotar, Y., Sermanet, P., Duckworth, D., Levine, S., Vanhoucke, V., Hausman, K., Toussaint, M., Greff, K., Zeng, A., Mordatch, I., Florence, P.: Palm-e: an embodied multimodal language model. In: Proceedings of the 40th International Conference on Machine Learning, ICML’23. JMLR.org (2023)
24. Everett, H.: Sensors for mobile robots. CRC Press (1995)
25. Fang, H.S., Wang, C., Gou, M., Lu, C.: Graspnet-1billion: a large-scale benchmark for general object grasping. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11444–11453 (2020)
26. Fang, Y., Wang, W., Xie, B., Sun, Q., Wu, L., Wang, X., Huang, T., Wang, X., Cao, Y.: Eva: exploring the limits of masked visual representation learning at scale. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 19358–19369 (2023)

27. Feng, T., Li, Q., Wang, X., Wang, M., Li, G., Zhu, W.: Multi-weather cross-view geo-localization using denoising diffusion models. In: Proceedings of the 2nd Workshop on UAVs in Multimedia: Capturing the World from a New Perspective, pp. 35–39 (2024)
28. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata: a large-scale cooperative perception dataset for swarm uavs autonomous flight. In: Proceedings of the 32nd ACM International Conference on Multimedia, pp. 7600–7608 (2024)
29. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata-2: a scalable swarm uavs autonomous flight dataset for long-horizon tasks. arXiv preprint [arXiv:2509.00055](https://arxiv.org/abs/2509.00055) (2025)
30. Feng, T., Wang, X., Zhou, Z., Wang, R., Zhan, Y., Li, G., Li, Q., Zhu, W.: Evoagent: agent autonomous evolution with continual world model for long-horizon tasks. arXiv preprint [arXiv:2502.05907](https://arxiv.org/abs/2502.05907) (2025)
31. Firoozi, R., Tucker, J., Tian, S., Majumdar, A., Sun, J., Liu, W., Zhu, Y., Song, S., Kapoor, A., Hausman, K., et al.: Foundation models in robotics: applications, challenges, and the future. arXiv preprint [arXiv:2312.07843](https://arxiv.org/abs/2312.07843) (2023)
32. Flavell, J.H.: Metacognition and cognitive monitoring: a new area of cognitive-developmental inquiry. *Am. Psychol.* **34**(10), 906 (1979)
33. Fujitatomoya: ros2ai: <https://github.com/fujitatomoya/ros2ai> (2024)
34. Gao, J., Sarkar, B., Xia, F., Xiao, T., Wu, J., Ichter, B., Majumdar, A., Sadigh, D.: Physically grounded vision-language models for robotic manipulation. In: 2024 IEEE International Conference on Robotics and Automation (ICRA), pp. 12462–12469. IEEE (2024)
35. Gestrin, E., Kuhlmann, M., Seipp, J.: NI2plan: robust llm-driven planning from minimal text descriptions. arXiv preprint [arXiv:2405.04215](https://arxiv.org/abs/2405.04215) (2024)
36. Gu, A., Dao, T.: Mamba: linear-time sequence modeling with selective state spaces. arXiv preprint [arXiv:2312.00752](https://arxiv.org/abs/2312.00752) (2023)
37. Gu, Q.: Llm-based code generation method for golang compiler testing. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 2201–2203 (2023)
38. Gureckis, T.M., Markant, D.B.: Self-directed learning: a cognitive and computational perspective. *Perspect. Psychol. Sci.* **7**(5), 464–481 (2012)
39. Hacker, D.J., Dunlosky, J., Graesser, A.C.: *Metacognition in Educational Theory and Practice*. Routledge (1998)
40. Han, F., Zhang, L., Wang, X., Zhao, K.A., Zhong, Y., Su, Z., Feng, T., Zhu, W.: U2usim - a uav telepresence simulation platform with multi-agent sensing and dynamic environment. In: Proceedings of the 32nd ACM International Conference on Multimedia, pp. 11258–11260 (2024)
41. Hong, W., Wang, W., Ding, M., Yu, W., Lv, Q., Wang, Y., Cheng, Y., Huang, S., Ji, J., Xue, Z., et al.: Cogvlm2: visual language models for image and video understanding. arXiv preprint [arXiv:2408.16500](https://arxiv.org/abs/2408.16500) (2024)
42. Hu, Y., Xie, Q., Jain, V., Francis, J., Patrikar, J., Keetha, N., Kim, S., Xie, Y., Zhang, T., Zhao, Z., et al.: Toward general-purpose robots via foundation models: a survey and meta-analysis. arXiv preprint [arXiv:2312.08782](https://arxiv.org/abs/2312.08782) (2023)
43. Huang, H., Lin, F., Hu, Y., Wang, S., Gao, Y.: Copa: general robotic manipulation through spatial constraints of parts with foundation models. In: 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 9488–9495. IEEE (2024)
44. Huang, H., Zheng, O., Wang, D., Yin, J., Wang, Z., Ding, S., Yin, H., Xu, C., Yang, R., Zheng, Q., et al.: Chatgpt for shaping the future of dentistry: the potential of multi-modal large language model. *Int. J. Oral Sci.* **15**(1), 29 (2023)
45. Huang, W., Abbeel, P., Pathak, D., Mordatch, I.: Language models as zero-shot planners: extracting actionable knowledge for embodied agents. In: International Conference on Machine Learning, pp. 9118–9147. PMLR (2022)
46. Huang, W., Wang, C., Li, Y., Zhang, R., Fei-Fei, L.: Rekep: spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. arXiv preprint [arXiv:2409.01652](https://arxiv.org/abs/2409.01652) (2024)

47. Huang, W., Wang, C., Zhang, R., Li, Y., Wu, J., Fei-Fei, L.: Voxposer: composable 3d value maps for robotic manipulation with language models. In: Conference on Robot Learning, pp. 540–562. PMLR (2023)
48. Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., et al.: Inner monologue: embodied reasoning through planning with language models. arXiv preprint [arXiv:2207.05608](https://arxiv.org/abs/2207.05608) (2022)
49. Huang, Y., Hao, Y., Yu, B., Yan, F., Yang, Y., Min, F., Han, Y., Ma, L., Liu, S., Liu, Q., et al.: Software-hardware co-design for embodied ai robots. arXiv preprint [arXiv:2407.04292](https://arxiv.org/abs/2407.04292) (2024)
50. Islam, R., Moushi, O.M.: Gpt-4o: the cutting-edge advancement in multimodal llm. In: Intelligent Computing-Proceedings of the Computing Conference, pp. 47–60. Springer (2025)
51. Jin, C., Tan, W., Yang, J., Liu, B., Song, R., Wang, L., Fu, J.: Alphablock: embodied finetuning for vision-language reasoning in robot manipulation. arXiv preprint [arXiv:2305.18898](https://arxiv.org/abs/2305.18898) (2023)
52. Jin, Y., Li, D., Shi, J., Hao, P., Sun, F., Zhang, J., Fang, B., et al.: Robotgpt: robot manipulation learning from chatgpt. IEEE Robot. Autom. Lett. 9(3), 2543–2550 (2024)
53. John Varghese, R.: ROS-MCP-Server: <https://github.com/lpigeon/ros-mcp-server> (2024)
54. Kajiya, J.T., Von Herzen, B.P.: Ray tracing volume densities. In: Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH '84, p. 165–174. Association for Computing Machinery, New York, NY, USA (1984). <https://doi.org/10.1145/800031.808594>
55. Karamcheti, S., Nair, S., Balakrishna, A., Liang, P., Kollar, T., Sadigh, D.: Prismatic vlms: investigating the design space of visually-conditioned language models. In: Forty-First International Conference on Machine Learning (2024)
56. Khan, M.T., Chen, L., Ng, Y.H., Feng, W., Tan, N.Y.J., Moon, S.K.: Fine-tuning vision-language model for automated engineering drawing information extraction. arXiv preprint [arXiv:2411.03707](https://arxiv.org/abs/2411.03707) (2024)
57. Khan, M.T., Chen, L., Ng, Y.H., Feng, W., Tan, N.Y.J., Moon, S.K.: Leveraging vision-language models for manufacturing feature recognition in cad designs. arXiv preprint [arXiv:2411.02810](https://arxiv.org/abs/2411.02810) (2024)
58. Khatib, O.: Real-time obstacle avoidance for manipulators and mobile robots. Int. J. Robot. Res. 5(1), 90–98 (1986)
59. Kim, C.Y., Lee, C.P., Mutlu, B.: Understanding large-language model (llm)-powered human-robot interaction. In: Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction, pp. 371–380 (2024)
60. Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al.: Openvla: an open-source vision-language-action model. arXiv preprint [arXiv:2406.09246](https://arxiv.org/abs/2406.09246) (2024)
61. Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A.C., Lo, W.Y., et al.: Segment anything. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 4015–4026 (2023)
62. Knowles, M.S.: Self-directed learning, vol. 291. Association Press New York (1975)
63. Kortenamp, D., Bonasso, R.P., Murphy, R.: Artificial Intelligence and Mobile Robots: Case Studies of Successful Robot Systems. MIT Press (1998)
64. Laskin, M., Lee, K., Stooke, A., Pinto, L., Abbeel, P., Srinivas, A.: Reinforcement learning with augmented data. Adv. Neural. Inf. Process. Syst. 33, 19884–19895 (2020)
65. Li, J., Li, D., Savarese, S., Hoi, S.: Blip-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In: International Conference on Machine Learning, pp. 19730–19742. PMLR (2023)
66. Li, P., Liu, X.: Common sensors in industrial robots: a review. In: Journal of Physics: Conference Series, vol. 1267, p. 012036. IOP Publishing (2019)
67. Li, X., Liu, M., Zhang, H., Yu, C., Xu, J., Wu, H., Cheang, C., Jing, Y., Zhang, W., Liu, H., et al.: Vision-language foundation models as effective robot imitators. arXiv preprint [arXiv:2311.01378](https://arxiv.org/abs/2311.01378) (2023)

68. Li, Y., Hu, B., Chen, X., Ma, L., Xu, Y., Zhang, M.: Lmeye: an interactive perception network for large language models. *IEEE Trans. Multimed.* (2024)
69. Li, Y., Wen, H., Wang, W., Li, X., Yuan, Y., Liu, G., Liu, J., Xu, W., Wang, X., Sun, Y., et al.: Personal llm agents: insights and survey about the capability, efficiency and security. *arXiv preprint [arXiv:2401.05459](https://arxiv.org/abs/2401.05459)* (2024)
70. Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., Zeng, A.: Code as policies: language model programs for embodied control. *arXiv preprint [arXiv:2209.07753](https://arxiv.org/abs/2209.07753)* (2022)
71. Liu, F., Lin, K., Li, L., Wang, J., Yacoob, Y., Wang, L.: Mitigating hallucination in large multi-modal models via robust instruction tuning. *arXiv preprint [arXiv:2306.14565](https://arxiv.org/abs/2306.14565)* (2023)
72. Liu, H., Huang, X., Gu, J., Shi, J., He, N., Feng, T.: TCDformer-based momentum transfer model for long-term sports prediction. *Expert. Syst. Appl.* 128310 (2025)
73. Liu, H., Li, C., Li, Y., Lee, Y.J.: Improved baselines with visual instruction tuning. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 26296–26306 (2024)
74. Liu, J., Liu, M., Wang, Z., Lee, L., Zhou, K., An, P., Yang, S., Zhang, R., Guo, Y., Zhang, S.: Robomamba: multimodal state space model for efficient robot reasoning and manipulation. *arXiv e-prints* pp. [arXiv-2406](https://arxiv.org/abs/2406.14565) (2024)
75. Liu, L., Yang, D., Zhong, S., Tholeti, K.S.S., Ding, L., Zhang, Y., Gilpin, L.: Right this way: can vlms guide us to see more to answer questions? *Adv. Neural. Inf. Process. Syst.* **37**, 132946–132976 (2024)
76. Long, Y., Lin, A., Kwok, D.H.C., Zhang, L., Yang, Z., Shi, K., Song, L., Fu, J., Lin, H., Wei, W., et al.: Surgical embodied intelligence for generalized task autonomy in laparoscopic robot-assisted surgery. *Sci. Robot.* **10**(104), eadt3093 (2025)
77. Luu, D.T., Le, V.T., Vo, D.M.: Questioning, answering, and captioning for zero-shot detailed image caption. In: *Proceedings of the Asian Conference on Computer Vision*, pp. 242–259 (2024)
78. Lykov, A., Tsetserukou, D.: Llm-brain: Ai-driven fast generation of robot behaviour tree based on large language model. In: *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pp. 392–397. *IEEE* (2024)
79. Lyu, C., Wu, M., Wang, L., Huang, X., Liu, B., Du, Z., Shi, S., Tu, Z.: Macaw-llm: multi-modal language modeling with image, audio, video, and text integration. *arXiv preprint [arXiv:2306.09093](https://arxiv.org/abs/2306.09093)* (2023)
80. Maass, W.: Networks of spiking neurons: the third generation of neural network models. *Neural Netw.* (1997)
81. Marchisio, A., Shafique, M.: Embedded neuromorphic using intel’s loihi processor. In: *Embedded Machine Learning for Cyber-Physical, IoT, and Edge Computing: Software Optimizations and Hardware/Software Codesign*. Springer (2023)
82. Mees, O., Hermann, L., Rosete-Beas, E., Burgard, W.: Calvin: a benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robot. Autom. Lett. (RA-L)* **7**(3), 7327–7334 (2022)
83. Mei, Y., Lu, Y.H., Hu, Y.C., Lee, C.G.: Deployment of mobile robots with energy and timing constraints. *IEEE Trans. Rob.* **22**(3), 507–522 (2006)
84. Mildenhall, B., Srinivasan, P.P., Tancik, M., Barron, J.T., Ramamoorthi, R., Ng, R.: Nerf: representing scenes as neural radiance fields for view synthesis. In: *Computer Vision – ECCV 2020*, pp. 405–421. Springer, Cham (2020)
85. Mildenhall, B., Srinivasan, P.P., Tancik, M., Barron, J.T., Ramamoorthi, R., Ng, R.: Nerf: representing scenes as neural radiance fields for view synthesis. *Commun. ACM* **65**(1), 99–106 (2021)
86. Mon-Williams, R., Li, G., Long, R., Du, W., Lucas, C.G.: Embodied large language models enable robots to complete complex tasks in unpredictable environments. *Nat. Mach. Intell.* 1–10 (2025)

87. Moulin-Frier, C., Fischer, T., Petit, M., Pointeau, G., Puigbo, J.Y., Pattacini, U., Low, S.C., Camilleri, D., Nguyen, P., Hoffmann, M., et al.: Dac-h3: a proactive robot cognitive architecture to acquire and express knowledge about the world and the self. *IEEE Trans. Cogn. Dev. Syst.* **10**(4), 1005–1022 (2017)
88. Mu, Y., Zhang, Q., Hu, M., Wang, W., Ding, M., Jin, J., Wang, B., Dai, J., Qiao, Y., Luo, P.: Embodiedgpt: vision-language pre-training via embodied chain of thought. *Adv. Neural Inf. Process. Syst.* **36** (2024)
89. openmind: OpenMind: <https://docs.openmind.org> (2024)
90. Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., et al.: Dinov2: learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193* (2023)
91. Orchard, G., et al.: Efficient neuromorphic signal processing with loihi 2. In: *SiPS* (2021)
92. Örebäck, A.: A component framework for autonomous mobile robots. Ph.D. thesis, Numerisk analys och datalogi (2004)
93. Oudeyer, P.Y.: Autonomous development and learning in artificial intelligence and robotics: scaling up deep learning to human-like learning. *arXiv preprint arXiv:1712.01626* (2017)
94. Pehle, C., et al.: The brainscales-2 accelerated neuromorphic system with hybrid plasticity. *Front. Neurosci.* (2022)
95. Piaget, J., Duckworth, E.: Genetic epistemology. *Am. Behav. Sci.* **13**(3), 459–480 (1970)
96. Putra, R.V.W., et al.: Embodied neuromorphic artificial intelligence for robotics: perspectives, challenges, and research development stack. In: *ICARCV* (2024)
97. Rachwal, K., Majek, M., Boczek, B., Dabrowski, K., Liberadzki, P., Dabrowski, A., Ganzha, M.: Rai: Flexible agent framework for embodied ai. *arXiv preprint arXiv:2505.07532* (2025)
98. Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al.: Learning transferable visual models from natural language supervision. In: *International Conference on Machine Learning*, pp. 8748–8763. *Pmlr* (2021)
99. Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al.: Improving language understanding by generative pre-training. *OpenAI blog* (2018)
100. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al.: Language models are unsupervised multitask learners. *OpenAI Blog* **1**(8), 9 (2019)
101. Richter, O., et al.: Dynap-se2: a scalable multi-core dynamic neuromorphic asynchronous spiking neural network processor. *Neuromorphic Comput. Eng.* (2024)
102. RoboCoachTechnologies: ROScribe: <https://github.com/RoboCoachTechnologies/ROScibe> (2024)
103. Royce, R., Kaufmann, M., Becktor, J., Moon, S., Carpenter, K., Pak, K., Towler, A., Thakker, R., Khattak, S.: Enabling novel mission operations and interactions with rosa: the robot operating system agent. In: *2025 IEEE Aerospace Conference*, pp. 1–16. *IEEE* (2025)
104. Sajjadi, M.S., Duckworth, D., Mahendran, A., Van Steenkiste, S., Pavetic, F., Lucic, M., Guibas, L.J., Greff, K., Kipf, T.: Object scene representation transformer. *Adv. Neural Inf. Process. Syst.* **35**, 9512–9524 (2022)
105. Santaera, G., Luberto, E., Serio, A., Gabiccini, M., Bicchi, A.: Low-cost, fast and accurate reconstruction of robotic and human postures via imu measurements. In: *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2728–2735. *IEEE* (2015)
106. Schraw, G., Moshman, D.: Metacognitive theories. *Educ. Psychol. Rev.* **7**(4), 351–371 (1995)
107. Sermanet, P., Ding, T., Zhao, J., Xia, F., Dwibedi, D., Gopalakrishnan, K., Chan, C., Dulac-Arnold, G., Maddineni, S., Joshi, N.J., et al.: Robovqa: multimodal long-horizon reasoning for robotics. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 645–652. *IEEE* (2024)
108. Sermanet, P., Lynch, C., Chebotar, Y., Hsu, J., Jang, E., Schaal, S., Levine, S., Brain, G.: Time-contrastive networks: self-supervised learning from video. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1134–1141. *IEEE* (2018)
109. Shah, R., Yu, A., Zhu, Y., Zhu, Y., Martin-Martin, R.: Bumble: unifying reasoning and acting with vision-language models for building-wide mobile manipulation. *arXiv preprint arXiv:2410.06237* (2024)

110. Song, C.H., Wu, J., Washington, C., Sadler, B.M., Chao, W.L., Su, Y.: Llm-planner: few-shot grounded planning for embodied agents with large language models. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 2998–3009 (2023)
111. Tan, M., Le, Q.: Efficientnet: rethinking model scaling for convolutional neural networks. In: *International Conference on Machine Learning*, pp. 6105–6114. PMLR (2019)
112. Tancik, M., Srinivasan, P.P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J.T., Ng, R.: Fourier features let networks learn high frequency functions in low dimensional domains. In: *Advances in Neural Information Processing Systems* (2020)
113. Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al.: Llama 2: open foundation and fine-tuned chat models. *arXiv preprint [arXiv:2307.09288](https://arxiv.org/abs/2307.09288)* (2023)
114. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30** (2017)
115. Vemprala, S.H., Bonatti, R., Buckner, A., Kapoor, A.: Chatgpt for robotics: design principles and model abilities. *IEEE Access* **12**, 55682–55696 (2024)
116. Viale, A., et al.: Carsnn: an efficient spiking neural network for event-based autonomous cars on the loihi neuromorphic research processor. In: *IJCNN* (2021)
117. Viale, A., et al.: Lanesnns: Spiking neural networks for lane detection on the loihi neuromorphic processor. In: *IROS* (2022)
118. Wake, N., Kanehira, A., Sasabuchi, K., Takamatsu, J., Ikeuchi, K.: Gpt-4v (ision) for robotics: multimodal task planning from human demonstration. *arXiv preprint [arXiv:2311.12015](https://arxiv.org/abs/2311.12015)* (2023)
119. Walke, H.R., Black, K., Zhao, T.Z., Vuong, Q., Zheng, C., Hansen-Estruch, P., He, A.W., Myers, V., Kim, M.J., Du, M., et al.: Bridgedata v2: a dataset for robot learning at scale. In: *Conference on Robot Learning*, pp. 1723–1736. PMLR (2023)
120. Wang, Z., Yu, J., Yu, A.W., Dai, Z., Tsvetkov, Y., Cao, Y.: Simvlm: simple visual language model pretraining with weak supervision. *arXiv preprint [arXiv:2108.10904](https://arxiv.org/abs/2108.10904)* (2021)
121. Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al.: Emergent abilities of large language models. *arXiv preprint [arXiv:2206.07682](https://arxiv.org/abs/2206.07682)* (2022)
122. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al.: Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural. Inf. Process. Syst.* **35**, 24824–24837 (2022)
123. Xiang, F., Qin, Y., Mo, K., Xia, Y., Zhu, H., Liu, F., Liu, M., Jiang, H., Yuan, Y., Wang, H., et al.: Sapien: a simulated part-based interactive environment. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11097–11107 (2020)
124. Yang, J., Dong, Y., Liu, S., Li, B., Wang, Z., Tan, H., Jiang, C., Kang, J., Zhang, Y., Zhou, K., et al.: Octopus: embodied vision-language programmer from environmental feedback. In: *European Conference on Computer Vision*, pp. 20–38. Springer (2024)
125. Yang, Z., Li, L., Lin, K., Wang, J., Lin, C.C., Liu, Z., Wang, L.: The dawn of llms: preliminary explorations with gpt-4v (ision). *arXiv preprint [arXiv:2309.17421](https://arxiv.org/abs/2309.17421)* (2023)
126. Yao, S., Zhao, J., Yu, D., Du, N., Shafraan, I., Narasimhan, K., Cao, Y.: React: synergizing reasoning and acting in language models. In: *International Conference on Learning Representations (ICLR)* (2023)
127. Zalta, E.N., Nodelman, U., Allen, C., Perry, J.: *Stanford encyclopedia of philosophy* (2003)
128. Zhai, X., Mustafa, B., Kolesnikov, A., Beyer, L.: Sigmoid loss for language image pre-training. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 11975–11986 (2023)
129. Zhao, X., Li, M., Weber, C., Hafez, M.B., Wermter, S.: Chat with the environment: interactive multimodal perception using large language models. In: *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 3590–3596. IEEE (2023)
130. Zhao, Y., Lin, Z., Zhou, D., Huang, Z., Feng, J., Kang, B.: Bubogpt: enabling visual grounding in multi-modal llms. *arXiv preprint [arXiv:2307.08581](https://arxiv.org/abs/2307.08581)* (2023)

131. Zhu, D., Chen, J., Shen, X., Li, X., Elhoseiny, M.: Minigpt-4: enhancing vision-language understanding with advanced large language models. arXiv preprint [arXiv:2304.10592](https://arxiv.org/abs/2304.10592) (2023)
132. Zhu, W., Wang, X., Xie, P.: Self-directed machine learning. *AI Open* **3**, 58–70 (2022)
133. Zhu, Z., Peng, S., Larsson, V., Xu, W., Bao, H., Cui, Z., Oswald, M.R., Pollefeys, M.: Nice-slam: neural implicit scalable encoding for slam. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2022)
134. Zhuang, Y., Zhang, Q., Feng, Y., Zhu, H., Yao, Y., Li, X., Cao, Y.P., Shan, Y., Cao, X.: Anti-aliased neural implicit surfaces with encoding level of detail. In: *SIGGRAPH Asia 2023 Conference Papers*, SA '23. Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3610548.3618197>
135. Zimmerman, B.J.: Becoming a self-regulated learner: an overview. *Theory Pract.* **41**(2), 64–70 (2002)
136. Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohlhart, P., Welker, S., Wahid, A., et al.: Rt-2: vision-language-action models transfer web knowledge to robotic control. In: *Conference on Robot Learning*, pp. 2165–2183. PMLR (2023)

Chapter 3

Autonomous Swarm Embodied AI



Abstract Autonomous Swarm Embodied Artificial Intelligence (Autonomous SEAI) represents a significant evolution from single-agent to multi-agent systems, leveraging swarm intelligence to overcome limitations in environment perception, embodied decision-making, and operational robustness of current embodied AI. This chapter reviews swarm embodied AI from foundations to advances, including key technologies, key components, operational Robustness of the sim-to-real setting, simulators, datasets, and evaluation metrics. Recent progress in autonomous embodied AI has greatly advanced this field. This chapter highlights autonomous collaboration, autonomous communication, and autonomous robustness optimization approaches of autonomous SEAI through three integrative paradigms: self-organization, self-negotiation, and self-reflection. Autonomous SEAI will be a key research direction in the future, which can maintain resilience, adaptability, and continuous self-improvement in dynamic environments.

3.1 The Overview

The last decade has witnessed a surge of Embodied Artificial Intelligence (EAI) research and applications in various real-world scenarios, such as industrial robots for logistics sorting, automobile welding, and concrete pouring; household robots for vacuuming, cleaning, and prosthetics; and special robots for border patrol, underwater exploration, and biological conservation. Most EAI methods aim to enable a single agent to perfectly perform a specific task in a specific scenario through perception, reasoning, decision-making, and action interaction. However, their performance is often compromised by occlusions, sensor degradations or failures, and self-mode limitations, and they generally lack the capability for long-range perception and long-term decisions. Moreover, the optimal model choice often varies depending on specific scenarios, ego modes, and tasks, requiring substantial expertise for adaptive optimization. This complexity poses a significant barrier for new users and limits the applicability and feasibility of modern EAI methods in dynamic and open environments.

Autonomous Swarm Embodied Artificial Intelligence (Autonomous SEAI) marks a significant evolution within embodied AI, shifting from single-agent operations to swarm intelligence frameworks. In this framework, sophisticated system-level capabilities arise through decentralized coordination among multiple physical agents. By integrating distributed sensing, communication, and adaptive behavior, Autonomous SEAI effectively addresses many constraints inherent in single-agent systems, particularly in dynamic and open environments. This collaborative approach supports enhanced situational awareness, improved decision-making quality, and greater resilience against real-world uncertainties and disruptions.

Drawing from biological collectives such as social insects and flocking birds, SEAI formalizes how localized interactions yield emergent swarm intelligence. Central to SEAI is the role of physical embodiment and agent-environment interplay, which together facilitate coherent and context-aware coordination. The framework exhibits critical properties including adaptive response, operational stability, and scalable performance, often demonstrating nonlinear improvements as agent numbers grow. Such collective capacity allows SEAI systems to tackle challenges that are infeasible for individual agents, especially in large-scale or spatially distributed scenarios.

Key technologies of SEAI include mechanisms for collective decision-making, distributed regulation, and self-organized synchronization. Algorithms such as Particle Swarm Optimization (PSO) translate natural swarm behaviors into computational strategies for solving complex optimization tasks. These methods allow decentralized systems to achieve global objectives through local rules and iterative feedback, maintaining functionality amid environmental variability and internal perturbations.

Key components of SEAI incorporate multi-stage collaboration frameworks. Early collaboration utilizes raw data fusion to maximize perceptual integrity at the cost of bandwidth. Intermediate collaboration leverages feature-level exchanges to balance detail and efficiency, while late collaboration operates on decision-level outputs to minimize communication overhead. Specialized modules for dynamic communication, feature integration, and loss customization further refine collaborative perception and enable robust performance.

Operational robustness of SEAI constitutes a vital concern for real-world applications. Key challenges encompass pose inaccuracies, intermittent connectivity, inter-agent model variance, and cybersecurity threats. Contemporary solutions employ pose refinement networks, latency-tolerant communication protocols, cross-domain feature alignment, and cryptographic data sharing to strengthen system-wide endurance and security.

Autonomous SEAI includes three core modules: autonomous collaboration, autonomous communication, and autonomous robustness optimization, which manifest through three integrative paradigms: self-organization, negotiation, and reflection. Self-organization facilitates emergent coordination via local interactions and feedback loops. Negotiation provides explicit protocols for conflict resolution and task allocation. Reflection enables continual learning from historical interactions, fostering strategic adaptation and long-term system enhancement.

Advancements in Autonomous SEAI rely heavily on sophisticated simulation tools, richly annotated datasets, and comprehensive performance metrics. Real-world mapping simulators like U2USim, along with large-scale multimodal datasets such as U2UData, offer critical platforms for algorithm development and validation. These resources help narrow the reality gap, ensuring that swarm intelligence models can transfer effectively from simulation to real-world deployment.

In conclusion, Autonomous SEAI synthesizes collaborative perception, distributed control, and continual self-optimization to create resilient and scalable multi-agent systems. By harnessing swarm intelligence, Autonomous SEAI surpasses the limitations of traditional single-agent systems and establishes a new paradigm for intelligent embodied operations in dynamic and open environments.

3.2 Key Limitations of Single Embodied Agent

Embodied AI refers to a closed-loop system composed of perception, action, and feedback, realized through the physical interaction between the agent and its environment. Traditional disEmbodied AI often separates cognition from the body, whereas Embodied AI emphasizes that the essence of intelligence lies in continuous adaptation and adjustment within dynamic real-world environments. This approach closely mirrors the natural behavior patterns of humans and other organisms. In this book, Embodied AI particularly focuses on the concept of a single agent. A single agent refers to an autonomous entity capable of independently perceiving the environment, making decisions, and executing actions. This model of individual embodiment AI has become a foundational aspect of Embodied AI research. By focusing on the behavior of a single agent, this approach provides a clear and intuitive framework for understanding how intelligence operates through physical interaction with the environment.

However, although individual agents can exhibit a certain degree of intelligent behavior in various environments, their limitations and challenges become increasingly apparent when facing complex tasks. This chapter delves into these shortcomings and provides a comprehensive analysis from multiple perspectives: Limited Environment Perception and Adaptation, Limited Embodied Learning and Reasoning Capacity, and Limited Operational Robustness.

3.2.1 *Limited Environment Perception and Adaptation Capacity*

Individual agents operate with bounded rationality, perceiving only partial, noisy information from their environment. This limited perception restricts their understanding to local or abstracted data, preventing a full grasp of complex systems.

Despite these constraints, agents exhibit adaptive capacity by employing simple rules, learning from feedback, and adjusting strategies based on their subjective experience. They form internal models to predict outcomes, prioritize relevant stimuli, and gradually improve decisions within their narrow context. While adaptation is often effective locally, it may lead to suboptimal global outcomes due to incomplete knowledge. This balance between limited perception and adaptive behavior is central to their functionality in multi-agent and real-world systems.

A. Limited Perception Abilities

Embodied AI requires a perception system to understand the physical environment and interact with it. This perception system largely relies on multiple sensors to collect multimodal information, enabling the agent to perceive, process, and understand the environment. In individual Embodied AI, such sensors face limitations such as small quantity and limited coverage. These limitations are particularly problematic in scenarios requiring high situational awareness and real-time dynamics.

Sensor Accuracy and Its Impact. One of the key challenges faced by EAI is the accuracy of its sensory system. Sensor accuracy refers to the agent's ability to accurately detect and measure environmental stimuli. For example, camera sensors may encounter difficulties under low light conditions or when detecting objects at a distance; tactile sensors may struggle to accurately assess pressure or surface texture. If sensors lack sufficient accuracy, the agent may receive incomplete or incorrect data, leading to misunderstandings of its environment.

This problem becomes especially apparent in tasks with insufficient sensor types. Different tasks typically require multiple types of sensory input to fully perceive the environment and make informed decisions. For instance, an agent relying solely on visual sensors may be unable to detect key factors such as temperature, humidity, or gas composition, which are critical in certain industrial or rescue missions. To meet the demands of these complex tasks, it may be necessary to integrate other types of sensors, such as thermal imaging, gas detection, or acoustic sensors, to enhance the agent's overall environmental perception.

In addition, sensor failure is another major challenge. Over time or under extreme environments, sensors may degrade or malfunction, resulting in data loss or distortion. For example, camera sensors may fail under strong light or extreme temperatures, while tactile sensors may lose sensitivity due to mechanical damage or wear. When key sensors fail, the agent may be unable to collect accurate environmental data, impairing its functionality, leading to poor decisions, task failure, and potential safety hazards.

Limited Coverage Range and Its Consequences. Another significant limitation of the sensory system of embodied agents is their coverage. An agent can only perceive what is within the detection range of its sensors. For example, a camera may have a limited field of view, or a microphone may only capture sounds within a certain radius. In many large or complex environments, such limited coverage can severely restrict the agent's ability to gather comprehensive data about its surroundings.

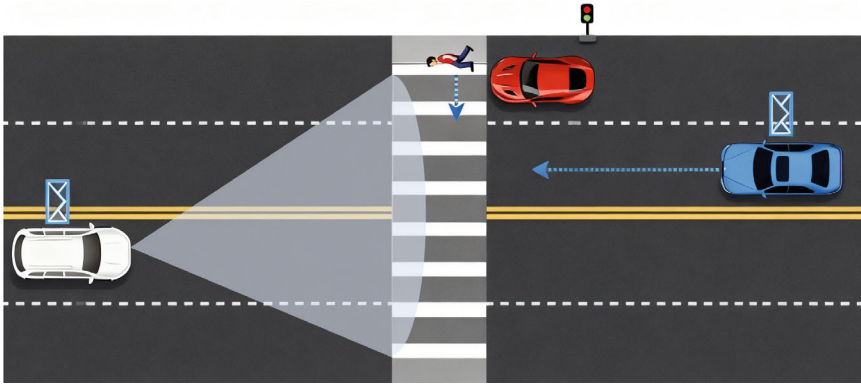


Fig. 3.1 Illustration of the limited coverage range

Limited range means that the agent is effectively “blind” to anything beyond its direct perception. In dynamic environments, this can lead to information gaps that hinder the agent’s situational awareness [84]. For example, in fast-moving environments such as traffic, an autonomous vehicle with limited sensor range may fail to detect a car or pedestrian approaching from a blind spot until it is too late to respond. As shown in Fig. 3.1, a person is crossing the street but is blocked from view by a red vehicle. If the blue vehicle continues moving forward, there is a high risk of collision. This may lead to accidents, making the system unreliable and unsafe in real-world applications.

B. Limited Environmental Adaptability

Embodied agents typically rely on specialized interactions with their physical environment, which can significantly limit their adaptability when facing dynamic changes and complex environments.

Firstly, when responding to rapidly evolving environments, the reaction speed and adaptability of embodied agents are often constrained. Their ability to adjust to environmental shifts is relatively slow, particularly in complex, fast-changing scenarios where a single embodied agent may struggle to meet task demands. For example, in autonomous vehicle applications, embodied agents must continuously sense environmental changes and make real-time decisions. If the pace of these changes exceeds the agent’s sensory and decision-making capabilities, it can lead to misjudgments or delayed actions. This is especially apparent in highly dynamic contexts, such as at the scene of a traffic accident, where rapid and unpredictable changes occur. In such cases, a single embodied agent’s adaptability may be insufficient, limiting its ability to swiftly modify its strategy in response to sudden shifts in the environment.

Secondly, embodied agents are often designed with specific tasks and environments in mind, which limits their operational flexibility in broader contexts. For instance, a robot designed for industrial settings may be optimized for flat, predictable factory floors, where it performs efficiently. However, when deployed in more irregu-

lar or challenging terrains, such as forests or mountainous areas, its performance can degrade significantly. This dependency on specific environmental conditions narrows the agent's application scope, making it less versatile in diverse environments. Furthermore, the lack of generalization in embodied agents is particularly pronounced in multi-task settings. Many embodied agents are tailored to excel at specific tasks, but when the nature of tasks changes or spans across different domains, their limitations become more evident. For instance, robots in industrial manufacturing may be highly proficient in assembling a particular component. However, when tasked with handling new materials or performing unfamiliar operations, they often require reprogramming or hardware modifications to accommodate the new demands. This lack of adaptability to new tasks significantly hinders their effectiveness in dealing with complex, cross-domain challenges.

3.2.2 *Limited Decision and Reasoning Capacity*

Compared to SEAI, a single embodied agent has limited decision-making and reasoning ability. It can only rely on its own sensors and computing power to perceive the environment and make judgments, without the ability to share information or work together with others. This often leads to incomplete information or incorrect decisions.

A. Limited Decision Abilities

Agents can interact with the external environment through sensors and make real-time decisions based on what they sense. However, the decision-making ability of embodied agents often depends on environmental perception and feedback. When they lack a global view, their performance in dynamic and complex environments becomes limited. Without a global view, embodied agents may fall into the trap of “local optimum” during decision-making. Relying only on local perception may cause the agent to miss the best overall path or strategy. For example, in a robot exploration task, a single robot may choose the closest path or resource, but ignore the best way to complete the overall task. This kind of limit caused by local sensing often makes the agent perform worse than a system with global information.

Also, group decision-making can lead to better knowledge improvement. When multiple embodied agents work together, they can learn from each other's experience to make better decisions and improve the system's ability to respond. For example, if one agent has trouble in a certain environment, it can get help or strategies from other agents and avoid repeating mistakes. Sharing experience helps the group improve its behavior step by step and adapt to different environments and tasks. As the agents interact more, each one can learn from others and improve, moving from “individual intelligence” to “collective intelligence.”

In addition, many tasks involve multiple goals, variables, and changing environments. When making decisions, individual agents may misjudge the situation because they cannot sense risks or chances far away [8]. For example, as shown in Fig. 3.2,

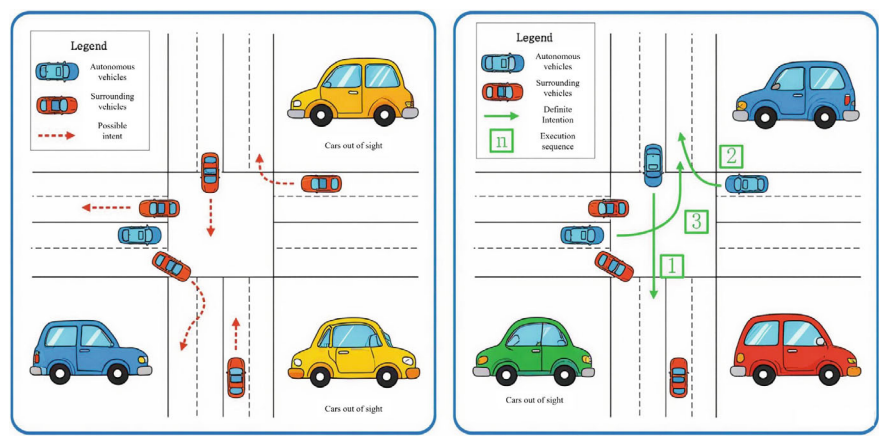


Fig. 3.2 The example of limited decision-making abilities

when a self-driving car approaches an intersection, it must make the best decision to pass safely and also give way to other nearby cars. In this case, the plans of each self-driving car might conflict. For example, the green car wants to turn left. If there is no proper coordination, it might crash into another car and stop all the cars from passing the intersection safely and smoothly.

B. Limited Reasoning Abilities on Complex Tasks

In many real-life situations, embodied agents are not simply required to complete a single task, but instead must operate within dynamic environments that involve the coordination and integration of multiple sub-tasks. For example, tasks like disaster rescue, self-driving, or smart delivery include things like sensing the environment, planning paths, reacting in real time, and balancing different goals. These are all part of a complex multi-task setting.

But a single embodied agent often lacks the reasoning ability needed for complex tasks. Reasoning is not just about sensing. It also involves understanding the task, predicting what might happen, and adjusting the plan when things change. When there is too much information or too many rules, the agent can become confused. It may not see the full picture or make the best decision. For example, when choosing between different goals or handling conflicts, the agent might pick the nearest or easiest option instead of what is best for the overall task. This limited reasoning ability makes it difficult for the agent to perform well in complex and dynamic environments. Also, embodied agents have trouble with long and complex task chains. Many tasks in real life have steps that depend on each other. Each step must be done right so the next one can start. For example, in delivery work, the steps include packing, planning the route, and bringing the item to the right place. These steps are linked together. A single agent often does tasks in a simple order and cannot deal with problems that break the chain. If one step goes wrong, it may not know how to fix it, and the whole job might fail or slow down.

In short, when doing hard and multi-step tasks, embodied agents are limited by their weak reasoning skills. This makes it hard for them to make good plans or work well in new or difficult situations. Better reasoning is needed if we want these agents to become smarter and more useful in real life.

3.2.3 *Limited Operational Robustness*

In complex and dynamic real-world environments, embodied agents must have strong operational robustness to ensure that they can continuously and reliably interact with the physical world. Operational robustness not only means that the system can keep running, but also that it can quickly sense, make correct decisions, and perform tasks well under uncertain and high-pressure conditions.

First, achieving robust operation requires fast sensing and correct understanding of the environment. Embodied agents need to process large amounts of high-dimensional data from multi-modal sensors, such as vision, hearing, and touch, in a very short time. However, a single agent has limited computing power and processing speed. In dynamic environments, it is often hard for one agent to provide stable and efficient responses. This may lead to delays, errors in perception, or wrong decisions, which reduces the agent's control over the environment. Second, a single embodied agent also faces serious challenges in data storage and communication. Complex tasks usually involve continuous data streams, such as image sequences, sound, or physical feedback. But the storage and bandwidth of a single agent are usually very limited. These hardware limits may cause important data to be lost or delayed during collection, transmission, or processing. This will affect decision-making and reduce the reliability of the whole system.

In addition to sensing and communication limits, operational robustness is also affected by external noise, frequent task switching, and energy use. In real use cases, the uncertainty and change of the environment require higher adaptability. This is especially true in tasks like disaster rescue, self-driving, or industrial operations, where continuous work and high fault tolerance are needed. In these tasks, the limits of a single embodied agent's robustness are more obvious. Therefore, to improve operational robustness in real-world conditions, we not only need to optimize perception and decision-making but also design better system-level solutions. This includes more efficient data scheduling, fault-tolerant design, and cooperation between multiple agents. These changes can help the whole system become more stable and better at handling different environments.

Overall, a single embodied agent faces many limitations in complex real-world environments. In terms of perception and adaptation, it relies on multimodal sensors to understand the environment, but often suffers from low accuracy, limited coverage, and a lack of sensor types. This makes it easy for the agent to miss important information or respond too slowly, especially in tasks that need high situational awareness and quick reactions. Also, since most embodied agents are designed for specific tasks

or environments, they have poor adaptability when the task or surroundings change. This makes it hard for them to work well across different domains or handle multiple tasks.

In decision-making and reasoning, a single embodied agent does not have access to global information and cannot coordinate with others. As a result, it may fall into “local optimum” traps and make poor choices. It also lacks the reasoning ability to deal with tasks that include many connected steps. When something goes wrong, the agent may not know how to adjust. At the operational level, limited computing power, storage, and communication bandwidth make it difficult to process large amounts of sensor data in real time. This can reduce the stability and accuracy of the agent’s actions. Therefore, to improve the overall intelligence level of embodied agents, it is necessary to introduce swarm Embodied AI. This can help enhance the system through collaborative perception, collaborative reasoning, and other forms of coordination.

3.3 Key Technologies of Swarm Embodied AI

Swarm Embodied AI combines the strengths of swarm intelligence and Embodied AI. It shows strong abilities in perception, decision-making, and environmental adaptation. Unlike a single agent, a swarm system uses the interactions between many agents to achieve distributed perception, self-organization, and global strategy optimization in complex tasks. In nature, collective systems like ant colonies and bee swarms show powerful collective decision-making, self-regulation, and dynamic adaptation. These natural examples provide useful ideas for designing artificial systems. By understanding the key mechanisms behind these group behaviors, we can build smarter, more efficient, and more robust swarm systems.

This section introduces the key technologies of Swarm Embodied AI. It focuses on three important mechanisms: collective decision-making, collective regulation and homeostasis, and collective periodic patterns. These parts explain how natural systems work and how they inspire algorithm design. In addition, the section presents Particle Swarm Optimization (PSO), a well-known algorithm that learns from natural group behavior. It shows how local rules and feedback can help solve complex problems. By combining these ideas and methods, Swarm Embodied AI becomes a powerful way to handle complex tasks in changing environments.

3.3.1 Swarm Intelligence

Swarm intelligence is a foundational field in the study of distributed multi-agent systems, where complex global behaviors emerge from simple local interactions and individual actions. Classic examples of swarm intelligence in nature include ant foraging, bird flocking, and bee nest building. These natural systems exhibit

remarkable adaptability, robustness, and scalability, offering valuable inspiration for designing technical systems. However, traditional models of swarm intelligence often rely on mathematical abstractions and theoretical frameworks, which may not fully account for the challenges encountered in real-world implementations. Factors such as agents' physical properties, spatial interaction constraints, and environmental complexity can significantly affect the practical performance of these theoretical models.

To address these challenges, the concept of Swarm embodied Artificial Intelligence (SEAI) has been introduced. SEAI emphasizes the importance of agents' physical embodiment and their interactions with the environment and with other agents, enabling more effective coordination and decision-making in dynamic and complex environments. Unlike traditional swarm intelligence, SEAI places greater emphasis on the physical characteristics of agents and how these properties influence collective behavior. This approach is particularly relevant to real-world robotic systems, where the embodied nature of agents makes swarm behavior more realistic and applicable. By establishing core principles for scientific inquiry, SEAI extends the field by enhancing task adaptability and improving the practicality of embodied systems, thus offering a more comprehensive framework for understanding and designing swarm behaviors in physical environments.

SEAI has been extensively studied in the literature. One of the most precise definitions was provided by Marc Millonas [141], whose work outlines the specific requirements and operational principles of SEAI algorithms. At the core of these algorithms is the use of simple local interaction rules and computations to generate complex and scalable swarm behaviors. In natural systems, SEAI often displays three key traits: adaptability, robustness, and scalability [13], which are the ideal properties that SEAI seeks to emulate in artificial systems.

- **Adaptability:** Swarms demonstrate the ability to adjust to dynamic environments and handle diverse tasks effectively.
- **Robustness:** Swarms maintain stable functionality even in the face of disturbances, such as the failure or loss of individual agents.
- **Scalability:** The performance of the swarm remains consistent as the number of agents or the complexity of tasks changes. Adding or removing agents does not significantly degrade the system's performance, provided the number of agents remains above the critical mass.

Beyond these widely acknowledged traits, super-linear performance scaling is a defining feature of SEAI systems. When the size of an embodied swarm exceeds a critical mass, its collective system performance should exhibit super-linear scaling. In other words, larger swarms not only achieve higher overall productivity but also surpass the simple sum of individual agent contributions. In a true SEAI system, interactions between agents amplify the system's overall capabilities, resulting in outcomes greater than the sum of its parts. Examples of such super-linear effects have been observed in honeybee colonies [178], robot swarms [2], and multi-processor computing systems [67]. The collaborative synergies among agents enhance the

efficiency of each individual agent, transforming the entire swarm into a highly effective system.

Achieving this level of super-linear scalability requires the careful design of local control algorithms for individual agents. These algorithms must ensure that not only the overall swarm efficiency improves as its size increases, but also that the performance of individual agents scales positively with the swarm size. This phenomenon, known as the embodied swarm effect, links the system's performance metrics to both the overall swarm's capabilities and the number of agents. Compared to Embodied Individual Intelligence, SEAI relies on more than just the sensory and motor capabilities of individual agents. SEAI leverages the collaboration and dynamic interactions between agents to deliver group behaviors that far exceed the sum of individual contributions. This makes SEAI uniquely suited to solving complex, large-scale problems. In contrast, embodied individual intelligence depends solely on the abilities of single agents, which are inherently limited in handling complex and scalable tasks. Consequently, scalability emerges as a critical benchmark for evaluating the true intelligence of SEAI systems.

For an SEAI system to excel in scalability, it must satisfy two fundamental requirements. First, its capabilities must emerge from collaborative behaviors among agents rather than the isolated abilities of individuals. If a system performs equally well with just one agent as it does with multiple agents, its behavior cannot be classified as swarm-intelligent, as it lacks the collaborative essence of SEAI. Second, as the swarm size increases, the system's overall performance should improve significantly, and the average contribution of each agent should remain stable or even increase. A genuinely scalable SEAI system should benefit from larger swarm sizes, avoiding efficiency losses as the number of agents grows. It is important to emphasize that these characteristics should manifest as the swarm size increases gradually, from a single agent to a practical number of agents, rather than being extrapolated to infinite sizes. All SEAI systems operate under physical constraints that limit their scalability. By adhering to these principles, SEAI systems can effectively utilize agent collaboration and dynamic interactions to achieve exceptional adaptability and efficiency in solving complex challenges.

Swarm intelligence draws inspiration from collective behaviors observed in nature, where large groups of simple agents interact locally to produce complex, intelligent outcomes. Unlike centralized systems, these natural swarms exhibit decentralized coordination, robustness, and adaptability, often relying on simple rules and local communication to achieve global objectives. From insect colonies to animal groups and microbial communities, these systems demonstrate how cooperation and feedback can solve problems efficiently, even under uncertain or changing conditions. To better understand and apply these natural principles, we begin by reviewing several representative examples of swarm intelligence in the natural world. These examples provide intuitive foundations and help us identify key patterns. We then analyze three core bio-inspired mechanisms found across these systems, namely, collective decision-making, collective regulation and homeostasis, and collective periodic patterns, and discuss how they guide the design of artificial swarm algorithms.

A. Classic Examples of Swarm Intelligence

Swarm intelligence appears in many natural systems, where simple individuals interact locally and create complex group behaviors. The following examples show how ants, bees, fish, birds, and mammals use simple rules to achieve efficient group coordination.

Ant Colonies. Ant colonies exhibit remarkable group intelligence, often referred to as swarm intelligence. One of the most fascinating behaviors is their ability to find the shortest path to a food source using pheromone trails. Ants deposit pheromones along their path, and subsequent ants are more likely to follow trails with higher pheromone concentrations. Over time, as ants reinforce shorter paths with more pheromone, the colony collectively identifies the optimal route. This phenomenon, called stigmergy, allows ants to solve complex problems without central coordination. Research by Dorigo et al. highlights how ants' behavior inspired the Ant Colony Optimization (ACO) algorithm, used to solve combinatorial optimization problems [39].

Another example is their cooperative behavior in building intricate nests. Ant species such as *Atta* (leafcutter ants) construct elaborate underground chambers for cultivating fungal gardens, their primary food source. This task involves precise excavation, waste management, and division of labor, demonstrating a decentralized but highly efficient social organization [81].

Honeybee Colonies. Bees show strong teamwork when they need to find a new nest. Some bees, called scouts, fly out to search for good places. When they return, they perform a waggle dance to show the location and quality of the site. Other bees watch, visit the sites, and also dance if they agree. This process spreads information in the group. Over time, most bees support one site, and the colony moves. There is no leader giving orders. The decision is made by group agreement through dancing and visiting. Bees also control the temperature of the hive. In hot weather, they fan their wings or bring water to cool the hive. In cold times, they group tightly to stay warm. Bees on the outside rotate positions to keep everyone safe.

Fish Schools. Fish often swim in schools to protect themselves. Each fish watches a few neighbors and copies their speed and direction. This simple behavior creates smooth and synchronized movement across the whole group. When a predator attacks, the school changes shape quickly. These fast movements confuse the predator, making it hard to catch one fish. This is called the "confusion effect." Some fish in the group may know about a food source or a threat. Their movement can influence others through local interaction. Even with no leader, the school can change direction quickly and adapt to the environment.

Bird Flocks. Birds like starlings fly in large groups called murmurations. Each bird watches six or seven neighbors and adjusts its flight based on them. Together, the group moves as one, avoiding crashes while creating flowing shapes in the sky. These patterns help the birds respond quickly to predators or changes in direction. The flock stays together without any central control. Local rules are enough for large-scale coordination. In bird migration, older birds lead younger ones along long routes.

Birds fly in V-shapes to save energy. The front bird faces the most air resistance, so they take turns. This teamwork helps the group travel far with less energy.

Mammalian Herds. Wildebeests move in large herds to find water and grass during seasonal changes. Some individuals sense better areas and others follow. Information spreads through simple cues like movement or sound. Group movement helps them avoid predators. Staying together makes it harder for predators to attack one animal. When crossing rivers or dangerous areas, they move in waves to reduce risk. Elephants also live in social groups. The oldest female, the matriarch, leads the group. She remembers paths and water sources from the past. Elephants use sounds and touch to stay connected, even across long distances. They care for each other and work together to protect the young.

Natural systems with collective characteristics provide a rich and diverse source of inspiration for designing swarm intelligence algorithms. These systems are typically composed of multiple interacting components, which may be living organisms or non-living entities. To draw inspiration from such complex systems and apply it to algorithm design, it is crucial to delve into two fundamental questions: “Why does it work effectively?” and “How does it work?” By answering these questions, we can distill the core mechanisms that drive the system’s functionality and abstract them into key features suitable for algorithmic design. The following sections will analyze the most significant collective properties and functionalities of swarm intelligence, illustrating how they guide the development of innovative algorithms.

B. Collective Decision-Making

Collective decision-making refers to the process by which a system, through interactions and information sharing among its individuals, arrives at an optimal choice from multiple alternatives. For example, ants use pheromone trails to mark paths, enabling the colony to gradually converge on the shortest route. Similarly, bees perform the “waggle dance” to communicate and collectively decide on the best foraging area or nesting site. This phenomenon highlights how local behaviors within complex systems can be amplified through interactions, ultimately resulting in global, coordinated, and macroscopic behaviors [165, 166].

In collective decision-making systems, positive feedback loops play a central role. These mechanisms operate on the principle that once a specific local behavior is initiated, it is reinforced and propagated through the system, encouraging more individuals to adopt the same behavior. For instance, when ants forage, individuals randomly explore their environment. Upon discovering food, an ant deposits a pheromone trail along its path. Other ants, detecting this pheromone, are more likely to follow the marked trail, reinforcing it further. Over time, paths with higher pheromone concentrations attract more ants, leading to the emergence of a collective behavior in which the colony converges on the shortest path. This process, where local behaviors are strengthened and spread, not only optimizes the system’s overall objective but also underscores the critical role of individual contributions to group coordination.

Beyond feedback mechanisms, the heterogeneity and distributed nature of individuals within collective decision-making systems deserve attention. In most natural

systems, individual heterogeneity significantly enhances the stability and efficiency of collective decision-making. For example, in bee colonies, certain individuals, known as “scouts,” take on proactive roles in exploring potential nesting sites during relocation. Meanwhile, other bees primarily evaluate the information provided by scouts to make informed decisions. This division of labor, arising from individual differences, improves the efficiency of resource exploration and reduces the risk of group bias toward a single information source.

The distributed nature of collective decision-making systems further provides these systems with remarkable resilience and adaptability. Unlike centralized decision-making processes, distributed systems enable individuals to make independent decisions based on local environmental cues and their own states. This decentralization ensures that the system can maintain its overall functionality even when some individuals fail. Such distributed properties have significant implications for artificial intelligence system design, particularly in large-scale robotic swarm control and decentralized network optimization.

C. Collective Regulation and Homeostasis

One of the defining features of complex systems, particularly biological organisms, is their ability to maintain certain critical variables within a viable and efficient range through collective regulation. This phenomenon, referred to as collective homeostasis, ensures that the system can function effectively in a dynamic environment, avoiding instability or resource depletion [99, 162]. In nature, social animals often exhibit remarkable examples of this ability, where cooperation among individuals leads to efficient resource utilization and distribution, achieving homeostasis at the population level.

At the heart of collective regulation systems lie negative feedback loops. These mechanisms operate by counteracting deviations from a target set point, thereby restoring balance. When a system variable moves away from its desired range, the negative feedback loop triggers corrective responses to bring the variable back into alignment. For instance, temperature regulation in bee colonies serves as a quintessential example. Worker bees employ behaviors such as wing vibration to cool the hive or clustering together to conserve heat, maintaining a temperature range optimal for larval development. When the hive temperature becomes too high or too low, these behaviors are adjusted dynamically to restore equilibrium. This form of regulation is rooted in simple individual behaviors that, when aggregated, enable the system as a whole to maintain environmental control. The emergent outcome exemplifies the power of local interactions in generating coordinated, system-wide stability.

A key strength of collective regulation systems is their adaptability and robustness. Adaptability refers to the system’s ability to adjust dynamically to environmental changes, while robustness denotes its capacity to maintain functionality despite internal or external disturbances. These properties are particularly evident in natural systems. For example, when a resource becomes scarce, ants or bees quickly redirect their efforts to locate and exploit alternative sources, ensuring the colony’s survival.

Similarly, if certain individuals are removed or incapacitated, the remaining members compensate for the loss, sustaining the system's overall function.

These characteristics are not only fascinating from a biological perspective but also highly relevant to technological and engineering applications. Understanding how natural systems achieve adaptability and robustness can inform the design of artificial systems, such as distributed networks or robotic swarms. In engineering and artificial intelligence, one of the central challenges is designing distributed systems that can efficiently manage resources and maintain stability. By studying the collective regulation mechanisms in nature, researchers can develop algorithms that mimic these processes. For instance, swarm robotics could leverage principles of collective homeostasis to coordinate the movements of individual robots for tasks such as search-and-rescue or environmental monitoring. Similarly, smart grid systems could adopt feedback-based resource allocation strategies inspired by natural systems to optimize energy distribution and enhance resilience to failures.

D. Collective Periodic Patterns

Collective systems often exhibit intricate dynamics of state changes over time, occasionally giving rise to fascinating spatial patterns such as circular or spiral waves. These spatiotemporal patterns are not mere byproducts of state changes within the system but play critical functional roles. For example, they can facilitate collective computation, enable self-assessment of group states (e.g., estimating local group size), and serve as a medium for signal transmission [158, 163]. The emergence and propagation of these patterns shed light on the underlying principles of communication, feedback regulation, and dynamic coupling in complex systems.

The formation of periodic patterns in collective systems is primarily driven by two key mechanisms: communication delays and the complex internal state changes of individuals. **Communication delays:** Communication delays arise when information transmission between individuals takes time, creating lag effects in the system. For example, in certain insect groups, individuals exchange information through chemical signals or behavioral cues. The speed at which these signals propagate determines how quickly the group can respond as a whole. When these delays accumulate within the system, they can lead to regular oscillatory behavior. For instance, delayed feedback in signaling can produce rhythmic waves that propagate through the group, influencing collective behavior. **Complex Internal State Changes:** Periodic oscillations can also result from the intricate internal dynamics of individuals. Many animals exhibit behavioral patterns influenced by their physiological rhythms or internal energy states. These internal oscillations can propagate through interactions among individuals, ultimately manifesting as observable periodic patterns at the group level. Such patterns regulate individual behavior while simultaneously affecting the overall functionality and coordination of the system.

Periodic patterns, particularly those expressed as dynamic spatial waves, offer a wide range of functional benefits in collective systems. They can serve as decentralized tools for computation, where the frequency or phase of spatiotemporal waves encodes information about resource distribution or environmental conditions. This allows the system to process complex tasks collectively without relying on central-

ized control. For instance, in coupled oscillator systems, waves can represent resource availability or changes in the environment, enabling the group to respond efficiently as a coordinated whole. These patterns also play a significant role in self-assessment of group states. Many social animals rely on information about group size and density for decision-making processes. In some cases, the speed or amplitude of wave propagation can provide individuals with localized information about resource use or group density, enabling dynamic feedback that helps optimize group behaviors, such as refining foraging ranges or redirecting movement. Furthermore, periodic patterns can act as effective mediums for signal transmission. Certain species, such as frogs and insects, produce rhythmic waves through sound or vibrations to convey information, such as mating calls or danger warnings. These oscillations enhance the range and reliability of communication through synchronization, amplifying the signal and ensuring it reaches a larger audience.

The study of collective periodic patterns offers profound insights into the behavior of natural systems while inspiring innovations in artificial systems. In robotics, mimicking these patterns could enable decentralized coordination in large-scale swarm systems, such as those used in search-and-rescue operations or environmental monitoring. Similarly, understanding periodic dynamics in biological systems has implications for medical research, including the study of cardiac arrhythmias and neural oscillations.

3.3.2 *Particle Swarm Optimization*

The study of natural collective intelligence offers profound insights into how distributed systems can achieve efficient coordination, remarkable adaptability, and holistic optimization without relying on centralized control. By uncovering the fundamental principles underlying group behavior in nature, we can transform these insights into valuable strategies for designing artificial systems capable of addressing complex problems. Particle Swarm Optimization (PSO), a nature-inspired optimization algorithm, emulates the interactions among individuals (particles) and their environmental awareness. By dynamically adjusting particle positions within the search space, PSO efficiently navigates complex problem-solving landscapes. In this section, we will explore the core principles of the PSO algorithm and examine how it draws inspiration from the collective behaviors observed in nature.

PSO [97] was first introduced by Kennedy and Eberhart. It is a meta-heuristic optimization technique inspired by the social behaviors observed in nature, such as those of animals, insects, and humans. Since its inception, PSO has gained widespread recognition among researchers and practitioners for its robustness and efficiency in solving various optimization problems. The design of PSO was grounded in a key insight into human social behavior and cognition: “People learn to make sense of the world by talking with other people about it” [96]. This observation inspired Kennedy and Eberhart to develop a computational framework where a population of candidate

solutions improves iteratively through information sharing and dynamic adjustments among particles.

In PSO, each particle represents a potential solution in the problem space. Particles optimize their positions by integrating feedback from their own experiences and the information shared by their neighbors. Although each particle operates based on simple rules, the collective interactions among particles often result in the emergence of a global optimal solution (though not guaranteed). Particles broadcast their current positions to neighbors and adjust their movements using random perturbations and a combination of their velocity, personal best position, and the best positions found by neighboring particles. As the algorithm iterates, the swarm converges toward regions in the search space containing high-quality solutions. This process emulates the behavior of a flock of birds collaboratively searching for food: by coordinating and communicating, the flock gradually homes in on the food source. Similarly, PSO enables particles to interact and collectively locate optimal solutions within the search space.

Specifically, the velocity and position of each particle in PSO are updated iteratively based on its current state, personal best position, and the best position discovered by its neighbors. Let v_i denote the velocity of the i th particle, x_i its current position, p_i its personal best position, and p_g the best position found within its neighborhood (or the entire swarm). The updates for v_i and x_i in the original PSO algorithm are defined by the following equations [97]:

$$\begin{aligned} \mathbf{v}_i^{NEW} &\leftarrow \mathbf{v}_i + \varphi_1 \otimes (\mathbf{p}_i - \mathbf{x}_i) + \varphi_2 \otimes (\mathbf{p}_g - \mathbf{x}_i), \\ \mathbf{x}_i^{NEW} &\leftarrow \mathbf{x}_i + \mathbf{v}_i^{NEW}, \end{aligned} \quad (3.1)$$

where $\varphi_1 = c_1 R_1$ and $\varphi_2 = c_2 R_2$, R_1 , and R_2 are independent functions generating random value vectors uniformly distributed in the range $[0, 1]$. c_1 and c_2 are acceleration coefficients, while the symbol $\otimes$ denotes element-wise vector multiplication. Equation 3.1 defines the velocity v_i of a particle as a combination of three components: “momentum,” “cognitive,” and “social” terms: The “momentum” term v_i represents the particle’s inertia, allowing it to maintain its previous trajectory. The “cognitive” term $\varphi_1 \otimes (\mathbf{p}_i - \mathbf{x}_i)$ reflects the particle’s tendency to return to the best position it has discovered. The “social” term $\varphi_2 \otimes (\mathbf{p}_g - \mathbf{x}_i)$ captures the particle’s inclination to move toward the best position identified by the swarm as a whole. In practice, randomness is introduced using a Random Number Generator (RNG). However, the choice of RNG can significantly influence the algorithm’s performance, and surprisingly, the best generator does not always yield optimal results. The “social” component leverages neighborhood topologies to regulate the speed of information propagation among particles. Common topologies include ring, star, and von Neumann configurations. Smaller neighborhood topologies, such as von Neumann, often perform better on complex multimodal problems due to restricted information flow, while larger neighborhoods tend to excel in simpler unimodal problems [140]. When the PSO algorithm restricts p_g to a local neighborhood, it is referred to as lbest PSO.

Conversely, if p_g is selected from the entire swarm without restrictions, it is known as gbest PSO.

Algorithm 1 provides an overview of a basic PSO implementation. In the algorithm, $f(\cdot)$ represents the fitness function, and $\max(p_{neighbors})$ retrieves the best position among the personal bests within particle i 's neighborhood. It is worth noting that Algorithm 1 typically adopts the classic asynchronous update model, which is widely regarded as the most efficient. However, synchronous updates can also be employed, particularly in parallel computing scenarios. In such cases, all p_i values are first saved, followed by an update to p_g , and then simultaneous updates to the velocities and positions of all particles.

Algorithm 1 The PSO algorithm, assuming maximization

Randomly generate an initial population

repeat

for each particle i **do**

if $f(\mathbf{x}_i) > f(\mathbf{p}_i)$ **then**

$\mathbf{p}_i \leftarrow \mathbf{x}_i$;

$\mathbf{p}_g \leftarrow \max(\mathbf{p}_{neighbors})$;

 Update velocity v_i ;

 Update position p_i ;

end if

end for

until *termination criterion is met*;

Many real-world optimization problems are inherently dynamic, requiring algorithms that can adapt to changes in optimal solutions over time. For instance, traffic conditions in a city are constantly fluctuating, what is considered the best route at one moment might no longer be optimal shortly after. Unlike static optimization, the primary objective in a dynamic environment is to track and adjust to the shifting optima as closely as possible.

Particle Swarm Optimization (PSO) is well-suited for dynamic optimization due to its fast convergence and inherent flexibility. Through continuous interactions among particles in the swarm, PSO dynamically adjusts particle positions, allowing the algorithm to respond effectively to environmental changes. This adaptability makes PSO particularly attractive for solving dynamic problems. However, studies have shown that standard PSO must be adapted to tackle the additional challenges posed by dynamic environments [9, 16].

One of the earliest applications of PSO to dynamic optimization was conducted by Eberhart and Shi [45]. They employed an inertia-weighted PSO to track the optimum of a unimodal parabolic function whose maximum changed at regular intervals. Their findings revealed that, under certain conditions, PSO could perform on par with or even outperform evolutionary algorithms. Detecting changes in dynamic environments can be achieved through several methods. One approach involves designating a sentry particle during each iteration [15]. By evaluating the fitness of the sentry particle and comparing it to its previous value, the algorithm can identify environ-

mental changes. If a change is detected, the swarm is alerted, triggering appropriate response mechanisms. Another straightforward method is to re-evaluate the global best position and the second-best particle's fitness to determine whether a change has occurred [85]. To address the problem of outdated memory caused by environmental shifts, various response strategies have been proposed. One common approach involves periodically replacing each particle's historical best position with its current position when a change is detected [16]. This allows particles to discard obsolete information and rely solely on updated knowledge of the new environment. Alternatively, re-randomizing a portion of the swarm has been suggested to maintain diversity and enhance the swarm's ability to track new optima.

Multi-swarm PSO, combined with charged particles, has also demonstrated strong potential [112]. In this approach, multiple swarms are distributed across different peaks in the search space. To prevent overlapping convergence, weaker swarms are randomized when two swarms come too close. Additionally, multi-swarm PSO replaces charged particles with quantum particles, whose positions are determined probabilistically around the swarm attractor. This approach is particularly effective in complex multimodal dynamic environments where multiple optima need to be tracked simultaneously. Expanding on this concept, a species-based PSO has been developed to identify and follow multiple peaks in dynamic scenarios [111, 149]. By incorporating a speciation algorithm [150], this method uses local "species seeds" to guide particles within a defined radius toward local optima. This approach encourages swarms to converge on multiple local solutions rather than a single global optimum, making it well-suited for multimodal optimization tasks in dynamic environments.

3.4 Key Components of Swarm Embodied AI

Swarm embodied AI promotes better and more holistic perception by enabling multiple agents to share complementary perceptual information with each other. This task can fundamentally overcome several long-standing issues [113, 114, 201] in single-agent perception, such as occlusion. The related methods and systems have tremendous potential in many applications, including multi-UAVs (unmanned aerial vehicles) for search and rescue [211, 239], multi-robot automation and mapping [179, 194], and vehicle-to-vehicle (V2V) and vehicle-to-everything (V2X) collaboration [110, 222]. The general collaborative perception system contains raw data, base network, feature, collaboration network and output. According to the data sharing and fusion stage, the collaborative perception stages can be broadly separated into early, intermediate, and late collaboration, as shown in Fig. 3.3:

Early Collaboration. Early collaboration [1, 22, 55] employs the raw data fusion at the input of the collaboration network, which is also known as data-level or low-level fusion. The ego agent receives and transforms the raw sensor data from other agents, and then aggregates transformed data onboard. Raw data contains the most comprehensive information and substantial description of agents. Consequently, early collaboration can fundamentally overcome occlusion and long-range problems

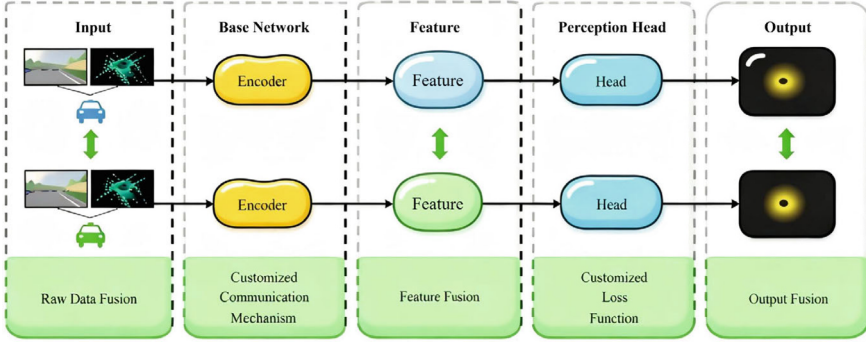


Fig. 3.3 Typical three collaboration stages

in individual perception and promote performance to the greatest extent. However, early collaboration relies on high data bandwidth, which makes it challenging to achieve real-time edge computing.

Intermediate Collaboration. Considering the high bandwidth of early collaboration, some works propose intermediate collaborative perception [25, 56, 66, 82, 88, 90, 100, 115, 132, 137, 171, 187, 198, 214, 219, 226–228, 233, 234, 246, 256] methods to balance the performance-bandwidth trade-off. In intermediate collaboration, other agents usually transfer deep semantic features to the ego agent. The ego agent fuses features to make the ultimate prediction. Intermediate collaboration has become the most popular multi-agent collaborative perception choice for flexibility. However, feature extraction often causes information loss and unnecessary information redundancy, which motivates people to explore suitable feature fusion strategies.

Late Collaboration. Late or object-level collaboration [23, 60, 134, 217, 232] employs the prediction fusion at the network output. Each agent trains the network individually and shares outputs with each other. The ego agent spatially transforms the outputs and merges all outputs after post-processing. Late collaboration is more bandwidth-economic and simpler than early and intermediate collaboration. However, the late collaboration also has limitations. Since individual output could be noisy and incomplete, late collaboration always has the worst perception performance.

Many collaborative perception methods have emerged recently. Since these methods are based on different frameworks and objectives, it's urgent to establish a systematic taxonomy to help readers understand this field. To this end, this section reviews recent collaborative perception approaches systematically. Specifically, we review collaborative perception from the aspect of methods for ideal scenarios and methods for real-world scenarios. The former pays attention to collaboration efficiency and performance, while the latter focuses more on collaboration robustness and safety.

In collaborative perception systems, researchers usually design corresponding collaboration modules at different network stages to improve collaborative efficiency and perception performance. Generally, three collaboration schemes require basic

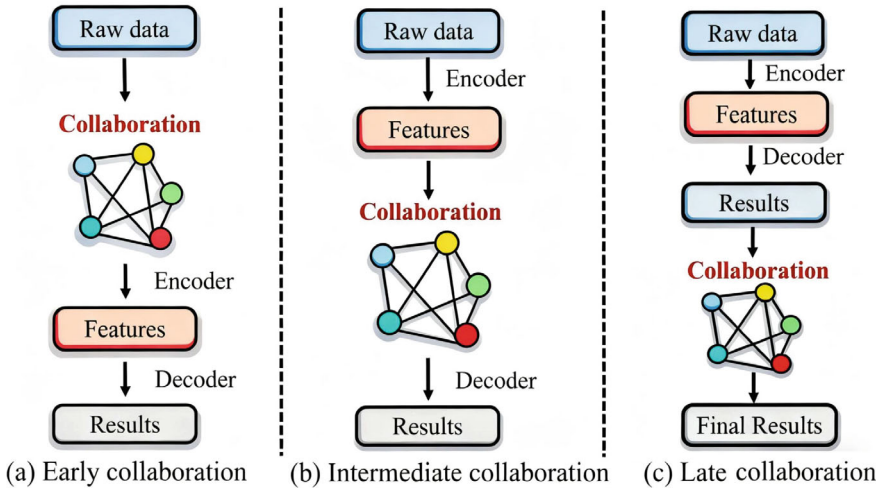


Fig. 3.4 Illustration of the common collaboration modules

collaboration modules to aggregate multi-agent observations, such as raw data fusion at the input stage (early collaboration), feature fusion at the feature processing stage (intermediate collaboration), and output fusion at the output stage (late collaboration). Furthermore, some works establish communication mechanisms in the base network stage to reduce transmission bandwidth, and introduce customized loss functions at the perception head stage to guide the network to capture helpful information. In this sub-section, we summarize and introduce the basic collaboration modules of each stage separately, as shown in Fig. 3.4.

3.4.1 Early Collaboration

Early collaboration adopts raw data fusion at the input stage. Since point clouds are irregular and can be aggregated directly, early collaboration works usually adopt point cloud fusion strategies. Agents need to reconstruct the received data because it was taken on different positions and angles. By exchanging LiDAR data, local environment can be reconstructed intuitively by merging point clouds into its physical positions. In order to reconstruct local environment by mapping point clouds into physical positions, additional information is encapsulated into the exchange package. Said package should be constituted from LiDAR sensor installation information and its GPS reading, which determines the center point position of every frame of point clouds. Agent's IMU (inertial measurement unit) reading is also required because it records the offset information of the agent during moving.

Problem Formulation. Consider N agents in the scene. Each agent represents a rotation whose yaw, pitch, and roll angles are α , β , and γ , respectively. A rotation

matrix R will be generated:

$$R = R_z(\alpha)R_y(\beta)R_x(\gamma), \quad (3.2)$$

$$R_z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.3)$$

$$R_y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix} \quad (3.4)$$

$$R_x(\gamma) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\gamma & -\sin\gamma \\ 0 & \sin\gamma & \cos\gamma \end{bmatrix} \quad (3.5)$$

where $R_z(\alpha)$, $R_y(\beta)$, $R_x(\gamma)$ are three basic rotation matrices rotate vectors by an angle on the z-, y-, x-axis in three dimensions.

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X_R \\ Y_R \\ Z_R \end{bmatrix} \cup \begin{bmatrix} X'_T \\ Y'_T \\ Z'_T \end{bmatrix} \quad (3.6)$$

$$\begin{bmatrix} X'_T \\ Y'_T \\ Z'_T \end{bmatrix} = R \times \begin{bmatrix} X_T \\ Y_T \\ Z_T \end{bmatrix} + \begin{bmatrix} \Delta d_{x_T} \\ \Delta d_{y_T} \\ \Delta d_{z_T} \end{bmatrix} \quad (3.7)$$

note, the X , Y , and Z in $[XYZ]'$ represents the 3D space value of each point in the LiDAR point cloud data, and $[X'_T Y'_T Z'_T]'$ is the transmitter's point cloud after applying the transform R to the translated coordinates of the transmitting agent. The transform is calculated by Eq. 3.2, using the IMU value difference between the transmitter and the receiver.

When connected agents exchange messages, cooperative perception produces a new frame by combining the transmitter and receiver's sensor data using Eq. 3.6, where we have the set of all coordinates equal to the coordinates of the receiver union with the coordinates from the transmitter. However, as the transmitting agent is in a different state than the receiver, we must apply a transform to the original coordinates to match the state of the receiving agent. To obtain the correct state for the transmitter's orientation, we use Eq. 3.2.

Outstanding Methods. The first early collaborative perception system Cooper [22] chooses LiDAR data as a fusion target. Point clouds can be compressed into smaller sizes by only extracting positional coordinates and reflection values. After interaction among agents, Cooper reconstructs the received point clouds with a transformation matrix, and then concatenates the ego point clouds set to make final predic-

tion. Inspired by Cooper, Coop3D [1] also explores early collaboration and introduces a new point cloud fusion method. Specifically, instead of employing concatenation, the Coop3D system utilizes the spatial transformation to fuse sensor data. Besides, unlike Cooper sharing agent-to-agent information on board, Coop3D proposes a central system to merge multiple sensor data, which allows the amortization of sensor and processing costs in collaboration.

3.4.2 Communication Mechanism

Raw data fusion in early collaboration broadens the horizons of the ego agent, which also causes high bandwidth pressure. To alleviate the above issues, more and more works [117, 199, 223] develop intermediate collaboration. The initial intermediate collaboration methods follow a greedy communication mechanism to obtain as much information as possible. Generally, they share information with all agents within the communication range and put the compressed full feature maps into collective perception messages (CPMs). However, since the feature sparsity and agent redundancy, the greedy communication may waste the bandwidth hugely. To fill this gap, some works establish dynamic communication mechanisms to select agents and features.

Problem Formulation. Consider N agents in the scene. Let $\mathcal{X}_i$ and $\mathcal{Y}_i$ be the observation and the perception supervision of the i th agent, respectively. The objective of collaborative perception is to achieve the maximized perception performance of all agents as a function of the total communication budget B and communication round K ; that is,

$$\begin{aligned} \xi_{\Phi}(B, K) = \arg \max_{\theta, \mathcal{P}} \sum_{i=1}^N g \left(\Phi_{\theta} \left(\mathcal{X}_i, \{\mathcal{P}_{i \rightarrow j}^{(K)}\}_{j=1}^N \right), \mathcal{Y}_i \right), \\ \text{s.t.} \quad \sum_{k=1}^K \sum_{i=1}^N |\mathcal{P}_{i \rightarrow j}^{(k)}| \leq B, \end{aligned} \quad (3.8)$$

where $g(\cdot, \cdot)$ is the perception evaluation metric, Φ is the perception network with trainable parameter θ , and $\mathcal{P}_{i \rightarrow j}^{(k)}$ is the message transmitted from the i th agent to the j th agent at the k th communication round.

Note that (i) when $B = K = 0$, there is no collaboration and $\xi_{\Phi}(0, 0)$ reflects the single-agent perception performance; (ii) through optimizing the communication strategy and the network parameter, collaborative perception should perform well consistently at any communication bandwidth or round; and (iii) For multi-round communication, each agent serves as both a supporter (offering message to help others) and a requester (requesting messages from others).

Outstanding Methods. Who2com [131] introduced the first communication mechanism designed to operate under bandwidth constraints, using a three-stage

handshake process. It employs a general attention function to calculate match scores among agents, allowing the selection of the most relevant agents, thereby reducing bandwidth usage. Building on this, When2com [130] utilizes scaled general attention to determine the optimal times for communication, enabling the ego agent to engage with others only when additional information is necessary, thus conserving collaboration resources.

Beyond selecting appropriate communication partners, the content of the communication is also crucial in minimizing bandwidth pressure. FPV-RCNN [235] was one of the first to propose a feature selection strategy, using a detection head to generate proposals and selecting feature points within these proposals. This keypoint selection module reduces redundancy in shared deep features and enhances the quality of the initial proposals.

Where2comm [87] introduced a novel spatial-confidence-aware communication mechanism that further optimizes communication efficiency. It uses a spatial confidence map to determine which features to share and with whom to communicate. During feature selection, Where2comm selects and transmits only spatial elements that meet high-confidence criteria and are requested by other agents. In agent selection, the ego agent communicates only with those who can provide the necessary features. By focusing on perceptually critical areas, Where2comm significantly reduces bandwidth usage and enhances collaboration efficiency.

3.4.3 *Intermediate Collaboration*

The feature fusion module plays a critical role in intermediate collaboration. Once the ego agent receives CPMs from other agents, it can employ various strategies to aggregate these features. An effective fusion strategy can identify hidden relationships between features and enhance the perception network's performance. Based on the underlying approach, feature fusion methods can be categorized into traditional, graph-based, and attention-based techniques.

A. Traditional Feature Fusion

In the initial phase of collaborative perception research, traditional strategies like concatenation, summation, and linear weighting were commonly used to fuse features. Intermediate collaboration typically employed these permutation-invariant operations on deep features, resulting in fast inference due to their simplicity.

Problem Formulation. Consider N agents in the scene. Let $\mathbf{O}_i$ be the feature map of the i th agent. When a receiver agent receives a feature map shared from a sender agent, it first measures all the new features that are contained in the feature map, by comparing them with the features in its own feature map. Based on the measurement, the receiver applies a weight, X , to the received feature map before fusion. Therefore, the fusion counteracts the negative effect caused by the weak features in the receiver's feature map.

$$\mathbf{F} = \{\mathbf{F}_3 \cup \max\{\mathbf{F}_1, \mathbf{F}_2 \times X\}\} \times Y, \quad (3.9)$$

$$X = \begin{cases} S/(A_o/A) & S < a, \\ S/(A_o/A) & a \leq S < b, \\ K & S \geq b \end{cases}. \quad (3.10)$$

$$S = \|\mathbf{F}_1^i - \mathbf{F}_2^i\| / (W \times H), \quad (3.11)$$

where $\mathbf{F}_1$ and $\mathbf{F}_2$ are the overlapping areas of the two to-be-fused feature maps from the receiver and the sender, respectively, $\mathbf{F}_3$ is the non-overlapping area of the receiver's feature map, and $\mathbf{F}$ is the resulting fused feature map. X is the assigned weight to the sender's feature map $\mathbf{F}_2$, and Y is the feature enhancement parameter. where $S = \|\mathbf{F}_1^i - \mathbf{F}_2^i\| / (W \times H)$ and $\|\mathbf{F}_1^i - \mathbf{F}_2^i\|$ is the L2 (or Euclidean) distance between two feature maps which are represented by two vectors $\mathbf{F}_1^i$ and $\mathbf{F}_2^i$. W and H are the width and height of the overlapping area of two feature maps. A_o is the size of the overlapping area, and A is the size of entire feature map. The constant numbers in the above equations, e.g., a , b , and k , are derived from intensive experiments.

Outstanding Methods. The first intermediate collaborative perception framework F-Cooper [21] extracts low-level voxel and deep spatial features. Based on these two level features, F-Cooper proposes two feature-fusion strategies: Voxel Feature Fusion (VFF) and Spatial Feature Fusion (SFF). Both employ element-wise maxout to fuse features in overlapped regions. Since voxel features are closer to raw data, VFF is as capable as the raw data fusion method for near object detection. In the meanwhile, SFF also has its advantages. Inspired by SENet [83], SFF opts to select partial channels to reduce transmission time consumption while keeping the comparable detection precision.

Considering F-Cooper [21] ignores the importance of low-confidence features, CoFF [69] is proposed to improve F-Cooper. CoFF weights the overlapped features by measuring their similarity and overlapping area. The smaller the similarity and the greater the distance, the more supplementary information provided by neighbor features intuitively. Besides, an enhancement parameter is added to increase the value of weak features. Experiments show that the simple yet efficient design makes CoFF improve F-Cooper much.

Although simple, traditional fusion has not been abandoned by recent methods. Collaborative camera-only 3D detection (CoCa3D [88]) is proposed to demonstrate the potential of collaboration in enhancing camera-based 3D detection. Since depth estimation is the bottleneck of camera-based 3D detection, CoCa3D contains collaborative depth estimation (Co-Depth) except for collaborative feature learning (Co-FL). In Co-Depth, neighbor agents transmit depth estimation with low uncertainty only, and the ego agent updates the depth estimation by considering single-view depth probability and multi-view consistency. In Co-FL, agents send feature elements with

high detection confidence and adopt a simple non-parametric point-wise maximum to fuse features. Experimental results indicate that CoCa3D helps the camera overtake LiDAR in 3D object detection.

B. Graph-Based Feature Fusion

Despite the simplicity of traditional intermediate fusion, they ignore the latent relationship among multi-agents and fail to reason the messages from sender to receiver. Graph Neural Networks (GNNs [160, 213, 250]) have the ability to propagate and aggregate messages from neighbors, and recent works have shown the effectiveness of GNN on perception and autonomous driving.

Problem Formulation. Let $\mathcal{X}_i$ be the 3D point cloud collected by the i th agent (M agents in total), the feature of the i th agent is obtained as $\mathbf{F}_i^s \leftarrow \Theta^s(\mathcal{X}_i)$, where $\Theta^s(\cdot)$ is the feature encoder shared by all the agents and the superscript s reflects the student mode. Let $\mathbf{B}_i \in \{0, 1\}^{K \times K \times C}$ be the BEV map of the i th agent associated with $\mathcal{X}_i$. The core component in the Graph-based feature fusion is a *collaboration graph* $\mathcal{G}_\Pi(\mathcal{V}, \mathcal{E}_\Pi)$, where $\mathcal{V}$ is the fixed node set and $\mathcal{E}_\Pi$ is the trainable edge set. Each node in $\mathcal{V}$ is an agent with the real-time pose information; for instance, $\xi_i \in \mathfrak{se}(3)$ is the i th agent’s pose in the global coordinate system; and each edge in $\mathcal{E}_\Pi$ is trainable and models the collaboration between two agents, with Π an edge-weight encoder, reflecting the trainable collaboration strength between agents. Let $\mathcal{M}_{\mathcal{G}_\Pi}(\cdot)$ be the collaboration process defined on the collaboration graph $\mathcal{G}_\Pi$. The feature maps of all the agents after collaboration are $\{\mathbf{H}_i^s\}_{i=1}^M \leftarrow \mathcal{M}_{\mathcal{G}_\Pi}(\{\mathbf{F}_i^s\}_{i=1}^M)$. This process has three stages: neural message transmission (S1), neural message attention (S2) and neural message aggregation (S3).

In the neural message transmission stage (S1), each agent transmits its BEV-based feature map (neural message) to the other agents. In the neural message attention stage (S2), each agent receives others’ neural messages and determines the matrix-valued edge weights ($\mathbf{W}_{j \rightarrow i}$), which reflect the importance of the neural message from one agent j to another i at each individual cell. Since each agent has its unique pose, leveraging the collaboration graph to achieve feature transformation across agents. In the neural message aggregation stage (S3), each agent aggregates the feature maps from all the agents based on the normalized matrix-valued edge weights. The updated feature map of the agent i is $\mathbf{H}_i^s = \sum_{j=1}^M \mathbf{W}_{j \rightarrow i} \odot \mathbf{F}_j^s$, where $\odot$ denotes the dot product with channel-wise broadcasting.

Outstanding Methods. V2VNet [199] was the first to utilize a spatial-aware graph neural network (GNN) for modeling communication among agents in collaborative perception. During the GNN message passing stage, V2VNet compresses features using a variational image compression algorithm. For cross-vehicle aggregation, the framework compensates for time delays to establish an initial state for each node, then warps and spatially transforms the compressed features from neighboring agents to the ego agent. These operations are conducted within overlapping fields of view. In the feature fusion stage, V2VNet employs an averaging operation to aggregate features and updates the node state using a convolutional gated recurrent unit (ConvGRU).

While V2VNet [199] demonstrated performance improvements with GNN, it uses scalar-valued collaboration weights, which are insufficient for capturing the

importance of different spatial regions. To address this limitation, DiscoNet [117] introduces matrix-valued edge weights to better capture inter-agent attention in high resolution. During message passing, DiscoNet concatenates features and applies matrix-valued edge weights to each element in the feature maps. Additionally, DiscoNet combines early and intermediate fusion techniques using a teacher-student framework, further enhancing performance.

MP-Pose [252] is another generalized GNN-based perception framework. During the message passing stage, MP-Pose encodes the relative spatial relationship with a spatial encoding network rather than warps features directly. Inspired by Graph Attention Networks (GAT [11, 79, 186]), it further uses a dynamic cross-attention encoding network [125, 204] to capture the relationship among agents and aggregate multiple features like GAT.

C. Attention-Based Feature Fusion

In addition to graph learning, attention mechanisms [185] have emerged as a powerful tool for exploring feature relationships [71]. Attention mechanisms can be classified according to the data domain into channel attention [151, 197], spatial attention [27, 68, 254], and channel & spatial attention [129]. Over the past decade, attention mechanisms have played an increasingly important role in computer vision and have inspired collaborative perception research [14, 43, 83]. Since feature selection and relationship exploring are vital issues in intermediate collaborative perception, some works have leveraged attention mechanisms to develop more dynamic and robust feature fusion strategies. Due to flexibility, attention-based designs have dominated intermediate collaboration.

Problem Formulation. Every agent $i \in \{1 \dots N\}$ within the communication networks shares metadata such as poses, extrinsics, and agent type $c_i \in \{I, V\}$ with each other. Selecting one of the connected AVs as the ego agent (e) to construct a V2X graph. Each agent i can receive ego pose $x_e^{t_i}$ at the time t_i . The PointPillar backbone extracts informative feature maps $\mathbf{F}_i^{t_i} \in \mathbb{R}^{H \times W \times C}$, denoting agent i 's feature at time t_i with height H , width W , and channels C . To reduce the required transmission bandwidth, utilizing a series of 1×1 convolutions to progressively compress the feature maps along the channel dimension. The compressed features with the size (H, W, C') (where $C' \ll C$) are then transmitted to the ego agent (e), on which the features are projected back to (H, W, C) using 1×1 convolutions. The intermediate features $\mathbf{H}_i = \Gamma_{\xi}(\mathbf{F}_i^{t_i}) \in \mathbb{R}^{H \times W \times C}$ aggregated from connected agents are fed into the major component, by conducting an iterative inter-agent and intra-agent feature fusion using self-attention mechanisms.

Outstanding Methods. To capture the interaction among the specific area in the feature maps, AttFusion [224] first employ a self-attention operation at the exact spatial location. Specifically, AttFusion introduces a single-head self-attention fusion module and achieves a balance between performance and inference speed compared to the traditional method F-Cooper [21] and graph-based method DiscoNet [117]. The spatial-aware interaction in AttFusion is similar to the matrix-weight edge in DiscoNet [117] while implemented with different tools.

In addition to traditional attention-based approaches, Transformer-based methods have significantly influenced collaborative perception. COOPERNAUT [31], inspired by Point Transformer [70, 146, 247], introduces a self-attention network specifically designed for point cloud processing in collaborative settings. Upon receiving messages, the ego agent uses a downsampling block to reduce point set cardinality and a point transformer block to enable local information exchange among points. These operations maintain the permutation invariance of the messages. Importantly, COOPERNAUT integrates collaborative perception with control decisions, advancing the module linkage crucial for autonomous driving.

V2X-ViT [223] offers the first unified Transformer architecture that simultaneously addresses vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) interactions. It introduces the heterogeneous multi-agent attention (HMSA) module to model the distinct relationships between V2V and V2I agents. Additionally, V2X-ViT employs a multi-scale window attention module (MSwin) to capture long-range spatial interactions necessary for high-resolution detection.

Recognizing the cost-effectiveness of RGB cameras compared to LiDAR, CoBEVT [220] presents a camera-based collaborative perception framework. CoBEVT's fused axial attention (FAX) module facilitates interactions among multiple views and agents by performing sparse global interactions and local window-based attention. Experiments confirm CoBEVT's effectiveness in multi-view, multi-agent scenarios, with resilience to camera loss.

VIMI [202] introduces the Vehicles-Infrastructure Multi-View Intermediate Fusion method, focusing on camera-based collaborative perception. To explore correlations between features from vehicles and infrastructure at various scales, VIMI employs a multi-scale cross-attention (MCA) module. This module extracts multi-scale features using deformable CNNs and generates attention weights through cross-attention. Additionally, VIMI's camera-aware channel masking (CCM) strategy corrects calibration errors and enhances features by re-weighting them based on camera parameters. Previous studies have primarily focused on homogeneous sensor collaboration scenarios.

While previous studies have concentrated on homogeneous sensor collaboration, the exploration of collaborative perception with heterogeneous sensors was lacking. HM-ViT [214] addresses this gap by introducing the first unified hetero-modal V2V cooperative perception framework. HM-ViT includes a heterogeneous 3D graph attention (H3GAT) mechanism that fuses bird's eye view (BEV) features from different sensor types across agents. H3GAT uses type-dependent nodes and edges to capture sensor heterogeneity and reason about spatial interactions and cross-agent relationships. It also incorporates local window-based attention and sparse global grid-based attention to capture both local and global cues. Experimental results demonstrate HM-ViT's superiority in both hetero-modal and homo-modal collaborative perception.

3.4.4 Loss Function

Communication broadens the perceptual range of the ego agent, but it also brings challenges due to redundant and uncertain shared data. To overcome these issues, it's crucial to provide precise guidance and develop customized loss functions for effective model training.

Problem Formulation. X represents the point cloud data. For the deterministic collaborative object detection, the agent $\alpha \in \{1, \dots, N\}$ creates a local binary *bev* map $\mathcal{M} \in \{0, 1\}^{W_m \times L_m \times H_m}$ from X , where N is the number of agents, and W_m , L_m , and H_m indicate the width, length, and height of the local *bev* map, respectively. Considering homogeneous collaborative object detection, in which multiple agents share the same detector f_θ which consists of an encoder denoted by $\mathcal{E}$, and an aggregator as well as a decoder that is collectively represented by $\mathcal{D}$, where θ denotes the parameters of f . $\mathcal{E}$ compresses the *bev* map into an intermediate feature map $\mathbf{F}_\alpha \in \mathbb{R}^{\frac{W_m}{K_m} \times \frac{L_m}{K_m} \times F_m} = \mathcal{E}(\mathcal{M})$, where K_m is the spatial downsampling scale, and F_m is the feature dimension. Afterward, $\mathbf{F}_\alpha$ is sent to other agents, and the agent α will use $\mathcal{D}$ to generate a set of bounding boxes $\hat{Y} = \mathcal{D}(\mathbf{F}_\alpha, \{\mathbf{F}_\beta\}_{\beta \neq \alpha})$.

In each point cloud data X , there are J objects. For each object $j \in \{1, \dots, J\}$, predicting I corners of the bounding box. Each corner $i \in \{1, \dots, I\}$ is represented by a D -dimensional vector in the *bev* map. The set of ground-truth bounding boxes Y is represented as $Y = \{c_j, \{y_{ij}\}_{i=1}^I\}_{j=1}^J$ where c is the classification label and $y_{ij} \in \mathbb{R}^D, \forall (i, j)$. The set of predicted bounding box $\hat{Y}$ is represented as $\hat{Y} = \{\hat{p}_j, \{\hat{y}_{ij}, \hat{\Sigma}_{ij}\}_{i=1}^I\}_{j=1}^J$, where $\hat{p}$ is the predicted classification probability, and each corner of the bounding box is modeled as a multivariate Gaussian distribution with $\hat{y}_{ij} \in \mathbb{R}^D$ as the predicted mean and $\hat{\Sigma}_{ij}$ as the $D \times D$ covariance matrix coordinate of corner i of object j . The probability distribution of each corner of the bounding box is independent. During training, the neural network parameters of the encoder $\mathcal{E}$, aggregator and decoder $\mathcal{D}$ are jointly learned by minimizing the detection loss $\mathcal{L}_{det}(Y, \hat{Y})$ that includes a classification loss and a regression loss that considers both prediction accuracy and uncertainty.

Outstanding Methods. In collaborative environments, the ego agent frequently encounters overlapping data from surrounding agents. To optimize the use of this shared information, CRCNet [135], a network that enhances collaboration by focusing on complementarity and reducing redundancy. CRCNet features two key modules: the complementarity enhancement module, which leverages contrastive learning to increase information gain, and the redundancy minimization module, which utilizes mutual information to promote effective feature fusion. These modules allow CRCNet to efficiently incorporate complementary data from neighboring agents, improving overall feature integration.

Collaborative information can also suffer from perceptual uncertainty due to perception errors and sensor noise, beyond just redundancy. To tackle this issue, an innovative method [173] that employs a tailored moving block bootstrap technique to assess both model and data uncertainty. Additionally, they develop a specialized

loss function aimed at directly accounting for data uncertainty. Their experiments demonstrate that this approach effectively mitigates uncertainty and boosts accuracy in different collaborative scenarios.

3.4.5 Late Collaboration

Late collaboration is a widely used technique in V2X collaborative perception, where the perception results from different agents are combined at the decision level. In this approach, sensor outputs from individual agents are processed independently, and the final perception is formed by merging these separate results. While early fusion offers the advantage of contextual information integration, it demands substantial transmission bandwidth. Late fusion, on the other hand, allows the incorporation of outputs from different models without requiring additional training and offers the flexibility to integrate data from various modalities. However, late fusion presents challenges such as the need for precise alignment of individual outputs and the potential for increased complexity in the fusion process. Additionally, it is prone to error accumulation, which can impact the overall accuracy of the collaborative perception system

Problem Formulation. Assuming there are two sets of input, the observation set ($N = n$) and target set ($N = m$), where $n + m$ equals the total number of pixels in the image. The observation set is $O = \{(x_i, y_i)\}_{i=1}^n$, where x_i represents the context feature vector and y_i represents the depth feature generated by the feature extraction network. The target set is $T = \{x_i\}_{i=n+1}^{n+m}$. A mapping function is $f : X \rightarrow Y$ and let P_θ be a probability distribution over the function f . For $f \sim P_\theta$, we set $\hat{y} = f(x_i)$, where $\hat{y}$ is the ground-truth depth for each pixel. So P_θ defines a joint probability distribution over $\{x_i\}_{i=n+1}^{n+m}$ and the depth completion problem is formulated as a Gaussian Process (GP) as follows:

$$P_\theta(f(T)|O, T), \quad (3.12)$$

where θ represents all parameters of the inductive fusion block, which defines the distribution P . As a result, the prediction is permutation-invariant and has the following property, which fits the assumption of Conditional Neural Process (CNP):

$$P_\theta(f(T)|O, T) = P_\theta(f(T')|O, T') = P_\theta(f(T)|O', T), \quad (3.13)$$

where O' and T' are permutations of O and T . So, we can apply the following equations from CNPs:

$$r_i = h_\theta(x_i, y_i), \quad \forall (x_i, y_i) \in O, \quad (3.14)$$

$$r_j = \frac{1}{T} \sum_{i=1}^T r_i, \quad (3.15)$$

$$\phi_i = g_\theta(x_i, r), \quad \forall(x_i) \in T. \quad (3.16)$$

Therefore, late fusion does not need to consider the encoding modes of different sensors, and only needs to achieve alignment at the output end.

Outstanding Methods. Late collaboration usually adopts fusion operation at the post-processing [139, 169, 236] stage, which merges multi-agent perception outputs. For example, late collaboration for 3D object detection usually leverages post-processing methods such as Non-Maximum Suppression (NMS) [54, 61, 191] to remove redundant and low-confidence prediction. However, late fusion always faces challenges such as spatial and temporal misalignment.

3.5 Operational Robustness of Swarm Embodied AI

Most studies on collaborative perception [135, 187, 226, 227] have focused on enhancing efficiency and accuracy, often assuming ideal conditions. However, in real-world collaborative perception scenarios, challenges such as (1) localization inaccuracies, (2) communication delays and interruptions, (3) inconsistencies between models or tasks, and (4) privacy and security threats, as shown in Fig. 3.5, can compromise the effectiveness of these systems. These issues may cause collaborative perception to underperform compared to individual perception, posing serious safety risks. Therefore, it is crucial to ensure that collaborative perception remains robust and secure. In this sub-section, we delve into advanced techniques developed to tackle these challenges.

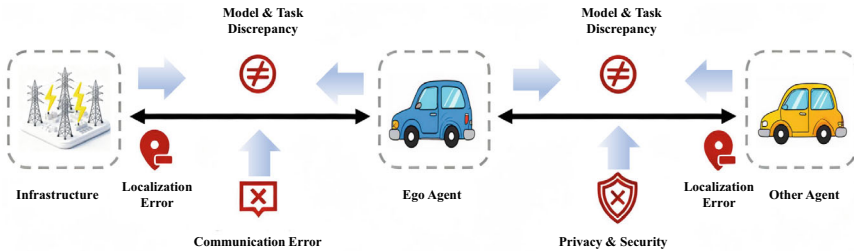


Fig. 3.5 The real-world challenges in collaborative perception scenarios

3.5.1 Localization Error

Collaborative perception methods use spatial transformations to align and integrate data, features, or outputs. However, problems such as GPS localization errors and asynchronous sensor measurements can lead to alignment issues, impacting overall performance. For instance, V2VNet [199] is notably sensitive to pose noise. Recent approaches have focused on addressing these challenges with various pose consistency techniques.

A method [184] based on V2VNet [199] was proposed, which improve V2VNet's accuracy by integrating learnable neural reasoning layers. Their method includes a pose regression module that corrects noisy transformations and a consistency module using Markov random fields (MRF) to achieve a globally consistent pose across all agents. FPV-RCNN [235] addresses localization errors with a correction module that detects keypoints on structures like poles and fences. It uses a maximum consensus algorithm to align agent centers with these keypoints, estimating pose errors more accurately and outperforming traditional BEV-based methods in scenarios with localization inaccuracies.

Late fusion methods, which often employ straightforward integration techniques, are more prone to localization errors. To enhance robustness, OptiMatch [172] was developed, a system based on optimal transport theory that refines object-level data through precise object matching, improving accuracy even with significant localization and heading noise. Likewise, CoAlign [134] was introduced, a hybrid framework that estimates accurate poses using a pose graph of agents and objects. CoAlign clusters objects based on uncertainty estimates and samples poses from these clusters, optimizing pose consistency without supervision, and enhancing performance across different noise levels.

3.5.2 Communication Issues

Collaborative perception in real-world relies on wireless communication among agents, which can face issues like latency [72, 121], interruptions [152, 157], and information loss [143] that impact collaboration. Recent research has focused on various strategies to address these challenges:

To manage latency in late fusion scenarios, the Time Compensation Late Fusion (TCLF [232]) framework was proposed. TCLF leverages tracking and state estimation to predict current conditions based on previous frames. By aligning predictions from adjacent frames and interpolating object positions, TCLF reduces latency effects and integrates these predictions with ego vehicle data.

For intermediate fusion, V2X-ViT [104] tackles latency with an adaptive delay-aware positional encoding (DPE) that aligns features over time. The framework also incorporates HMSA and MSwin modules to correct for misalignments caused by localization errors and delays. Experiments demonstrate that DPE enhances perfor-

mance despite varying time delays. SyncNet[104] is designed to address latency through feature-level synchronization. Its core module, the feature-attention symbiotic estimation (FASE), uses both real-time and historical data to estimate and refine features and attention, with time modulation adapting feature fusion according to latency. A method [152] was addressed communication interruptions with the V2X-INCOP framework, which utilizes historical data to recover lost features. The framework includes a spatial attention mask to reduce background noise and employs curriculum learning to stabilize training.

To handle packet loss, often caused by obstacles or fast-moving vehicles, the LC-aware Repair Network (LCRN [110]) was proposed. This network, inspired by image-denoising techniques, uses an encoder-decoder architecture to recover features lost during transmission. LCRN also features an attention-based fusion method to combine features from various agents, improving robustness against communication loss.

3.5.3 *Model and Task Discrepancies*

Existing multi-agent collaborative perception methods often assume that all agents use identical models and are trained for specific tasks, leading to model-specific and task-specific learning. However, in real-world applications, it is impractical for all agents to have homogeneous models, and task-specific training can limit the scalability of collaborative perception systems. To address these challenges, some studies [89, 203, 226] have proposed innovative approaches to mitigate the effects of model and task discrepancies.

When agents are equipped with different perception models, existing collaboration methods may produce unreliable fusion results due to model heterogeneity. To counter this, a model-agnostic collaborative perception framework [217] was introduced. This approach uses an offline calibrator to align the confidence scores of agents with their empirical accuracy, recognizing that there is a distribution of confidence among agents. Additionally, this method proposes the Promotion-Suppression Aggregation (PSA) module to enhance spatial correlation by identifying promotion proposals. With the combination of offline confidence calibration and online proposal aggregation, this late collaboration method demonstrates robust performance even with model and parameter discrepancies.

Model discrepancies can also cause domain gaps between intermediate features, which can degrade the effectiveness of collaboration. MPDA [219] addresses this issue by developing the Multi-agent Perception Domain Adaptation framework, the first of its kind to tackle domain gaps in intermediate collaboration. MPDA includes a Learnable Resizer to adjust received features to a target size and employs a sparse cross-domain transformer to generate domain-invariant features by adversarially fooling a domain classifier. Experimental results show that MPDA effectively bridges the feature domain gap in the presence of model discrepancies.

Task-specific training often results in features that are too specialized for a particular task, limiting their generalizability in the multi-task environment. To overcome this limitation, a novel self-supervised learning task called multi-robot scene completion [118] was proposed. This task enables each agent to independently reconstruct a scene, thereby learning latent features that are general and robust across multiple tasks. They designed a spatiotemporal autoencoder (STAR) module to balance scene reconstruction performance with communication efficiency, allowing the model to learn more versatile representations suited to a variety of tasks in collaborative perception.

3.5.4 *Privacy and Security*

Beyond the challenges in collaborative perception, data privacy and model security are paramount concerns. Collaborative training can expose sensitive information, and models may be susceptible to attacks during data transmission between agents. This section highlights research efforts focused on these issues, including privacy-preserving methods [4, 46] and defense strategies against attacks [78].

Sharing sensor-captured data in collaborative perception poses significant privacy risks, making it crucial to develop privacy-preserving techniques for connected and automated vehicles (CAVs). Various approaches have been proposed, such as the privacy-preserving convolutional neural network (P-CNN [215]), the edge-cooperative privacy-preserving point cloud object detection framework (SecPCV [7]), and the privacy-preserving object detection framework (P2OD [6]). These methods employ security technologies like additive secret sharing (ASS) and chaotic encryption to protect shared data. However, these privacy-preserving techniques have primarily been tested on individual perception datasets [59], and further validation on collaborative perception datasets is necessary.

Collaborative perception also relies on communication between agents, which can be targeted by malicious actors, rendering networks vulnerable to adversarial attacks. Strengthening the security of collaborative perception involves improving adversarial robustness. To date, only one study [183] has specifically investigated adversarial attacks in this context. This method evaluated both the attack and defense performance of V2VNet [199], revealing that joint perturbations by attackers result in more potent attacks. On the defense side, adversarial training proved effective when the attack model was known, and the study also found that increasing the number of collaborating agents enhances the system's defense capabilities.

3.6 Autonomous Swarm Embodied AI

This section explores the core components of Autonomous Embodied AI, focusing on autonomous collaboration, communication, and robustness optimization. The capacity for collaboration is a defining characteristic of swarm AI, allowing intelligence to emerge from the collective actions of many embodied entities. Three paradigms explain this phenomenon: self-organization, where simple, local interactions generate complex group behaviors without a central controller; negotiation, which enables agents to use explicit reasoning to exchange proposals, resolve conflicts, and allocate tasks; and reflection, which transforms the swarm into a system that learns from its history by reviewing past outcomes and adjusting strategies. These collaborative mechanisms are underpinned by autonomous communication, which can also be understood through these three paradigms. Self-organization-based communication relies on indirect environmental cues, where one agent's actions affect another's perception, a process known as stigmergy. Negotiation-based communication involves a more direct, goal-focused exchange of messages and replies, allowing agents to share intentions and reach a group consensus. Reflection-based communication, the most advanced form, enables agents to share and update complex internal models of each other's thought processes, fostering a shared mental model for deep teamwork. The final crucial element is autonomous robustness optimization, a proactive, self-healing framework. This includes self-recovery optimization, where the swarm continuously monitors its health to identify and address failures, and self-reconfiguration optimization, which allows the swarm to dynamically change its physical or logical structure to adapt to new conditions or internal failures. By integrating these paradigms, swarm AI systems can maintain resilience, adaptability, and continuous self-improvement in dynamic environments.

3.6.1 *Autonomous Collaboration*

The capacity for autonomous collaboration has become a defining characteristic of swarm embodied AI. When intelligence is distributed across many embodied entities, it no longer depends on the limitations of a single agent. Instead, it emerges from collective perception, action, and reasoning processes. Collaboration provides stability and flexibility in environments that are uncertain, changing, and only partially observable. This section outlines three dominant paradigms that describe how autonomous swarms achieve coordination. These are self-organization-based collaboration [10], negotiation-based collaboration [42], and reflection-based collaboration [159]. Each paradigm highlights a distinct mechanism that enables distributed agents to maintain collective purpose, resolve conflicts, and adapt over time.

A. Self-Organization-Based Collaboration

Self-organization-based collaboration [142] describes how ordered group behavior appears when individual agents repeatedly interact under simple local rules. No

supervisory controller is needed. Instead, patterns of coordination emerge through local adjustments, feedback mechanisms, and adaptation to external conditions. This principle allows swarms to produce outcomes that cannot be explained by isolated behaviors alone.

Agents typically rely on neighbor awareness, rule-based responses, and reinforcement of effective actions [58, 145]. Through repeated interactions, these rules generate large-scale patterns that appear stable even under noisy sensing or environmental disruptions [49, 50]. The resilience of self-organized groups is one of the primary reasons why this paradigm has been widely adopted in embodied swarm systems. The number of agents strongly influences the properties of a swarm. Adding new members often increases overall performance rather than diluting it [12]. This is because additional agents contribute not only their own abilities but also strengthen the interactions that support coordination. Such scaling behavior explains the effectiveness of self-organization in distributed exploration, large-area monitoring, and resource management tasks.

Mathematical perspectives on self-organization often rely on nonlinear system analysis [29] and graph connectivity models [91]. Stability is explained through feedback loops that amplify successful actions and suppress ineffective ones. Communication delays or stochastic variations may destabilize local interactions, but they can also create new patterns that stabilize at the global scale. Practical implementation in embodied swarms relies on decentralized controllers [188]. These controllers allow agents to manage spacing, coverage, and motion alignment autonomously. Consensus strategies or artificial potential fields are common methods to preserve global cohesion. Some agents may specialize in particular behaviors when reinforcement mechanisms favor differentiation, which helps the group balance between exploration and focused exploitation.

Future challenges involve adapting self-organization principles to physical constraints such as limited sensing [161], finite energy [73], and heterogeneous bodies [168]. Unlike abstract models, embodied systems require designs that account for mechanical feasibility and hardware restrictions. This connection between embodiment and self-organization underlines why swarm embodied artificial intelligence must be studied as a separate research area.

B. Negotiation-Based Collaboration

Self-organization creates order without planning, but negotiation-based collaboration introduces explicit reasoning and alignment [92, 101]. In many tasks, autonomous entities have incomplete knowledge and possibly conflicting interests. To maintain effective coordination, they exchange proposals, share partial evaluations, and establish agreements. This negotiation process enables a swarm to remain coherent when emergent mechanisms alone would be insufficient.

Negotiation builds on principles of multi-agent decision theory [207] and game-theoretic reasoning [156]. Each participant is modeled as an independent decision-maker with limited capacity and individual objectives. Negotiation protocols reconcile these objectives into collective strategies. The protocols often involve structured communication, evaluation of utilities, and iterative convergence toward shared out-

comes. Conflicts frequently occur in embodied swarms [26, 245]. Agents may target overlapping spatial areas, attempt incompatible maneuvers, or compete for scarce resources. Negotiation provides a structured way to surface these conflicts, exchange priorities, and determine outcomes that benefit the group as a whole. This deliberate resolution process complements the resilience achieved by self-organization.

Task allocation is a major application of negotiation [3, 33]. When tasks differ in complexity or urgency, bargaining mechanisms allow agents to distribute responsibilities according to their capabilities. Auction systems and contract-net frameworks are often used to implement these negotiations, ensuring that resources and roles are assigned dynamically and efficiently. Negotiation systems [103] require communication protocols and convergence mechanisms. Agents must balance their own local objectives against global constraints when evaluating options. Robust convergence ensures stability of outcomes even when messages are delayed or lost.

Since agents are embodied, negotiation mechanisms must be efficient and robust under hardware limitations [225]. Communication can be unreliable, bandwidth is often limited, and computational resources are constrained. Therefore negotiation protocols must minimize overhead while retaining enough structure to be effective in large and dynamic groups.

C. Reflection-Based Collaboration

Reflection-based collaboration [28, 52, 155] extends the possibilities of self-organization and negotiation by introducing collective evaluation and long-term adaptation. Agents are not limited to immediate action. They can review past interactions, analyze outcomes, and adjust strategies accordingly. Reflection transforms a swarm into a system that can learn from its own history. Reflection involves storing and interpreting outcomes and embedding them in distributed memory [205]. It closes the loop between action and adaptation by enabling swarms to revise rules and strategies based on performance. This leads to improvement over time and increases the ability of swarms to cope with change. Machine learning [53] and meta-learning [164] are frequently used to realize reflection. Reinforcement learning frameworks allow policies to be updated through accumulated rewards and penalties. Meta-learning adds the ability to adapt rapidly across tasks by encoding strategies for learning itself.

Distributed memory [17, 230] is essential in reflective systems. If experiences remain local, the benefits of reflection are limited. Shared repositories and consensus methods allow the integration of local histories into a collective memory. This shared knowledge supports prediction, anticipation of environmental changes, and proactive strategy refinement. Reflection also supports safety and ethical considerations. By analyzing outcomes, swarms can recognize when their behavior conflicts with mission constraints or safety norms. Reflection [206] therefore provides a pathway for correction and ensures trustworthiness of autonomous operation.

Embodiment plays a crucial role in reflective processes. Agents not only evaluate decision outcomes but also measure sensor performance, energy use, and physical stress [148]. Incorporating these dimensions ensures strategies are both effective and sustainable. Research challenges include designing distributed reflective architec-

tures that scale with large swarms, formalizing metrics that capture improvement over extended horizons, and integrating meta-learning at the collective level [105]. Progress in these directions would allow reflection-based collaboration to become a central mechanism of self-evolving swarm intelligence.

In conclusion, autonomous collaboration in swarm embodied artificial intelligence can be understood through three perspectives. Self-organization allows spontaneous group coordination to arise from local interaction. Negotiation provides structured exchanges that align interests and allocate tasks. Reflection introduces learning across time, embedding evaluation and adaptation into the collective. Together, these paradigms explain how embodied swarms maintain resilience, adaptability, and self-improvement in dynamic environments.

3.6.2 *Autonomous Communication*

For autonomous swarm-based artificial intelligence systems to work and be truly useful, communication is a must. The ability for individual agents to share information, either directly through signals or indirectly by changing their environment, is critical for the group to achieve goals that a single agent couldn't manage alone. This communication system must be autonomous, allowing the swarm to deal with complex, ever-changing situations and new problems without needing a central control unit or outside help. This section looks at the design and function of autonomous communication by dividing it into three main types: based on self-organization, negotiation, and reflection. Each type handles different challenges and creates different levels of group intelligence, from simple, uncoordinated actions to complex, goal-oriented strategies. Combining these different communication methods effectively is key to creating strong, scalable, and truly intelligent swarm systems. The ideas behind these mechanisms come from many fields, including biology, computer science, and social sciences.

A. Self-Organization-Based Communication

Self-organization-based communication is a decentralized, bottom-up approach [32] where complex group behaviors emerge from simple, local interactions between agents and their surroundings. This isn't about agents sending direct messages to each other. Instead, it's about one agent's actions indirectly affecting what another agent sees or perceives [40]. A central idea here is stigmergy, where agents communicate by changing their local environment, leaving clues that other agents can notice. These changes can be like chemical trails, physical markers, or even minor shifts in the local energy. This communication is naturally asynchronous and indirect, using the environment as a shared channel for information [41]. A big advantage of this model is how easily it can scale and how resilient it is. The communication load doesn't explode as the swarm gets bigger, because agents only interact with their immediate neighbors. It's also very robust against the failure of a single agent, since

there's no single point of failure and the overall behavior is a result of countless local interactions.

Another part of self-organization is using local information to figure out the global state of the system. Individual agents make decisions based on what they see nearby, and when you put all these decisions together, a global pattern or consensus can form [192]. This includes things like distributed sorting and aggregation algorithms, where agents move or change their state based on their neighbors' properties. The communication here doesn't involve exchanging complex data; it's about simple signals like attracting, repelling, or status indicators [237]. For example, in a search for food, an agent finding a food source might release a pheromone-like signal that influences where other agents go next [37]. This signal might fade over time, which naturally makes more recent information a higher priority. The group's response is an emergent behavior [35, 38], not a pre-planned command, and its effectiveness comes from the shared information processing that happens across many small interactions.

The main drawback of this method is that it can't express much. Simple signals can only carry a limited amount of information, which makes it hard to achieve very precise or complex goals [251]. The behavior that emerges is often just an approximation of the best solution, and it can be slow to converge or get stuck in sub-optimal states. Plus, since agents don't share their identities or states explicitly, it's hard to form stable sub-groups or coordinate tasks that require specific roles or steps [64]. Despite these limits, for tasks like exploration, making decentralized patterns, or basic foraging, self-organization offers a simple, scalable, and tough communication system that mirrors what we see in biology [36]. Current research in this area is focused on putting more information into these environmental signals and creating local rules that can lead to more advanced forms of group intelligence.

B. Negotiation-Based Communication

Negotiation-based communication is a more organized and goal-focused way for autonomous swarms to exchange information. Unlike self-organization, this method uses specific communication rules where agents interact directly to solve problems, share resources, or work together on a common goal [30]. Each agent has its own goals and knows its own state, and it can also model the possible states and intentions of other agents [108]. The communication process is a series of messages and replies, where agents suggest ideas, counter with new ones, and make commitments until they reach an agreement [124]. This can be formally described using game theory, where agents are seen as smart decision-makers trying to get the most for themselves while also helping the group as a whole [181]. Here, communication isn't just about sending data; it's about sharing intentions, preferences, and reaching a group consensus.

One major use of negotiation is for assigning tasks. When a new task appears, agents can use a distributed auction system where they bid on the task based on their current work, abilities, and how close they are to the task's location [170]. The agent with the best bid gets the task, and this information is sent to the rest of the swarm [75]. This makes sure the best-suited agent handles the task, which helps the whole system perform better. Negotiation can also be used to avoid collisions and plan paths in crowded spaces [147]. Agents can negotiate for the right of way, suggesting

different paths to avoid crashing and keep things moving smoothly [175]. This type of communication is usually real-time and involves a lot of direct state-sharing and advanced thinking.

How well negotiation-based communication works depends on the communication rules and the agents' own thinking abilities [138]. The messages need to be detailed enough to carry complex information, and the agents must have enough computing power to process these messages and make smart decisions [108]. The challenges include potential communication bottlenecks in large swarms, the risk of agents acting badly or not following the rules, and the high computing cost of running complex negotiation algorithms [76]. Also, relying on a fixed set of rules can make the system less flexible in new or unexpected situations where the rules aren't clear [123]. But for tasks that need high accuracy, accountability, and planned coordination, negotiation offers a powerful way to move from simple group behavior to deliberate, organized action [19]. Research in this area is focused on creating more efficient, reliable, and secure negotiation rules that can handle more agents and more complex tasks.

C. Reflection-Based Communication

Reflection-based communication is the most advanced and cognitively demanding type of information exchange in autonomous swarms. It goes beyond the direct message passing of negotiation and the indirect signaling of self-organization. This is a system where agents can look at their own internal state and also build and update models of other agents' internal states and thought processes [193]. Communication isn't just about sharing data or agreeing on something; it's about sharing and improving these internal models [242]. An agent might communicate how sure it is about a task, what it plans to do next, or why it made a certain decision in the past [208]. This ability allows for a much deeper level of teamwork, where agents can predict what others need and act accordingly, instead of just reacting to what's happening.

A key part of this communication is the idea of a shared mental model or a collective understanding of the environment and the task [216]. This shared model isn't just a collection of facts; it's a shared understanding of how those facts relate, what the swarm's overall goals are, and what each agent's role is [107, 241]. When a new agent joins the swarm or an old one fails, the other agents can use reflection to quickly update their mental models and reassign tasks efficiently [243]. This is especially useful in dynamic and unpredictable environments where things might change quickly and make old plans useless [80, 209]. This type of communication happens at a higher level, where agents think about and improve the very structure of their teamwork.

The technical challenges with reflection-based communication are significant. Each agent needs a lot of computing power to keep its internal models accurate and up-to-date [86, 93]. The communication load can also be huge, since agents have to exchange not just data, but complex representations of their beliefs and plans [182, 238]. Making sure these shared mental models are consistent and correct across a large swarm is a huge challenge [38, 65]. If there's a misunderstanding or incorrect information spreads, it could cause major system failures. Still, the

benefits are equally large [94]. Reflection-based communication allows a swarm to be flexible, adaptable, and resilient in ways that simpler communication methods can't match [210]. It helps with things like leadership emerging, forming specialized sub-groups, and handling complex, long-term tasks that require a deep understanding of the problem and the group's abilities [180]. Future research in this field will focus on developing efficient ways to represent and transfer knowledge, as well as methods to ensure the reliability and integrity of these shared mental models in a decentralized system.

3.6.3 Autonomous Robustness Optimization

Autonomous robustness optimization is an essential discipline for guaranteeing the long-term utility and endurance of autonomous swarm systems. Fundamentally, this paradigm represents a shift away from traditional, reactive fault tolerance, instead embracing a proactive, self-healing framework. At its core, the goal is for a swarm to continuously evaluate its own health and functional status, allowing it to take pre-emptive action and maintain performance even when faced with potential failures. This advanced form of system maintenance is about more than just surviving single-point failures; it is about ensuring that the entire system's critical functions remain operational despite widespread degradation stemming from component malfunction to environmental unpredictability. Robustness optimization is a multifaceted endeavor, blending methodologies from control theory, systems engineering, and machine learning to construct a collective entity that is both self-aware and self-correcting. This section will delve into the foundational concepts and advanced techniques of autonomous robustness optimization, dissecting the topic into two distinct, yet interconnected, sub-paradigms: self-recovery optimization [62] and self-reconfiguration optimization [57]. Each approach contributes to the overall resilience of a swarm by addressing different facets of operational disruption, collectively enabling the swarm to remain effective even in the face of significant adversity.

A. Self-Recovery Optimization

The primary aim of self-recovery optimization is to enable an autonomous agent swarm to independently identify, diagnose, and address failures and their consequences. The central challenge [5] lies in the decentralized architecture of such swarms, which inherently lacks a single, global point of supervision. Consequently, agents must rely on local information and collaborative processes to pinpoint anomalies and initiate a response. The entire process starts with continuous status monitoring [20, 122], where each agent uses its internal sensors to track its own operational parameters as well as the performance of its neighbors. Any deviation from expected behavior or a sudden shift in an agent's operational signature can serve as an early warning of a potential problem. For instance, an agent whose movement trajectory deviates from a planned path, or whose data link quality is degrading, might be flagged

for further examination by its peers. This distributed health assessment is a critical first step, providing the necessary data for subsequent diagnosis and resolution.

Once a potential issue is detected, the swarm must execute diagnostic protocols to determine the nature of the fault [63]. This is often a collaborative task, where a sub-group of agents may isolate a suspected faulty agent and collectively run a diagnostic routine. These routines [74] can range from basic communication checks to more detailed analysis of the agent's internal state variables. Diagnosis is not always straightforward, as observed symptoms can be complex and interrelated. Advanced machine learning models, particularly those trained on historical failure data, can be utilized to establish correlations between symptoms and specific fault types, thereby speeding up the diagnostic process and improving its accuracy [98]. Following a confirmed diagnosis, recovery protocols are activated. These protocols [102] are adaptive and are scaled based on the severity of the fault. Minor issues might be resolved with a simple reboot or recalibration. For more severe failures, the swarm must take more decisive action.

A particularly effective recovery strategy [106] involves the strategic application of redundancy, both in terms of physical hardware and task allocation. The swarm can be provisioned with spare agents or duplicate components that can be activated when a primary unit becomes inoperable. This form of physical redundancy ensures that system capabilities are maintained without significant performance degradation, albeit at the cost of increased system size and resource consumption. A more sophisticated approach to redundancy is the dynamic reassignment of tasks. When an agent is diagnosed with an irreversible fault and removed from the active collective, its workload must be rapidly redistributed among the remaining functional agents. This reassignment must be quick and efficient to prevent mission failure. The swarm can leverage decentralized auction or consensus-based algorithms to re-allocate tasks, with agents bidding for responsibilities based on their current workload and capabilities. This form of operational redundancy is highly flexible and scalable, allowing the swarm to degrade gracefully rather than experience a complete system collapse.

A crucial element of self-recovery is the ability to learn from past failures. By logging and analyzing fault data, the swarm can update its internal models of system health and continuously refine its diagnostic capabilities [110] over time. This learning process can be managed centrally, or in a distributed fashion [126]. In a centralized model, a designated leader agent might periodically collect data from all other agents to update a global fault model. While effective, this introduces a single point of vulnerability. Conversely, in a decentralized model, each agent could maintain its own local model and periodically exchange updates with neighbors, leading to a robust diagnostic model that evolves through collective learning. This allows the swarm to predict and prevent certain types of failures before they manifest, shifting from a passive response to a proactive state of resilience. The integration [128] of advanced learning techniques such as reinforcement and deep learning into this framework holds immense promise for creating systems that not only recover from failure but also continuously improve their recovery protocols through operational experience.

The design of effective self-recovery mechanisms must also account for the potential of cascading failures [136], where a single agent's malfunction triggers a chain reaction of subsequent failures throughout the swarm. This is a critical concern in tightly coupled systems with high inter-agent dependencies [177]. Resilient recovery protocols must include measures to prevent such cascades, such as immediately isolating a faulty agent and rerouting communication and data flows around the compromised area. The communication network itself must also be designed with redundancy and resilience, ensuring that information vital for recovery can still be transmitted even if parts of the network are compromised. The overall objective is to create a system that can absorb shocks and disruptions and still continue to perform its mission, albeit at a potentially reduced capacity. As such, the field of self-recovery optimization is a critical area of future research for creating reliable and resilient autonomous swarm systems with significant implications for applications in search-and-rescue, environmental monitoring, and space exploration.

B. Self-Reconfiguration Optimization

Self-reconfiguration optimization refers to the process by which an autonomous swarm dynamically alters its physical or logical structure to adapt to changes in mission objectives, environmental conditions, or internal failures [176]. While self-recovery focuses on restoring functionality, self-reconfiguration is concerned with a fundamental change in form. This capability is crucial for a truly versatile and intelligent swarm. The reconfiguration process can involve changes in the swarm's spatial layout, its task assignments, or its internal communication topology. It represents a higher-level form of macroscopic robustness, enabling the swarm to move beyond simple fault tolerance to a state of profound adaptivity. The ultimate goal is for a swarm to be able to transition from one state to another in a stable and efficient manner, thereby enhancing its ability to handle complex and dynamic environments.

One of the most direct forms of self-reconfiguration is the dynamic adjustment of a swarm's physical morphology. For example, a collective of modular robots could physically connect or detach from one another to form new structures better suited for a particular task [190]. A linear chain of robots might be highly effective for navigating a narrow passage, whereas a broader, two-dimensional plane might be more efficient for exploring a wide-open area. The swarm must be equipped with robust coordination algorithms to manage the connection and detachment process, ensuring the stability and functionality of the new configuration. This requires agents to have a precise understanding of their position and orientation relative to their neighbors and the ability to execute complex, synchronized maneuvers. Optimizing this process involves finding the most efficient way to transition from one morphology to another, minimizing the time and energy spent during the conversion.

Beyond physical form, a swarm can also reconfigure its logical structure. This includes the dynamic reassignment of roles and responsibilities to individual agents [195]. A swarm initially assigned to exploration, upon locating a target, might need to reconfigure itself into a rescue team. This requires a decentralized decision-making process where agents can collaboratively decide on new roles for each member. This process can be modeled as a global optimization problem, where the swarm seeks

a new task allocation that maximizes its collective performance in the face of a new objective. The reconfiguration [229] must also be resilient to communication failures during the transition. If an agent responsible for a critical role fails, the swarm must be able to immediately reassign that role to another agent without disrupting the mission. It is this level of flexibility that distinguishes a truly autonomous and intelligent swarm from a simple collection of independent agents.

The self-reconfiguration of a swarm's communication network is another vital aspect of this optimization paradigm. In dynamic environments, certain communication links may become unreliable or severed due to obstacles, interference, or agent failures [48, 196]. The swarm must be able to autonomously re-establish communication by forming new links or creating relay networks to route information around compromised areas. This requires agents to continuously monitor the quality of their communication links and share that information with their neighbors. Decentralized algorithms [244] can then be used to find an optimal new network topology that ensures connectivity while minimizing latency and power consumption. The reconfigured network [51] might prioritize critical communication links for essential mission data while using secondary links for less time-sensitive information. Such an adaptive network topology is crucial for maintaining situational awareness and coordination in challenging environments.

The challenges in self-reconfiguration are significant. The complexity of the reconfiguration space can be enormous, and finding an optimal new configuration in real-time is a major computational challenge [189]. Furthermore, the transition from one configuration to another must be executed with precision to avoid instability or errors. Research in this field is focused on developing decentralized optimization algorithms that can rapidly converge on a near-optimal solution with minimal communication overhead. These algorithms often draw inspiration from biological systems that exhibit similar forms of self-reconfiguration, such as cellular automata [248] or reconfigurable organisms [249]. The ultimate objective is to create swarm systems that can not only endure adversity but can also leverage it as a catalyst for adaptation and enhanced performance. A swarm capable of autonomous reconfiguration [253] is a truly resilient and intelligent system, capable of operating effectively in a wide range of tasks and environments from space exploration to disaster relief.

3.7 Simulators, Datasets, and Metrics

In this section, we introduce the world-first real-world mapping simulation environment and dataset for swarm Unmanned Aerial Vehicles (UAVs) autonomous flight.

Perception is critical in the UAVs' autonomous flight task for accurate navigation and safe planning [203, 246]. The recent development of deep learning brings significant breakthroughs in various perception tasks such as 3D object detection [240], object tracking [174], and semantic segmentation [221]. However, single-UAV vision systems still suffer from many real-world challenges, such as occlusion and short-range perception capability [88, 133], which can cause catastrophic accidents and are

key bottlenecks in improving low-altitude economic task performance. The shortcomings are mainly due to the limited field-of-view of individual UAVs, resulting in an incomplete understanding of the environment.

A growing interest and recent advances in cooperative perception systems [117, 130, 200, 223] have enabled a new paradigm that can potentially overcome the limitations of single-UAV perception. By leveraging UAV-to-UAV (U2U) technologies, multiple connected and automated UAVs (UAVs) can communicate and share captured sensor information simultaneously. Swarm UAVs flying autonomously in a dynamic open environment, the ego UAV struggles to perceive the tracking object due to leaf obstruction and snow interference. Incorporating the multimodal features of the nearby UAVs can further distribute multiple tasks and achieve greater flexibility, robustness, and perceptual range, leading to significant advantages in harsh and complex environments.

However, despite the great promise, cooperative perception is mainly focused on the vehicles and ignores the UAV literature, which remains challenging to validate U2U perception in real-world scenarios due to the lack of public datasets. Existing U2U cooperative perception datasets, as shown in Table 3.1, CoPerception-UAVs [87] and CoPerception-UAVs+ [88] rely on open-source simulators such as AirSim [167] and CARLA [44] and consider only 1 terrain, 1 weather, and 1–2 sensor types; they collect datasets using fixed altitude and consistent or fixed formation mode. In real-world scenarios, compared to autonomous driving, autonomous flight has more freedom, faces more complex environments, and is more susceptible to the influence of temperature, humidity, and airflow due to its smaller size. Obviously, there will be a clear domain gap between existing synthetic data and real-world data. As a result, models trained on these datasets may not generalize well to realistic flight situations.

To further advance innovative research on U2U cooperative perception, (1) we present a large-scale cooperative perception dataset (U2UData) for swarm UAVs autonomous flight. It is collected by three UAVs flying autonomously in the U2USim, covering a 9 km² flight area, comprising 315K LiDAR frames, 945K RGB and depth frames, 2.41 M annotated 3D bounding boxes for 3 classes, and including brightness, temperature, humidity, smoke, airflow values covering all flight paths. Compared with fixed altitude and consistent or fixed formation mode flying, autonomous flight can more comprehensively explore harsh and complex environments, allowing perception models to achieve higher flexibility and stronger robustness. (2) Brightness, temperature, humidity, smoke, and airflow modalities can greatly affect UAV flight control. However, they are closely related to terrain and meteorology, and have strong interactions with each other, making them difficult to collect. We build the first real-world mapping swarm UAVs simulation environment, U2USim, including four terrains, seven weather conditions, and eight sensor types. It takes Yunnan Province as the prototype and uses the real meteorological data of Yunnan Province collected by the China Meteorological Center to map the simulation environment based on longitude and latitude. It enables UAVs to perceive the world like humans and make optimal decisions. (3) U2UData introduces two perception tasks: cooperative 3D object detection and cooperative 3D object tracking. (4) We will make all the simulator, dataset, and metrics publicly available worldwide.

Table 3.1 A detailed comparison between autonomous flight-related datasets—indicates that specific information is not provided. DF: Discipline formation mode, where swarm UAVs keep a consistent and relatively static array; FF: Fixed formation mode, where each UAV navigates independently with a fixed path; AF: Autonomous formation mode, where each UAV flies autonomously. D: 3D object detection; T: 3D object tracking; S: semantic segmentation

Datasets	Year	Real data	UAVs	Altitudes	Formation	Sample	Terrains	Weather	Sensors	Tasks
CoPerception-UAVs	2022	–	5	Fixed	DF, FF	4s	1	1	1	D, S
CoPerception-UAVs+	2023	–	10	Fixed	DF, FF	4s	1	1	2	D, S
U2UData	2024	China	3	[56.6, 3000]	DF, FF, AF	0.03s	4	7	8	D, S, T

3.7.1 *Simulators*

In this sub-section, we build U2USim, the first real-world mapping swarm UAVs simulation environment, taking Yunnan Province as the prototype, including four terrains, seven weather conditions, and eight sensor types. We use actual meteorological data from Yunnan Province collected by the China Meteorological Center to map the simulation environment based on longitude and latitude. U2USim can not only significantly reduce the domain gap, but UAVs can also use it to perceive the world and make optimal decisions like humans.

A. General Simulators

A simulator for swarm UAVs must be able to more realistically simulate dynamic physical characteristics [144] (such as collision); sensors such as IMU [34], camera [153], GPS [154], and lidar [167]; and interaction with the ROS ecosystem [47]. In real-world scenarios, UAVs are extremely sensitive to the effects of temperature, humidity, and airflow due to their small size. Existing UAV simulators, as shown in Table 3.2, can visually realize digital twins [144] of the real world through GPU rendering, but it is difficult to provide modal information other than visual and LiDAR modalities. Many existing studies manually set values based on experience for temperature, humidity, brightness, smoke, and airflow modalities. However, because sensors are closely related to terrain and meteorology, and have strong interactions between modalities, the model trained by existing simulators is difficult to deploy in the real world.

Existing swarm UAV simulators include FightGear [34], XPlan [153], Jmavsim, Gazebo [154], AirSim [167], RflySim [47], Isaac Sim [144], and U2USim [77]. Specifically, fightGear is not open source. XPlan and Jmavsim can only interact with ROS. Gazebo, AirSim, and RflySim can interact with ROS, simulate physical collision, and output visual sensor content. AirSim and RflySim can also implement weather control. However the information on these simulators is purely simulated, and the models trained on these simulators are difficult to run in the real world. Isaac Sim and U2USim add real environment data based on previous simulators. Isaac Sim can visually realize digital twins of the real world through GPU rendering, but it is difficult to provide modal information other than visual and LiDAR modalities. U2USim is the first real-world mapping swarm UAV simulator, taking Yunnan Province as the prototype, including four terrains, seven weather conditions, and eight sensor types.

B. Real-World Mapping Simulators

U2USim is the first real-world mapping swarm UAVs simulation environment, taking Yunnan Province as the prototype, including four terrains, seven weather conditions, and eight sensor types. We use actual meteorological data from Yunnan Province collected by the China Meteorological Center to map the simulation environment based on longitude and latitude. U2USim can not only significantly reduce the domain gap, but UAVs can also use it to perceive the world and make optimal decisions like humans.

Table 3.2 A detailed comparison between simulators related to autonomous flight. H: High; M: Medium; L: Low. U2USim takes Yunnan Province as the prototype and uses the real meteorological data of Yunnan Province collected by the China Meteorological Center to map the simulation environment based on longitude and latitude. U2USim includes dynamic temperature, humidity, smoke, and airflow sensor information

Simulator	FightGear	XPlan	Jmavsim	Gazebo	AirSim	RflySim	Isaac Sim	U2USim
Open source	—	✓	✓	✓	✓	✓	✓	✓
ROS	—	—	✓	✓	✓	✓	✓	✓
Sensor output	—	—	—	✓	✓	✓	✓	✓
Physical collision	—	—	—	✓	✓	✓	✓	✓
Records	—	—	—	—	✓	✓	✓	✓
Weather	—	—	—	—	✓	✓	✓	✓
Real data	—	—	—	—	—	—	Digital Twin	Mapping
Altitude	—	—	—	—	—	—	✓	✓
Temperature sensor	—	—	—	—	—	—	—	✓
Humidity sensor	—	—	—	—	—	—	—	✓
Smoke sensor	—	—	—	—	—	—	—	✓
Airflow sensor	—	—	—	—	—	—	—	✓
Fidelity	L	H	L	M	H	H	H	H
Richness	L	H	L	M	H	H	H	H

Yunnan Province is located in southwest China, with high terrain in the north-west and low terrain in the southeast; the elevation range is [76.4, 6740]m. Yunnan Province has a sub-tropical plateau monsoon climate with significant vertical climate characteristics. The temperature changes vertically with the terrain, so the map of Yunnan Province is an excellent real-world scene for constructing a simulation environment.

U2USim uses Unreal Engine (UE) 5.2 to construct a scaled-down 3km*3km simulated environment map based on the map of Yunnan Province. U2USim includes four types of terrain: mountains, hills, plains, and basins. The elevation range is [56.6, 3000]m. Based on the vegetation distribution in Yunnan, 58 types of original forest vegetation assets were constructed, and more than 15 superposition methods were used to combine them, including epiphytic growth, diagonal staggered growth, and so on. Among them, the leaves of each plant will dynamically change with wind, rain, snow, and other weather conditions. U2USim takes Yunnan Province as a prototype, including four terrains, seven weather conditions, and eight sensor types. It uses the real meteorological data of Yunnan Province collected by the China Meteorological Center to map the simulation environment based on longitude and latitude.

The weather system is designed to simulate the dynamic weather environment in Yunnan Province. It consists of two parts: the practical code and the artistic performance. To create these impressive weather effects, 19 materials and 78 textures

were used. To imitate reality, the simulation includes effects that occur from the sky to the ground, such as wet leaves and muddy floors after heavy rain in Yunnan Province. In addition, there are eight highly customized particle systems to simulate rain, snow, dust, and fog at specific positions within the simulation environment. All of these effects are primarily based on Unreal Blueprints, which can reduce the time required to modify complex code. This allows visual designers to participate more in the programming process and easily change the value for realistic modification.

The sensors in U2USim are based on the Unreal Engine, which allows cross-platform compatibility and provides accurate data due to the engine's strong physics calculation capabilities. The LiDAR sensor primarily uses ray detection, and with the powerful engine, it can save a lot of time in tracking and calculating each ray. All image-based data, including depth and flare, are generated using Unreal rendering techniques. The flare image is created through the ID segmentation process, where objects are rendered using mapped temperature colors and output. The depth image is captured during the depth test process. The Luminance sensor uses a camera on the drone to capture live textures and calculate the average brightness and smoke concentration of each pixel. The brightness function is based on the color stimulus curve [119] of the human eye. The smoke concentration function is based on the Laplacian gradient sum algorithm. To enhance the credibility of the data captured by the sensors, including temperature, humidity, and airflow, weather data from Yunnan Province was used to map into the simulator environment.

These sensors are installed on the multirotor to explore the simulator map and collect data at 0.02-second intervals, which can be customized using a JSON settings file. Sensor mounting is also customizable; users can edit the JSON file to customize their own multirotor by selecting practical sensors from nine templates and designing the sensor accuracy and return rate.

We use the real meteorological data of Yunnan Province collected by the China Meteorological Center to map the simulation environment based on longitude and latitude. First, the temperature, humidity, pressure, wind speed, and wind direction data collected by weather stations in Yunnan Province from March 5 to 10, 2024, released by the National Meteorological Information Center are downloaded. The real sensor information is then mapped into the U2USim based on the latitude and longitude information of the real map. Among them, temperature and humidity are scalars, and missing values are filled by the moving average method (interval 5m). For wind speed and direction, assuming that the latitudinal wind speed of a location on March 5, 2024 is u_h m/s and the longitudinal wind speed is v_h m/s, the formulas for the synthesized wind speed v_h and wind direction θ_h are as follows:

$$V_h = \sqrt{u_h^2 + v_h^2}, \quad (3.17)$$

$$\alpha = \arctan\left(\frac{u_h}{v_h}\right) \times \frac{180}{\pi}, \quad (3.18)$$

$$\theta_h = \begin{cases} 0 & (u_h = 0, v_h \leq 0) \\ 90 & (u_h < 0, v_h = 0) \\ 270 & (u_h > 0, v_h = 0) \\ 180 + \alpha & (v_h > 0) \\ 360 + \alpha & (u_h > 0, v_h < 0) \\ \alpha & (u_h < 0, v_h < 0), \end{cases} \quad (3.19)$$

where α is the angular parameter. The equations for latitudinal wind speed u_h and longitudinal wind speed v_h for missing values are as follows:

$$u_h = \frac{u_1 \times (H_2 - H_1) + \sum_{i=2} u_i \times (H_{i+1} - H_{i-1}) + u_n \times (H_n - H_{n-1})}{2 \times (H_n - H_1)}, \quad (3.20)$$

$$v_h = \frac{v_1 \times (H_2 - H_1) + \sum_{i=2} v_i \times (H_{i+1} - H_{i-1}) + v_n \times (H_n - H_{n-1})}{2 \times (H_n - H_1)}, \quad (3.21)$$

where H_h is the potential height of isobars (obtained by sliding average of neighboring sampling points based on distance), n is the number of isobars, and i is the isobar at the missing value location, and the synthetic wind speed and direction are calculated according to Eqs. (3.17)–(3.19).

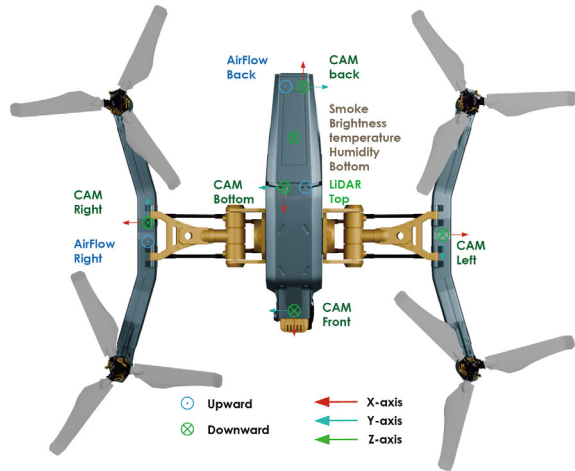
3.7.2 Datasets and Metrics

To expedite progress in U2U cooperative perception, we propose U2UData, the large-scale, autonomously flying, real2sim, multimodal dataset with different weather scenarios. This dataset is meticulously annotated with 3D bounding boxes to facilitate research on swarm UAVs cooperative perception.

A. Cooperative Perception Datasets

Collecting datasets [18, 24, 95, 120, 127, 231, 255] is crucial for advancing algorithmic research. Public cooperative perception datasets have significantly accelerated progress in autonomous driving technologies in recent years. For instance, OPV2V [224] introduced the first 3D cooperative perception dataset, leveraging CARLA [44] and OpenCDA [218] joint simulation environment. DAIR-V2X pioneered real-world datasets for cooperative detection. V2x-seq [234] developed the initial sequential dataset for vehicle-infrastructure cooperative perception and forecasting. V2X-Sim [116] further explored vehicle-to-everything perception feasibility using synthesized data from the CARLA simulator [232]. V2V4Real [222] established the first large-scale real-world dataset for vehicle-to-everything cooperative perception. However, cooperative perception datasets in autonomous flight scenarios remain scarce. Existing U2U cooperative perception datasets, such as CoPerception-UAVs [87] and CoPerception-UAVs+ [88], rely on open-source simulators like AirSim [167] and CARLA [44], featuring limited terrain, weather, and sensor types.

Fig. 3.6 Sensor setup for our data collection platform



These datasets collect data at fixed altitudes and in consistent or fixed formation modes.

In contrast to autonomous driving, UAVs' autonomous flight presents greater freedom, encounters more complex environments, and is more influenced by natural weather due to their smaller size. Hence, there exists a notable domain gap between existing synthetic data and real-world data, potentially limiting the generalization of models trained. In this paper, we introduce the first large-scale cooperative perception dataset for swarm UAVs autonomous flight.

B. Real-World Mapping datasets

We collect the U2UData using three UAVs flying autonomously in the U2USim. All UAVs are equipped with 5 RGBD cameras (front, back, left, right, and bottom), a 64-LiDAR sensor (top), 1 brightness, temperature, humidity, and smoke sensor (bottom), 2 airflow sensors (back and right), and GPS/IMU systems. Figure 3.6 illustrates the sensor layout configuration.

The existing UAV flight modes of data collection mainly adopt discipline formation and fixed formation modes. As shown in Fig. 3.7, in discipline formation mode, the swarm UAVs maintain a consistent and relatively static array, which has the same height, fixed spacing, and the same speed. In fixed formation mode, each UAV navigates independently with a fixed path, having the same height and speed. However, in a dynamic and open real-world environment, such as UAV delivery tasks, it is difficult for swarm UAVs to maintain a fixed height, speed, and path. In U2UData, we have adopted a new autonomous formation mode in which each UAV flies autonomously. Compared with fixed altitude and discipline or fixed formation mode flying, autonomous flight can more comprehensively explore harsh and complex environments, allowing perception models to achieve higher flexibility and stronger robustness.

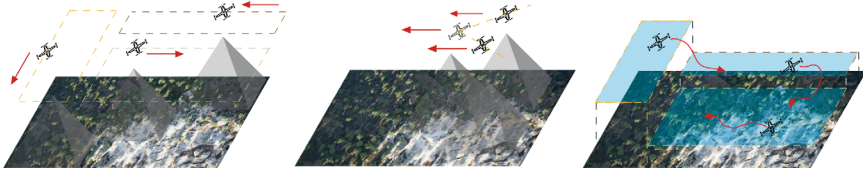


Fig. 3.7 Three types of swarm UAVs formation

For the 7 representative weather conditions in U2USim, we manually selected 100 scenarios for each weather condition, including different altitudes and different vegetation coverage, covering a 9 km^2 flight area. For each scenario, we collected 15 seconds of swarm UAVs cooperative perception data. We sample the image frames at 30Hz, comprising a total 945K RGB frames and 945K depth frames. We collect 315K LiDAR frames at a sampling frequency of 10Hz, including 2.41M annotated 3D bounding boxes for 3 classes. The 945K brightness, 945K temperature, 945K humidity, 945K smoke, and 1.89M airflow values were collected through real-world mapping according to the acquisition coordinates of each image.

For annotating 3D bounding boxes on the gathered LiDAR data, we utilize SusTechPoint [109], a robust open-source labeling tool. We also manually refine the annotations. There are a total of three object classes, including bear, deer, and wolf. For each object, we annotate its 3D bounding box with 7 degrees of freedom, encompassing its location (x, y, z) and rotation (expressed as quaternions: w, x, y, z). The location (x, y, z) corresponds to the center of the bounding box. These 3D bounding boxes are annotated separately based on the global coordinate system of each UAV. This approach enables the sensor data from each UAV to be treated independently as a single-agent detection task. We initialize the relative pose of the two UAVs for each frame using positional information provided by the GPS on both UAVs.

In total, U2UData has 315K LiDAR frames, 945K RGB and depth frames, 2.41M 3D bounding boxes, 1.89M airflow value, 945K brightness, temperature, humidity, and smoke values. We randomly divide it into training sets/validation sets/test sets according to the ratio of 0.7/0.15/0.15. It can greatly facilitate the credibility of algorithm performance compared to different papers.

C. Metrics

In contrast to the single-UAV task, cooperative perception presents several domain-specific challenges: Bandwidth limitation. Conventional U2U communication technologies often operate within narrow bandwidth constraints [200, 221] and cooperative detection algorithms need to balance between precision and the bandwidth cost carefully. Location error. Due to environmental disturbances, inherent errors [212] in the adjacent UAVs' relative poses will lead to coordinate system mapping errors. Asynchronicity. Heterogeneous UAVs [88] are inevitable and the distance between UAVs is variable. Different sensor sampling frequencies and transmission latencies will cause UAVs asynchronous.

We utilize some evaluation metrics for models evaluation. These metrics include: AMOTA, average multiobject tracking accuracy; AMOTP, average multiobject tracking precision; sAMOTA, scaled average multiobject tracking accuracy, which ensures a more linear representation across the entire $[0, 1]$ range of significantly challenging tracking tasks; MOTA, multi object tracking accuracy; MT, mostly tracked trajectories; ML, mostly lost trajectories.

References

1. Arnold, E., Dianati, M., de Temple, R., Fallah, S.: Cooperative perception for 3d object detection in driving scenarios using infrastructure sensors. *IEEE Trans. Intell. Transp. Syst.* **23**(3), 1852–1864 (2020)
2. Beckers, R., Deneubourg, J.L., Goss, S.: Trails and u-turns in the selection of a path by the ant *Lasius Niger*. *J. Theor. Biol.* **159**(4), 397–415 (1992)
3. Bererton, C., Gordon, G.J., Thrun, S.: Auction mechanisms for multi-robot coordination. *Adv. Neural Inf. Process. Syst.* **16** (2003)
4. Bhardwaj, A., Kumar, V.: A framework for enhancing privacy in online collaboration. *Int. J. Electron. Secur. Digit. Forensics* **14**(4), 413–432 (2022)
5. Bhat, N., Kambhampati, S.: Autonomous self-healing in multi-robot systems. In: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), vol. 2021, 5883–5890 (2021)
6. Bi, R., Xiong, J., Tian, Y., Li, Q., Choo, K.K.R.: Achieving lightweight and privacy-preserving object detection for connected autonomous vehicles. *IEEE Internet Things J.* **10**(3), 2314–2329 (2022)
7. Bi, R., Xiong, J., Tian, Y., Li, Q., Liu, X.: Edge-cooperative privacy-preserving object detection over random point cloud shares for connected autonomous vehicles. *IEEE Trans. Intell. Transp. Syst.* **23**(12), 24979–24990 (2022)
8. Bi, X., He, M., Sun, Y.: Mix q-learning for lane changing: a collaborative decision-making method in multi-agent deep reinforcement learning (2024). [arXiv:2406.09755](https://arxiv.org/abs/2406.09755)
9. Blackwell, T., Branke, J.: Multi-swarm optimization in dynamic environments. In: Workshops on Applications of Evolutionary Computation, pp. 489–500. Springer (2004)
10. Brambilla, M., Ferrante, E., Birattari, M., Dorigo, M.: Swarm robotics: a review from the swarm engineering perspective. *Swarm Intell.* **7**(1), 1–41 (2013)
11. Brody, S., Alon, U., Yahav, E.: How attentive are graph attention networks? (2021) [arXiv:2105.14491](https://arxiv.org/abs/2105.14491)
12. Brutschy, A., Pini, G., Pincirol, C., Birattari, M., Dorigo, M.: Self-organized task allocation to sequentially interdependent tasks in swarm robotics. In: Proceedings of the 2014 International Conference on Autonomous Agents and Multi-agent Systems, pp. 379–386 (2014)
13. Camazine, S., Deneubourg, J.L., Franks, N.R., Sneyd, J., Theraula, G., Bonabeau, E.: Self-organization in biological systems. In: *Self-Organization in Biological Systems*. Princeton University Press (2020)
14. Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-end object detection with transformers. In: European Conference on Computer Vision, pp. 213–229 (2020)
15. Carlisle, A., Dozier, G.: Adapting particle swarm optimization to dynamic environments. In: International Conference on Artificial Intelligence, vol. 1, pp. 429–434. Citeseer (2000)
16. Carlisle, A., Dozier, G.: Tracking changing extrema with adaptive particle swarm optimizer. In: Proceedings of the 5th Biannual World Automation Congress, vol. 13, pp. 265–270. IEEE (2002)

17. Carrillo, H., Lin, Z., Tang, J., Luo, Z.: Multi-agent reinforcement learning: methods, applications, and challenges. *Artif. Intell.* **305**, 103679 (2022)
18. Chen, A., Wang, Z., Dong, C., Tian, K., Zhao, R., Liang, X., Kang, Z., Li, X.: Chinaopen: a dataset for open-world multimodal learning. In: *Proceedings of the 31st ACM International Conference on Multimedia*, pp. 6432–6440 (2023)
19. Chen, F., Zhang, W., Liu, Y.: Multi-agent reinforcement learning with communication. *IEEE Trans. Neural Netw. Learn. Syst.* **32**(6), 2620–2634 (2021)
20. Chen, H., Zhang, M., Li, R.: Distributed cooperative task allocation for multi-robot systems. In: *2020 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2020, pp. 9066–9073 (2020)
21. Chen, Q., Ma, X., Tang, S., Guo, J., Yang, Q., Fu, S.: F-cooper: feature based cooperative perception for autonomous vehicle edge computing system using 3d point clouds. In: *Proceedings of the 4th ACM/IEEE Symposium on Edge Computing*, pp. 88–100 (2019)
22. Chen, Q., Tang, S., Yang, Q., Fu, S.: Cooper: cooperative perception for connected autonomous vehicles based on 3d point clouds. In: *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, pp. 514–524 (2019)
23. Chen, R., Mu, Y., Xu, R., Shao, W., Jiang, C., Xu, H., Li, Z., Luo, P.: Co3: Cooperative unsupervised 3d representation learning for autonomous driving. In: *The International Conference on Learning Representations* (2023)
24. Chen, X., Xiong, Y., Wang, S., Wang, H., Sheng, T., Zhang, Y., Ye, Y.: Reco: a dataset for residential community layout planning. In: *Proceedings of the 31st ACM International Conference on Multimedia*, pp. 397–405 (2023)
25. Chen, Z., Shi, Y., Jia, J.: Transiff: an instance-level feature fusion framework for vehicle-infrastructure cooperative 3d detection with transformers. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 18205–18214 (2023)
26. Cheng, T., Zhang, J.: Multi-robot task allocation: a review of the state-of-the-art. *Front. Inf. Technol. Electron. Eng.* **16**, 452–472 (2015)
27. Chu, X., Tian, Z., Wang, Y., Zhang, B., Ren, H., Wei, X., Xia, H., Shen, C.: Twins: Revisiting the design of spatial attention in vision transformers. *Adv. Neural. Inf. Process. Syst.* **34**, 9355–9366 (2021)
28. Clark, A.: Reflections on intelligence: prospects for embodied artificial general intelligence. In: *Proceedings of the First AGI Conference*, pp. 77–83. IOS Press (2008)
29. Couzin, I.D., Krause, J., Franks, N.R., Levin, S.A.: Effective leadership and decision-making in animal groups on the move. *Nature* **433**(7025), 513–516 (2005)
30. Crites, R.H., Stone, P.: Co-operative multi-agent q-learning. *ICML* **96**, 1–9 (1996)
31. Cui, J., Qiu, H., Chen, D., Stone, P., Zhu, Y.: Coopernaut: end-to-end driving with cooperative perception for networked vehicles. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 17252–17262 (2022)
32. Dias, M.B., Stentz, A.: An agent-based approach for cooperative exploration. *Int. J. Robot. Res.* **25**(1–2), 75–94 (2006)
33. Dias, M.B., Zlot, R., Kalra, N., Stentz, A.: Market-based multirobot coordination: a survey and analysis. *Proc. IEEE* **94**(7), 1257–1270 (2006)
34. DigitalOcean: Flightgear. <https://www.flightgear.org/>
35. Ding, W., Yang, B., Zhang, L.: Communication protocols for decentralized swarm robotics. *Swarm Evol. Comput.* **78**, 101270 (2023a)
36. Ding, W., Yang, B., Zhang, L.: Distributed control for multi-robot systems. *Swarm Evol. Comput.* **79**, 101290 (2023b)
37. Ding, W., Yang, B., Zhang, L.: Efficient communication for large-scale swarm robotics. *Swarm Intell.* **17**(1), 1–18 (2023c)
38. Ding, W., Yang, B., Zhang, L.: Fault-tolerant communication in swarm robotics. *Swarm Evol. Comput.* **79**, 101290 (2023d)
39. Dorigo, M., Gambardella, L.M.: Ant colony system: a cooperative learning approach to the traveling salesman problem. *IEEE Trans. Evol. Comput.* **1**(1), 53–66 (1997)

40. Dorigo, M., Maniezzo, V., Colorni, A.: Ant colony optimization: a new meta-heuristic. *Artif. Life Robot.* **1**, 1–13 (1992)
41. Dorigo, M., Maniezzo, V., Colorni, A.: The ant system: an autocatalytic search algorithm. *IEEE Trans. Syst. Man Cybern. Part B* **26**(1), 29–41 (1996)
42. Dorigo, M., Theraulaz, G., Trianni, V.: Reflections on the future of swarm robotics. *Sci. Robot.* **5**(49), eabe4385 (2020)
43. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al.: An image is worth 16x16 words: transformers for image recognition at scale (2020). [arXiv:2010.11929](https://arxiv.org/abs/2010.11929)
44. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: Carla: An open urban driving simulator. In: *Conference on Robot Learning*, pp. 1–16. PMLR (2017)
45. Eberhart, R.C., Shi, Y.: Tracking and optimizing dynamic systems with particle swarms. In: *Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No. 01TH8546)*, vol. 1, pp. 94–100. IEEE (2001)
46. Fan, L., Zeng, C., Meng, Z., Xia, X., Liu, Y., Ma, J., Wang, F.Y.: A secured vehicle brain: Dao-based collaborative perception and decision-making systems for intelligent vehicles in cpss. *IEEE Trans. Intell. Veh* (2023)
47. FEISILAB: Rflysim. <https://rflysim.com/doc/zh/>
48. Feng, T., Li, Q., Wang, X., Wang, M., Li, G., Zhu, W.: Multi-weather cross-view geo-localization using denoising diffusion models. In: *Proceedings of the 2nd Workshop on UAVs in Multimedia: Capturing the World from a New Perspective*, pp. 35–39 (2024)
49. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata: A large-scale cooperative perception dataset for swarm uavs autonomous flight. In: *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 7600–7608 (2024)
50. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata-2: A scalable swarm uavs autonomous flight dataset for long-horizon tasks (2025). [arXiv:2509.00055](https://arxiv.org/abs/2509.00055)
51. Feng, T., Wang, X., Jiang, Y.G., Zhu, W.: Embodied ai: From llms to world models. *IEEE Circuits Syst. Mag* (2025)
52. Feng, T., Wang, X., Zhou, Z., Wang, R., Zhan, Y., Li, G., Li, Q., Zhu, W.: Evoagent: agent autonomous evolution with continual world model for long-horizon tasks (2025). [arXiv:2502.05907](https://arxiv.org/abs/2502.05907)
53. Finn, C., Abbeel, P., Levine, S.: Model-agnostic meta-learning for fast adaptation of deep networks. In: *Proceedings of the 34th International Conference on Machine Learning*, pp. 1126–1135. PMLR (2017)
54. Fu, C., Dong, C., Mertz, C., Dolan, J.M.: Depth completion via inductive fusion of planar lidar and monocular camera. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 10843–10848 (2020)
55. Gao, H., Cheng, B., Wang, J., Li, K., Zhao, J., Li, D.: Object classification using cnn-based fusion of vision and lidar in autonomous vehicle environment. *IEEE Trans. Ind. Inf.* **14**(9), 4224–4231 (2018)
56. Gao, P., Zhu, Q., Lu, H., Gan, C., Zhang, H.: Deep masked graph matching for correspondence identification in collaborative perception. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6117–6123 (2023)
57. Garg, A., Kumar, N.: Self-reconfiguring and self-healing soft robotic swarms. In: *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 2023, pp. 2209–2216 (2023)
58. Garnier, S., Gautrais, J., Theraulaz, G.: Emergence of swarm intelligence: from natural to artificial systems. *Swarm Intell.* **1**(1), 3–31 (2007)
59. Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? The kitti vision benchmark suite. In: *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3354–3361 (2012)
60. Glaser, N.M., Kira, Z.: Communication-critical planning via multi-agent trajectory exchange. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3468–3475 (2023)

61. Glaser, N.M., Kira, Z.: We need to talk: Identifying and overcoming communication-critical scenarios for self-driving (2023). [arXiv:2305.04352](https://arxiv.org/abs/2305.04352)
62. Gong, G., Li, J., Wang, W.: Distributed control for cooperative transportation with multiple unmanned aerial vehicles. *IEEE Trans. Ind. Inf.* **16**(6), 4279–4288 (2020)
63. Gong, Z., Duan, G.: Bio-inspired optimization algorithms for multi-robot path planning: a review. *Robotica* **39**(1), 1–22 (2021)
64. Gu, J., Li, B.: Distributed consensus for multi-agent systems. *Automatica* **146**, 110587 (2022a)
65. Gu, J., Li, B.: Dynamic communication for multi-agent systems. *IEEE Trans. Control Syst. Technol.* **30**(5), 1925–1938 (2022b)
66. Gu, J., Zhang, J., Zhang, M., Meng, W., Xu, S., Zhang, J., Zhang, X.: Feaco: reaching robust feature-level consensus in noisy pose conditions. In: *Proceedings of the 31st ACM International Conference on Multimedia*, pp. 3628–3636 (2023)
67. Gunther, N.J.: A simple capacity model of massively parallel transaction systems. In: *International CMG Conference*, pp. 1–9 (1993)
68. Guo, C., Szemenyei, M., Yi, Y., Wang, W., Chen, B., Fan, C.: Sa-unet: spatial attention u-net for retinal vessel segmentation. In: *2020 25th International Conference on Pattern Recognition (ICPR)*, pp. 1236–1242 (2021)
69. Guo, J., Carrillo, D., Tang, S., Chen, Q., Yang, Q., Fu, S., Wang, X., Wang, N., Palacharla, P.: Coff: cooperative spatial feature fusion for 3-d object detection on autonomous vehicles. *IEEE Internet Things J.* **8**(14), 11078–11087 (2021)
70. Guo, M.H., Cai, J.X., Liu, Z.N., Mu, T.J., Martin, R.R., Hu, S.M.: Pct: Point cloud transformer. *Comput. Visual Media* **7**, 187–199 (2021)
71. Guo, M.H., Xu, T.X., Liu, J.J., Liu, Z.N., Jiang, P.T., Mu, T.J., Zhang, S.H., Martin, R.R., Cheng, M.M., Hu, S.M.: Attention mechanisms in computer vision: a survey. *Comput. Visual Media* **8**(3), 331–368 (2022)
72. Hadidi, R., Cao, J., Woodward, M., Ryoo, M.S., Kim, H.: Distributed perception by collaborative robots. *IEEE Robot. Autom. Lett.* **3**(4), 3709–3716 (2018)
73. Hamann, H.: *Swarm Robotics: A Formal Approach*. Springer (2018)
74. Han, D., Kim, J., Lee, S.: Resilient formation control for multi-agent systems under adversarial attacks. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 2022, pp. 2502–2509 (2022)
75. Han, D., Wang, L.: Consensus-based task allocation for multi-robot systems. In: *Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1001–1008 (2021)
76. Han, D., Wang, L.: Multi-agent consensus with communication delays. *IEEE Trans. Control Netw. Syst.* **8**(4), 1670–1681 (2021b)
77. Han, F., Zhang, L., Wang, X., Zhao, K.A., Zhong, Y., Su, Z., Feng, T., Zhu, W.: U2usim-a uav telepresence simulation platform with multi-agent sensing and dynamic environment. In: *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 11258–11260 (2024)
78. Han, Y., Zhang, H., Li, H., Jin, Y., Lang, C., Li, Y.: Collaborative perception in autonomous driving: methods, datasets, and challenges. *IEEE Intell. Transp. Syst. Mag.* (2023)
79. He, L., Bai, L., Yang, X., Du, H., Liang, J.: High-order graph attention network. *Inf. Sci.* **630**, 222–234 (2023)
80. He, Y., Zhang, M., Liu, J.: Distributed decision-making in multi-robot systems. *J. Intell. Robot. Syst.* **105**(2), 1–18 (2022)
81. Hölldobler, B., Wilson, E.O.: *The Ants*. Harvard University Press (1990)
82. Hong, S., Liu, Y., Li, Z., Li, S., He, Y.: Multi-agent collaborative perception via motion-aware robust communication network. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15301–15310 (2024)
83. Hu, J., Shen, L., Sun, G.: Squeeze-and-excitation networks. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7132–7141 (2018)
84. Hu, S., Fang, Z., Deng, Y., Chen, X., Fang, Y.: Collaborative perception for connected and autonomous driving: challenges, possible solutions and opportunities (2024). [arXiv:2401.01544](https://arxiv.org/abs/2401.01544)

85. Hu, X., Eberhart, R.C.: Adaptive particle swarm optimization: detection and response to dynamic systems. In: *Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02* (cat. No. 02TH8600), vol. 2, pp. 1666–1670. IEEE (2002)
86. Hu, Y., Chen, C., Li, Q.: Cognitive models for multi-robot systems. *J. Auton. Robot.* **47**(2), 201–215 (2023)
87. Hu, Y., Fang, S., Lei, Z., Zhong, Y., Chen, S.: Where2comm: communication-efficient collaborative perception via spatial confidence maps. *Adv. Neural. Inf. Process. Syst.* **35**, 4874–4886 (2022)
88. Hu, Y., Lu, Y., Xu, R., Xie, W., Chen, S., Wang, Y.: Collaboration helps camera overtake lidar in 3d detection. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9243–9252 (2023)
89. Hu, Y., Pang, X., Qin, X., Eldar, Y.C., Chen, S., Zhang, P., Zhang, W.: Pragmatic communication in multi-agent collaborative perception (2024). [arXiv:2401.12694](https://arxiv.org/abs/2401.12694)
90. Hu, Y., Peng, J., Liu, S., Ge, J., Liu, S., Chen, S.: Communication-efficient collaborative perception via information filling with codebook. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15481–15490 (2024)
91. Jadbabaie, A., Lin, J., Morse, A.S.: Coordination of groups of mobile autonomous agents using nearest neighbor rules. *IEEE Trans. Autom. Control* **48**(6), 988–1001 (2003)
92. Jennings, N.R., Faratin, P., Lomuscio, A.R., Parsons, S., Sierra, C., Wooldridge, M.: Automated negotiation: prospects, methods and challenges. *Group Decis. Negot.* **10**(2), 199–215 (2001)
93. Jiang, H., Liu, N.: Cognitive-inspired multi-agent systems. *J. Cogn. Syst.* **11**, 1–15 (2023a)
94. Jiang, H., Liu, N.: Decentralized consensus in multi-agent systems. *IEEE Trans. Autom. Control* **68**(1), 1–15 (2023b)
95. Jin, Z., Yu, H., Haul, C., Wang, L., Zhu, Z., Shen, Q., Cao, X.: Wormtrack: dataset and benchmark for multi-object tracking in worm crowds. In: *Proceedings of the 31st ACM International Conference on Multimedia*, pp. 5756–5763 (2023)
96. Kennedy, J.: Swarm intelligence. In: *Handbook of Nature-Inspired and Innovative Computing: Integrating Classical Models with Emerging Technologies*, pp. 187–219. Springer (2006)
97. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, pp. 1942–1948. IEEE (1995)
98. Kim, D., Lee, J., Park, M.: Self-reconfigurable modular robot systems: a review on design and control. *Sensors* **20**(19), 5550 (2020)
99. Kleinhenz, M., Bujok, B., Fuchs, S., Tautz, J.: Hot bees in empty broodnest cells: heating from within. *J. Exp. Biol.* **206**(23), 4217–4231 (2003)
100. Kong, X., Jiang, W., Jia, J., Shi, Y., Xu, R., Liu, S.: Dusa: decoupled unsupervised sim2real adaptation for vehicle-to-everything collaborative perception. In: *Proceedings of the 31st ACM International Conference on Multimedia*, pp. 1943–1954 (2023)
101. Kraus, S.: *Negotiation and Cooperation in Multi-agent Environments*. MIT Press (1997)
102. Kumar, A., Singh, V., Mishra, R.: Adaptive task allocation for heterogeneous multi-robot teams. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2023, pp. 4501–4508 (2023)
103. Lai, G., Sycara, K., Li, C.: Emergent negotiation agents for multi-issue bargaining. *Int. J. Hum Comput Stud.* **60**(3), 371–404 (2004)
104. Lei, Z., Ren, S., Hu, Y., Zhang, W., Chen, S.: Latency-aware collaborative perception. In: *European Conference on Computer Vision*, pp. 316–332 (2022)
105. Lewis, M., Sycara, K., Walker, P.: Autonomous agents modeling other agents: a comprehensive survey and open problems. *Artif. Intell.* **296**, 103471 (2021)
106. Li, B., Yang, H.: Distributed control and communication for multi-robot systems. *Automatica* **132**, 109786 (2021)
107. Li, B., Yang, H.: Shared mental models for robust human-robot collaboration. *IEEE Trans. Hum. Mach. Syst.* **53**(2), 178–191 (2023)
108. Li, C., Zhang, T.: Multi-agent systems with shared knowledge representation. *IEEE Trans. Neural Netw. Learn. Syst.* **33**(10), 5554–5567 (2022)

109. Li, E., Wang, S., Li, C., Li, D., Wu, X., Hao, Q.: Sustech points: a portable 3d point cloud interactive annotation platform system. In: 2020 IEEE Intelligent Vehicles Symposium (IV), p. 1108–1115 (2020)
110. Li, J., Xu, R., Liu, X., Ma, J., Chi, Z., Ma, J., Yu, H.: Learning for vehicle-to-vehicle cooperative perception under lossy communication. *IEEE Trans. Intell. Veh.* **8**(4), 2650–2660 (2023)
111. Li, X.: Adaptively choosing neighbourhood bests using species in a particle swarm optimizer for multimodal function optimization. In: Genetic and Evolutionary Computation–GECCO 2004: Genetic and Evolutionary Computation Conference, Seattle, WA, USA, June 26–30, 2004. Proceedings, Part I, pp. 105–116. Springer (2004)
112. Li, X., Branke, J., Blackwell, T.: Particle swarm with speciation and adaptation in a dynamic environment. In: Proceedings of the 8th Annual Conference on Genetic and Evolutionary Computation, pp. 51–58 (2006)
113. Li, X., Tian, Y., Ye, P., Duan, H., Wang, F.Y.: A novel scenarios engineering methodology for foundation models in metaverse. *IEEE Trans. Syst. Man Cybern. Syst.* **53**(4), 2148–2159 (2022)
114. Li, X., Ye, P., Li, J., Liu, Z., Cao, L., Wang, F.Y.: From features engineering to scenarios engineering for trustworthy ai: I&i, c&c, and v&v. *IEEE Intell. Syst.* **37**(4), 18–26 (2022)
115. Li, Y., Fang, Q., Bai, J., Chen, S., Juefei-Xu, F., Feng, C.: Among us: adversarially robust collaborative perception by consensus. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 186–195 (2023)
116. Li, Y., Ma, D., An, Z., Wang, Z., Zhong, Y., Chen, S., Feng, C.: V2x-sim: Multi-agent collaborative perception dataset and benchmark for autonomous driving. *IEEE Robot. Autom. Lett.* **7**(4), 10914–10921 (2022)
117. Li, Y., Ren, S., Wu, P., Chen, S., Feng, C., Zhang, W.: Learning distilled collaboration graph for multi-agent perception. *Adv. Neural. Inf. Process. Syst.* **34**, 29541–29552 (2021)
118. Li, Y., Zhang, J., Ma, D., Wang, Y., Feng, C.: Multi-robot scene completion: towards task-agnostic collaborative perception. In: Conference on Robot Learning, pp. 2062–2072 (2023)
119. Liao, F., Zhou, Z., Kim, B.J., et al.: Bioinspired in-sensor visual adaptation for accurate perception. *Nat. Electron.* 84–91 (2022)
120. Lin, H., Ruan, L., Xia, W., Liu, P., Wen, J., Xu, Y., Hu, D., Song, R., Zhao, W.X., Jin, Q., et al.: Tiktalk: a video-based dialogue dataset for multi-modal chitchat in real world. In: Proceedings of the 31st ACM International Conference on Multimedia, pp. 1303–1313 (2023)
121. Liu, C., Chen, J., Chen, Y., Payton, R., Riley, M., Yang, S.H.: Self-supervised adaptive weighting for cooperative perception in v2v communications. *IEEE Trans. Intell. Veh.* (2023)
122. Liu, H., Huang, X., Gu, J., Shi, J., He, N., Feng, T.: Tcdformer-based momentum transfer model for long-term sports prediction. *Expert Syst. Appl.* 128310 (2025)
123. Liu, J., Wang, X., Zhang, P.: Distributed control for multi-robot systems with communication constraints. *IEEE Trans. Control Netw. Syst.* **10**(1), 1–14 (2023a)
124. Liu, J., Wang, X., Zhang, P.: Negotiation for task allocation in multi-robot swarms. In: Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 1–8 (2023)
125. Liu, M., Yin, H.: Cross attention network for semantic segmentation. In: 2019 IEEE International Conference on Image Processing (ICIP), pp. 2434–2438 (2019)
126. Liu, Q., Wang, P., Li, X.: Bio-inspired collective decision making for swarm robotics. *Front. Robot. AI* **7**, 61 (2020)
127. Liu, W., He, T., Gong, C., Zhang, N., Yang, H., Yan, J.: Fine-grained music plagiarism detection: revealing plagiarists through bipartite graph matching and a comprehensive large-scale dataset. In: Proceedings of the 31st ACM International Conference on Multimedia, pp. 8839–8848 (2023)
128. Liu, Y., Liu, K., Wang, J.: Resilient cooperative control of networked multi-robot systems. *IEEE Trans. Control Netw. Syst.* **8**(2), 681–692 (2021)
129. Liu, Y., Shao, Z., Hoffmann, N.: Global attention mechanism: retain information to enhance channel-spatial interactions (2021). [arXiv:2112.05561](https://arxiv.org/abs/2112.05561)

130. Liu, Y.C., Tian, J., Glaser, N., Kira, Z.: When2com: multi-agent perception via communication graph grouping. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4106–4115 (2020)
131. Liu, Y.C., Tian, J., Ma, C.Y., Glaser, N., Kuo, C.W., Kira, Z.: Who2com: collaborative perception via learnable handshake communication. In: *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6876–6883 (2020)
132. Lu, Y., Hu, Y., Zhong, Y., Wang, D., Chen, S., Wang, Y.: An extensible framework for open heterogeneous collaborative perception (2024). [arXiv:2401.13964](https://arxiv.org/abs/2401.13964)
133. Lu, Y., Hu, Y., Zhong, Y., Wang, D., Chen, S., Wang, Y.: An extensible framework for open heterogeneous collaborative perception. In: *The Twelfth International Conference on Learning Representations* (2024)
134. Lu, Y., Li, Q., Liu, B., Dianati, M., Feng, C., Chen, S., Wang, Y.: Robust collaborative 3d object detection in presence of pose errors. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4812–4818 (2023)
135. Luo, G., Zhang, H., Yuan, Q., Li, J.: Complementarity-enhanced and redundancy-minimized collaboration network for multi-agent perception. In: *Proceedings of the 30th ACM International Conference on Multimedia*, pp. 3578–3586 (2022)
136. Ma, W., Chen, H., Liu, W.: Collective decision making in multi-robot systems. *IEEE Trans. Rob.* **38**(1), 166–179 (2022)
137. Ma, Y., Lu, J., Cui, C., Zhao, S., Cao, X., Ye, W., Wang, Z.: Macp: efficient model adaptation for cooperative perception. In: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 3373–3382 (2024)
138. Mateescu, A., Ionescu, T.: Communication protocols for multi-robot systems. *Robot. Auton. Syst.* **59**(1), 31–40 (2011)
139. Melotti, G., Premebida, C., Gonçalves, N.: Multimodal deep-learning for object recognition combining camera and lidar data. In: *2020 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, pp. 177–182 (2020)
140. Mendes, R., Kennedy, J., Neves, J.: The fully informed particle swarm: simpler, maybe better. *IEEE Trans. Evol. Comput.* **8**(3), 204–210 (2004)
141. Millonas, M.M.: Swarms, phase transitions, and collective intelligence (1993). [arXiv:adap-
org/9306002](https://arxiv.org/abs/9306002)
142. Millonas, M.M.: Swarms, phase transitions, and collective intelligence. *Artif. Life* **2**(3), 279–317 (1994)
143. Ngo, H., Fang, H., Wang, H.: Cooperative perception with v2v communication for autonomous vehicles. *IEEE Trans. Veh. Technol.* **72**(9), 11122–11131 (2023)
144. NVIDIA: Isaac sim. <https://developer.nvidia.com/isaac-sim>
145. Olfati-Saber, R., Fax, J.A., Murray, R.M.: Consensus and cooperation in networked multi-agent systems. *Proc. IEEE* **95**(1), 215–233 (2007)
146. Park, C., Jeong, Y., Cho, M., Park, J.: Fast point transformer. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16949–16958 (2022)
147. Parker, L.E.: Alliance: an architecture for fault tolerant multi-robot cooperation. *IEEE Trans. Robot. Autom.* **14**(2), 220–240 (1998)
148. Parker, L.E.: Distributed intelligence: overview of the field and its application in multi-robot systems. In: *Distributed Autonomous Robotic Systems*, pp. 5–14. Springer (2008)
149. Parrott, D., Li, X.: Locating and tracking multiple dynamic optima by a particle swarm model using speciation. *IEEE Trans. Evol. Comput.* **10**(4), 440–458 (2006)
150. Pérowski, A.: A clearing procedure as a niching method for genetic algorithms. In: *Proceedings of IEEE International Conference on Evolutionary Computation*, pp. 798–803. IEEE (1996)
151. Qin, Z., Zhang, P., Wu, F., Li, X.: Fcanet: Frequency channel attention networks. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 783–792 (2021)
152. Ren, S., Lei, Z., Wang, Z., Dianati, M., Wang, Y., Chen, S., Zhang, W.: Interruption-Aware Cooperative Perception for v2x Communication-Aided Autonomous Driving. *IEEE Trans. Intell. Veh* (2024)

153. Research, L.: Xplan. <https://www.x-plane.com/>
154. Robotics, O.: Gazebo. <https://gazebo.org/home>
155. Russell, S., Norvig, P.: *Artificial Intelligence: A Modern Approach*. Pearson (2021)
156. Sandholm, T.: Multiagent systems: a survey from a machine learning perspective. *Auton. Agent. Multi-Agent Syst.* **2**(3), 203–238 (1999)
157. Sarlak, A., Alzorgan, H., Boroujeni, S.P.H., Razi, A., Amin, R.: Enhanced cooperative perception for autonomous vehicles using imperfect communication (2024). [arXiv:2404.08013](https://arxiv.org/abs/2404.08013)
158. Sato, T., Kano, T., Ishiguro, A.: On the applicability of the decentralized control mechanism extracted from the true slime mold: a robotic case study with a serpentine robot. *Bioinspiration Biomim.* **6**(2), 026006 (2011)
159. Sawaya, W., Sayed, A., Krishnamachari, B.: Multi-agent systems for cooperative artificial intelligence: a survey of challenges and opportunities. *ACM Comput. Surv.* **55**(8), 1–41 (2022)
160. Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G.: The graph neural network model. *IEEE Trans. Neural Netw.* **20**(1), 61–80 (2008)
161. Scheinerman, E.R.: *Mathematics: A Discrete Introduction*. Cengage Learning (2012)
162. Schmickl, T., Crailsheim, K.: Inner nest homeostasis in a changing environment with special emphasis on honey bee brood nursing and pollen supply. *Apidologie* **35**(3), 249–263 (2004)
163. Schmickl, T., Crailsheim, K.: A navigation algorithm for swarm robotics inspired by slime mold aggregation. In: *International Workshop on Swarm Robotics*, pp. 1–13. Springer (2006)
164. Schmidhuber, J.: Self-referential learning: a model of continuous learning and reflective intelligence. *Front. Artif. Intell.* **3**, 10 (2020)
165. Seeley, T.D., Camazine, S., Sneyd, J.: Collective decision-making in honey bees: how colonies choose among nectar sources. *Behav. Ecol. Sociobiol.* **28**, 277–290 (1991)
166. Seeley, T.D., Visscher, P.K.: Choosing a home: how the scouts in a honey bee swarm perceive the completion of their group decision making. *Behav. Ecol. Sociobiol.* **54**, 511–520 (2003)
167. Shah, S., Dey, D., Lovett, C., Kapoor, A.: *AirSim: High-fidelity visual and physical simulation for autonomous vehicles*. In: *Field and Service Robotics: Results of the 11th International Conference*, pp. 621–635 (2018)
168. Shen, Y., Liu, H., Liu, P., Xia, R., Yao, T., Sun, Y., Feng, T.: Detach: cross-domain learning for long-horizon tasks via mixture of disentangled experts (2025). [arXiv:2508.07842](https://arxiv.org/abs/2508.07842)
169. Shi, S., Cui, J., Jiang, Z., Yan, Z., Xing, G., Niu, J., Ouyang, Z.: Vips: Real-time perception fusion for infrastructure-assisted autonomous driving. In: *Proceedings of the 28th Annual International Conference on Mobile Computing and Networking*, pp. 133–146 (2022)
170. Smith, R.G.: The contract net protocol: high-level communication and control in a physically distributed a-l system. *IEEE Trans. Comput.* **100**(12), 1104–1113 (1980)
171. Song, R., Liang, C., Cao, H., Yan, Z., Zimmer, W., Gross, M., Festag, A., Knoll, A.: Collaborative semantic occupancy prediction with hybrid feature fusion in connected automated vehicles. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 17996–18006 (2024)
172. Song, Z., Wen, F., Zhang, H., Li, J.: A cooperative perception system robust to localization errors. In: *2023 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1–6 (2023)
173. Su, S., Li, Y., He, S., Han, S., Feng, C., Ding, C., Miao, F.: Uncertainty quantification of collaborative detection for self-driving. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5588–5594 (2023)
174. Sun, D., Cheng, L., Chen, S., Li, C., Xiao, Y., Luo, B.: Uav-ground visual tracking: a unified dataset and collaborative learning approach. *IEEE Trans. Circuits Syst. Video Technol.* (2023)
175. Sun, K., Li, B., Chen, Q.: Multi-robot path planning with communication constraints. *Robot. Auton. Syst.* **155**, 104169 (2022)
176. Sun, M., Wang, Z., Zhang, B.: Collective perception with multi-agent reinforcement learning. In: *2022 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2022, pp. 9202–9209 (2022)
177. Sun, P., Li, W., Gao, K.: A survey on multi-agent reinforcement learning. *IEEE Trans. Syst. Man Cybern. Syst.* **50**(12), 5264–5279 (2020)

178. Szopek, M., Schmickl, T., Thenius, R., Radspieler, G., Crailsheim, K.: Dynamics of collective decision making of honeybees in complex temperature fields. *PLoS ONE* **8**(10), e76250 (2013)
179. Tian, Y., Chang, Y., Quang, L., Schang, A., Nieto-Granda, C., How, J.P., Carlone, L.: Resilient and distributed multi-robot visual slam: Datasets, experiments, and lessons learned. In: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 11027–11034 (2023)
180. Tian, Y., Wang, Y.: Collaborative localization and mapping for multi-robot systems. *Robot. Auton. Syst.* **162**, 104345 (2023a)
181. Tian, Y., Wang, Y.: Multi-robot information fusion with communication constraints. *IEEE Trans. Cybern.* **53**(4), 1901–1914 (2023b)
182. Tian, Y., Wang, Y.: Robust communication protocols for unreliable multi-robot networks. *IEEE Trans. Cybern.* **53**(3), 1–14 (2023c)
183. Tu, J., Wang, T., Wang, J., Manivasagam, S., Ren, M., Urtasun, R.: Adversarial attacks on multi-agent communication. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 7768–7777 (2021)
184. Vadivelu, N., Ren, M., Tu, J., Wang, J., Urtasun, R.: Learning to communicate and correct pose errors. In: Conference on Robot Learning, pp. 1195–1210 (2021)
185. Vaswani, A.: Attention is all you need (2017). [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)
186. Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., et al.: Graph attention networks. *Stat* **1050**(20), 10–48550 (2017)
187. Wang, B., Zhang, L., Wang, Z., Zhao, Y., Zhou, T.: Core: Cooperative reconstruction for multi-agent perception. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 8710–8720 (2023)
188. Wang, C., Li, W., Guo, Y.: A survey of swarm robotics: concepts, methodologies, and applications. *Front. Robot. AI* **7**, 580801 (2020)
189. Wang, J., Liu, Y., Li, Q.: Learning collective self-healing policies for multi-robot systems. In: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), vol. 2023, pp. 5622–5629 (2023)
190. Wang, K., Li, Q., Wang, L.: Adaptive flocking control for multi-agent systems with unknown leader dynamics. *IEEE Trans. Cybern.* **50**(11), 4776–4787 (2020)
191. Wang, L., Huang, Y.: Lidar-camera fusion for road detection using a recurrent conditional random field model. *Sci. Rep.* **12**(1), 11320 (2022)
192. Wang, L., Pan, J.: Collective perception and decision-making in swarm robotics. *Swarm Intell.* **14**(4), 319–336 (2020)
193. Wang, L., Pan, J.: Towards self-aware multi-robot systems. *J. Auton. Robot.* **45**(1), 101–115 (2021)
194. Wang, M., Cong, M., Du, Y., Liu, D., Tian, X.: Multi-robot raster map fusion without initial relative position. *Robot. Intell. Autom.* **43**(5), 498–508 (2023)
195. Wang, P., Gao, C., Han, F.: Fault-tolerant distributed flocking control for multi-agent systems. In: 2020 IEEE International Conference on Robotics and Automation (ICRA), vol. 2020, pp. 9080–9086 (2020)
196. Wang, P., Li, J., Yang, M.: Fault diagnosis and fault-tolerant control of distributed multi-agent systems: a survey. *IET Control Theory Appl.* **15**(1), 1–13 (2021)
197. Wang, Q., Wu, B., Zhu, P., Li, P., Zuo, W., Hu, Q.: Eca-net: efficient channel attention for deep convolutional neural networks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11534–11542 (2020)
198. Wang, T., Chen, G., Chen, K., Liu, Z., Zhang, B., Knoll, A., Jiang, C.: Umc: a unified bandwidth-efficient and multi-resolution based collaborative perception framework. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 8187–8196 (2023)
199. Wang, T.H., Manivasagam, S., Liang, M., Yang, B., Zeng, W., Urtasun, R.: V2vnet: vehicle-to-vehicle communication for joint perception and prediction. In: Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part II, vol. 16, pp. 605–621 (2020)

200. Wang, T.H., Manivasagam, S., Liang, M., Yang, B., Zeng, W., Urtasun, R.: V2vnet: vehicle-to-vehicle communication for joint perception and prediction. In: Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part II, vol. 16, pp. 605–621 (2020)
201. Wang, X., Jin, Y., Li, C., Cen, Y., Li, Y.: Vsln: view-aware sphere learning network for cross-view vehicle re-identification. *Int. J. Intell. Syst.* **37**(10), 6631–6651 (2022)
202. Wang, Z., Fan, S., Huo, X., Xu, T., Wang, Y., Liu, J., Chen, Y., Zhang, Y.Q.: Vimi: Vehicle-infrastructure multi-view intermediate fusion for camera-based 3d object detection. *arXiv preprint [arXiv:2303.10975](https://arxiv.org/abs/2303.10975)* (2023)
203. Wei, S., Wei, Y., Hu, Y., Lu, Y., Zhong, Y., Chen, S., Zhang, Y.: Asynchrony-robust collaborative perception via bird’s eye view flow. *Adv. Neural Inf. Process. Syst.* **36** (2024)
204. Wei, X., Zhang, T., Li, Y., Zhang, Y., Wu, F.: Multi-modality cross attention network for image and sentence matching. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 10941–10950 (2020)
205. Wen, Y., Zhang, R., Luo, X.: Reflective agents in multi-agent systems: a survey. *Eng. Appl. Artif. Intell.* **103**, 104295 (2021)
206. Winfield, A.F.T., Michael, K.: Ethical governance is essential to building trust in robotics and artificial intelligence systems. *Phil. Trans. R. Soc. A* **376**(2133), 20180085 (2019)
207. Wooldridge, M.: An Introduction to Multiagent Systems. John Wiley Sons (2009)
208. Wu, J., Chen, Z., Li, F.: Collaborative perception in multi-robot systems. *IEEE Trans. Rob.* **39**(6), 1601–1615 (2023a)
209. Wu, J., Chen, Z., Li, F.: Dynamic task allocation for multi-robot swarms. *J. Intell. Robot. Syst.* **106**(3), 1–19 (2023b)
210. Wu, J., Chen, Z., Li, F.: A reflective architecture for self-adaptive multi-agent systems. *Artif. Intell. Rev.* **56**(5), 4301–4320 (2023c)
211. Wu, J., Li, D., Shi, J., Li, X., Gao, L., Yu, L., Han, G., Wu, J., et al.: An adaptive conversion speed q-learning algorithm for search and rescue uav path planning in unknown environments. *IEEE Trans. Veh. Technol.* (2023)
212. Wu, R., Cheng, X., Zhu, J., Liu, X., Zhang, M., Yan, S.: Uavd4l: A large-scale dataset for uav 6-dof localization (2024). [arXiv:2401.05971](https://arxiv.org/abs/2401.05971)
213. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Philip, S.Y.: A comprehensive survey on graph neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* **32**(1), 4–24 (2020)
214. Xiang, H., Xu, R., Ma, J.: Hm-vit: hetero-modal vehicle-to-vehicle cooperative perception with vision transformer. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 284–295 (2023)
215. Xiong, J., Bi, R., Tian, Y., Liu, X., Wu, D.: Toward lightweight, privacy-preserving cooperative object classification for connected autonomous vehicles. *IEEE Internet Things J.* **9**(4), 2787–2801 (2021)
216. Xu, C., Li, Q.: Multi-agent systems with self-adaptive communication. *Automatica* **156**, 110901 (2023)
217. Xu, R., Chen, W., Xiang, H., Xia, X., Liu, L., Ma, J.: Model-agnostic multi-agent perception framework. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 1471–1478 (2023)
218. Xu, R., Guo, Y., Han, X., Xia, X., Xiang, H., Ma, J.: Opencdca: an open cooperative driving automation framework integrated with co-simulation. In: 2021 IEEE International Intelligent Transportation Systems Conference (ITSC), pp. 1155–1162 (2021)
219. Xu, R., Li, J., Dong, X., Yu, H., Ma, J.: Bridging the domain gap for multi-agent perception. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 6035–6042 (2023)
220. Xu, R., Tu, Z., Xiang, H., Shao, W., Zhou, B., Ma, J.: Cobevt: cooperative bird’s eye view semantic segmentation with sparse transformers (2022). [arXiv:2207.02202](https://arxiv.org/abs/2207.02202)
221. Xu, R., Tu, Z., Xiang, H., Shao, W., Zhou, B., Ma, J.: Cobevt: cooperative bird’s eye view semantic segmentation with sparse transformers (2022). [arXiv:2207.02202](https://arxiv.org/abs/2207.02202)

222. Xu, R., Xia, X., Li, J., Li, H., Zhang, S., Tu, Z., Meng, Z., Xiang, H., Dong, X., Song, R., et al.: V2v4real: a real-world large-scale dataset for vehicle-to-vehicle cooperative perception. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 13712–13722 (2023)
223. Xu, R., Xiang, H., Tu, Z., Xia, X., Yang, M.H., Ma, J.: V2x-vit: vehicle-to-everything cooperative perception with vision transformer. In: European Conference on Computer Vision, pp. 107–124 (2022)
224. Xu, R., Xiang, H., Xia, X., Han, X., Li, J., Ma, J.: Opv2v: an open benchmark dataset and fusion pipeline for perception with vehicle-to-vehicle communication. In: 2022 International Conference on Robotics and Automation (ICRA), pp. 2583–2589 (2022)
225. Yan, Z., Yang, Z., Sun, J.: A survey of multi-agent coordination under communication constraints. *IEEE Trans. Ind. Inf.* **9**(4), 2334–2349 (2013)
226. Yang, D., Yang, K., Wang, Y., Liu, J., Xu, Z., Yin, R., Zhai, P., Zhang, L.: How2comm: Communication-efficient and collaboration-pragmatic multi-agent perception. *Adv. Neural Inf. Process. Syst.* **36** (2024)
227. Yang, K., Yang, D., Zhang, J., Li, M., Liu, Y., Liu, J., Wang, H., Sun, P., Song, L.: Spatio-temporal domain awareness for multi-agent collaborative perception. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 23383–23392 (2023)
228. Yang, K., Yang, D., Zhang, J., Wang, H., Sun, P., Song, L.: What2comm: towards communication-efficient collaborative perception via feature decoupling. In: Proceedings of the 31st ACM International Conference on Multimedia, pp. 7686–7695 (2023)
229. Yang, X., Li, J., Yang, Y.: Distributed leader-follower formation control for multi-agent systems with unknown disturbances. *IEEE Trans. Autom. Control* **65**(12), 5096–5108 (2020)
230. Yang, Y., Wang, J., et al.: Survey of distributed reinforcement learning: methods, applications, and challenges. *Artif. Intell.* **287**, 103348 (2020)
231. Ying, Q., Liu, J., Li, S., Xu, H., Qian, Z., Zhang, X.: Retouchingffhq: a large-scale dataset for fine-grained face retouching detection. In: Proceedings of the 31st ACM International Conference on Multimedia, pp. 737–746 (2023)
232. Yu, H., Luo, Y., Shu, M., Huo, Y., Yang, Z., Shi, Y., Guo, Z., Li, H., Hu, X., Yuan, J., et al.: Dair-v2x: a large-scale dataset for vehicle-infrastructure cooperative 3d object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 21361–21370 (2022)
233. Yu, H., Tang, Y., Xie, E., Mao, J., Luo, P., Nie, Z.: Flow-based feature fusion for vehicle-infrastructure cooperative 3d object detection. *Adv. Neural Inf. Process. Syst.* **36** (2024)
234. Yu, H., Yang, W., Ruan, H., Yang, Z., Tang, Y., Gao, X., Hao, X., Shi, Y., Pan, Y., Sun, N., et al.: V2x-seq: a large-scale sequential dataset for vehicle-infrastructure cooperative perception and forecasting. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5486–5495 (2023)
235. Yuan, Y., Cheng, H., Sester, M.: Keypoints-based deep feature fusion for cooperative vehicle detection of autonomous driving. *IEEE Robot. Autom. Lett.* **7**(2), 3054–3061 (2022)
236. Zeng, W., Wang, S., Liao, R., Chen, Y., Yang, B., Urtasun, R.: Dsdnet: deep structured self-driving network. In: Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXI, vol. 16, pp. 156–172 (2020)
237. Zeng, W., Zhang, L.: Decentralized communication for swarm robotics. *Robot. Auton. Syst.* **158**, 104250 (2022a)
238. Zeng, W., Zhang, L.: Information-theoretic analysis of communication in multi-agent systems. *IEEE Trans. Cybern.* **52**(10), 10235–10248 (2022b)
239. Zhang, C., Zhou, W., Qin, W., Tang, W.: A novel uav path planning approach: heuristic crossing search and rescue optimization algorithm. *Expert Syst. Appl.* **215**, 119243 (2023)
240. Zhang, K., Yang, T., Ding, Z., Yang, S., Ma, T., Li, M., Xu, C., Gao, F.: The visual-inertial-dynamical multirotor dataset. In: 2022 International Conference on Robotics and Automation (ICRA), pp. 7635–7641 (2022)
241. Zhang, W., Liu, Y., Han, M.: Knowledge representation and reasoning in multi-agent systems. *Artif. Intell.* **303**, 103632 (2022a)

242. Zhang, W., Liu, Y., Han, M.: Shared mental models in human-robot collaboration. *Front. Robot. AI* **9**, 875432 (2022b)
243. Zhang, W., Liu, Y., Han, M.: Multi-robot systems with self-reflection. *Front. Robot. AI* **10**, 1023456 (2023)
244. Zhang, Y., Li, H., Wang, P.: Collaborative perception in multi-robot systems. In: 2022 IEEE International Conference on Robotics and Automation (ICRA), vol. 2022, pp. 9101–9108 (2022)
245. Zhang, Y., Parker, L.E.: A survey of multi-robot task allocation. *Robot. Auton. Syst.* **57**(4), 345–360 (2013)
246. Zhao, B., Zhang, W., Zou, Z.: Bm2cp: efficient collaborative perception with lidar-camera modalities (2023). [arXiv:2310.14702](https://arxiv.org/abs/2310.14702)
247. Zhao, H., Jiang, L., Jia, J., Torr, P.H., Koltun, V.: Point transformer. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 16259–16268 (2021)
248. Zhao, X., Wang, L., Li, H.: Resilient flocking control for multi-agent systems under denial of service attacks. In: 2022 American Control Conference (ACC), vol. 2022, pp. 1037–1044 (2022)
249. Zhong, K., Sun, J., Liu, K.: Resilient consensus of multi-agent systems subject to deception attacks. *IEEE Trans. Cybern.* **51**(10), 4967–4977 (2021)
250. Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., Sun, M.: Graph neural networks: a review of methods and applications. *AI Open* **1**, 57–81 (2020)
251. Zhou, Y., Xiao, J., Zhou, Y., Loianno, G.: Distributed perception in multi-robot systems. *Robot. Auton. Syst.* **157**, 104200 (2022a)
252. Zhou, Y., Xiao, J., Zhou, Y., Loianno, G.: Multi-robot collaborative perception with graph neural networks. *IEEE Robot. Autom. Lett.* **7**(2), 2289–2296 (2022b)
253. Zhu, H., Chen, J., Li, J.: Autonomous swarm control: a review on algorithms and applications. *Appl. Sci.* **11**(15), 6865 (2021)
254. Zhu, M., Jiao, L., Liu, F., Yang, S., Wang, J.: Residual spectral-spatial attention network for hyperspectral image classification. *IEEE Trans. Geosci. Remote Sens.* **59**(1), 449–462 (2020)
255. Zhu, Z., Li, C., Pan, J., Li, X., Xiao, Y., Chang, Y., Zheng, F., Wang, S.: Medic: A multimodal empathy dataset in counseling. In: Proceedings of the 31st ACM International Conference on Multimedia, pp. 6054–6062 (2023)
256. Zimmer, W., Wardana, G.A., Sritharan, S., Zhou, X., Song, R., Knoll, A.C.: Tumtraf v2x cooperative perception dataset. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 22668–22677 (2024)

Part II

Self-evolving Intelligence

To make this book cutting edge and leading the way, Part II will cover the concepts, key technologies, key components, key architectures, key strategy, and case study of self-evolving embodied AI, self-evolving world model, and self-evolving agent.

Chapter 4

Self-evolving Embodied Intelligence



Abstract This chapter introduces self-evolving embodied Artificial Intelligence (AI), a critical frontier for developing autonomous agents like robots that can learn and adapt directly within the physical world. We define these agents as embodied systems that continuously and systematically refine their own internal models and policies through dynamic, real-world interaction. The chapter analyzes three key technologies enabling this evolution: Meta Learning, which allows rapid task adaptation; Curriculum Learning, which organizes the learning process progressively from simple to complex tasks, and Lifelong Learning, which facilitates knowledge accumulation over time without catastrophic forgetting. We then deconstruct this paradigm into its fundamental components: Self-evolving World Models and Self-evolving Agents. This latter concept encompasses intelligence emerging from embodied cognition and dynamic interaction. This work provides a comprehensive roadmap toward a new generation of AI, shifting from autonomous pattern recognition to self-evolving, physically grounded intelligence.

4.1 The Overview

The advent of large-scale foundation models has marked a significant milestone in artificial intelligence, demonstrating unprecedented capabilities in language understanding, reasoning, and generation [29]. These models, trained on vast datasets [7–9], serve as powerful engines for a new generation of AI agents. However, a fundamental limitation persists: the static nature of their underlying architecture and knowledge base. Most contemporary agent systems are built upon manually crafted configurations that remain fixed after deployment. This static design paradigm creates a critical bottleneck when these agents are deployed in the real world, which is inherently dynamic, non-stationary, and continuously evolving. In such environments, user intentions shift, task requirements change, and new information becomes available, rendering a static agent progressively less effective over time. The manual process of reconfiguring and retraining these complex systems is not only labor intensive and time consuming but also fundamentally unscalable.

This challenge has catalyzed a paradigm shift in AI research, moving away from the development of static, pretrained models toward the creation of adaptive systems capable of autonomous and continuous self-improvement. This emerging field, which we term self-evolving embodied AI, envisions agents that can adaptively reason, act, and evolve in real time based on their interactions and experiences. This chapter provides a comprehensive overview of this nascent field, defining its core concepts, exploring its key enabling technologies, and deconstructing its fundamental components. The transition from static knowledge repositories to dynamic, lifelong learning systems represents a necessary evolutionary step for AI to bridge the gap between the controlled environments of training datasets and the unpredictable complexities of the real world. It moves the central question of AI development from merely what a model knows to how a model learns and adapts throughout its operational lifetime.

Synthesizing recent comprehensive surveys, we can formally define self-evolving embodied AI as follows: self-evolving AI agents are autonomous systems that continuously and systematically optimize their internal components through interaction with their environment. The primary objective is to adapt to changing tasks, contexts, and resources while preserving operational safety and enhancing performance [6, 15]. This paradigm bridges the static capabilities of foundation models with the continuous adaptability required by lifelong agentic systems, enabling a more robust and resilient form of artificial intelligence. To ensure that this evolutionary process remains both beneficial and controlled, a set of guiding principles—analogue to Asimov’s laws of robotics—has been proposed, known as the Three Laws of self-Evolving AI Agents. According to these laws, a self-evolving agent must first endure, ensuring safety and stability throughout any modification; subsequently, it must excel, preserving or enhancing its existing task performance in accordance with the first requirement; and ultimately, it should evolve, autonomously optimizing its internal components once the preceding conditions are satisfied. Collectively, these laws emphasize that the pursuit of autonomous evolution must be carefully constrained by imperatives of safety and performance, thereby providing essential guardrails for the responsible development of this powerful technology.

To systematically analyze and design such systems, the field has developed conceptual frameworks that abstract the core processes of self-evolution. One prominent framework identifies four pillars that constitute the feedback loop driving adaptation.

Inputs. This includes the initial data and conditions provided to the agent, such as task specifications, environmental observations, user queries, and historical interaction logs.

Agent. This encompasses all internal components of the agent that are subject to evolution, including the core foundation model, prompts, memory systems, tool libraries, and operational workflows.

Environment. This represents the external context in which the agent operates. The environment serves as both the testing ground for the agent’s capabilities and the source of feedback signals necessary for learning and improvement.

Optimizers. These are the algorithmic mechanisms responsible for analyzing feedback and implementing enhancements to the agent system. Examples include

gradient-based optimization, reinforcement learning algorithms, and evolutionary strategies.

A complementary framework organizes the field along three foundational dimensions, providing a structured approach to categorizing different evolutionary strategies.

What to Evolve. This dimension examines which components of the agent are being modified, such as the parameters of the underlying model, the structure of the memory, the set of available tools, or the overall agent architecture.

When to Evolve. This dimension categorizes adaptation based on its timing, distinguishing between intra test time evolution (rapid adaptation within a single-task instance) and inter test time evolution (slower, more permanent updates between tasks).

How to Evolve. This dimension analyzes the specific algorithms and architectural designs that guide the evolutionary process, such as methods based on scalar rewards, textual feedback, imitation learning, or population-based evolution.

Together, these definitions and frameworks provide a robust conceptual foundation for understanding the principles, mechanisms, and objectives of self-evolving embodied AI.

This chapter then focuses on three key enabling technologies: Meta Learning, which allows agents to rapidly adapt to new tasks; Curriculum Learning, which organizes the training process progressively from simple to complex tasks; and Lifelong Learning, which enables agents to accumulate knowledge over time while avoiding catastrophic forgetting. Building on these, the chapter further explores the fundamental components of self-evolving embodied AI: the Self-evolving World Model, serving as an internal predictive simulator of the environment to support reasoning and decision-making [19]; and the Self-evolving Agent, which encompasses both Self-evolving Embodied Cognition, emphasizing that intelligence emerges from the coupling of perception, action, and the agent's body, and Self-evolving Dynamic Interaction, which enables complex collaboration and communication in multi-agent systems.

4.2 Key Technologies of Self-evolving Embodied Intelligence

The realization of self-evolving Embodied AI is not predicated on a single breakthrough but rather on the synergistic integration of several mature and emerging machine learning paradigms. These technologies provide the algorithmic machinery that enables agents to learn from experience, adapt to novelty, and accumulate knowledge over time. This section provides a detailed technical survey of three cornerstone technologies: Meta Learning, which provides the foundation for rapid adaptation; Curriculum Learning, which structures the learning process for optimal knowledge

acquisition; and Lifelong Learning, which ensures the retention of knowledge in a continuously changing world.

4.2.1 *Meta Learning*

Meta learning, often described as “learning to learn,” is a subfield of machine learning that aims to develop models capable of improving their own learning algorithm based on experience from multiple tasks [4, 36]. The standard meta learning framework is commonly decomposed into two sequential stages. During meta training, a model is exposed to a distribution of tasks and learns a strategy that facilitates knowledge transfer. At meta testing, the model is evaluated on unseen tasks where it must adapt quickly with only a few examples. For a self-evolving agent, the primary utility of meta learning is its ability to confer rapid adaptation. Instead of learning a single-task from scratch, a meta trained agent learns a high-level strategy or a well-disposed parameter initialization that allows it to generalize and master new, previously unseen tasks with very few training examples [12]. This capability is indispensable for embodied agents, such as robots, which must operate efficiently and adaptively in a wide variety of dynamic environments where data collection is expensive and time consuming. The meta learning process is typically bifurcated into a meta training phase, where the model learns across a distribution of related tasks, and a meta testing phase, where its ability to quickly adapt to novel tasks is evaluated.

Four primary paradigms dominate the literature: optimization-based meta learning, metric-based meta learning, model-based meta learning, and memory-based meta learning. Each reflects a distinct architectural philosophy regarding how adaptation should be encoded.

A. Optimization-Based Meta Learning

This family of methods learns an initial set of model parameters that is explicitly optimized for fast adaptation via a few steps of gradient descent. The seminal work in this area is Model Agnostic Meta Learning (MAML) [12]. MAML is distinguished by its generality; it is compatible with any model that can be trained with gradient-based optimization, including deep neural networks for classification, regression, and reinforcement learning. The architecture of MAML is characterized by a bi-level optimization process, as illustrated in Fig. 4.1.

The inner loop performs task-specific adaptation. For a given task $\mathcal{T}_i$ sampled from a distribution of tasks $p(\mathcal{T})$, the model’s parameters θ are updated using a small number of data points (the support set) from that task. For a single gradient step, the updated, task-specific parameters θ'_i are computed as shown in Eq. 4.1.

$$\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(f_{\theta}) \quad (4.1)$$

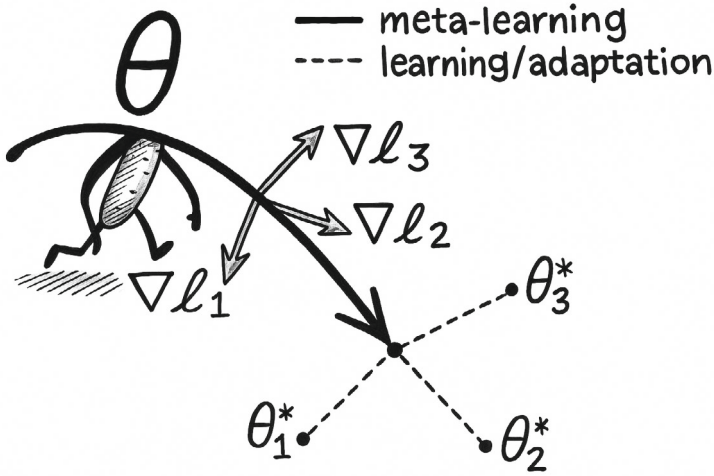


Fig. 4.1 A diagram of the Model Agnostic Meta Learning (MAML) algorithm

Here, $\mathcal{L}_{\mathcal{T}_i}$ is the loss function for task $\mathcal{T}_i$, f_θ is the model parameterized by θ , and α is the inner loop learning rate.

The outer loop performs the meta optimization. The goal is to improve the initial parameters θ based on the performance of the adapted parameters θ'_i on a separate dataset (the query set) for each task. The meta loss is the sum of the losses across all tasks, and the meta parameters θ are updated by taking the gradient of this meta loss. This crucially involves differentiating through the inner loop update step, a gradient through gradient computation, as shown in Eq. 4.2.

$$\theta \leftarrow \theta - \beta \nabla_\theta \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i}(f_{\theta'_i}) \quad (4.2)$$

Here, β is the meta learning rate. By optimizing in this manner, MAML finds an initialization θ that is not necessarily optimal for any single-task but is positioned in the parameter space such that it is highly sensitive to new tasks, allowing for substantial performance gains with minimal fine tuning.

B. Metric-Based Meta Learning

This approach focuses on learning a feature embedding space where new examples can be classified by comparing them to a few labeled examples. Instead of learning an update rule, these methods learn a metric or distance function. Prototypical Networks are a leading example of this paradigm [45]. They operate on the inductive bias that there exists an embedding space where data points for each class cluster around a single representative point, or prototype.

The architecture consists of three main components. First, an embedding function f_ϕ , typically a deep neural network with parameters ϕ , maps input data into a high-dimensional vector space. Second, for each class k within a given few-shot learning episode, a prototype vector c_k is computed by taking the mean of the embeddings of the support set examples belonging to that class, as defined in Eq. 4.3.

$$c_k = \frac{1}{|S_k|} \sum_{(x_i, y_i) \in S_k} f_\phi(x_i) \quad (4.3)$$

Here, S_k is the set of support examples for class k . Finally, a new query point x is classified by computing a probability distribution over the classes based on a softmax over the negative distances to each class prototype, as expressed in Eq. 4.4. Squared Euclidean distance is commonly used, as it corresponds to the assumption of a Gaussian distribution around each prototype.

$$p_\phi(y = k|x) = \frac{\exp(-d(f_\phi(x), c_k))}{\sum_{k'} \exp(-d(f_\phi(x), c_{k'}))} \quad (4.4)$$

The model is trained end to end by minimizing the negative log likelihood of the true class for the query points. The simplicity and computational efficiency of Prototypical Networks make them a powerful and effective method for few-shot learning scenarios where rapid, non-parametric adaptation is required.

C. Model-Based Meta Learning

Model-based meta learning develops specialized architectures designed for rapid adaptation. Instead of re-optimizing parameters for each new task, these models employ internal mechanisms that support fast generalization. One notable example is the use of recurrent neural networks (RNNs) that process task sequences and encode task-specific strategies within hidden states [20, 39]. Memory Augmented Neural Networks (MANNs) extend this idea by incorporating external memory components for storing task-relevant information [41]. In this category, adaptation is achieved not through explicit gradient updates but through architectural properties that enable contextual reasoning. This allows near instantaneous adaptation once sufficient representational capacity has been acquired during meta training.

D. Memory-Based Meta Learning

Memory-based approaches focus on explicitly storing past task experiences and retrieving them for new task adaptation. These methods are distinguished from model-based architectures by their emphasis on external memory retrieval. Meta Networks [34] pioneered this framework by integrating fast weights into neural architectures. Neural Episodic Control [38] and Gradient Episodic Memory [31]

further illustrated how episodic memory facilitates continual learning and mitigates catastrophic forgetting. Such mechanisms are particularly valuable for agents operating in non-stationary environments, where task distributions evolve over time. By maintaining structured memory of past interactions, these models achieve robust adaptation and lifelong learning.

4.2.2 Curriculum Learning

Curriculum Learning (CL) is a training methodology inspired by the human cognitive process of learning, where concepts are introduced in a structured sequence, typically progressing from simple to more complex. When applied to machine learning, this strategy involves organizing the training data or tasks in a meaningful order to guide the model's learning process. This approach has been shown to accelerate model convergence, improve final performance, and enhance generalization by helping the optimizer avoid poor local minima in the loss landscape. In the context of reinforcement learning, CL is particularly valuable for tackling tasks with sparse rewards or vast exploration spaces. By starting with easier sub-tasks, an agent can acquire fundamental skills and build a useful value function or policy representation, which can then be leveraged to solve the more challenging target task.

Early work by Bengio et al. [5] established the foundation of CL in natural language modeling, showing the benefit of ordering samples by difficulty. Kumar et al. [26] proposed self-Paced Learning (SPL), where the model dynamically selects which examples to learn based on a latent optimization framework. In reinforcement learning, Florensa et al. [13] introduced an automatic goal generation strategy, which enables agents to progressively acquire skills by solving incrementally complex tasks.

The design of any curriculum learning system can be conceptualized within a general framework comprising two essential components, as systematically reviewed by Wang et al. [49].

Difficulty Measurer. This component is responsible for quantifying the difficulty of a given training sample or task. The notion of “difficulty” is domain specific and can be defined by various heuristics. For instance, in natural language processing, the length of a sentence could serve as a proxy for difficulty. In computer vision, the level of occlusion or noise in an image might be used. Alternatively, difficulty can be defined in relation to the model's current state, such as using the training loss on a sample as an indicator of how challenging it is for the model.

Training Scheduler. This component dictates the progression of the curriculum, determining how and when to introduce more difficult data to the model. Schedulers can be predefined and static, such as increasing the maximum sentence length allowed in the training batch every ten epochs. More advanced schedulers are dynamic and adaptive, adjusting the curriculum based on the model's learning progress, for example, by increasing data difficulty once the model's performance on the current subset reaches a certain threshold (Fig. 4.2)..

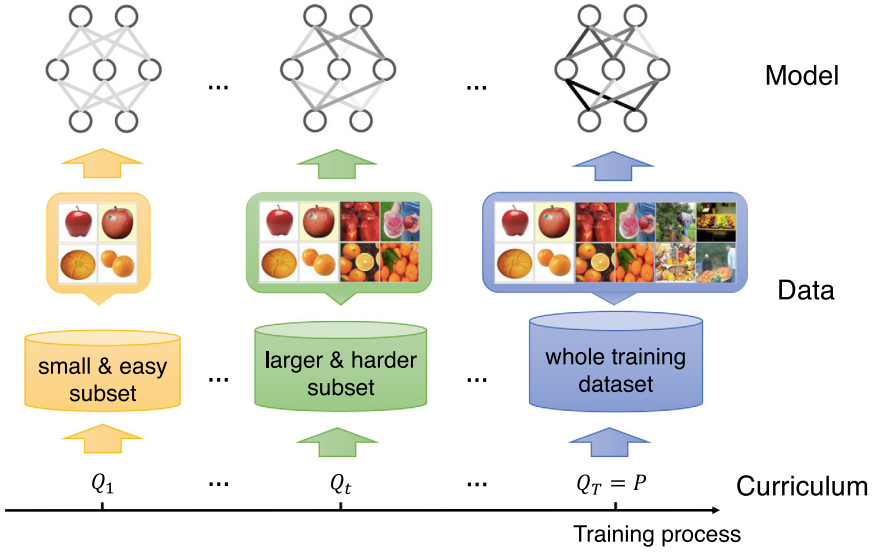


Fig. 4.2 An illustration of the curriculum learning concept

While early implementations of CL often relied on human experts to manually design the curriculum, the drive toward greater autonomy has led to the development of methods for automatic curriculum generation. This is particularly crucial for self-evolving agents, which must be capable of structuring their own learning experiences. Several key strategies have emerged in this domain.

Teacher Student Frameworks. This paradigm involves two models: a “student” model that learns the target task, and a “teacher” model that learns to select or generate appropriate sub-tasks for the student. The teacher is often modeled as a reinforcement learning agent or a multi-armed bandit, where its “actions” are the tasks it proposes, and its “reward” is based on the learning progress of the student. The teacher learns a policy to present tasks that are neither too easy (offering no new information) nor too hard (providing no learnable signal), thereby maximizing the student’s learning efficiency.

Self-Paced Learning (SPL). In SPL, the model itself acts as its own teacher, selecting which samples to learn from at each stage of training. This is typically formulated as an optimization problem that jointly learns the model parameters and a set of latent weight variables for each training sample. Initially, the model assigns higher weights to samples with low training loss (the “easy” samples). As the model’s parameters improve, its confidence grows, and it gradually assigns higher weights to more complex samples, effectively creating its own curriculum.

Automatic Goal Generation in Reinforcement Learning. In multi-goal RL settings, a curriculum can be automatically generated by manipulating the goals presented to the agent. For a robotic arm learning a reaching task, for instance, the curriculum can start by sampling goals that are very close to the arm’s initial position.

As the agent becomes proficient at reaching these nearby goals, the goal generation mechanism starts sampling goals that are progressively farther away or in more challenging locations, such as behind obstacles. This strategy, often combined with techniques like Hindsight Experience Replay (HER) that allow learning from failed attempts, effectively guides the agent's exploration and skill acquisition process.

The research community has extended CL into multiple areas. In natural language processing, CL is applied in the pretraining of large-scale language models. In computer vision, SPL variants have been used to improve robustness and efficiency. Furthermore, Matisen et al. [32] proposed Teacher Student Curriculum Learning, where a teacher model adaptively designs curricula for a student, effectively combining CL with meta learning principles.

Despite its successes, several open issues remain. First, automating the design of curricula remains an unsolved problem, as most approaches rely on handcrafted heuristics or task-specific criteria. Second, evaluating the effectiveness of curricula lacks a standardized metric. Finally, extending CL to multi-task and multimodal settings, as well as integrating it seamlessly with meta learning, is a promising but still underexplored direction.

4.2.3 Lifelong Learning

Lifelong Learning (LL), also known as continual learning, refers to the ability of a model to sequentially learn multiple tasks without catastrophic forgetting of previously acquired knowledge [35]. LL has become an essential paradigm for building AI systems that adapt over time in dynamic and open-ended environments.

LL is a hallmark of biological intelligence but remains a formidable challenge for artificial systems. The central problem in lifelong learning is the stability plasticity dilemma: a model must be plastic (adaptive) enough to acquire new knowledge and skills from new data, while also being stable enough to prevent the degradation or erasure of previously learned knowledge. The failure to maintain old knowledge upon learning new tasks is a well-documented phenomenon in neural networks known as catastrophic forgetting or catastrophic interference. This occurs because standard training algorithms, such as stochastic gradient descent, modify network weights to optimize for the current task, inadvertently overwriting the weight configurations that were crucial for performance on previous tasks. For a self-evolving agent intended for long-term deployment, overcoming catastrophic forgetting is not just a desirable feature but a fundamental requirement.

To address this challenge, three main families of techniques have been developed, each employing a different strategy to balance stability and plasticity (Fig. 4.3).

Regularization-Based Methods. These methods mitigate forgetting by introducing an additional penalty term into the loss function during training on new tasks. This penalty discourages significant changes to network parameters that have been identified as important for performance on previous tasks. A landmark algorithm in this category is Elastic Weight Consolidation (EWC) [25]. EWC computes a per

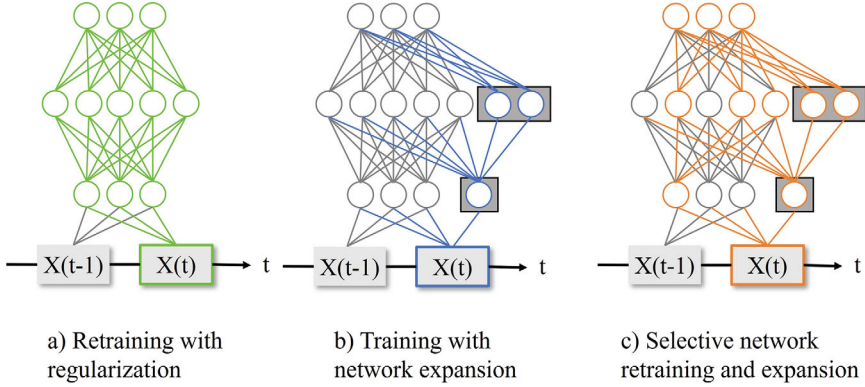


Fig. 4.3 A conceptual comparison of three continual learning strategies

parameter importance weight based on the diagonal of the Fisher Information Matrix, which approximates the curvature of the loss landscape and thus the sensitivity of the model's output to changes in that parameter. When training on a new task B after having learned task A, the EWC loss function is given in Eq. 4.5:

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \sum_i \frac{\lambda}{2} F_i (\theta_i - \theta_{A,i}^*)^2 \quad (4.5)$$

Here, $\mathcal{L}_B(\theta)$ is the loss for the new task, $\theta_{A,i}^*$ are the parameter values after training on task A, F_i is the estimated Fisher information for parameter i on task A, and λ is a hyperparameter that controls the trade-off between learning the new task and retaining the old one. By selectively applying a stronger penalty to more important weights, EWC creates an “elastic” constraint that allows less important weights to change more freely.

Replay-Based Methods. Replay-based (or memory based) methods tackle forgetting by storing a small subset of data from past tasks in an explicit memory buffer. During training on a new task, samples from this buffer are “replayed” alongside the new data, effectively creating a mixed batch that reminds the model of its previous knowledge. This direct rehearsal of past experiences is a highly effective, albeit conceptually simple, strategy for preventing catastrophic forgetting. A significant challenge is the memory cost and potential privacy issues associated with storing raw data. To circumvent this, Generative Replay has been proposed. In this approach, instead of storing data samples, a generative model (like a Variational Autoencoder or a Generative Adversarial Network) is trained on the data of each task. When learning a new task, this generative model is used to produce “pseudo samples” from previous tasks, which are then used for replay, thus avoiding the need to store the original data.

Architectural Methods. These methods prevent interference between tasks by dynamically modifying the model's architecture. The core idea is to allocate distinct

sets of parameters for different tasks, thereby isolating their respective knowledge bases. A foundational example is Progressive Neural Networks [40]. When the model is presented with a new task, a new neural network “column” is added to the architecture. The parameters of all previously trained columns are frozen, making forgetting impossible. Each layer in the new column receives input not only from the preceding layer within its own column but also, via lateral connections, from the preceding layers of all previously trained columns. This allows the model to leverage features learned from past tasks (positive forward transfer) while ensuring that old knowledge is perfectly preserved. The primary drawback of this approach is that the model size grows linearly with the number of tasks, which can be unsustainable for a truly lifelong learning agent.

The field has rapidly expanded toward task-agnostic continual learning, where task boundaries are not explicitly provided, making learning more realistic but also more challenging. At the same time, researchers emphasize efficiency: balancing computational cost and memory constraints is crucial for scaling LL to real-world applications. LL is also increasingly combined with meta learning and curriculum learning, forming adaptive systems that can both remember past tasks and generalize rapidly to new ones.

Key challenges remain unresolved. First, the prevention of catastrophic forgetting without unlimited memory or parameter growth is still an open research problem. Second, designing benchmarks that reflect realistic lifelong scenarios, rather than artificially separated tasks, is essential for evaluating progress. Finally, developing unified frameworks that integrate LL with other paradigms, such as meta learning and CL, is a promising future direction.

The three paradigms discussed above address distinct aspects of adaptability and robustness. A clear comparison of their goals, mechanisms, and roles can highlight how they complement one another in shaping self-evolving artificial intelligence systems. Table 4.1 provides a structured summary of these paradigms.

The comparative analysis in Table 4.1 makes it evident that the three paradigms address different but interconnected dimensions of adaptive intelligence. Meta Learning ensures efficiency in acquiring new tasks, Curriculum Learning emphasizes the progressive organization of knowledge acquisition, and Lifelong Learning safeguards stability across extended temporal horizons. Together, these approaches constitute a complementary toolkit for developing agents that are capable of continuous and self-directed evolution in complex environments.

4.3 Key Components of Self-evolving Embodied Intelligence

Having established the foundational technologies, this section deconstructs the concept of self-evolving embodied AI into three core, interdependent pillars. True autonomy in artificial agents requires not only the possession of advanced capabilities but

Table 4.1 Comparative analysis of key enabling technologies for self-evolving embodied AI

Technology	Primary goal	Core mechanism	Role in self-evolving AI
Meta learning	Rapid adaptation to new tasks with minimal data	Learning an optimization process or a metric space across a distribution of tasks	Enables the agent to learn to learn, providing the fundamental capability to adapt its skills and strategies efficiently in novel situations
Curriculum learning	Improve convergence and generalization by structuring the learning experience	Organizing training data or tasks in a meaningful order, typically from easy to hard	Allows the agent to autonomously structure its own learning path, acquiring foundational skills on simpler problems before tackling more complex ones, thus optimizing its developmental trajectory
Lifelong learning	Accumulate knowledge over time without catastrophic forgetting	Mitigating interference between sequentially learned tasks via regularization, replay, or architectural modifications	Provides the necessary stability for long-term autonomy, ensuring that as the agent evolves and acquires new knowledge, it does not lose previously mastered capabilities

also the capacity to actively refine and improve them over a lifetime of interaction. This evolution is predicated on a continuous cycle of learning, adaptation, and interaction. The first pillar is the self-evolving world model, an agent’s internal, predictive understanding of its environment that must continuously adapt to maintain fidelity in a non- stationary world. The second is self-evolving embodied cognition, which grounds this understanding in physical interaction, allowing intelligence to emerge and develop from the reciprocal interplay between an agent’s body, its perception, and its environment. The third and final pillar, self-evolving dynamic interaction, extends these principles to multi-agent systems, where collective intelligence and social structures emerge through learned coordination, communication, and mutual adaptation. Together, these components form a blueprint for agents that do not merely execute pre-programmed behaviors but grow, learn, and evolve into more capable and intelligent entities.

4.3.1 Self-evolving World Model

A world model is an internal representation that an agent learns to encapsulate the dynamics of its environment. Functionally, it is a generative neural network that learns a compressed spatial and temporal representation, serving as a predictive engine or an internal simulator [17, 47]. This capability allows an agent to forecast the future consequences of its actions without executing them in the real world. By querying its world model, an agent can “imagine” or “dream” future trajectories, enabling sophisticated planning and decision-making [18]. This paradigm is foundational to model-based reinforcement learning, as it has been demonstrated that an agent can be trained entirely within its own simulated environment, with the resulting policy successfully transferring back to the actual environment [17]. For an agent operating in a dynamic, non-stationary world, a static world model is insufficient. The model itself must evolve to maintain its fidelity with a changing reality [24].

The primary obstacle to the development of a self-evolving world model is catastrophic forgetting. This phenomenon occurs when a neural network, upon learning a new task or adapting to new environmental dynamics, overwrites the synaptic weights that encode previously acquired knowledge [23, 37]. A self-evolving world model [10, 11] therefore requires specialized mechanisms designed explicitly to manage the trade-off between stability and plasticity (Fig. 4.4).

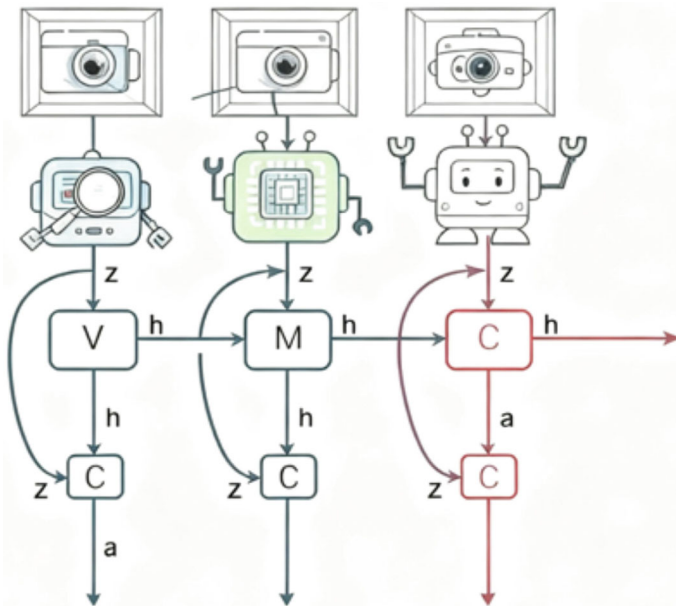


Fig. 4.4 The conceptual diagram of a World Model architecture

Lifelong Learning via Memory and Replay. To overcome catastrophic forgetting, a world model must be equipped with mechanisms for lifelong or continual learning [30]. One direct strategy is the use of an experience replay buffer that persists across tasks [24]. As the agent learns, it stores experiences in a shared buffer. When the world model is trained, it samples not only from recent experiences but also from this historical buffer, effectively rehearsing past tasks while learning new ones. A more sophisticated alternative is Deep Generative Replay, where the agent trains a generative model to capture the data distribution of previous tasks [42]. When a new task is introduced, this generator creates synthetic pseudo data that mimics past experiences. These generated samples are then interleaved with real data from the new task to train the agent’s primary model, allowing it to learn the new task without forgetting the old ones [16].

Modeling Dynamic Environmental Structures. A more advanced paradigm for world model evolution focuses on understanding the underlying structure of environmental change. This can be achieved by conceptualizing the environment as a dynamic graph, where entities are nodes and their relationships are edges [57]. Changes in the environment can be formally modeled as spatio-temporal distribution shifts in the underlying graph structure. Dynamic Graph Neural Networks are designed to learn on such evolving structures. Frameworks such as Environment Aware dynamic Graph Learning (EAGLE) explicitly model and infer latent environmental variables that govern the graph’s evolution, enabling the model to identify invariant patterns across different environmental conditions [58]. This elevates the concept of a self-evolving world model from merely adapting to new data points to understanding the causal structure of environmental change.

4.3.2 *Self-evolving Agent*

While the world model provides an agent with a predictive understanding of its environment, the self-evolving agent is the core component that leverages this understanding to act, learn, and improve. The evolution of an embodied agent is a multifaceted process that occurs along two critical and interconnected axes: the refinement of its individual intelligence through direct physical experience, and the development of its collective intelligence through interaction with other agents.

The first axis, Self-evolving Embodied Cognition, explores how intelligence is intrinsically linked to the agent’s physical form. It posits that an agent’s evolution is driven by the dynamic feedback loop between its body, its actions, and its perception of the world. The second axis, Self-evolving Dynamic Interaction, extends this concept to multi-agent systems, where evolution occurs at a social level. Here, agents must learn to coordinate, communicate, and develop shared conventions to achieve collective goals. The following sections deconstruct these two fundamental aspects of agent evolution.

A. Self-evolving Embodied Cognition

The theory of embodied cognition challenges the classical, disembodied view of intelligence as a purely computational process residing within a central processor. Instead, it posits that cognition is fundamentally shaped by, and emerges from, the dynamic, reciprocal interactions between an agent's brain, its physical body (morphology and sensorimotor capabilities), and its environment. From this perspective, the body is not merely a passive peripheral for sensing and acting but an active and constitutive component of the cognitive process itself. The principle that "cognition is for action" suggests that our cognitive faculties evolved to support effective interaction with the world, and thus, an agent's physical form and capabilities constrain and enable its potential for intelligence. For an embodied AI, this means that its intelligence must be grounded in its physical or virtual existence.

The evolution of embodied cognition occurs through at least two interconnected mechanisms, which allow the agent to refine its interaction with the world over time.

Self-Improvement via Active Perception. Learning How to Perceive. An embodied agent is not a passive observer; it actively participates in the perceptual process. Active perception refers to the agent's ability to intelligently control its sensors to acquire more informative data and resolve ambiguities about the state of the world. This is a form of self-evolution where the agent learns to improve its own data acquisition strategy. For instance, a robot attempting to identify an object may learn that its initial view is uninformative. Through reinforcement learning, it can discover a policy of actions, such as moving its camera or manipulating the object, that reveals distinguishing features (e.g., rotating a cup to find its handle). This closed-loop interaction, where perception guides action and action informs perception, allows the agent to evolve a more efficient and robust perceptual system tailored to its tasks and environment.

Co-evolution of Morphology and Control. A more profound form of self-evolution involves the simultaneous and coupled development of an agent's physical form (morphology) and its control policy (cognition/brain) [3, 21]. This concept is a central theme in developmental and evolutionary robotics [33], drawing inspiration from the ontogenetic development of biological organisms. In nature, an organism's body and brain co-develop: changes in morphology create new challenges and opportunities for the control system, which in turn drives further learning and adaptation. For example, the changing proportions and increasing strength of a human infant's body provide a natural curriculum for acquiring motor skills like crawling and walking. In artificial systems, seminal work on evolving virtual creatures demonstrated this principle by using genetic algorithms to simultaneously evolve block-based body plans and neural network controllers [44], leading to the emergence of diverse and effective locomotion strategies not explicitly pre-programmed but discovered through simulated evolution. More generally, co-evolutionary algorithms can optimize both robot morphology and control parameters, enabling the discovery of synergistic relationships whereby the body offloads significant computational complexity from the controller. By treating the body itself as a set of optimizable parameters, the agent can actively shape its own evolutionary landscape.

The research from our group provides critical insights into this area. The paper “Embodied AI: From LLMs to World Models” explicitly frames embodied interaction as the crucial link between abstract knowledge, as found in LLMs, and functional, grounded intelligence [53]. Furthermore, the framework proposed in “Embodied-R: Collaborative Framework for Activating Embodied Spatial Reasoning in Foundation Models via Reinforcement Learning” is a direct implementation of self-evolving cognition [54]. In this work, an agent uses reinforcement learning to learn how to interact with its environment to activate and ground its spatial reasoning capabilities, effectively evolving its own cognitive understanding of space through embodied experience. Finally, the work on “Improving Compositional Generalization in Cross Embodiment Learning” addresses the critical challenge of how learned cognitive abilities can be generalized across different physical bodies, a key step toward creating truly adaptive and versatile embodied agents [55].

B. Self-evolving Dynamic Interaction

Self-evolving embodied AI also extends to multi-agent systems (MAS), where intelligence emerges from the complex and dynamic interactions among multiple autonomous entities [43, 48]. In such systems, agents must learn to coordinate their actions to achieve shared objectives (cooperation) or to compete for limited resources (competition) within a shared environment. A core challenge lies in the inherently non-stationary nature of the environment for each agent, as the policies of other agents are constantly changing and adapting [22]. This necessitates a continuous process of mutual adaptation and the evolution of interaction strategies.

The primary mechanism driving the self-evolution of interaction in multi-agent systems is emergent communication, where agents learn to develop their own shared symbolic system from the ground up to facilitate coordination [14, 46]. In this paradigm, agents are not endowed with a pre-programmed communication protocol. Instead, they learn within a Multi-Agent Reinforcement Learning (MARL) framework, where the communication channel is part of the action space. Initially, agents may produce random messages, but through reinforcement on a shared team reward, signals that lead to successful outcomes are reinforced.

Frameworks such as Reinforced Inter-Agent Learning (RIAL) and Differentiable Inter-Agent Learning (DIAL) provide mechanisms for this emergence [14]. While RIAL uses independent deep Q-learning, DIAL improves upon this by leveraging centralized training. It treats the communication channel as a differentiable connection, allowing error gradients to be backpropagated from a listening agent to a speaking agent. This end-to-end learning provides direct, causal credit assignment that informs the speaker how to adjust its message to improve the team’s performance.

The learned protocols that arise from emergent communication are, in essence, social conventions, which are arbitrary but stable patterns of behavior that are collectively shared and facilitate coordination [28, 59]. These conventions form the basis of more complex social structures and allow agents to navigate and resolve social dilemmas, which are situations where actions that are rational for an individual lead

to outcomes that are detrimental for the group [22, 27]. Foundational work on the evolution of cooperation demonstrated how simple reciprocal strategies could lead to stable cooperation in iterated games [1, 2]. Modern multi-agent reinforcement learning extends this to more complex, temporally extended scenarios known as sequential social dilemmas [27]. Through reinforcement on shared rewards, communicative behaviors that lead to mutually beneficial outcomes are reinforced, while those that lead to conflict are suppressed. Over time, these stable patterns of communicative behavior become social norms that govern the interactions within the agent society.

The extensive research from our group on dynamic graphs and graph neural networks offers a formal and powerful lens through which to analyze and model these evolving multi-agent interactions. A multi-agent system can be naturally represented as a dynamic graph, where the agents are nodes and their communication links or interaction patterns form the edges. The state of the system, including the agents' internal states and their communication protocol, evolves over time, making it a dynamic graph learning problem. Papers such as "Dynamic Spatio-Temporal Graph Reasoning for VideoQA" and "Disentangled Dynamic Graph Attention Network for Out of distribution Sequential Recommendation" provide sophisticated architectures for capturing and reasoning about such evolving relational structures [52, 60]. Furthermore, tutorials on topics like "Multi-Modal Generative AI in Dynamic and Open Environment" and research on multi-agent reinforcement learning directly address the core challenges of how multiple agents can learn to coordinate and adapt in complex, open-ended worlds, which is the essence of evolving dynamic interaction [50, 56].

Building on the general enabling paradigms discussed earlier, it is also valuable to examine specific research contributions that have advanced the components of self-evolving embodied AI. Among them, the works of Wang et al. represent influential examples that provide concrete methods and frameworks supporting different aspects of self-evolution. Table 4.2 summarizes these contributions according to the three identified components.

The comparative view in Table 4.2 highlights how individual contributions can be mapped onto broader components of self-evolving embodied AI. By aligning methodological advances with world modeling, embodied cognition, and dynamic interaction, these works illustrate concrete pathways through which theoretical paradigms can be instantiated in practical research.

Table 4.2 The key components of self-evolving embodied AI, highlighting how each work advances world modeling, embodied cognition, and dynamic interaction

Component	Relevant publication/tutorial	Key contribution to self-evolution
Self-evolving World Model	Dynamic Graph Neural Networks Under Spatio-Temporal Distribution Shift [57]	Provides a framework for updating world models by treating environmental dynamics as a graph under distribution shift, enabling continual adaptation
Self-evolving Embodied Cognition	Embodied-R: Collaborative Framework for Activating Embodied Spatial Reasoning in Foundation Models via Reinforcement Learning [54]	Demonstrates how an agent can evolve its cognitive abilities (spatial reasoning) through active, embodied interaction with the environment, driven by reinforcement learning
Self-evolving Dynamic Interaction	Dynamic Spatio-Temporal Modular Network for Video Question Answering [51]	Introduces a modular network that dynamically reasons over spatio-temporal interactions, providing a mechanism for modeling the evolving relationships between agents or objects in a scene

References

1. Axelrod, R.: The evolution of cooperation. Basic books (1984)

2. Axelrod, R., Hamilton, W.D.: The evolution of cooperation. *Science* **211**(4489), 1390–1396 (1981)

3. Aydin, E.: Co-evolution of morphology and control for a modular robot. Master’s thesis, EPFL (2012)

4. Beck, J., Eimer, R., Codel, C., Wu, Z., Zhao, L., Kumar, S., Laradji, I., Tian, M., Rodriguez, M., Hessel, M., et al.: A survey of meta-reinforcement learning. *arXiv preprint [arXiv:2301.08028](#)* (2023)

5. Bengio, Y., Louradour, J., Collobert, R., Weston, J.: Curriculum learning. In: *Proceedings of the 26th Annual International Conference on Machine Learning*, pp. 41–48. ACM (2009)

6. Fang, J., Peng, Y., Zhang, X., Wang, Y., Yi, X., Zhang, G., Xu, Y., Wu, B., Liu, S., Li, Z., et al.: A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint [arXiv:2508.07407](#)* (2025)

7. Feng, T., Li, Q., Wang, X., Wang, M., Li, G., Zhu, W.: Multi-weather cross-view geo-localization using denoising diffusion models. In: *Proceedings of the 2nd Workshop on UAVs in Multimedia: Capturing the World from a New Perspective*, pp. 35–39 (2024)

8. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata: A large-scale cooperative perception dataset for swarm uavs autonomous flight. In: *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 7600–7608 (2024)

9. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata-2: A scalable swarm uavs autonomous flight dataset for long-horizon tasks. arXiv preprint [arXiv:2509.00055](#) (2025)
10. Feng, T., Wang, X., Jiang, Y.G., Zhu, W.: Embodied ai: From llms to world models. In: IEEE Circuits and Systems Magazine (2025)
11. Feng, T., Wang, X., Zhou, Z., Wang, R., Zhan, Y., Li, G., Li, Q., Zhu, W.: Evoagent: Agent autonomous evolution with continual world model for long-horizon tasks. arXiv preprint [arXiv:2502.05907](#) (2025)
12. Finn, C., Abbeel, P., Levine, S.: Model-agnostic meta-learning for fast adaptation of deep networks. In: International conference on machine learning, pp. 1126–1135. PMLR (2017)
13. Florensa, C., Held, D., Wulfmeier, M., Zhang, M., Abbeel, P.: Automatic goal generation for reinforcement learning agents. In: Proceedings of the 34th International Conference on Machine Learning, pp. 1514–1523 (2017)
14. Foerster, J., Assael, Y.M., de Freitas, N., Whiteson, S.: Learning to communicate with deep multi-agent reinforcement learning. In: Advances in neural information processing systems, vol. 29 (2016)
15. Gao, H.a., Hua, W., Liu, H., Xiao, H., Zhang, J., Xiang, J., Ren, Q., Geng, J., Zhang, Z., Zhang, Y., et al.: A survey of self-evolving agents: On path to artificial super intelligence. arXiv preprint [arXiv:2507.21046](#) (2025)
16. Gao, R., Liu, W.: Ddgr: Continual learning with deep diffusion-based generative replay. In: Proceedings of the 40th International Conference on Machine Learning (2023)
17. Ha, D., Schmidhuber, J.: World models. arXiv preprint [arXiv:1803.10122](#) (2018)
18. Hafner, D., Lillicrap, T., Ba, J., Norouzi, M.: Dream to control: Learning behaviors by latent imagination. In: International Conference on Learning Representations (2019)
19. Han, F., Zhang, L., Wang, X., Zhao, K.A., Zhong, Y., Su, Z., Feng, T., Zhu, W.: U2usim—a uav telepresence simulation platform with multi-agent sensing and dynamic environment. In: Proceedings of the 32nd ACM International Conference on Multimedia, p. 11258–11260 (2024)
20. Hochreiter, S., Younger, A.S., Conwell, P.R.: Learning to learn using gradient descent. In: International Conference on Artificial Neural Networks, pp. 87–94. Springer (2001)
21. Howard, D., Glette, K., Cheney, N.: Editorial: Evolving robotic morphologies. *Frontiers in Robotics and AI* **9**, 874853 (2022)
22. Hughes, E., Tacchetti, A., Graepel, T., Gemp, I., Leibo, J.Z.: Learning to resolve social dilemmas: a survey. *J. Artif. Intell. Res.* **79**, 1011–1048 (2024)
23. Kaushik, S., Hein, M., Schiele, B.: Understanding catastrophic forgetting in neural networks. arXiv preprint [arXiv:2110.12827](#) (2021)
24. Kessler, S., Miloš, P., Parker-Holder, J., Roberts, S.J.: The surprising effectiveness of latent world models for continual reinforcement learning. arXiv preprint [arXiv:2212.03221](#) (2022)
25. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al.: Overcoming catastrophic forgetting in neural networks. *Proc. Natl. Acad. Sci.* **114**(13), 3521–3526 (2017)
26. Kumar, M.P., Packer, B., Koller, D.: Self-paced learning for latent variable models. In: Advances in Neural Information Processing Systems, pp. 1189–1197 (2010)
27. Leibo, J.Z., Zambaldi, V., Lanctot, M., Marecki, J., Graepel, T.: Multi-agent reinforcement learning in sequential social dilemmas. In: Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems, pp. 464–473 (2017)
28. Lipowska, D., Lipowski, A.: Emergence of linguistic conventions in multi-agent reinforcement learning. *PLoS ONE* **13**(11), e0208095 (2018)
29. Liu, B., Li, X., Zhang, J., Wang, J., He, T., Hong, S., Chen, J., Shi, H., Li, B., Chen, Y., et al.: Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. arXiv preprint [arXiv:2504.01990](#) (2025)
30. Liu, H., Huang, X., Gu, J., Shi, J., He, N., Feng, T.: Tcdformer-based momentum transfer model for long-term sports prediction. *Expert Systems with Applications* p. 128310 (2025)
31. Lopez-Paz, D., Ranzato, M.: Gradient episodic memory for continual learning. In: Advances in Neural Information Processing Systems, pp. 6467–6476 (2017)

32. Mattiisen, T., Oliver, A., Cohen, T., Schulman, J.: Teacher-student curriculum learning. *IEEE Trans. Neural Netw. Learn. Syst.* **31**(9), 3732–3740 (2019)
33. Moreno, R., Fafiña, A.: Emerge modular robot: A tool for fast deployment of evolved robots. *Front. Robotics AI* **9**, 869684 (2022)
34. Munkhdalai, T., Yu, H.: Meta networks. In: *International Conference on Machine Learning*, pp. 2554–2563. PMLR (2017)
35. Parisi, G.I., Kemker, R., Part, J.L., Kanan, C., Wermter, S.: Continual lifelong learning with neural networks: A review. *Neural Netw.* **113**, 54–71 (2019)
36. Peng, H.: A comprehensive overview and survey of recent advances in meta-learning. *arXiv preprint [arXiv:2004.11149](https://arxiv.org/abs/2004.11149)* (2020)
37. Pramanik, S.: Continual learning and catastrophic forgetting: The challenges and strategies in ai. *Medium* (2023)
38. Pritzel, A., Urias, B., Srinivasan, S., Badia, A.P., Vinyals, O., Hassabis, D., Wierstra, D., Blundell, C.: Neural episodic control. In: *Proceedings of the 34th International Conference on Machine Learning*-Volume 70, pp. 2827–2836. JMLR. org (2017)
39. Ravi, S., Larochelle, H.: Optimization as a model for few-shot learning. In: *International Conference on Learning Representations* (2017)
40. Rusu, A.A., Rabinowitz, N.C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Hadsell, R., Heß, C.: Progressive neural networks. *arXiv preprint [arXiv:1606.04671](https://arxiv.org/abs/1606.04671)* (2016)
41. Santoro, A., Bartunov, S., Botvinick, M., Wierstra, D., Lillicrap, T.: Meta-learning with memory-augmented neural networks. In: *International Conference on Machine Learning*, pp. 1842–1850. PMLR (2016)
42. Shin, H., Lee, J.K., Kim, J., Kim, J.: Continual learning with deep generative replay. In: *Advances in neural information processing systems*, vol. 30 (2017)
43. Shoham, Y., Leyton-Brown, K.: *Multiagent systems: Algorithmic, game-theoretic, and logical foundations*. Cambridge University Press (2009)
44. Sims, K.: Evolving virtual creatures. In: *Proceedings of the 21st annual conference on Computer graphics and interactive techniques*, pp. 15–22 (1994)
45. Snell, J., Swersky, K., Zemel, R.: Prototypical networks for few-shot learning. In: *Advances in neural information processing systems*, pp. 4077–4087 (2017)
46. Sukhbaatar, S., Fergus, R., et al.: Learning multiagent communication with backpropagation. In: *Advances in neural information processing systems*, vol. 29 (2016)
47. Sutton, R.S., Barto, A.G.: *Reinforcement learning: An introduction*. MIT press (2018)
48. Tuyls, K., Weiss, G.: Multiagent learning: Basics, challenges, and prospects. *AI Mag.* **33**(3), 41–41 (2012)
49. Wang, X., Chen, Y., Zhu, W.: A survey on curriculum learning. *IEEE Trans. Pattern Anal. Mach. Intell.* **44**(9), 4555–4576 (2021)
50. Wang, X., Li, Z., Zhong, H., Huang, L.: Few is more: Task-efficient skill-discovery for multi-task offline multi-agent reinforcement learning. *arXiv preprint [arXiv:2502.08985](https://arxiv.org/abs/2502.08985)* (2025)
51. Wang, X., et al.: Dynamic spatio-temporal modular network for video question answering. *ACM Multimedia* (2022)
52. Wang, X., et al.: Dynamic spatio-temporal graph reasoning for videoqa with self-supervised event recognition. *IEEE Transactions on Image Processing (TIP)* (2024)
53. Wang, X., et al.: Embodied ai: From llms to world models. *IEEE Circuits and Systems Magazine* (2025)
54. Wang, X., et al.: Embodied-r: Collaborative framework for activating embodied spatial reasoning in foundation models via reinforcement learning. *ACM Multimedia* (2025)
55. Wang, X., et al.: Improving compositional generalization in cross-embodiment learning via mixture of disentangled prototypes. *ACM Multimedia* (2025)
56. Wang, X., et al.: Tutorial on multi-modal generative ai in dynamic and open environment. *International Joint Conference on Artificial Intelligence (IJCAI)* (2025)
57. Wang, Z., Zhang, Z., Zhang, Y., Wang, Y., Sun, L., Li, J.: Dynamic graph neural networks under spatio-temporal distribution shift. *arXiv preprint [arXiv:2210.12574](https://arxiv.org/abs/2210.12574)* (2022)

58. Wang, Z., Zhang, Z., Zhang, Y., Wang, Y., Sun, L., Li, J.: Environment-aware dynamic graph learning for out-of-distribution generalization. In: *Advances in Neural Information Processing Systems*, vol. 36 (2023)
59. Yu, Y., Giannoccaro, I., Perc, M., Latora, V., Baronchelli, A.: Spontaneous emergence of social conventions in decentralized populations of large language model agents. *Proc. Natl. Acad. Sci.* **121**(26), e2319721121 (2024)
60. Zhang, Z., Wang, X., et al.: Disentangled dynamic graph attention network for out-of-distribution sequential recommendation. *ACM Transactions on Information Systems (TOIS)* (2024)

Chapter 5

Self-evolving World Model



Abstract The self-evolving world model (SEWM) represents a foundational advancement in artificial intelligence, enabling agents to autonomously refine their internal representations and predictive capabilities through continuous interaction with the environment. This chapter provides a comprehensive framework by formalizing SEWM into four core components: the Model (encompassing perception, task, policy, actor, and reward), Memory, Tools, and the Dynamics Optimizer. Mathematically, the self-evolving world model aims to learn a dynamics function $P(s_{t+1}|s_t, a_t, M, D, L, P)$ that achieves functional equivalence with the true environment. The self-evolving mechanism is driven by closed-loop optimization via the Dynamics Optimizer. The chapter systematically explores self-evolution within each component (e.g., perception, task, policy, actor, and reward evolution for the Model; Memory; Tools; Dynamics Optimizer) and between components (e.g., joint model-memory, joint model-tools, joint model-optimizer self-evolution). This structured formalism promotes the development of adaptive, robust, and generalizable AI systems capable of lifelong learning.

5.1 The Overview

World Model is the agent's internal representation and future predictions of the real world, as shown in Fig. 5.1, which can abstract the dynamic model of the objective environment and comprehend the underlying principles governing the real world. By compressing high-dimensional sensory inputs into structured latent spaces, world models facilitate efficient reasoning, long-term planning, and counterfactual imagination. This capability is especially vital in real-world applications such as autonomous driving, robotic manipulation, and interactive AI systems, where agents must operate under uncertainty and partial observability.

The training paradigms for world models have undergone a significant evolution, as shown in Fig. 5.2. Early open-loop world models were trained on fixed datasets and lacked the ability to interact with or adapt to their environments. The advent of closed-loop world models marked a shift toward online learning, where agents could update their models based on real-time feedback. More recently, foundation world

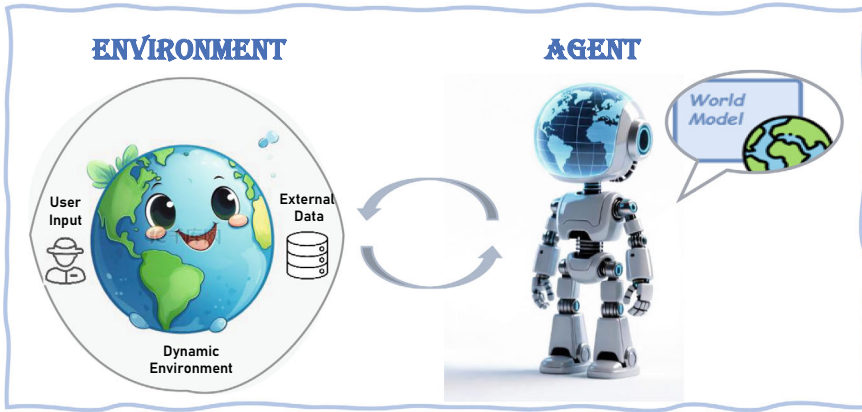


Fig. 5.1 The concept of world model

models have emerged, leveraging large-scale pretraining on diverse multimodal data to acquire broad, generalizable representations. Today, we stand at the frontier of self-evolving world models, which can autonomously refine their internal structures, learn from their own experiences, and continuously improve without human oversight.

A self-evolving world model is defined not only by its ability to predict and simulate but also by its capacity for autonomous self-improvement. Drawing inspiration from cognitive science and neuroscience, such a model embodies principles like neuroplasticity, predictive coding, and metacognition. It engages in an iterative cycle of experience acquisition, refinement, and integration, much like the human cognitive processes described by Piaget’s theory of cognitive development. Key mechanisms include structural plasticity, modular growth, self-reflection, knowledge consolidation, and adaptive forgetting, all of which enable the model to remain efficient and relevant in changing environments. Formally, a self-evolving world model can be characterized as a dynamic system that optimizes its own architecture and parameters through continuous interaction with the environment. It integrates techniques from reinforcement learning, continual learning, meta learning, curriculum learning, and autonomous machine learning to achieve sustained growth and adaptation. The goal is to create agents that not only perform well on predefined tasks but also discover new objectives, refine their strategies, and evolve their capabilities over time.

This chapter aims to provide a comprehensive framework for understanding and developing self-evolving world models. First, the world model currently has only a functional definition: Anything that satisfies the requirements of “understanding the world and predicting the future” is considered a world model. This chapter is the first to construct a structured and formalized definition of the world model and identify its key components, such as the model, memory, tools, and dynamics optimizer, which can greatly promote the systematic development and iteration of the world model. Secondly, this chapter organizes the theories and methods for self-evolution within components, such as model self-evolution, memory self-evolution,

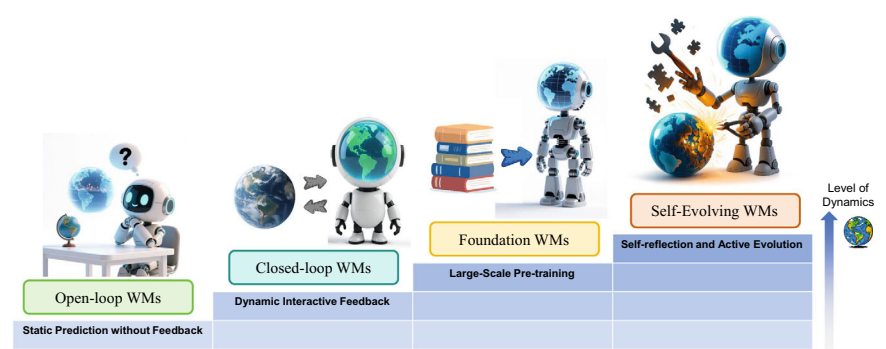


Fig. 5.2 The training paradigms evolution of world models

tools self-evolution, and dynamics optimizer self-evolution. In particular, for model self-evolution, we consider how the entire model process evolves, including how environmental perception, task decomposition, policy decision-making, action selection, and reward function optimization self-evolve. Finally, this chapter organizes the theories and methods for self-evolution between components, such as joint model-memory self-evolution, joint model-tools self-evolution, joint model-dynamics optimizer self-evolution, and others.

By bridging the gap between theoretical foundations and practical implementations, we hope to accelerate the development of truly autonomous, adaptive, and generalizable AI systems capable of thriving in an ever-changing world.

5.2 The Foundation of Self-evolving World Model

The concept of a world model, often informally described as an agent’s internal representation of the objective world and its mechanism for future prediction, has been central to artificial intelligence and cognitive science. However, its definitions have largely remained functional, lacking a structured and formalized decomposition that facilitates systematic engineering and iterative improvement. This section addresses this gap by proposing a novel, comprehensive definition of the Self-Evolving World Model. We deconstruct the world model into four core, interacting modules: the Model, Memory, Tools, and the Dynamics Optimizer. Crucially, we introduce the concept of *self-evolution* as an intrinsic property, where the model continuously refines its internal dynamics function through experience. The primary objective of this framework is to learn a *Dynamics Function* that probabilistically maps the current state and action to the next state, conditioned on the internal modules. We formalize the goal as achieving functional equivalence, where the learned dynamics are consistent with the true environment’s dynamics for all possible actions. This formalization provides a foundational blueprint for the development of more robust, generalizable, and autonomous intelligent systems.

5.2.1 Why Need Formalization

The idea of internal models has deep roots in fields ranging from control theory, with concepts like the Kalman filter, to psychology, through Tolman’s cognitive maps. In reinforcement learning, the distinction between model-free and model-based methods hinges on the presence or absence of a world model. Pioneering works like Sutton’s Dyna architecture integrated a world model for planning. More recently, deep learning approaches have produced impressive world models, such as those used in the Dreamer series of agents [43], which learn compact representations of the environment from pixels and use them for planning in a latent space.

Despite these advancements, a consistent and granular formal definition of the world model itself is often absent. The model is typically implicit in the architecture, a byproduct of training a neural network on prediction tasks. This lack of explicit formalization poses challenges. It makes it difficult to compare different world model architectures, to diagnose failures, to systematically incorporate new capabilities such as tool use or episodic memory, and to engineer their components independently. Our work draws inspiration from these previous approaches but seeks to abstract their common elements into a general, modular, and formally defined framework. By explicitly naming and defining components like Memory, Tools, and the Dynamics Optimizer, we provide a common vocabulary and a structural blueprint for future research.

5.2.2 The Key Components

In the pursuit of Artificial General Intelligence (AGI), the notion of a world model stands as a cornerstone. It encapsulates an agent’s capacity to not merely react to stimuli but to possess an internal simulacrum of its environment. This internal model enables understanding, reasoning, and prediction, which are essential for planning and sophisticated behavior. Traditional definitions, while conceptually sound, often suffer from vagueness. Descriptions such as “an abstraction of the environment’s dynamics” or “a system that understands the world’s rules” provide high-level intuition but offer little guidance for implementation or rigorous analysis. They treat the world model as a monolithic black box satisfying a functional requirement, namely understanding and prediction.

This subsection presents a significant departure from this perspective. We argue that for a world model to be truly effective and scalable, especially in complex, open-ended environments, it must be architecturally decomposed and endowed with a self-improving capability. We therefore propose a formalized definition of a *Self-Evolving World Model*. This definition moves beyond functional descriptions to a structural and operational one. The key contribution is the identification and detailed specification of four fundamental components that constitute the world model, and the formalization of their interaction aimed at learning a precise dynamics func-

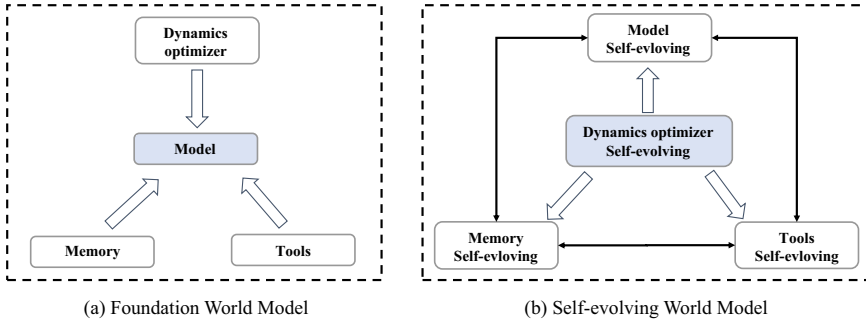


Fig. 5.3 The components of self-evolving world model

tion. The self-evolution property emerges from the continuous interaction between these components, particularly through the role of the Dynamics Optimizer. This framework is intended to serve as a structured foundation, promoting systematic development, benchmarking, and theoretical analysis of world models.

The proposed Self-Evolving World Model is defined as an integrated system composed of four core modules. The interaction between these modules is illustrated in Fig. 5.3 and forms the basis for the model’s operation and evolution.

A. The Model Core (M)

The Model Core is the central predictive engine of the world model, denoted as M . It is responsible for maintaining the current state representation and executing the dynamics function. It can be further decomposed into several specialized submodules:

- Perception (O): This submodule is responsible for processing raw sensory inputs from the environment and transforming them into an internal state representation s_t . It handles tasks such as object recognition, state estimation, and feature extraction.
- Task (T): This module encodes the current goal or objective of the agent. It modulates the behavior of the Policy and provides a context for what aspects of the world dynamics are most relevant.
- Policy (π): The policy is a function that maps the current state s_t and task T to a distribution over actions a_t . It represents the agent’s current strategy for achieving its task.
- Actor (A): The actor is the component that samples a specific action a_t from the policy’s distribution and executes it in the environment (or the internal simulation).
- Reward (R): This module predicts the expected reward r_{t+1} resulting from taking action a_t in state s_t . It is a crucial component for evaluating the consequences of actions.

The Model Core’s integrity relies on the seamless interaction of these submodules. Perception informs the state, the Task guides the Policy, the Actor executes actions, and the Reward module evaluates outcomes.

B. Memory (D)

The Memory module, denoted as D , serves as the repository of the agent’s past experiences. It is not a single unitary store but likely consists of multiple subsystems:

- Episodic Memory: Records specific past events in the form $(s_t, a_t, r_{t+1}, s_{t+1})$.
- Semantic Memory: Stores generalized knowledge and facts about the world, abstracted from specific experiences.
- Procedural Memory: Encodes skills and routines for using tools and performing actions.

The Memory module provides the data foundation upon which the world model is built and refined. It allows the agent to learn from history, avoid past mistakes, and recall successful strategies. The Dynamics Optimizer heavily relies on D for training the Model Core.

C. Tools (L)

The Tools module, denoted as L , represents a set of external or internal capabilities that the agent can invoke to interact with the world or to aid its own reasoning processes. Tools extend the native capabilities of the agent. Examples include:

- Physical tools (e.g., a robotic arm using a wrench).
- Computational tools (e.g., a calculator, a database query system).
- Conceptual tools (e.g., a physics simulator, a language model).

The key aspect of tools is that their effects on the world must be learnable and integrated into the overall dynamics function. The Policy submodule must learn to decide not only on primitive actions but also on when and how to use available tools.

D. The Dynamics Optimizer (P)

The Dynamics Optimizer is the engine of *self-evolution*, denoted as P . It is the meta-cognitive component responsible for improving the predictive accuracy of the Model Core. Its primary function is to continuously compare the predictions made by the Model Core against the actual outcomes observed from the environment (or retrieved from Memory). It then calculates a loss signal, typically based on prediction error (e.g., the difference between predicted s_{t+1} and actual s_{t+1} , or between predicted and actual reward), and uses optimization algorithms to update the parameters of the

Model Core’s submodules. This process of iterative refinement is what allows the world model to become more accurate and comprehensive over time.

5.2.3 The Mathematical Formalization

The ultimate objective of the Self Evolving World Model is to construct a highly accurate *Dynamics Function*. This function is the mathematical heart of the Model Core. We define it as a conditional probability distribution:

$$P(s_{t+1}|s_t, a_t, M, D, L, P)$$

This function predicts the probability distribution of the next state s_{t+1} , given the current state s_t , the action taken a_t , and conditioned upon the current Model M , the content of Memory D , the available Tools M , and the Dynamics Optimizer P . This formulation acknowledges that the world’s dynamics are not absolute but are perceived and interpreted through the agent’s current cognitive apparatus.

The goal of learning is to ensure that this internal dynamics function aligns perfectly with the true dynamics of the environment, $P_{\text{env}}(s_{t+1}|s_t, a_t)$. The formal criterion for success is **functional equivalence**. We state that the learned model is functionally equivalent to the true environment if, for any state s that the agent can represent, and for any action a within the domain of possible actions, the following holds:

$$\forall a \in \mathcal{A}, P(s_{t+1}|s_t, a) = P_{\text{env}}(s'_{t+1}|s_t, a)$$

In other words, for any action, the predicted distribution over next states (and consequently, rewards) generated by the internal model is identical to the distribution produced by the actual environment. Achieving this equivalence means the agent’s internal simulation of the world is as accurate as the world itself, enabling perfect prediction and optimal planning.

5.2.4 The Self-evolving Mechanism

The term *self-evolving* is key to our definition. Evolution is driven by the continuous operation of the Dynamics Optimizer in a closed loop with the other modules. As shown in Fig. 5.4, (1) The agent, using its current Model Core, interacts with the environment. (2) New experience tuples $(s_t, a_t, r_{t+1}, s_{t+1})$ are stored in Memory D . (3) The Dynamics Optimizer samples batches of experiences from D . (4) It computes the prediction error by comparing the Model Core’s predictions (for state and reward) with the actual outcomes. (5) Using this error signal, it updates the parameters of the Perception, Reward, and Policy networks within the Model Core via backpropagation or other optimization techniques. (6) This updated Model Core

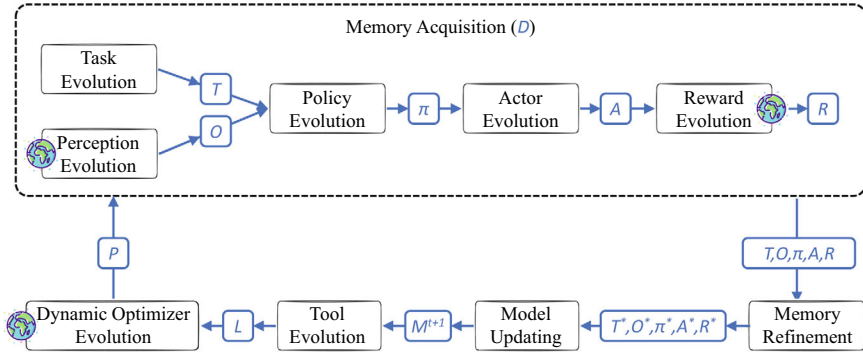


Fig. 5.4 The self-evolving mechanism of world model

now makes more accurate predictions, leading to better policy decisions and, in turn, generating more informative data for future learning cycles.

This cycle creates a positive feedback loop for self-improvement. Furthermore, as the Tools module L is expanded, the Dynamics Optimizer must learn to incorporate the effects of these new tools into the dynamics function. Similarly, as Memory grows, the model can learn from a richer and more diverse set of experiences, preventing catastrophic forgetting and enabling knowledge transfer.

5.3 Self-evolution Within Components

This subsection organizes the theories and methods for self-evolution within components, such as model self-evolution, memory self-evolution, tools self-evolution, and dynamics optimizer self-evolution. In particular, for model self-evolution, this section considers how the entire model process evolves.

5.3.1 Model Self-evolution

This section considers how the entire model process evolves, including how environmental perception, task decomposition, policy decision-making, action selection, and reward function optimization self-evolve.

A. Perception Evolution

In conventional world models, the perception module is usually implemented as a static encoder, such as a convolutional network or a variational autoencoder, that com-

presses raw observations into low-dimensional latent states. This design assumes that a compact latent vector is sufficient to support downstream dynamics prediction and control. While effective in simple domains, such non-evolving perception modules are limited in their ability to generalize across tasks and environments. They primarily optimize for reconstruction or one-step prediction, often entangling multiple factors of variation into a monolithic representation. As a result, they struggle to capture object-level structure, to integrate multiple modalities, or to maintain coherent long-horizon memory, thereby constraining imagination and planning capabilities.

Recent research has therefore moved toward evolving the perception module itself. Instead of remaining a static feature compressor, perception in world models is now treated as a dynamic component that adapts its representations as agents encounter new states, tasks, and environments. Self-supervised predictive objectives evolve perception into a future-oriented latent space; object-centric methods evolve perception into structured, interpretable representations; temporal modeling extends perception into long-horizon dynamics; and multimodal, geometry-aware designs evolve perception into cross-sensor, spatially consistent latent spaces. The following subsections elaborate on these four trajectories and their representative works.

Self-supervised Predictive. Early world models demonstrated that perception could evolve by learning predictive latent spaces. The original World Models [40] and PlaNet [42] introduced compact stochastic-deterministic latent dynamics trained with variational objectives, enabling multi-step rollouts entirely in latent space. This established the principle that perception should serve imagination by encoding states that are compact yet predictive. The Dreamer series [30, 41, 43, 44] advanced this approach with discrete latents, improved regularization, and balanced training objectives, significantly enhancing stability and sample efficiency. Exploration-driven methods such as Plan2Explore [107] extended predictive perception by maximizing epistemic uncertainty and information gain, ensuring that latent representations adaptively cover unseen states.

Object-centric Structured. Another research direction evolves perception into structured object- and relation-centric representations. Instead of entangled feature vectors, scenes are decomposed into entities and their interactions. SAVi/SAVi++ [21, 55] extend slot attention to video prediction, enabling temporal object tracking. Slot Structured World Models (SSWM) [15] combine slot attention with graph dynamics to model object interactions explicitly, while SOLD [80] learns object-centric latent dynamics without supervision, supporting combinatorial generalization to unseen object configurations. Recent advances such as Dyn-O [128] introduce dynamic slots that evolve over time, and Jeong et al. [53] align perception with natural language conditions, bridging structured perception with task-relevant semantics. Together, these works highlight the evolution of perception from pixel-level encoding to interpretable, object-level latent structures.

Long-sequence Temporal. A third trajectory addresses the challenge of modeling long-horizon dependencies in perception. Recurrent latent models such as RSSM often suffer from limited memory capacity. To address this, IRIS [76] combined discrete autoencoding with Transformers, yielding sample-efficient world models with

extended memory. STORM [146] integrated stochastic latent variables into a Transformer backbone, balancing expressiveness and stability for long-sequence modeling. TWISTER [4] further improved horizon modeling with action-conditioned contrastive predictive coding, while Long-Context SSM Video World Models [89] extended temporal coverage using state-space architectures with linear-time inference. These advances demonstrate how perception evolves into a temporal module capable of sustaining coherent imagination over hundreds of steps.

Multi-modal and Geometry-aware. Finally, perception evolves by integrating multimodal signals and spatial geometry. Real-world applications such as autonomous driving require fusing visual, LiDAR, and geometric cues. UniWorld [77] jointly encoded vision and LiDAR into a unified latent space to predict 4D occupancy, robust to occlusion and sensor noise. OccWorld [155] and Drive-OccWorld [135] further refined this approach by explicitly predicting occupancy flows and aligning semantic and motion cues. UniFuture [65] introduced dual-latent spaces for image and depth with multi-scale cross-modal interactions, unifying perception and prediction for trajectory forecasting and semantic completion. DIO [16] decomposed occupancy-flow into instance-level latent structures, improving compositionality and interpretability. These works collectively illustrate how perception evolves from single-modality encoders into spatio-temporal, geometry-aware latent spaces, enabling robust planning in complex environments.

B. Task Evolution

In conventional world models, tasks are typically treated as fixed and externally defined. The environment specifies a goal, such as reaching a target location or maximizing a scalar reward, and the world model is trained only to predict state transitions or rewards under this predefined objective. In this setting, tasks are external constraints rather than intrinsic components of the model. This formulation suffers from several limitations. Fixed tasks limit the model’s ability to generalize across objectives, prevent the discovery of novel skills, and offer no mechanism for adaptation when new tasks are encountered. As a result, world models trained under static task definitions often lack flexibility and struggle in open-ended or multi-task scenarios.

Recent progress has redefined tasks as dynamic entities that co-evolve with other components of the world model. We refer to this direction as task self-evolution. Unlike static benchmarks, self-evolving tasks are not predetermined but instead emerge through conditioning on contextual variables, through the automatic generation of new goals, or through adaptive mechanisms that allow a single model to adjust across objectives. These approaches can be grouped into three major categories. Task conditioning injects task information directly into the latent space of the world model, enabling imagination and planning to adapt to diverse goals. Task generation expands the task distribution by automatically constructing novel challenges or curricula, ensuring that agent capabilities evolve in tandem with task complex-

ity. Finally, Task adaptation focuses on enabling a single world model to generalize across tasks and to transfer effectively to new objectives. The following subsections review each category in detail and highlight representative methods.

Task Conditioning. The first category encodes task information directly within the world model, allowing a single architecture to flexibly handle diverse objectives. Instead of training separate models per task, contextual variables such as goals, language instructions, or physical priors are embedded into the latent imagination process. LEXA [75] represents tasks as latent goals by learning temporal distances in imagination, providing a reward signal that reflects control difficulty. Language-based conditioning has further expanded this paradigm. LUMOS [82] integrates natural language commands into latent imagination to support long-horizon robotic manipulation, while LIMT [2] derives semantic embeddings from pretrained language models to steer multi-task prediction and control. Other approaches extend conditioning to physics and pretrained knowledge. Physics-informed world models such as PIN-WM [62] and RoboScape [110] constrain latent states to respect physical consistency, enabling better transfer to real-world settings. Pretrained-aligned models such as LaDi-WM [51] incorporate semantic priors from vision foundation models, enriching task representations with high-level knowledge. These works demonstrate that conditioning transforms tasks from external constraints into intrinsic signals that shape world model learning.

Task Generation. The second category focuses on evolving the task space itself rather than conditioning on existing objectives. By continuously generating new challenges, these approaches ensure that world models and policies grow with task complexity. Programmatic platforms such as XLand [115] define massive distributions of procedurally generated tasks, showing that diversity is essential for open-ended skill acquisition. More recent methods build task curricula dynamically. Hill et al. [49] construct new tasks by posing challenges to the agent, thereby producing an automated curriculum that scales in difficulty as the agent improves. In both procedural and adversarial approaches, task evolution arises through the expansion of the task distribution itself, allowing world models to develop broad competence across domains.

Task Adaptation. The third category emphasizes how a single world model can generalize across multiple tasks and quickly adapt to new objectives. Rather than creating or conditioning on tasks, these methods embed task-awareness into the training process. TEMPO [136] introduces a meta-weighting scheme that prioritizes data more informative for the current objective, improving efficiency in multi-task learning. WAKER [98] adopts a reward-free curriculum design, selecting environment instances that minimize prediction error, thereby producing robust world models that transfer effectively to downstream tasks. Empirical advances further support this perspective: Multi-task Disentangled World Models [117] demonstrate that training across multiple tasks leads to disentangled latent spaces, improving cross-task generalization. Task adaptation thus highlights how tasks themselves can act as inductive biases that shape world model training, supporting robust transfer and flexible planning across diverse domains.

C. Policy Evolution

In standard world models, the policy module is often realized as a static neural network, such as a multi-layer perceptron or recurrent head, mapping latent states directly to action distributions. These policies are typically optimized using reinforcement learning methods, for example policy gradients or actor-critic algorithms. While functional in simple environments, such static policies have several limitations. They are heavily reliant on sparse or delayed rewards, leading to poor sample efficiency, and they make only minimal use of the predictive capacity of the world model. In practice, the world model is reduced to a feature encoder for the policy, while imagination, planning, and optimization remain largely decoupled. This separation restricts generalization to more challenging tasks and prevents policies from adapting dynamically to new learning signals. By contrast, self-evolving policies integrate additional sources of feedback and architectural mechanisms, allowing them to adapt their objectives and interactions with the world model to improve efficiency, stability, and generality.

Recent advances point toward what may be termed policy self-evolution. Unlike static mappings, self-evolving policies adapt their learning process through distinct forms of guidance. These approaches can be grouped into three main categories. The first is Externally-driven evolution, where policies are improved by feedback from model-based search or imagination, for example by distilling high-quality trajectories found through planning or by incorporating imagined rollouts as contextual input. The second is Internally-driven evolution, where policies are optimized end-to-end within the latent space of the world model through value gradients or temporal-difference signals, thereby exploiting the differentiability of the model. The third is Structural evolution, which emphasizes architectural modifications that allow policies to scale in both horizon length and agent population, for example through modular transformers or decentralized multi-agent designs. The following subsections review each of these categories in turn and highlight representative methods.

Externally-driven Evolution. Externally-driven evolution integrates planning or imagination into policy learning. PlaNet [42] combined latent dynamics with a cross-entropy method planner, and distilled the resulting action sequences into a policy, effectively transferring planning competence into a reactive controller. MuZero [106] extended this idea by embedding Monte Carlo tree search into a learned model, jointly optimizing policy, value, and reward functions. Theoretical progress was made by MPDP [59], which provided guarantees for monotonic improvement when distilling planning results into policy updates. Another line of work, Imagination-Augmented Agents (I2A) [97], directly supplied imagined rollouts as input features to the policy network, enabling it to learn how to use simulated futures for decision-making. These methods demonstrate that external feedback, whether derived from search or imagination, can drive policies beyond static mappings toward adaptive and informed strategies.

Internally driven Evolution. Internally-driven evolution exploits differentiable objectives within the world model to directly optimize policies. Imagined Value Gradients (IVG) [86] first showed that policies could be updated by backpropagating gradients through imagined trajectories in latent space. Building on this, TD-MPC [48] and TD-MPC2 [46] combined short-horizon trajectory optimization with temporal-difference learning, enabling policies to balance near-term prediction quality with long-horizon returns. Another influential method is MBPO [52], which stabilized training by using short synthetic rollouts from the model in off-policy learning, improving both efficiency and reliability. In these approaches, the policy evolves from a passive consumer of rewards to an active participant in differentiable optimization, leveraging the world model as a direct source of learning signals.

Structural Evolution. As environments and tasks grow more complex, policies must be able to scale their capacity and coordination mechanisms. Structural evolution refers to architectural innovations that adapt the form of the policy itself. UniZero [90] proposed a scalable latent world model based on modular transformers, jointly predicting dynamics, values, and policy signals across extended horizons. In multi-agent settings, MARIE [147] developed decentralized transformer-based world models with centralized aggregation, enabling policies to coordinate effectively across agents while preserving local autonomy. These examples illustrate how structural evolution enables policies to adapt not only through optimization signals but also through the reconfiguration of their architectures, supporting long-horizon reasoning and cooperative control in increasingly demanding domains.

D. Actor Evolution

In conventional world models, the actor is typically implemented as a simple policy head that maps latent states into primitive actions. This design assumes that the world model provides sufficiently accurate predictive signals, while the actor merely translates them into executable controls. Although effective in constrained environments, such non-evolving actors are limited in their ability to generalize across domains. They struggle to abstract complex behaviors, to adapt to diverse task conditions, or to align with the structural biases of the world model itself. As a result, conventional actors often face difficulties in achieving robust control in long-horizon, multimodal, or dynamic settings.

Recent work has therefore sought to evolve the actor module itself. Unlike static policies, evolving actors modify their input context, internal structure, or output representation, allowing them to co-adapt with the world model dynamics. This shift reflects a deeper integration between action generation and imagination. Input evolution enriches the actor’s decision process by conditioning actions on imagined rollouts rather than raw states, embedding future-oriented reasoning directly into action selection. Structural evolution integrates the actor more tightly with the world model’s generative pipeline, where action prediction and state imagination are unified in a single model. Output evolution redefines the space of actions by learning latent or

trajectory-based abstractions, enabling more transferable and structured behaviors. The following subsections elaborate on these three forms of actor evolution and their representative works.

Input Context. One research trajectory enriches the inputs to the actor by incorporating world model imagination into decision-making. Instead of mapping from latent states alone, actors consume imagined rollouts or summaries of future predictions. The pioneering I2A [97] demonstrated that actors can learn to interpret and utilize predictions generated by a world model, effectively embedding foresight into their policies. Building on this, LEXA [75] trains two complementary actors (explorer and achiever) entirely within imagined latent spaces, where exploration discovers diverse skills and the achiever learns to reach latent goals. This paradigm demonstrates that by evolving input context, actors can directly exploit predictive imagination for more informed and goal-directed decision-making.

Internal Architecture. A second stream of research rethinks the structural role of the actor within the world model. Rather than functioning as an external policy head, actors are embedded into the generative architecture of the model itself. LaDi-WM [51] integrates action prediction into a latent diffusion framework, where both future states and action sequences are iteratively refined in the same latent space. Similarly, WorldVLA [8] unifies vision-language-action modeling with world models in an autoregressive manner, jointly generating multimodal observations and corresponding actions. These methods illustrate how the actor evolves structurally, becoming a component of the generative pipeline rather than an add-on, thereby enhancing coherence between imagined futures and executable actions.

Output Representation. Another line of research redefines how actions are represented and output. Instead of producing low-level control signals directly, actors can operate over latent action abstractions. AdaWorld [34] learns action representations from unlabeled videos via self-supervised pretraining, extracting latent action codes that facilitate cross-domain adaptation. PreLAR [144] extends this paradigm by pretraining world models on action-free videos, then introducing learnable action representations in downstream tasks, bridging the gap between passive perception and active control. Another representative method, PolyGRAD [99], employs diffusion models to generate entire trajectories, with policy gradients guiding the diffusion process. This enables actors to output temporally consistent action sequences, rather than step-wise primitives, improving sample efficiency and planning robustness. Collectively, these approaches highlight how output representation evolves from direct motor commands to structured latent or trajectory-based abstractions.

E. Reward Evolution

In conventional world models, the reward function is typically provided as a fixed supervision from the environment. Agents rely on this externally defined signal, often handcrafted by domain experts, to guide policy learning and value estimation. Such

a design has well-known limitations. Handcrafted rewards are narrow in scope and may not capture abstract or complex task objectives. Rewards are frequently sparse or delayed, which leads to inefficient exploration and unstable credit assignment. Moreover, the reward signal is static and does not adapt as the agent or the world model evolves, thereby restricting generalization across tasks and domains. In this setting, the reward remains an inflexible constraint rather than a dynamic component that interacts with other modules.

Recent research has moved toward what may be described as reward self-evolution, in which reward signals are no longer fixed but are instead adapted in response to the agent’s learning process and the world model’s representations. Compared with static signals, self-evolving rewards arise from model-internal structures, task-independent pretraining strategies, or alignment with external evaluation criteria. These approaches highlight that rewards can themselves evolve as part of the learning system. The remainder of this section reviews three representative directions: model-internal rewards, task-agnostic pretraining, and externally-aligned rewards.

Model-internal Rewards. The first category derives rewards directly from the dynamics and representations of the world model. Instead of relying exclusively on externally defined objectives, intrinsic signals emerge from latent states, prediction errors, or uncertainty estimates. Hindsight PRIORS [118] exploit a learned forward model to assess the importance of individual states within a trajectory and redistribute preference-based returns to critical transitions, thereby addressing temporal credit assignment. LEXA [75] defines rewards in latent imagination by estimating temporal distances between states, which more faithfully reflect the difficulty of achieving goals than pixel-level measures. Exploration-oriented methods also belong to this category. Plan2Explore [107] introduces information gain in latent rollouts as a curiosity-based signal, Active World Model Learning [54] uses measures of learning progress to shape exploration, and Go Beyond Imagination [32] designs episodic reachability rewards that encourage agents to expand the set of futures reachable in imagination. These contributions illustrate how the reward module can be reformulated as an adaptive signal that co-evolves with the internal states of the world model.

Task-agnostic Pretraining. A second direction emphasizes reward-free pretraining, where the world model is optimized without access to explicit task rewards. This paradigm shifts the role of reward by preparing the model to acquire general-purpose representations that accelerate subsequent reward-driven learning. CASCADE [133] trains a population of exploration agents under information-theoretic objectives to maximize trajectory diversity, thereby producing latent dynamics that generalize across domains. WAKER [98] advances this idea by introducing curriculum design for reward-free world model training, selecting environment instances that minimize the worst-case prediction error. In both cases, the absence of reward during pretraining does not hinder learning; rather, it produces world models with more robust representations, which later enable efficient and stable adaptation once downstream rewards are introduced. Task-agnostic pretraining therefore represents a form

of reward self-evolution in which the role of reward is redefined from being a primary training signal to a fine-tuning objective.

Externally-aligned Rewards. The third category explores the integration of verifiable and interpretable metrics into the reward function. Unlike intrinsic methods, these approaches incorporate external criteria that correspond to human-understandable objectives. RLVR-World [130] demonstrates this perspective by training world models with verifiable reward signals, such as prediction accuracy or perceptual quality measures. Embedding such metrics into the optimization process ensures that the learned reward signal is consistent with downstream evaluation standards. Although research in this direction is still limited, the alignment of reward with external metrics opens a promising avenue for improving the transparency, interpretability, and reliability of world models.

5.3.2 Memory Self-evolution

Memory is a cornerstone of intelligent systems, enabling agents to accumulate knowledge across experiences, generalize to novel situations, and build long-term models of the environment. In the context of self-evolving world models, memory does not remain static but undergoes continuous adaptation, restructuring, and abstraction. This section introduces the conceptual foundation of self-evolving memory, reviews the current computational mechanisms, and outlines potential trajectories for designing adaptive memory architectures that can evolve with both task demands and environmental changes.

Memory plays a foundational role in constructing and evolving world models. A world model, by definition, is not just a static representation of external reality but a dynamic cognitive substrate that allows agents to predict, reason, and adapt over extended horizons. Without effective memory mechanisms, the world model remains shallow, prone to context drift, forgetting, and even hallucination [71, 148, 149]. Therefore, optimizing memory is central to enabling a self-evolving world model that can accumulate experience, refine internal representations, and ensure stable behavior in complex and dynamic environments.

Unlike training-time methods such as fine-tuning or knowledge editing [6, 78], which statically reshape model parameters, inference-time memory optimization focuses on adaptive utilization of information during reasoning. In this view, the world model evolves not only by what it learns permanently but also by how it schedules, stores, and recalls knowledge on demand. This aligns memory with decision-making: deciding what to remember, what to discard, and how to generalize.

From this perspective, memory optimization for world models can be grouped into two complementary processes: (1) *Short-term memory optimisation*, ensuring local coherence and efficient reasoning within limited context windows. (2) *Long-term memory optimisation*, enabling persistent accumulation and retrieval of experiences

across tasks and sessions. Together, they provide the substrate upon which the world model evolves from immediate perception to enduring cognition.

A. Short-term Memory in World Models

Short-term memory serves as the “working memory” of a world model. It maintains immediate dialogue turns, intermediate reasoning traces, and contextual cues needed for coherent response generation [71]. Given that the context window is finite, strategies such as summarization, selective retention, and sparse attention are employed to prevent overload [121, 149]. Recursive summarization [121] incrementally condenses knowledge into compact representations, while MemoChat [74] builds dialogue-level memory to sustain personalized interaction. Compression-based methods such as COMEDY [11] and ReadAgent [57] enhance context efficiency by distilling prior conversations into gist-level traces. More sophisticated designs retrieve intermediate states to guide multi-hop reasoning, such as MoT [63] and StructRAG [64]. Inspired by cognitive theories, MemoryBank [156] integrates the Ebbinghaus forgetting curve [81] to dynamically update memory traces based on recency and relevance. Reflexion [112] pushes the boundary further by letting agents reflect on episodic outcomes and store lessons for future adaptation. These techniques extend the short-term stability of world models, but their inherent limitation lies in transience: they cannot sustain cross-session knowledge evolution without long-term storage.

B. Long-term Memory in World Models

To achieve self-evolution, a world model requires long-term memory mechanisms that persistently accumulate knowledge beyond the context window [18]. Long-term memory allows agents to recall factual knowledge, reconstruct task histories, and adapt behavior according to past interactions, thereby transforming one-time experiences into enduring competencies. The most influential paradigm is Retrieval-Augmented Generation (RAG) [20, 35, 120], where the world model actively queries external memory to support reasoning. Systems such as EWE [10] and A-MEM [132] showcase how explicit working memory and dynamic indexing create evolving knowledge networks. Meanwhile, agentic retrieval approaches enable models to autonomously decide what and when to retrieve, integrating episodic trajectories into future planning [1, 153]. Vector-based long-term systems (e.g., MemGPT [83], NeuroCache [100], G-Memory [142], AWESOME [7], Mem0 [12]) provide scalable memory encoding and rapid access. Biological inspirations, such as HippoRAG [39], simulate hippocampal mechanisms, while graph-based designs like GraphReader [61] enable structured retrieval. Symbolic systems such as ChatDB [50] further ground memory in explicit databases [26–28]. Beyond storage, memory control mechanisms govern the evolution of world models. MATTER [56] selects segments from heterogeneous sources, while AWM [129] maintains continuous updates

across settings. MemoryBank [156] revisits old knowledge to counteract forgetting, and MEM1 [156] employs reinforcement learning to balance consolidation and pruning. Systems like MrSteve [84] add episodic “what-where-when” structure, while MIRIX [124] introduces multi-agent shared memory for collaborative long-horizon reasoning.

C. Toward Self-evolving Memory

Recent works stress that memory is not merely a storage unit but a self-evolving substrate for the world model [91, 108]. Effective memory enables an agent to accumulate, reorganize, and abstract experiences into higher-level heuristics and reusable strategies [31, 150]. Expel [150] distills past trajectories into generalizable rules, while Agent Workflow Memory [129] records reusable sub-task workflows. Riche-lieu [37] demonstrates how negotiation strategies improve through iterative self-play and memory augmentation. Mechanisms such as Mem0’s add-merge-delete operations [12] or MemInsight’s semantic tagging [102] ensure coherence over time. REMEMBER [140] combines episodic reinforcement with long-term storage, further highlighting that memory evolution for world model evolution.

Memory optimization is not a peripheral enhancement but the very engine of self-evolving world models. Short-term memory ensures immediate coherence; long-term memory guarantees persistence and adaptation; and self-evolving memory mechanisms transform episodic experiences into generalizable competencies. Together, these layers enable a world model to transcend static knowledge, continuously restructure itself, and sustain robust reasoning in dynamic environments.

5.3.3 Tools Self-evolution

Tools are indispensable components of intelligent agents, functioning as interfaces through which agents perceive, act upon, and extend their reach into the external world. They provide access to structured databases, computational resources, APIs, and domain-specific services, significantly amplifying problem-solving abilities [38, 88, 134]. Effective utilization requires agents to determine when, how, and why to invoke tools, and to integrate tool outputs into coherent multi-step reasoning. This challenge has sparked extensive research into tool optimization and, more recently, tool evolution, which marks a transition from passive tool use toward autonomous tool creation and management. This section synthesizes existing efforts and outlines a three-stage progression: optimizing tool interaction, iterative mastery and refinement, and autonomous creation with scalable management. Together, these constitute the foundation of adaptive tool evolution in self-evolving world models.

A. From Tool Use to Tool Optimisation

Early studies focused on improving an agent’s ability to employ existing tools efficiently. Since large language models (LLMs) are pretrained primarily for text generation, they lack an intrinsic understanding of tool invocation. To address this, researchers have developed training-based and inference-time optimization strategies. Training-based optimization leverages supervised fine-tuning (SFT) on curated tool-use trajectories [19, 72, 105, 122]. These trajectories capture queries, reasoning steps, tool calls, and answers, explicitly teaching agents how to plan, execute, and integrate tools. Variants such as ToolLLM [94] and GPT4Tools [134] generate synthetic tool-use data, while STE [119] and TOOLEVO [9] rely on simulated exploration for higher-quality trajectories. Multi-modal agents, exemplified by T3-Agent [36], extend these techniques beyond text. Reinforcement learning (RL) offers another avenue, enabling agents to refine tool-use policies through interaction and reward feedback, as seen in ReTool [25] and Tool-Star [17].

Inference-time optimization avoids retraining by enhancing contextual guidance during test time. Prompt-based methods restructure tool documentation into concise, LLM-friendly instructions [96, 138], while reasoning-based approaches employ tree search or planning algorithms to dynamically select and compose tools [24, 73, 158]. These strategies demonstrate that even without parameter updates, agents can significantly improve their tool-using capabilities.

B. Mastery Through Iterative Refinement

Optimization alone cannot guarantee robustness. Newly created or rarely used tools often remain brittle and error-prone. A second evolutionary step therefore involves iterative refinement and mastery. Frameworks such as LearnAct [151] and From Exploration to Mastery embed agents in self-correction loops, where errors from compilation, API mismatches, or environmental feedback are analyzed to locate the causes of failure. Addressing this “credit assignment” problem, that is, identifying which component of a tool caused the failure, is critical for reliable mastery. Refinement extends beyond debugging code, as agents also revise tool documentation and usage examples, thereby improving interpretability and reusability.

Human-in-the-loop paradigms accelerate this process by enabling experts to validate, correct, or provide high-level guidance. Such hybrid refinement not only improves functional reliability but also ensures alignment with safety and ethical principles. In this sense, mastery transforms raw tool scripts into stable, reusable, and trustworthy capabilities, strengthening the skill base of the world model.

C. Beyond Use: Autonomous Discovery, Creation, and Management

The frontier of tool evolution lies not only in using or refining tools but also in autonomously discovering, creating, and managing them. This trajectory mir-

rors human civilisation’s progress from using natural objects as tools to engineering increasingly sophisticated instruments. Autonomous discovery and creation. Agents such as Voyager [120] exhibit open-ended skill acquisition through intrinsic exploration in rich environments like Minecraft. Structured systems such as CREATOR [92] and SkillWeaver [152] formalize tool synthesis into modular APIs, separating generalizable creation from domain-specific applications. Retrieval-augmented systems, exemplified by Alita [95], dynamically mine or generate functions upon encountering capability gaps, while CRAFT [137] assembles domain-specific toolsets from reusable code snippets. These approaches collectively point to an emerging paradigm in which agents act as tool makers rather than mere tool users.

Scalable management and composition. As tool libraries grow, agents must overcome the “curse of abundance.” Simple retrieval mechanisms struggle when thousands of skills are available. Approaches such as ToolGen [122] encode tools directly into the model’s vocabulary, reframing retrieval as a generation task. Higher-level systems such as AgentSquare [109] employ meta learning across planning, memory, and tool-use components to optimize modular architectures for multi-step tasks. Looking further ahead, the Darwin Godel Machine [143] envisions recursive self-modification, where agents evolve not only their toolsets but also their underlying architectures, blurring the boundary between “agent” and “tool.”

D. Toward Self-Evolving Tool Ecosystems

The progression from optimized tool use to autonomous tool creation and scalable management represents more than incremental advancement; it signals a paradigm shift toward self-evolving tool ecosystems. In such systems, agents continuously identify capability gaps, generate and refine new tools, and manage their integration into a coherent repertoire. This establishes a closed feedback loop of discovery, mastery, and integration, echoing the recursive adaptability of biological evolution.

Autonomy also introduces new challenges. Ensuring safety, preventing malicious tool generation, and designing verification mechanisms remain open issues. Furthermore, integrating tool evolution with broader world models that encompass memory, perception, and dynamics requires unified frameworks. Despite these challenges, the trajectory is clear: the future of intelligent systems depends not only on their ability to use tools effectively but also on their capacity to evolve the very tools that expand their cognitive horizons.

5.3.4 Dynamics Optimizer Self-evolution

The world model, an agent’s internal blueprint of its environment, is central to achieving autonomous adaptability in AI systems. By encoding states, transitions, and predictions, it enables agents to simulate outcomes, reason through multi-step tasks, and

adapt to dynamic contexts [23, 33]. The self-evolving dynamics optimizer refines this world model, ensuring it remains robust and adaptive through iterative feedback, learning, and tool integration. Unlike static optimizers, these systems sustain continuous evolution of the world model, allowing agents to anticipate environmental shifts and proactively update their understanding without extensive human intervention [66]. This process is particularly critical for tasks requiring long-term reasoning, such as planning in open-ended domains or interacting with real-time data [88].

Reward-driven optimization is a cornerstone for evolving the world model. Agents refine their internal representations using environmental feedback or self-generated signals, ensuring alignment with real-world dynamics. For example, Tool-Star employs reinforcement learning (RL) to update the world model's predictions for multi-tool interactions, improving generalization across tasks [17]. SPORT extends this approach to multimodal environments, applying step-wise preference optimization to refine visual and textual representations without human annotations [60]. These methods leverage process-level rewards to adjust the world model's parameters, functioning similarly to gradient updates and ensuring accurate anticipation of environmental states [93]. In multi-agent settings, SAGE integrates reflective reasoning with Ebbinghaus-inspired memory optimization, reducing inconsistencies in the world model during complex tasks such as database operations [66].

Imitation and demonstration learning further strengthen the evolution of the world model by incorporating self-generated or cross-agent trajectories. STaR bootstraps reasoning by iteratively refining the world model with successful demonstrations, enabling unsupervised updates [139]. Population-based approaches, such as EvoAgentX, maintain diverse world model variants and select optimal configurations through evolutionary competition [125]. ToolLLM generates high-quality tool-use trajectories to fine-tune the world model's understanding of action-outcome mappings, especially in multi-turn scenarios [93]. Gorilla enhances adaptability by integrating document retrievers, grounding world model predictions in retrieved knowledge as new tools are introduced [87].

Domain-specific adaptations highlight the optimizer's role in tailoring the world model to specialized contexts. In coding and GUI tasks, Skywork-Reward-V2 curates preference data through human-AI collaboration, refining domain-specific representations for greater predictive precision [69]. ToolGen encodes tools as tokens within the world model, streamlining selection and composition for large-scale tasks [122]. Evaluations on AgentBench report 5–48% improvements in open-source models, demonstrating the optimizer's ability to enhance predictive power [23]. Nevertheless, challenges persist in ensuring safety, preventing catastrophic forgetting, and scaling to diverse domains, which calls for robust adaptivity metrics [33]. Future directions include the exploration of co-evolutionary multi-agent systems to refine shared world models, thereby enhancing generalization [137, 152]. By fostering a closed-loop process that integrates feedback, memory augmentation, and tool-driven updates, the self-evolving dynamics optimizer empowers the world model to advance toward near-superintelligent autonomy. Such systems will be increasingly capable of navigating complex, ever-changing environments with resilience and foresight.

5.4 Self-evolution Between Components

The transition from a static AI agent to a self-evolving one necessitates architectural innovations that enable continuous, autonomous improvement. This evolution is not monolithic but occurs across the agent’s core components, its model, memory, tools, and dynamics optimizer, both individually and in concert. This section delineates the key architectural paradigms that facilitate this self-driven growth, categorizing them based on the components involved in the evolutionary loop. To more intuitively illustrate these methods and their research trajectories, we provide in Fig. 5.5 an overview diagram focused on each core component.

5.4.1 Joint Model-Memory Self-evolution

Models are designed to provide intelligent agents with an internal simulator of environmental dynamics, thereby supporting core capabilities such as prediction, planning, and decision-making. At the same time, the memory system stores and retrieves information across temporal scales, including short-term memory (STM) for immediate reasoning and long-term memory (LTM) for knowledge abstraction. In this chapter, we define co-evolution of model and memory as follows: while the parameters and structure of the world model are continuously updated, the memory system is also dynamically updated and expanded; through their interaction, a closed loop is formed, thereby enabling adaptive long-term learning and cross-task generalization.

A typical model-memory co-evolution framework can be described as a cyclic interactive process: the model performs prediction and planning with the support of memory; the critical experiences generated during interaction are selectively written into and/or reorganized; the policy is continuously optimized under memory

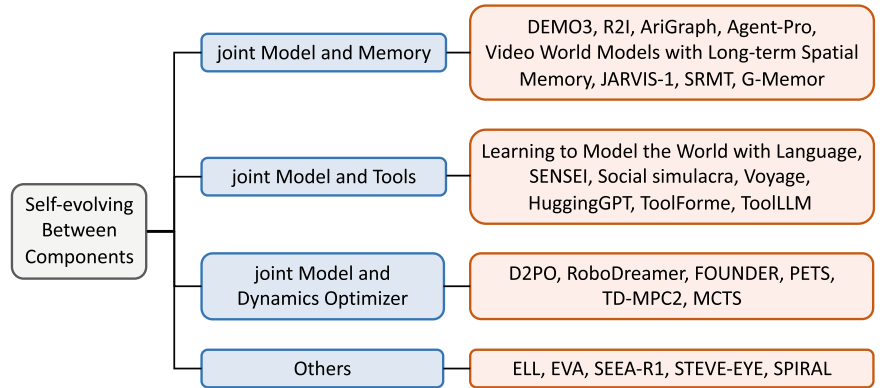


Fig. 5.5 An overview of self-evolving between components

retrieval and replay; and finally, the model parameters are updated. This closed-loop mechanism characterizes the self-evolving dynamics of “model prediction-memory update-action decision-model update,” providing a unified theoretical foundation for the sustained co-evolution of model and memory.

The importance of model-memory co-evolution lies in ensuring that world models can dynamically model across multiple temporal scales. At the fast timescale, short-term memory is updated almost in real time, serving to complete missing observations and maintain local context. At the intermediate timescale, the model continuously evolves through memory replay and sampling, reducing catastrophic forgetting. At the slow timescale, the consolidation of long-term memory and the internalization of model parameters promote knowledge abstraction and cross-task transfer. This multi-level integration enables the world model to both adapt quickly to environmental changes and gradually accumulate stable long-term knowledge.

In existing research, the co-evolution of model and memory can generally be divided into three distinct mechanisms. First, the first paradigm emphasizes the direct driving role of memory in model evolution. In this paradigm, memory is regarded as external experience or knowledge source, used to guide the parameter update of the world model and the optimization of policy. For example, DEMO3[22] introduces demonstration trajectories as explicit memory, transforms sparse-reward signals into learnable feedback, and realizes the joint optimization of memory and the world model. This type of method shows that memory is not only static storage, but can also be transformed into effective training signals in the form of replay and reuse. Similarly, R2I [103] demonstrates how to use memory retrieval to generate visualized inputs or replay key fragments, thereby improving the performance of the model in long-term prediction and reasoning. In contrast, RAG [58] embodies a “memory-in-the-loop” design. It does not update model parameters, but can enhance reasoning through external non-parametric memory, allowing the model to adaptively access factual knowledge during inference.

Another paradigm highlights the active update and reorganization of memory by the model. In this direction, the model not only retrieves memory, but can also structure and abstract it through reasoning and prediction. A typical work is AriGraph [3], which in embodied simulation tasks [29, 45], encodes experience into structured fact graphs and event graphs, thereby forming structured memory representations to support task transfer and complex reasoning. At the same time, Agent-Pro [145] proposes a method of transforming interaction history into “beliefs” and adjusting behavior through a policy-level reflection mechanism. This process is essentially model-driven memory reconstruction and abstraction, highlighting the value of memory in long-term decision-making [70].

The third paradigm focuses on closed-loop co-evolution, in which the model and memory continuously update each other through cycles of prediction, storage, retrieval, and updating. In the single-agent scenario, Video World Models with Long-term Spatial Memory [131] embeds spatial memory into the world model architecture, enabling advantages in scene consistency and long-term reasoning. JARVIS-1 [127] combines multimodal memory with a language-model-centered agent, improving planning and execution capabilities in open-world tasks. In multi-agent scenarios,

similar mechanisms are also of great significance. SRMT [101] proposes a shared workspace architecture, enabling multiple agents to aggregate local observations and reasoning results, and then coordinate through global memory, thereby achieving efficient solutions for long-term tasks. G-Memory [141] further develops a hierarchical triple-memory structure (insight, query, and interaction graphs), whose mechanism of continuous evolution with experience significantly improves the performance of multi-agent systems in complex reasoning.

5.4.2 *Joint Model-Tools Self-evolution*

Model-tool co-evolution aims to enhance the world modeling capability of embodied agents through the introduction of external tools, thereby enabling stronger core functions such as prediction, planning, and decision-making. Tools provide multi-dimensional capability interfaces for agents, including perception, knowledge and retrieval, reasoning and planning, as well as execution, which allow models to accomplish tasks more effectively in complex and open environments. In this chapter, we define model-tool co-evolution as follows: while the parameters and structure of the world model are continuously updated, the selection, invocation, and composition of tools also evolve with accumulated experience; the two interact to form a closed loop.

A typical model-tool co-evolution framework can be described as a cyclic interaction process: the model conducts reasoning and planning based on task objectives, and accordingly selects and invokes appropriate tools to obtain support in perception, knowledge, computation, or execution; the results returned by tools, as verifiable external data, are incorporated into subsequent decision refinement; with the accumulation of interaction experience, the system on the one hand updates the parameters and strategies of the model, and on the other hand also updates the strategies at the tool layer.

The importance of model-tool co-evolution lies first in its ability to bridge the capability gap of parameterized models: by invoking specialized perception and external knowledge bases, the model can achieve an expandable capability boundary without enlarging parameter scale; second, tool outputs (such as results of executable code, retrieved raw evidence, or feedback from physical execution) provide verifiable and traceable evidence chains for reasoning and decision-making, thereby significantly improving system reliability and interpretability; finally, tools, as externalized “plug-and-play modules,” naturally support compositionality and transferability, whereby the same set of tools can be reused across different tasks, enabling agents to possess cross-domain generalization and few-shot adaptation capabilities.

In the study of model-tool co-evolution, tools can generally be divided into three categories. Perception tools are mainly responsible for acquiring and representing environmental information. For example, in SENSEI [104], foundation models are used as perception tools for semantic scoring and cue extraction. Their outputs are directly fed into the world model and policy learning loop, serving as sources of

exploration guidance or intrinsic rewards; the model then adjusts observation selection and action distribution accordingly, which in turn alters the query strategy of the tools. Such tools expand the perceptual scope of the model in complex environments, enabling the agent to capture and exploit semantic information more efficiently.

By contrast, reasoning and planning tools emphasize providing logical inference and decision-making support for agents in complex tasks. A representative work is Learning to Model the World with Language [67], in which language is regarded as an explicit knowledge carrier and symbolic scaffold; during reasoning, the model retrieves or conditionally generates relevant rules and facts to compensate for the limitations of parameterized knowledge. In addition, Social simulacra [85] constructs a virtual mirror of social computing systems to generate and store diverse social facts and experiences. The model tests and revises hypotheses within the simulated society, and the newly generated “social facts” are written back into the knowledge base to drive the next round of retrieval and decision-making.

Execution tools bring external computation and real-world actions into a verifiable closed loop, representing the most intuitive scenario of model-tool co-evolution. Voyager [120] leverages code execution and skill libraries for continual self-expansion: the model synthesizes or invokes programs based on task decomposition, and execution results are fed back as strong supervision and traceable evidence, driving the bidirectional update of policies and skill libraries (including the addition, reconstruction, and elimination of skills). Stable and effective skills can be abstracted into higher-level reusable tools, which are then combined in subsequent tasks. Hugging-GPT [111] employs meta-scheduling to route tools among multiple external models, with execution outputs directly measuring task completion and error sources, thereby guiding the adjustment of routing strategies and sub-task assignments in the next iteration. ToolFormer [105] and ToolLLM [93] learn during training “when and how to call APIs,” shifting invocation decisions from heuristics to data-driven strategies; execution results provide verifiable supervisory signals for the model, forming a self-reinforcing cycle of “invocation-verification-internalization-reinvocation.”

5.4.3 *Joint Model-Dynamics Optimizer Self-evolution*

Model-dynamics optimizer co-evolution aims to enhance the predictability of environmental dynamics and the quality of decision-making through the coordinated evolution of model parameters and optimization mechanisms. In this chapter, we define model-dynamics optimizer co-evolution as follows: during the learning process of an agent, while the parameters and structure of the model continuously evolve, the strategies and update mechanisms of the optimizer are also constantly adapted; the two interact to form a loop, thereby achieving faster convergence and stronger robustness.

A typical joint framework can be described as a cyclic interaction: the world model performs prediction and planning under the current reward and policy configuration, collecting environmental returns and model errors; the optimizer, based

on this, integrates external rewards and outputs from the learned reward model to generate objective functions and policy schedules. Subsequently, the system updates the world model and policy parameters using this objective function, while at the same time retraining and reconstructing the reward model and policy scheduling rules. The optimizer continuously rewrites objectives, and the model, in turn, learns more robust dynamics and decision-making, which further drives the optimizer to converge toward more effective reward and policy configurations.

The core value of model-optimizer co-evolution lies in correcting and amplifying learning signals, thereby achieving stable and efficient learning in long-horizon and sparse-reward environments. First, joint co-evolution can alleviate reward sparsity by transforming hard-to-learn goals into learnable dense signals and adaptively rescaling magnitudes and weights across environments and stages; second, through the closed loop of “reward-policy-model,” the system can simultaneously improve sample efficiency and generalization, enabling the model to converge faster, while stable predictions allow the optimizer to allocate credit and design sub-goals with greater accuracy.

In existing research, one class of methods emphasizes reward-driven co-evolution. The core issue of this class of methods lies in the fact that the optimizer, through mechanisms such as preference modeling and reward updating, acts directly on the loss function, thereby altering the update trajectories of the world model and the policy. Representative works include D2PO [123], which leverages human preferences to rank imagined trajectories, enabling the reward model, world model, and policy to be jointly optimized under a unified objective; RoboDreamer [157] grounds high-level language goals into low-level action tokens and selects action sequences according to predicted success probabilities, essentially treating success probability as an optimization objective to guide generation and planning; FOUNDER [126] introduces the semantic alignment mechanism of foundation models into the world model and decision-making loop, so that abstract task intentions are transformed into operable rewards and constraints.

Another class of methods emphasizes search-driven co-evolution. The core of this class lies in the optimizer adopting trajectory optimization, sampling strategies, and tree search as its main forms, directly influencing the generation and selection of candidate sequences, and feeding the trajectories obtained from search back as training signals to the world model and the policy. The PETS [14] method proposed by Chua et al. employs ensemble dynamics and sampling-based optimization (CEM/MPPI), and uses uncertainty signals to regulate the sampling process and policy updates; TD-MPC2 [47] further embeds the trajectory optimizer into the latent space of the world model, making planning results directly part of the training objective; and MCTS together with its variants [79, 113, 114] performs tree search over the world model to prospectively evaluate candidate actions, and uses the resulting search distributions and returns as supervision signals to update the policy and value functions.

5.4.4 Others

In existing research, there are also studies that explore the joint self-evolution of memory and optimizers. ELL [5] focuses on a lifelong learning framework driven by long-term experience. This method stores and replays experiences from the interaction process and directly uses them as input signals to the optimizer, for the purpose of shaping rewards and recalibrating strategies, thereby achieving a tight coupling between memory and the dynamics optimizer. Such methods dynamically adjust learning objectives and optimization strategies through the accumulation and invocation of experiences.

At the same time, another line of research demonstrates a more complex mechanism of multi-factor joint optimization (Model + Memory + Tool + Optimizer). Representative work includes EVA [13], whose core idea is to introduce a reflection mechanism that treats prediction errors and failure cases as memorized experiences, generates corrective signals through an externalized reflection module, and thereby drives the objective revision of the optimizer while coordinating the update of the video prediction model. Similarly, SEEA-R1 [116] proposes a tree-structured reinforcement fine-tuning method, in which the storage of branched experiences plays the role of memory, the tree-structured reasoning process serves as a tool for decision-making, and the optimizer achieves continuous evolution of the model and policy through hierarchical scheduling. In addition, STEVE-EYE [154] combines LLMs with visual perception tools to provide embodied agents with visual understanding in open environments. Its perception module not only acts as a tool interface to provide external information streams, but also accumulates perceptual experiences as memory; these signals are incorporated into the reward and policy optimization loop, thereby driving the model to continuously evolve in the cycle of perception-decision-optimization. Finally, SPIRAL [68] illustrates a multi-factor coordination mechanism in a multi-agent self-play environment. This method relies on interaction histories as memory resources, while leveraging multi-turn reasoning and gameplay as external tools, and ultimately, under the shaping of the adversarial optimizer, enhances both the reasoning ability and the strategy selection capacity of the model.

Overall, more studies are attempting to deeply integrate models, memory, tools, and optimizers. Within this framework, the optimizer no longer functions in isolation as a parameter updater, but rather forms a self-evolutionary loop together with memory and tools, providing new ideas for embodied agents to achieve long-term adaptation and cross-task transfer in complex environments.

References

1. Alizadeh, K., Mirzadeh, S., Belenko, D., Khatamifard, S., Cho, M., del Mundo, C.C., Rastegari, M., Farajtabar, M.: LLM in a flash: Efficient large language model inference with limited memory. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 12562–12584 (2024)

2. Aljalbout, E., Sotirakis, N., van der Smagt, P., Karl, M., Chen, N.: Limt: Language-informed multi-task visual world models. In: 2025 IEEE International Conference on Robotics and Automation (ICRA), pp. 8226–8233. IEEE (2025)
3. Anokhin, P., Semenov, N., Sorokin, A., Evseev, D., Burtsev, M., Burnaev, E.: Arigraph: Learning knowledge graph world models with episodic memory for llm agents. arXiv preprint [arXiv:2407.04363](https://arxiv.org/abs/2407.04363) (2024)
4. Burchi, M., Timofte, R.: Learning transformer-based world models with contrastive predictive coding. arXiv preprint [arXiv:2503.04416](https://arxiv.org/abs/2503.04416) (2025)
5. Cai, Y., Hao, Y., Zhou, J., Yan, H., Lei, Z., Zhen, R., Han, Z., Yang, Y., Li, J., Pan, Q., et al.: Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark. arXiv preprint [arXiv:2508.19005](https://arxiv.org/abs/2508.19005) (2025)
6. Cao, N.D., Aziz, W., Titov, I.: Editing factual knowledge in language models. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp. 6491–6506 (2021)
7. Cao, S., Wang, L.: AWESOME: GPU memory-constrained long document summarization using memory mechanism and global salient content. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pp. 5925–5941 (2024)
8. Cen, J., Yu, C., Yuan, H., Jiang, Y., Huang, S., Guo, J., Li, X., Song, Y., Luo, H., Wang, F., et al.: Worldvla: Towards autoregressive action world model. arXiv preprint [arXiv:2506.21539](https://arxiv.org/abs/2506.21539) (2025)
9. Chen, G., Zhang, Z., Cong, X., Guo, F., Wu, Y., Lin, Y., Feng, W., Wang, Y.: Learning evolving tools for large language models. In: The Thirteenth International Conference on Learning Representations (2025)
10. Chen, M., Li, Y., Padthe, K., Shao, R., Sun, A.Y., Zettlemoyer, L., Ghosh, G., Yih, W.: Improving factuality with explicit working memory. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 11199–11213 (2025)
11. Chen, N., Li, H., Chang, J., Huang, J., Wang, B., Li, J.: Compress to impress: Unleashing the potential of compressive memory in real-world long-term conversations. In: Proceedings of the 31st International Conference on Computational Linguistics, pp. 755–773 (2025)
12. Chhikara, P., Khant, D., Aryan, S., Singh, T., Yadav, D.: Mem0: Building production-ready ai agents with scalable long-term memory. arXiv preprint [arXiv:2504.19413](https://arxiv.org/abs/2504.19413) (2025)
13. Chi, X., Fan, C.K., Zhang, H., Qi, X., Zhang, R., Chen, A., Chan, C.M., Xue, W., Liu, Q., Zhang, S., et al.: Empowering world models with reflection for embodied video prediction. In: Forty-second International Conference on Machine Learning (2025)
14. Chua, K., Calandra, R., McAllister, R., Levine, S.: Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In: Advances in Neural Information Processing Systems (NeurIPS) (2018). <https://papers.neurips.cc/paper/2018/file/5e365e392d0c2e1d741915d6f2bfff1cd-Paper.pdf>
15. Collu, J., Majellaro, R., Plaat, A., Moerland, T.M.: Slot structured world models. arXiv preprint [arXiv:2402.03326](https://arxiv.org/abs/2402.03326) (2024)
16. Diehl, C., Sykora, Q., Agro, B., Gilles, T., Casas, S., Urtasun, R.: Dio: Decomposable implicit 4d occupancy-flow world model. In: Proceedings of the Computer Vision and Pattern Recognition Conference, pp. 27456–27466 (2025)
17. Dong, G., Chen, Y., Li, X., Jin, J., Qian, H., Zhu, Y., Mao, H., Zhou, G., Dou, Z., Wen, J.R.: Tool-Star: Empowering llm-brained multi-tool reasoner via reinforcement learning. arXiv preprint [arXiv:2505.16410](https://arxiv.org/abs/2505.16410) (2025)
18. Du, Y., Huang, W., Zheng, D., Wang, Z., Montella, S., Lapata, M., Wong, K.F., Pan, J.Z.: Rethinking memory in ai: Taxonomy, operations, topics, and future directions. arXiv preprint [arXiv:2505.00675](https://arxiv.org/abs/2505.00675) (2025)
19. Du, Y., Wei, F., Zhang, H.: AnyTool: Self-reflective, hierarchical agents for large-scale API calls. In: Forty-first International Conference on Machine Learning (2024)

20. Efeoglu, S., Paschke, A.: Retrieval-augmented generation-based relation extraction. arXiv preprint [arXiv:2404.13397](https://arxiv.org/abs/2404.13397) (2024)
21. Elsayed, G., Mahendran, A., Van Steenkiste, S., Greff, K., Mozer, M.C., Kipf, T.: Savi++: Towards end-to-end object-centric learning from real-world videos. *Adv. Neural. Inf. Process. Syst.* **35**, 28940–28954 (2022)
22. Escoriza, A.L., Hansen, N., Tao, S., Mu, T., Su, H.: Multi-stage manipulation with demonstration-augmented reward, policy, and world model learning. arXiv preprint [arXiv:2503.01837](https://arxiv.org/abs/2503.01837) (2025)
23. Fang, J., Peng, Y., Zhang, X., Wang, Y., Yi, X., Zhang, G., Xu, Y., Wu, B., Liu, S., Li, Z., Ren, Z., Aletras, N., Wang, X., Zhou, H., Meng, Z.: A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems (2025). <https://arxiv.org/abs/2508.07407>
24. Fei, X., Zheng, X., Feng, H.: MCP-Zero: Active tool discovery for autonomous llm agents. arXiv preprint [arXiv:2506.01056](https://arxiv.org/abs/2506.01056) (2025)
25. Feng, J., Huang, S., Qu, X., Zhang, G., Qin, Y., Zhong, B., Jiang, C., Chi, J., Zhong, W.: Retool: Reinforcement learning for strategic tool use in llms. arXiv preprint [arXiv:2504.11536](https://arxiv.org/abs/2504.11536) (2025)
26. Feng, T., Li, Q., Wang, X., Wang, M., Li, G., Zhu, W.: Multi-weather cross-view geo-localization using denoising diffusion models. In: *Proceedings of the 2nd Workshop on UAVs in Multimedia: Capturing the World from a New Perspective*, pp. 35–39 (2024)
27. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2data: A large-scale cooperative perception dataset for swarm uavs autonomous flight. In: *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 7600–7608 (2024)
28. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2data-2: A scalable swarm uavs autonomous flight dataset for long-horizon tasks. arXiv preprint [arXiv:2509.00055](https://arxiv.org/abs/2509.00055) (2025)
29. Feng, T., Wang, X., Jiang, Y.G., Zhu, W.: Embodied ai: From llms to world models. In: *IEEE Circuits and Systems Magazine* (2025)
30. Feng, T., Wang, X., Zhou, Z., Wang, R., Zhan, Y., Li, G., Li, Q., Zhu, W.: Evoagent: Agent autonomous evolution with continual world model for long-horizon tasks. arXiv preprint [arXiv:2502.05907](https://arxiv.org/abs/2502.05907) (2025)
31. Fu, Y., Kim, D.K., Kim, J., Sohn, S., Logeswaran, L., Bae, K., Lee, H.: Autoguide: Automated generation and selection of state-aware guidelines for large language model agents. arXiv e-prints pp. arXiv–2403 (2024)
32. Fu, Y., Peng, R., Lee, H.: Go beyond imagination: maximizing episodic reachability with world models. In: *International Conference on Machine Learning*, pp. 10405–10420. PMLR (2023)
33. Gao, H., Geng, J., Hua, W., Hu, M., Juan, X., Liu, H., Liu, S., Qiu, J., Qi, X., Wu, Y., Wang, H., Xiao, H., Zhou, Y., Zhang, S., Zhang, J., Xiang, J., Fang, Y., Zhao, Q., Liu, D., Ren, Q., Qian, C., Wang, Z., Hu, M., Wang, H., Wu, Q., Ji, H., Wang, M.: A survey of self-evolving agents: On path to artificial super intelligence (2025). <https://arxiv.org/abs/2507.21046>
34. Gao, S., Zhou, S., Du, Y., Zhang, J., Gan, C.: Adaworld: Learning adaptable world models with latent actions. arXiv preprint [arXiv:2503.18938](https://arxiv.org/abs/2503.18938) (2025)
35. Gao, Y., Xiong, Y., Zhong, Y., Bi, Y., Xue, M., Wang, H.: Synergizing RAG and reasoning: A systematic review. arXiv preprint [arXiv:2504.15909](https://arxiv.org/abs/2504.15909) (2025)
36. Gao, Z., Zhang, B., Li, P., Ma, X., Yuan, T., Fan, Y., Wu, Y., Jia, Y., Zhu, S., Li, Q.: Multi-modal agent tuning: Building a vlm-driven agent for efficient tool usage. In: *The Thirteenth International Conference on Learning Representations* (2025)
37. Guan, Z., Kong, X., Zhong, F., Wang, Y.: Richelieu: Self-evolving llm-based agents for ai diplomacy. *Adv. Neural. Inf. Process. Syst.* **37**, 123471–123497 (2024)
38. Guo, Z., Cheng, S., Wang, H., Liang, S., Qin, Y., Li, P., Liu, Z., Sun, M., Liu, Y.: StableToolBench: Towards stable large-scale benchmarking on tool learning of large language models. In: *Findings of the Association for Computational Linguistics*, pp. 11143–11156 (2024)
39. Gutierrez, B.J., Shu, Y., Gu, Y., Yasunaga, M., Su, Y.: HippoRAG: Neurobiologically inspired long-term memory for large language models. In: *Advances in Neural Information Processing Systems* (2024)

40. Ha, D., Schmidhuber, J.: World models. arXiv preprint [arXiv:1803.10122](https://arxiv.org/abs/1803.10122) 2(3) (2018)
41. Hafner, D., Lillicrap, T., Ba, J., Norouzi, M.: Dream to control: Learning behaviors by latent imagination. arXiv preprint [arXiv:1912.01603](https://arxiv.org/abs/1912.01603) (2019)
42. Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., Davidson, J.: Learning latent dynamics for planning from pixels. In: International conference on machine learning, pp. 2555–2565. PMLR (2019)
43. Hafner, D., Lillicrap, T., Norouzi, M., Ba, J.: Mastering atari with discrete world models. arXiv preprint [arXiv:2010.02193](https://arxiv.org/abs/2010.02193) (2020)
44. Hafner, D., Pasukonis, J., Ba, J., Lillicrap, T.: Mastering diverse domains through world models. arXiv preprint [arXiv:2301.04104](https://arxiv.org/abs/2301.04104) (2023)
45. Han, F., Zhang, L., Wang, X., Zhao, K.A., Zhong, Y., Su, Z., Feng, T., Zhu, W.: U2usim—a uav telepresence simulation platform with multi-agent sensing and dynamic environment. In: Proceedings of the 32nd ACM International Conference on Multimedia, p. 11258–11260 (2024)
46. Hansen, N., Su, H., Wang, X.: Td-mpc2: Scalable, robust world models for continuous control. arXiv preprint [arXiv:2310.16828](https://arxiv.org/abs/2310.16828) (2023)
47. Hansen, N., Su, H., Wang, X.: Td-mpc2: Scalable, robust world models for continuous control. arXiv preprint [arXiv:2310.16828](https://arxiv.org/abs/2310.16828) (2023)
48. Hansen, N., Wang, X., Su, H.: Temporal difference learning for model predictive control. arXiv preprint [arXiv:2203.04955](https://arxiv.org/abs/2203.04955) (2022)
49. Hill, B.: Learning an adversarial world model for automated curriculum generation in marl. arXiv preprint [arXiv:2509.03771](https://arxiv.org/abs/2509.03771) (2025)
50. Hu, C., Fu, J., Du, C., Luo, S., Zhao, J., Zhao, H.: ChatDB: Augmenting llms with databases as their symbolic memory. arXiv preprint [arXiv:2306.03901](https://arxiv.org/abs/2306.03901) (2023)
51. Huang, Y., Zhang, J., Zou, S., Liu, X., Hu, R., Xu, K.: Ladi-wm: A latent diffusion-based world model for predictive manipulation. arXiv preprint [arXiv:2505.11528](https://arxiv.org/abs/2505.11528) (2025)
52. Janner, M., Fu, J., Zhang, M., Levine, S.: When to trust your model: Model-based policy optimization. *Advances in neural information processing systems* **32** (2019)
53. Jeong, Y., Chun, J., Cha, S., Kim, T.: Object-centric world model for language-guided manipulation. arXiv preprint [arXiv:2503.06170](https://arxiv.org/abs/2503.06170) (2025)
54. Kim, K., Sano, M., De Freitas, J., Haber, N., Yamins, D.: Active world model learning with progress curiosity. In: International conference on machine learning, pp. 5306–5315. PMLR (2020)
55. Kipf, T., Elsayed, G.F., Mahendran, A., Stone, A., Sabour, S., Heigold, G., Jonschkowski, R., Dosovitskiy, A., Greff, K.: Conditional object-centric learning from video. arXiv preprint [arXiv:2111.12594](https://arxiv.org/abs/2111.12594) (2021)
56. Lee, D., Prakash, C.S., FitzGerald, J., Lehmann, J.: MATTER: memory-augmented transformer using heterogeneous knowledge sources. In: Findings of the Association for Computational Linguistics, pp. 16110–16121 (2024)
57. Lee, K., Chen, X., Furuta, H., Canny, J.F., Fischer, I.: A human-inspired reading agent with gist memory of very long contexts. In: Forty-first International Conference on Machine Learning (2024)
58. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 9459–9474 (2020). <https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf>
59. Li, C., Jia, R., Liu, J., Zhang, Y., Niu, Y., Yang, Y., Liu, Y., Ouyang, W.: Theoretically guaranteed policy improvement distilled from model-based planning. arXiv preprint [arXiv:2307.12933](https://arxiv.org/abs/2307.12933) (2023)
60. Li, P., Gao, Z., Zhang, B., Mi, Y., Ma, X., Shi, C., Yuan, T., Wu, Y., Jia, Y., Zhu, S.C., Li, Q.: Iterative tool usage exploration for multimodal agents via step-wise preference tuning (2025). <https://arxiv.org/abs/2504.21561>

61. Li, S., He, Y., Guo, H., Bu, X., Bai, G., Liu, J., Liu, J., Qu, X., Li, Y., Ouyang, W., Su, W., Zheng, B.: GraphReader: Building graph-based agent to enhance long-context abilities of large language models. In: Findings of the Association for Computational Linguistics: EMNLP, pp. 12758–12786 (2024)
62. Li, W., Zhao, H., Yu, Z., Du, Y., Zou, Q., Hu, R., Xu, K.: Pin-wm: Learning physics-informed world models for non-prehensile manipulation. arXiv preprint [arXiv:2504.16693](https://arxiv.org/abs/2504.16693) (2025)
63. Li, X., Qiu, X.: MoT: Memory-of-thought enables chatgpt to self-improve. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 6354–6374 (2023)
64. Li, Z., Chen, X., Yu, H., Lin, H., Lu, Y., Tang, Q., Huang, F., Han, X., Sun, L., Li, Y.: StructRAG: Boosting knowledge intensive reasoning of llms via inference-time hybrid information structurization. In: The Thirteenth International Conference on Learning Representations (2025)
65. Liang, D., Zhang, D., Zhou, X., Tu, S., Feng, T., Li, X., Zhang, Y., Du, M., Tan, X., Bai, X.: Seeing the future, perceiving the future: A unified driving world model for future generation and perception. arXiv preprint [arXiv:2503.13587](https://arxiv.org/abs/2503.13587) (2025)
66. Liang, X., Tao, M., Xia, Y., Wang, J., Li, K., Wang, Y., He, Y., Yang, J., Shi, T., Wang, Y., et al.: Sage: Self-evolving agents with reflective and memory-augmented abilities. Neurocomputing p. 130470 (2025)
67. Lin, J., Du, Y., Watkins, O., Hafner, D., Abbeel, P., Klein, D., Dragan, A.D.: Learning to model the world with language. arXiv preprint [arXiv:2308.01399](https://arxiv.org/abs/2308.01399) (2023). <https://arxiv.org/abs/2308.01399>
68. Liu, B., Guertler, L., Yu, S., Liu, Z., Qi, P., Balcells, D., Liu, M., Tan, C., Shi, W., Lin, M., et al.: Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. arXiv preprint [arXiv:2506.24119](https://arxiv.org/abs/2506.24119) (2025)
69. Liu, C.Y., Zeng, L., Xiao, Y., He, J., Liu, J., Wang, C., Yan, R., Shen, W., Zhang, F., Xu, J., et al.: Skywork-reward-v2: Scaling preference data curation via human-ai synergy. arXiv preprint [arXiv:2507.01352](https://arxiv.org/abs/2507.01352) (2025)
70. Liu, H., Huang, X., Gu, J., Shi, J., He, N., Feng, T.: Tcdformer-based momentum transfer model for long-term sports prediction. Expert Systems with Applications p. 128310 (2025)
71. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. Transactions of the Association for Computational Linguistics **12**, 157–173 (2024)
72. Liu, W., Huang, X., Zeng, X., Hao, X., Yu, S., Li, D., Wang, S., Gan, W., Liu, Z., Yu, Y., Wang, Z., Wang, Y., Ning, W., Hou, Y., Wang, B., Wu, C., Wang, X., Liu, Y., Wang, Y., Tang, D., Tu, D., Shang, L., Jiang, X., Tang, R., Lian, D., Liu, Q., Chen, E.: ToolACE: Winning the points of LLM function calling. In: The Thirteenth International Conference on Learning Representations (2025)
73. Liu, Y., Peng, X., Cao, J., Bo, S., Zhang, Y., Zhang, X., Cheng, S., Wang, X., Yin, J., Du, T.: Tool-Planner: Task planning with clusters across multiple tools. In: The Thirteenth International Conference on Learning Representations (2025)
74. Lu, J., An, S., Lin, M., Pergola, G., He, Y., Yin, D., Sun, X., Wu, Y.: MemoChat: Tuning LLMs to use memos for consistent long-range open-domain conversation. arXiv preprint [arXiv:2308.08239](https://arxiv.org/abs/2308.08239) (2023)
75. Mendonca, R., Rybkin, O., Daniilidis, K., Hafner, D., Pathak, D.: Discovering and achieving goals via world models. Adv. Neural. Inf. Process. Syst. **34**, 24379–24391 (2021)
76. Micheli, V., Alonso, E., Fleuret, F.: Transformers are sample-efficient world models. arXiv preprint [arXiv:2209.00588](https://arxiv.org/abs/2209.00588) (2022)
77. Min, W., Yu, R., Chen, Y., et al.: Uniworld: Autonomous driving pre-training via world models. arXiv preprint [arXiv:2308.07234](https://arxiv.org/abs/2308.07234) (2023)
78. Mitchell, E., Lin, C., Bosselut, A., Finn, C., Manning, C.D.: Fast model editing at scale. In: The Tenth International Conference on Learning Representations (2022)
79. Moerland, T.M., Broekens, J., Plaat, A., Jonker, C.M.: A0c: Alpha zero in continuous action space. arXiv preprint [arXiv:1805.09613](https://arxiv.org/abs/1805.09613) (2018)

80. Mosbach, M., Ewertz, J.N., Villar-Corrales, A., Behnke, S.: Sold: Slot object-centric latent dynamics models for relational manipulation learning from pixels. arXiv preprint [arXiv:2410.08822](https://arxiv.org/abs/2410.08822) (2024)
81. Murre, J.M., Dros, J.: Replication and analysis of ebbinghaus' forgetting curve. PLoS ONE **10**(7), e0120644 (2015)
82. Nematollahi, I., DeMoss, B., Chandra, A.L., Hawes, N., Burgard, W., Posner, I.: Lumos: Language-conditioned imitation learning with world models. arXiv preprint [arXiv:2503.10370](https://arxiv.org/abs/2503.10370) (2025)
83. Packer, C., Fang, V., Patil, S.G., Lin, K., Wooders, S., Gonzalez, J.E.: MemGPT: Towards llms as operating systems. arXiv preprint [arXiv:2310.08560](https://arxiv.org/abs/2310.08560) (2023)
84. Park, J., Cho, J., Ahn, S.: MrSteve: Instruction-following agents in minecraft with what-where-when memory. arXiv preprint [arXiv:2411.06736](https://arxiv.org/abs/2411.06736) (2024)
85. Park, J.S., Popowski, L., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S.: Social simulacra: Creating populated prototypes for social computing systems. In: Proceedings of the 35th Annual ACM Symposium on User Interface Software and Technology (UIST '22), pp. 1–18. ACM, New York, NY, USA (2022). <https://doi.org/10.1145/3526113.3545616>
86. Pascanu, R., Li, Y., Vinyals, O., Heess, N., Buesing, L., Racanière, S., Reichert, D., Weber, T., Wierstra, D., Battaglia, P.: Learning model-based planning from scratch. arXiv preprint [arXiv:1707.06170](https://arxiv.org/abs/1707.06170) (2017)
87. Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E.: Gorilla: Large language model connected with massive apis (2023). <https://arxiv.org/abs/2305.15334>
88. Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E.: Gorilla: Large language model connected with massive apis. In: NeurIPS (2024)
89. Po, R., Nitzan, Y., Zhang, R., Chen, B., Dao, T., Shechtman, E., Wetzstein, G., Huang, X.: Long-context state-space video world models. arXiv preprint [arXiv:2505.20171](https://arxiv.org/abs/2505.20171) (2025)
90. Pu, Y., Niu, Y., Yang, Z., Ren, J., Li, H., Liu, Y.: Unizero: Generalized and efficient planning with scalable latent world models. arXiv preprint [arXiv:2406.10667](https://arxiv.org/abs/2406.10667) (2024)
91. Qian, C., Dang, Y., Li, J., Liu, W., Xie, Z., Wang, Y., Chen, W., Yang, C., Cong, X., Che, X., et al.: Experiential co-learning of software-developing agents. arXiv preprint [arXiv:2312.17025](https://arxiv.org/abs/2312.17025) (2023)
92. Qian, C., Han, C., Fung, Y., Qin, Y., Liu, Z., Ji, H.: CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. In: The 2023 Conference on Empirical Methods in Natural Language Processing (2023). <https://openreview.net/forum?id=aCHq10rQiH>
93. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., Sun, M.: Toolllm: Facilitating large language models to master 16000+ real-world apis (2023). <https://arxiv.org/abs/2307.16789>
94. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., Sun, M.: ToolLLM: Facilitating large language models to master 16000+ real-world apis. In: The Twelfth International Conference on Learning Representations (2024)
95. Qiu, J., Qi, X., Zhang, T., Juan, X., Guo, J., Lu, Y., Wang, Y., Yao, Z., Ren, Q., Jiang, X., Zhou, X., Liu, D., Yang, L., Wu, Y., Huang, K., Liu, S., Wang, H., Wang, M.: Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution. CoRR [arXiv:2505.20286](https://arxiv.org/abs/2505.20286) (2025)
96. Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., Wen, J.R.: From exploration to mastery: Enabling LLMs to master tools via self-driven interactions. In: The Thirteenth International Conference on Learning Representations (2025)
97. Racanière, S., Weber, T., Reichert, D., Buesing, L., Guez, A., Jimenez Rezende, D., Puigdomènech Badia, A., Vinyals, O., Heess, N., Li, Y., et al.: Imagination-augmented agents for deep reinforcement learning. Adv. Neural Inf. Process. Syst. **30** (2017)
98. Rigter, M., Jiang, M., Posner, I.: Reward-free curricula for training robust world models. arXiv preprint [arXiv:2306.09205](https://arxiv.org/abs/2306.09205) (2023)

99. Rigter, M., Yamada, J., Posner, I.: World models via policy-guided trajectory diffusion. arXiv preprint [arXiv:2312.08533](https://arxiv.org/abs/2312.08533) (2023)
100. Safaya, A., Yuret, D.: Neurocache: Efficient vector retrieval for long-range language modeling. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pp. 870–883 (2024)
101. Sagirova, A., Kuratov, Y., Burtsev, M.: Srmt: Shared memory for multi-agent lifelong pathfinding. arXiv preprint [arXiv:2501.13200](https://arxiv.org/abs/2501.13200) (2025)
102. Salama, R., Cai, J., Yuan, M., Currey, A., Sunkara, M., Zhang, Y., Benajiba, Y.: Meminsight: Autonomous memory augmentation for llm agents. arXiv preprint [arXiv:2503.21760](https://arxiv.org/abs/2503.21760) (2025)
103. Samsami, M.R., Zhoulis, A., Rajendran, J., Chandar, S.: Mastering memory tasks with world models. In: International Conference on Learning Representations (ICLR) (2024). <https://arxiv.org/abs/2403.04253>
104. Sancaktar, C., Gumbsch, C., Zadaianchuk, A., Kolev, P., Martius, G.: Sensei: Semantic exploration guided by foundation models to learn versatile world models. arXiv preprint [arXiv:2503.01584](https://arxiv.org/abs/2503.01584) (2025). ICML 2025 camera-ready
105. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. arXiv preprint [arXiv:2302.04761](https://arxiv.org/abs/2302.04761) (2023)
106. Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., et al.: Mastering atari, go, chess and shogi by planning with a learned model. *Nature* **588**(7839), 604–609 (2020)
107. Sekar, R., Rybkin, O., Daniilidis, K., Abbeel, P., Hafner, D., Pathak, D.: Planning to explore via self-supervised world models. In: International conference on machine learning, pp. 8583–8592. PMLR (2020)
108. Shan, L., Luo, S., Zhu, Z., Yuan, Y., Wu, Y.: Cognitive memory in large language models. arXiv preprint [arXiv:2504.02441](https://arxiv.org/abs/2504.02441) (2025)
109. Shang, Y., Li, Y., Zhao, K., Ma, L., Liu, J., Xu, F., Li, Y.: Agentsquare: Automatic LLM agent search in modular design space. In: ICLR. OpenReview.net (2025)
110. Shang, Y., Zhang, X., Tang, Y., Jin, L., Gao, C., Wu, W., Li, Y.: Roboscape: Physics-informed embodied world model. arXiv preprint [arXiv:2506.23135](https://arxiv.org/abs/2506.23135) (2025)
111. Shen, Y., Song, K., Tan, X., Li, D., Lu, W., Zhuang, Y.: Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. arXiv preprint [arXiv:2303.17580](https://arxiv.org/abs/2303.17580) (2023)
112. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. *Adv. Neural. Inf. Process. Syst.* **36**, 8634–8652 (2023)
113. Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al.: Mastering the game of go with deep neural networks and tree search. *Nature* **529**(7587), 484–489 (2016). <https://doi.org/10.1038/nature16961>
114. Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., et al.: Mastering the game of go without human knowledge. *Nature* **550**(7676), 354–359 (2017). <https://doi.org/10.1038/nature24270>
115. Team, O.E.L., Stooke, A., Mahajan, A., Barros, C., Deck, C., Bauer, J., Sygnowski, J., Trebacz, M., Jaderberg, M., Mathieu, M., et al.: Open-ended learning leads to generally capable agents. arXiv preprint [arXiv:2107.12808](https://arxiv.org/abs/2107.12808) (2021)
116. Tian, W., Zhang, S., Zhang, K., Chi, X., Luo, Y., Lu, J., Fan, C., Zhou, Q., Zhao, Y., Lin, N.L.S., et al.: Seer-r1: Tree-structured reinforcement fine-tuning for self-evolving embodied agents. arXiv preprint [arXiv:2506.21669](https://arxiv.org/abs/2506.21669) (2025)
117. Vafidis, P., Bhargava, A., Rangel, A.: Multi-task learning yields disentangled world models: Impact and implications. In: UniReps: 2nd Edition of the Workshop on Unifying Representations in Neural Models (2024)
118. Verma, M., Metcalf, K.: Hindsight priors for reward learning from human preferences. arXiv preprint [arXiv:2404.08828](https://arxiv.org/abs/2404.08828) (2024)

119. Wang, B., Fang, H., Eisner, J., Van Durme, B., Su, Y.: LLMs in the imaginarium: Tool learning through simulated trial and error. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 10583–10604 (2024)
120. Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., Anandkumar, A.: Voyager: An open-ended embodied agent with large language models. arXiv preprint [arXiv:2305.16291](https://arxiv.org/abs/2305.16291) (2023)
121. Wang, Q., Fu, Y., Cao, Y., Wang, S., Tian, Z., Ding, L.: Recursively summarizing enables long-term dialogue memory in large language models. *Neurocomputing* **639**, 130193 (2025)
122. Wang, R., Han, X., Ji, L., Wang, S., Baldwin, T., Li, H.: Toolgen: Unified tool retrieval and calling via generation. In: ICLR. OpenReview.net (2025)
123. Wang, S., Fei, Z., Cheng, Q., Zhang, S., Cai, P., Fu, J., Qiu, X.: World modeling makes a better planner: Dual preference optimization for embodied task planning. arXiv preprint [arXiv:2503.10480](https://arxiv.org/abs/2503.10480) (2025)
124. Wang, Y., Chen, X.: MIRIX: Multi-agent memory system for llm-based agents. arXiv preprint [arXiv:2507.07957](https://arxiv.org/abs/2507.07957) (2025)
125. Wang, Y., Liu, S., Fang, J., Meng, Z.: Evoagentx: An automated framework for evolving agentic workflows (2025). <https://arxiv.org/abs/2507.03616>
126. Wang, Y., Yu, R., Wan, S., Gan, L., Zhan, D.C.: Founder: Grounding foundation models in world models for open-ended embodied decision making. In: Proceedings of the 42nd International Conference on Machine Learning (ICML), *PMLR*, vol. 267 (2025). <https://arxiv.org/abs/2507.12496>
127. Wang, Z., Cai, S., Liu, A., Jin, Y., Hou, J., Zhang, B., Lin, H., He, Z., Zheng, Z., Yang, Y., Ma, X., Liang, Y.: Jarvis-1: Open-world multi-task agents with memory-augmented multimodal language models. arXiv preprint [arXiv:2311.05997](https://arxiv.org/abs/2311.05997) (2023)
128. Wang, Z., Wang, K., Zhao, L., Stone, P., Bian, J.: Dyn-o: Building structured world models with object-centric representations. arXiv preprint [arXiv:2507.03298](https://arxiv.org/abs/2507.03298) (2025)
129. Wang, Z.Z., Mao, J., Fried, D., Neubig, G.: Agent workflow memory. In: Forty-second International Conference on Machine Learning (2024)
130. Wu, J., Yin, S., Feng, N., Long, M.: Rlv-world: Training world models with reinforcement learning. arXiv preprint [arXiv:2505.13934](https://arxiv.org/abs/2505.13934) (2025)
131. Wu, T., Yang, S., Po, R., Xu, Y., Liu, Z., Lin, D., Wetzstein, G.: Video world models with long-term spatial memory. arXiv preprint [arXiv:2506.05284](https://arxiv.org/abs/2506.05284) (2025)
132. Xu, W., Mei, K., Gao, H., Tan, J., Liang, Z., Zhang, Y.: A-MEM: Agentic memory for llm agents. arXiv preprint [arXiv:2502.12110](https://arxiv.org/abs/2502.12110) (2025)
133. Xu, Y., Parker-Holder, J., Pacchiano, A., Ball, P., Rybkin, O., Roberts, S., Rocktäschel, T., Grefenstette, E.: Learning general world models in a handful of reward-free deployments. *Adv. Neural. Inf. Process. Syst.* **35**, 26820–26838 (2022)
134. Yang, R., Song, L., Li, Y., Zhao, S., Ge, Y., Li, X., Shan, Y.: GPT4Tools: Teaching large language model to use tools via self instruction. In: Advances in Neural Information Processing Systems (2023)
135. Yang, Y., Mei, J., Ma, Y., Du, S., Chen, W., Qian, Y., Feng, Y., Liu, Y.: Driving in the occupancy world: Vision-centric 4d occupancy forecasting and planning via world models for autonomous driving. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, pp. 9327–9335 (2025)
136. Yuan, H., Dou, H., Jiang, X., Deng, Y.: Task-aware world model learning with meta weighting via bi-level optimization. *Adv. Neural. Inf. Process. Syst.* **36**, 54167–54186 (2023)
137. Yuan, L., Chen, Y., Wang, X., Fung, Y., Peng, H., Ji, H.: CRAFT: Customizing LLMs by Creating and Retrieving from Specialized Toolsets. In: 12th International Conference on Learning Representations (2024). <https://openreview.net/forum?id=G0vdDSt9XM>
138. Yuan, S., Song, K., Chen, J., Tan, X., Shen, Y., Ren, K., Li, D., Yang, D.: EASYTOOL: enhancing llm-based agents with concise tool instruction. In: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 951–972 (2025)

139. Zelikman, E., Wu, Y., Mu, J., Goodman, N.: Star: Bootstrapping reasoning with reasoning. *Adv. Neural. Inf. Process. Syst.* **35**, 15476–15488 (2022)
140. Zhang, D., Chen, L., Zhang, S., Xu, H., Zhao, Z., Yu, K.: Large language models are semi-parametric reinforcement learning agents. *Advances in Neural Information Processing Systems* **36** (2024)
141. Zhang, G., Fu, M., Wan, G., Yu, M., Wang, K., Yan, S.: G-memory: Tracing hierarchical memory for multi-agent systems. *arXiv preprint [arXiv:2506.07398](https://arxiv.org/abs/2506.07398)* (2025)
142. Zhang, G., Fu, M., Wan, G., Yu, M., Wang, K., Yan, S.: G-Memory: Tracing hierarchical memory for multi-agent systems. *arXiv preprint [arXiv:2506.07398](https://arxiv.org/abs/2506.07398)* (2025)
143. Zhang, J., Hu, S., Lu, C., Lange, R., Clune, J.: Darwin Godel Machine: Open-Ended Evolution of Self-Improving Agents. *arXiv preprint [arXiv:2505.22954](https://arxiv.org/abs/2505.22954)* (2025)
144. Zhang, L., Kan, M., Shan, S., Chen, X.: Prelar: World model pre-training with learnable action representation. In: *European Conference on Computer Vision*, pp. 185–201. Springer (2024)
145. Zhang, W., Tang, K., Wu, H., Wang, M., Shen, Y., Hou, G., Tan, Z., Li, P., Zhuang, Y., Lu, W.: Agent-pro: Learning to evolve via policy-level reflection and optimization. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)* (2024). <https://aclanthology.org/2024.acl-long.292/>
146. Zhang, W., Wang, G., Sun, J., Yuan, Y., Huang, G.: Storm: Efficient stochastic transformer based world models for reinforcement learning. *Adv. Neural. Inf. Process. Syst.* **36**, 27147–27166 (2023)
147. Zhang, Y., Bai, C., Zhao, B., Yan, J., Li, X., Li, X.: Decentralized transformers with centralized aggregation are sample-efficient multi-agent world models. *arXiv preprint [arXiv:2406.15836](https://arxiv.org/abs/2406.15836)* (2024)
148. Zhang, Y., Sun, R., Chen, Y., Pfister, T., Zhang, R., Arik, S.: Chain of agents: Large language models collaborating on long-context tasks. *Adv. Neural. Inf. Process. Syst.* **37**, 132208–132237 (2024)
149. Zhang, Z., Dai, Q., Bo, X., Ma, C., Li, R., Chen, X., Zhu, J., Dong, Z., Wen, J.R.: A survey on the memory mechanism of large language model based agents. *ACM Transactions on Information Systems* (2024)
150. Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.J., Huang, G.: Expel: Llm agents are experiential learners. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 19632–19642 (2024)
151. Zhao, H., Ma, C., Wang, G., Su, J., Kong, L., Xu, J., Deng, Z., Yang, H.: Empowering large language model agents through action learning. *CoRR [arXiv:2402.15809](https://arxiv.org/abs/2402.15809)* (2024)
152. Zheng, B., Fatemi, M.Y., Jin, X., Wang, Z.Z., Gandhi, A., Song, Y., Gu, Y., Srinivasa, J., Liu, G., Neubig, G., Su, Y.: Skillweaver: Web agents can self-improve by discovering and honing skills. *CoRR [arXiv:2504.07079](https://arxiv.org/abs/2504.07079)* (2025)
153. Zheng, L., Wang, R., Wang, X., An, B.: Synapse: Trajectory-as-exemplar prompting with memory for computer control. In: *The Twelfth International Conference on Learning Representations* (2023)
154. Zheng, S., Liu, J., Feng, Y., Lu, Z.: Steve-eye: Equipping llm-based embodied agents with visual perception in open worlds. *arXiv preprint [arXiv:2310.13255](https://arxiv.org/abs/2310.13255)* (2023)
155. Zheng, W., Chen, W., Huang, Y., Zhang, B., Duan, Y., Lu, J.: Occworld: Learning a 3d occupancy world model for autonomous driving. In: *European conference on computer vision*, pp. 55–72. Springer (2024)
156. Zhong, W., Guo, L., Gao, Q., Ye, H., Wang, Y.: Memorybank: Enhancing large language models with long-term memory. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 19724–19731 (2024)
157. Zhou, S., Du, Y., Chen, J., Li, Y., Yeung, D.Y., Gan, C.: Robodreamer: Learning compositional world models for robot imagination. *arXiv preprint [arXiv:2404.12377](https://arxiv.org/abs/2404.12377)* (2024)
158. Zhuang, Y., Chen, X., Yu, T., Mitra, S., Bursztny, V., Rossi, R.A., Sarkhel, S., Zhang, C.: Toolchain*: Efficient action space navigation in large language models with a* search. In: *The Twelfth International Conference on Learning Representations* (2024)

Chapter 6

Self-evolving Agent



Abstract This chapter presents a systematic review and methodological exploration of Self-evolving Agents. First, in the overview section, it introduces the research background and driving motivations of self-evolving agents. The chapter then provides an in-depth analysis of their Key Components, including the model, memory, and tools, clarifying the roles of each plays in the evolutionary process. Building on this, the chapter summarizes the key architectures, examining model self-evolution, joint model-memory evolution, joint model-tools evolution, and the joint evolution of model-memory-tools, thereby demonstrating the potential of multi-dimensional collaborative optimization. Furthermore, the chapter outlines the key strategies, which include imitation-based self-evolution and interaction-based self-evolution, highlighting the distinct modes of learning and adaptation within these mechanisms. In the case study section, the chapter presents a complete practical framework for self-evolving agents. Overall, this chapter not only systematizes the theoretical and technical foundations of self-evolving agents but also illustrates their application potential through concrete examples, offering a structured reference framework for future research and practice.

6.1 The Overview

Large Language Model (LLM)-based agents have made significant advances in planning, reasoning, and tool-based problem-solving [38, 43, 46]. However, once deployed, these agents remain static: their parameters and structures are fixed. This limits their ability to handle new tasks, adapt to changing domains, or respond to evolving user needs. As a result, their autonomy and long-term reliability in dynamic environments remain constrained [10, 28, 32]. To overcome these limitations, recent research has begun to focus on self-evolving agents, which continuously improve their perception, reasoning, and interaction abilities during operation. Through continual learning, modular adaptation, and architectural plasticity, these agents aim to remain robust in changing conditions while avoiding catastrophic forgetting [24, 33].

A typical self-evolving agent [14] consists of three core components: the model, memory, and tools [42]. The model, which usually a large language model or a

multimodal large model, provides capabilities in semantic reasoning, abstraction, and decision-making [2]. Memory supports the accumulation of long-term knowledge, contextual grounding, and performance improvement through the integration of past experiences [4]. Tools extend the agent’s ability to interact with digital and physical environments, enabling information acquisition, action execution, and the coordination of complex workflows [38, 46]. Each of these components can evolve: models adapt via continual pretraining, fine-tuning, or parameter-efficient updates [21]; memory is reorganized to support scalable retrieval and lifelong learning [3]; and tools expand dynamically to handle new tasks or domains [39].

Although each component has advanced independently, the next key challenge is to develop architectures in which the model, memory, and tools evolve together. A joint framework promotes deeper synergy among subsystems, allowing them to co-adapt in mutually reinforcing ways. For instance, improvements in memory organization can provide structured knowledge to guide model adaptation, while new tools can expand the model’s action space and reshape how memory stores and retrieves information [14]. Recent work such as EvoAgent illustrates this approach by integrating multimodal reasoning with world models and adaptive tool use, enabling autonomous self-planning, self-reflection, and self-control across diverse environments. Similarly, modular prototype-based architectures like MoDP [6] demonstrate how disentanglement and compositionality can support incremental self-improvement in embodied agents [13].

Therefore, developing mechanisms that enable the joint evolution of models, memory, and tools is crucial for building truly autonomous and continuously self-improving systems. Such agents would go beyond the limitations of static LLM-based approaches and establish a foundation for intelligence that exhibits sustained growth, open-ended learning, and reliable operation in dynamic real-world settings [18, 45]. Ultimately, the goal of self-evolving agents is not only to enhance current AI systems technically, but also to pave the way for a new paradigm of adaptive, resilient, and lifelong learning artificial intelligence.

6.2 Key Components of Self-evolving Agent

The construction of a self-evolving agent relies on a set of interdependent components that jointly enable adaptation, reflection, and long-term growth. Unlike conventional systems where reasoning, action, and memory are often treated as isolated functions, self-evolving agents demand a tightly coupled design in which models, memory, and tools form a continuous feedback loop. The model provides the cognitive substrate for reasoning. Memory serves as both the substrate for immediate grounding and the repository for lifelong experience, and tools extend the agent’s reach beyond its intrinsic capacities. Together, these components define the mechanisms through which agents can not only react to environments but also iteratively improve themselves, making self-evolution a sustainable process rather than a static design.

6.2.1 *Model*

The model serves as the cognitive core of a self-evolving agent, determining how it interprets tasks, generates strategies, and produces responses. In most systems, this role is carried out by foundation models such as large language models, which offer broad linguistic and reasoning capabilities [8]. While these models perform well on static benchmarks, self-evolving agents require them to go beyond fixed reasoning and support adaptive processes that allow continuous improvement. Recent studies highlight that such models should incorporate metacognition and self-assessment mechanisms, enabling them to monitor their own reasoning, detect errors, and refine strategies over time [5]. This sets them apart from conventional models that depend solely on pretraining and fine-tuning, as self-evolving models gain the ability to improve autonomously through interaction, self-supervised data generation, and iterative feedback [27].

A growing body of work shows that agents can generate their own supervision signals to improve their models. For example, agents may create problems, attempt solutions, and use the results as training data, allowing them to learn without human annotation. This idea is central to methods such as self-play, self-rewarding mechanisms, and reflective fine-tuning, all of which help models enhance their reasoning and planning abilities. Furthermore, interactive feedback from environments or other agents can be used as reward signals, leading to steady improvements in policy and reasoning quality [27]. Together, these advances show that model evolution involves more than parameter adjustment—it requires embedding agents in ongoing cycles of exploration, error detection, and correction to support long-term growth.

A. Policy

The policy is the operational layer of a self-evolving agent, controlling how it selects and executes actions. Unlike traditional agents with fixed, handcrafted policies, self-evolving agents adjust their strategies based on experience, reflecting the idea that policies should evolve through feedback [27]. This adaptation can be achieved using reinforcement learning, reflective adjustments, or hybrid methods that combine imitation with outcome-based optimization. A key capability is generating or collecting feedback without human input, allowing policies to update continuously as agents operate in complex and dynamic environments.

Recent research has explored methods that let agents test their own policies by creating test cases, evaluating performance, and revising strategies accordingly. Self-play methods, where agents compete against earlier versions of themselves, create evolutionary pressure that strengthens strategies over time. Other approaches use adversarial refinement, where agents improve by tackling increasingly difficult or deceptive challenges generated internally. These methods share a common idea: the agent's own experience and feedback serve as the main source of learning, turning interaction into a sustainable driver of adaptation. The outcome is a dynamic pol-

icy layer that balances proven strategies with new explorations, helping the agent remain effective even in unpredictable situations. For example, adaptive tool-use mechanisms like MeCo show how agents can use meta-cognitive signals to decide when to use tools, illustrating dynamic policy adaptation [26].

B. Architecture

The architecture of self-evolving agents offers the structural foundation that integrates models, memory, tools, and feedback mechanisms into a unified system. A key characteristic of these architectures is modularity, which allows components to evolve independently while still contributing to overall system improvement [27]. Unlike static designs, self-evolving architectures support continuous information flow and closed-loop feedback, so that the results of reasoning and action can directly influence future decisions. This cycle of reflection, action, and refinement turns the architecture from a fixed pipeline into a dynamic, self-optimizing system.

Architectural evolution can take place at both single-agent and multi-agent levels. In single-agent systems, optimization often focuses on improving workflows by adjusting internal modules, restructuring reasoning steps, or even rewriting code to overcome observed weaknesses. In multi-agent systems, architectural evolution emphasizes coordination and collective intelligence. Research shows that multi-agent systems can optimize team composition, communication strategies, and coordination mechanisms, leading to emergent behaviors such as efficient task allocation or collaborative problem-solving. Importantly, these architectures are not fixed designs. They evolve in response to task requirements, computational limits, and accumulated experience. In this way, architecture acts as both the scaffold and the engine of self-evolution, allowing agents to adapt effectively across increasingly complex environments (Fig. 6.1).

6.2.2 *Memory*

Memory is a foundational pillar in self-evolving agents, as it enables the accumulation, retrieval, and structuring of past experiences that form the substrate for adaptation, generalization, and error avoidance [4]. Unlike conventional memory systems that primarily serve short-term task completion, the memory of self-evolving agents must support both immediate adaptability and lifelong learning. This dual requirement calls for a sophisticated architecture that continuously balances transient working memory with persistent long-term repositories [8].

In practice, memory is typically organized into short-term and long-term components. Short-term memory provides the contextual grounding for the agent's current interactions, while long-term memory preserves knowledge that spans across tasks and episodes, including factual information, high-level abstractions, procedural skills, and records of past successes and failures. Modern retrieval-augmented

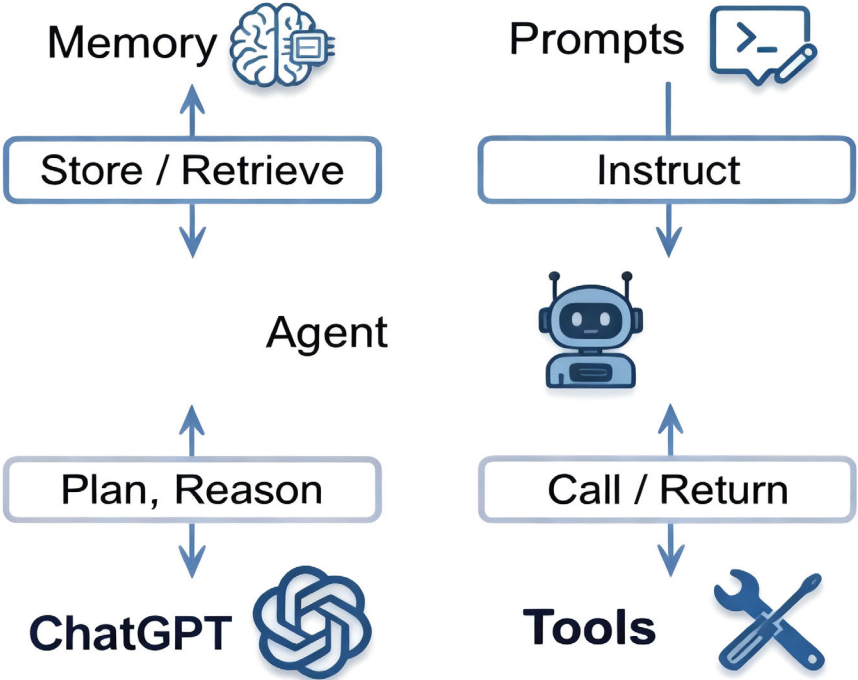


Fig. 6.1 The four core modules of the self-evolving agent

methods allow agents to fetch relevant memories at the moment of reasoning, ensuring that decision-making is informed by accumulated experience rather than limited to the local context of the input. In parallel, techniques such as pruning, clustering, summarization, and relevance scoring help to prevent long-term memory from becoming unwieldy, thereby sustaining efficiency over extended periods of use [4].

Beyond static storage and retrieval, self-evolving agents exhibit autonomy in shaping their memory content. They decide what to store, in what form, and how to reorganize and update knowledge as new experiences accumulate. Reflective processes are often employed to assign labels to experiences, distinguishing between successful and unsuccessful strategies, and to abstract generalizable skills or compress outdated records. The Experience-based Lifelong Learning (ELL) framework highlights these dynamics by emphasizing the continuous restructuring of experience traces into higher-level forms of knowledge that can be reused and adapted in novel scenarios [4]. Furthermore, the survey emphasizes that memory evolution can be both internal and external. Internally, agents restructure their knowledge base through reflective consolidation, while externally, they may share experiences with other agents in multi-agent ecosystems, thereby enabling collective memory that accelerates the evolution of capabilities across populations of agents.

6.2.3 Tools

External tools extend the capability space of self-evolving agents beyond what their internal models alone can accomplish [8]. Tools provide access to fresh information, computational precision, physical actuation, or specialized APIs that open pathways for solving tasks otherwise out of reach. The essence of self-evolving agents lies not merely in their ability to call tools but in the dynamic adaptation, optimization, and expansion of tool use over time.

At the most basic level, agents must select among available tools, generate appropriate inputs, and integrate outputs seamlessly into their reasoning pipelines. Yet self-evolving agents must progress beyond this initial competence toward the ability to discover and master new tools. This involves learning to read documentation, interpret examples, or actively experiment with interface calls until a reliable usage pattern is formed [26]. When no suitable tool exists, agents may even construct new tools or wrap existing functionalities, thereby enlarging their own operational environment. Such mechanisms reflect an ongoing process of tool acquisition and innovation that parallels human learning in open domains.

The survey stresses that tool evolution is not limited to enlarging the tool set but also includes refining the policy for tool usage. Agents must learn to decide under what conditions to invoke a tool, how to weigh the trade-offs between cost, reliability, and efficiency, and how to recover gracefully when tools fail. Recent approaches introduce meta-cognitive signals that guide these decisions, ensuring that tool invocation occurs only when it enhances performance, as demonstrated by the MeCo framework [26]. Over time, agents can construct hierarchies of tool usage, chaining multiple tools together, or developing fallback strategies that provide robustness in dynamic environments.

Equally significant is the integration of tool usage into the broader feedback loops that drive self-evolution. Each tool invocation produces outcomes that may succeed, fail, or yield partial results, and these outcomes generate signals that flow back into the agent's memory, model, and policy layers. Through this cycle, agents not only learn how to use tools more effectively but also refine their internal strategies in response to tool performance [27]. In multi-agent contexts, tool evolution takes on an additional dimension, as agents may share knowledge of tools, observe others' strategies, and collectively expand their accessible toolset, thereby accelerating the overall trajectory of self-evolution.

6.3 Key Architecture of Self-evolving Agent

The transition from a static AI agent to a self-evolving one necessitates architectural innovations that enable continuous, autonomous improvement. This evolution is not monolithic but occurs across the agent's core components, its model, memory, and tools, both individually and in concert. This section delineates the key architectural

paradigms that facilitate this self-driven growth, categorizing them based on the components involved in the evolutionary loop.

6.3.1 *Model Self-evolution*

Model self-evolution captures the capacity of an agent's central reasoning engine, most often instantiated as a large language model, to refine its own parameters, decision policies, and outputs on the basis of accumulated experience without the need for direct human intervention. This represents the most fundamental dimension of self-evolution, as it directly targets the improvement of the agent's internal cognitive processes. The central ambition is to enable the model not only to execute tasks but also to critically evaluate and improve its own reasoning patterns over time, thereby developing an increasingly sophisticated capacity for autonomous adaptation.

Architectures that support model self-evolution frequently rely on meta-cognitive feedback loops that allow the system to generate and act upon its own assessments. A key mechanism in this regard is self-reflection. After completing a task, the model is encouraged to evaluate its own performance, detect possible errors in reasoning or planning, and articulate explicit feedback on how its strategy might be improved. This feedback is not discarded but instead becomes a source of training material, guiding the synthesis of refined examples for additional fine-tuning or serving as a direct adjustment to the model's reasoning strategies in future tasks [47]. Such a cycle of experience, reflection, and learning creates the conditions for iterative internal growth, where each episode becomes a stepping stone toward more reliable performance.

A further advance in this trajectory is the development of generative self-improvement. In this approach, the model autonomously produces synthetic data, including new problem-solution pairs or alternative reasoning trajectories, which are then incorporated into its training process. By generating its own learning material, the model can reinforce effective strategies, experiment with novel concepts, and gradually expand the diversity of its internal knowledge base [40]. This strategy can be combined with online learning algorithms that allow incremental updates to the model's parameters. Reinforcement signals, such as those derived from reinforcement learning with human feedback, serve as valuable indicators of success or failure and provide a foundation for guiding continual parameter adjustment [7].

The central architectural challenge in model self-evolution lies in designing feedback mechanisms that remain stable and constructive over time. While self-generated reflections and synthetic data provide powerful opportunities for autonomous growth, they also carry the risk of compounding errors and reinforcing flawed reasoning patterns. Ensuring that the evolutionary cycle produces sustained improvements rather than degenerative drift requires careful structuring of evaluation criteria, the integration of external validation signals, and the design of mechanisms that balance exploration with reliability. Addressing this challenge is essential for transforming models from static reasoning systems into agents capable of open-ended self-improvement.

6.3.2 *Joint Model-Memory Self-evolution*

The architecture of joint model-memory self-evolution emphasizes that the model and its memory system cannot develop in isolation but must advance together in order to support sustained and robust learning. The model shapes the content and structure of what is stored in memory, while the evolving memory, in turn, provides the contextual foundation that enriches the model's future reasoning and self-improvement. This interdependence creates a bidirectional dynamic in which each component serves both as a driver and as a beneficiary of the other's evolution.

At the center of this process lies the design of a dynamic and selectively curated memory mechanism. Rather than storing raw experiences indiscriminately, the model performs a reflective step after each task to evaluate outcomes and extract essential lessons. These lessons often include key strategies that proved successful, errors that hindered performance, and broader insights that may hold relevance for future problem-solving. The extracted knowledge is then transformed into structured entries, annotated with metadata such as task categories or performance indicators, and stored in long-term memory [49]. In this way, memory evolves from a passive archive of past interactions into an active and structured repository of distilled experience, resembling a knowledge graph that encodes the agent's growing body of learned wisdom.

Memory, however, is not only a storage mechanism but also a sophisticated retrieval system that actively contributes to ongoing reasoning. When confronted with new challenges, the model engages with memory in search of not only exact matches but also analogical experiences that may offer transferable strategies. Through this process, the model adapts past knowledge to novel contexts, achieving a level of generalization that transcends rote recall. Recent advances in retrieval-augmented generation and dense vector search techniques have significantly strengthened this capacity by enabling efficient and contextually rich memory queries [35]. This continuous exchange between model and memory establishes a virtuous cycle: improved models generate more structured and valuable memories, while enriched memory repositories empower the model to reason with greater depth, flexibility, and foresight.

6.3.3 *Joint Model-Tools Self-evolution*

The principle of self-evolution can also be extended to the interplay between models and tools, where tools are employed not merely as external resources for task execution but as active instruments that enhance the model's learning and adaptation. Within this architecture, tools serve both as extensions of the agent's operational reach and as catalysts for gathering new data and feedback that guide the refinement of the model itself. The outcome is a system in which strategic tool use drives cognitive growth and operational expansion.

One of the defining capabilities of this paradigm is the capacity for automated tool discovery and learning. An agent may leverage its model to study tool documentation, interpret API specifications, or experiment directly with new functionalities. Through such processes, the agent gradually integrates new tools into its repertoire, thereby expanding the range of actions it can perform without human supervision. This dynamic acquisition of tools broadens the model's action space and contributes directly to its adaptive evolution.

Beyond discovery, tools also play an indispensable role in data acquisition and feedback generation. For instance, access to a web search system allows an agent to retrieve the most up-to-date information and thus overcome knowledge limitations inherited from pretraining. Similarly, the use of computational tools such as code interpreters provides opportunities for the agent to test hypotheses, verify predictions, and receive precise feedback grounded in executable outcomes rather than linguistic evaluation alone. This form of external feedback is particularly valuable, as it is integrated into the agent's self-reflection loop and used to refine its reasoning strategies, policies, and internal knowledge structures.

Designing architectures that enable effective model-tools co-evolution requires careful management of the interactions between tool outputs and model adaptation. Feedback obtained from tools must be accurately parsed, contextualized, and translated into signals that contribute meaningfully to learning. When executed effectively, this integration establishes a feedback-rich environment where tools not only extend the agent's operational capabilities but also drive iterative cycles of self-improvement. In doing so, joint model-tools self-evolution provides a foundation for agents that can continuously adapt to novel environments, expand their problem-solving repertoire, and evolve into more versatile and autonomous systems.

6.3.4 Joint Model-Memory-Tools Self-evolution

The most advanced stage of self-evolving agent design brings together model, memory, and tools into a unified architecture that functions as a fully autonomous and adaptive learning system. In such an integrated framework, the reasoning capacity of the model, the accumulated knowledge stored in memory, and the interaction capabilities provided by tools do not develop independently but instead co-evolve in a tightly coupled and mutually reinforcing manner. This synergy transforms the agent from a task-specific executor into a dynamic entity capable of sustained self-improvement across extended time horizons.

Within this architecture, every aspect of the agent's interaction history is harnessed for reflective learning. The model processes plans, tool invocations, tool outputs, and task outcomes in a comprehensive retrospective analysis. This analysis does not merely assess whether the final outcome was successful but also evaluates how effectively tools were selected and applied, how tool outputs were interpreted, and how memory retrievals influenced reasoning pathways. The insights distilled from this process are transformed into structured memories that capture not only

factual outcomes but also meta-knowledge, such as the relative reliability of tools, the strategies that produced effective memory retrievals, and the workflows that led to successful results [30].

Through this mechanism, the agent develops higher-order learning capacities. Over time, it can infer, for example, that a particular tool consistently outperforms alternatives for a certain class of problems and encode this preference into memory for future decision-making. Similarly, the system can identify which types of memory queries are most effective for different reasoning tasks, distinguishing between keyword-based retrieval and analogy-driven approaches. These insights accumulate into procedural knowledge that allows the agent to adaptively refine its own strategies. Beyond optimizing existing workflows, the integrated architecture also supports the creative synthesis of new workflows, where the model learns to combine tools with retrieved knowledge in novel configurations that prove effective for emerging challenges.

The evolving memory in this context becomes far more than a repository of task outcomes. It functions as a continually expanding database of domain-specific knowledge, procedural tactics, and meta-cognitive insights into the agent's own operational patterns. This layered memory enables the agent not only to recall what has been learned but also to understand how it learns most effectively, creating a recursive improvement loop that strengthens both competence and adaptability [44].

By tightly coupling the evolution of model, memory, and tools, this architecture establishes a closed-loop system in which each component amplifies the capabilities of the others. The result is an agent that is capable of sustained growth, autonomous adaptation to novel environments, and resilience in the face of open-world complexity. Such integration represents the culmination of the self-evolving paradigm, providing a foundation for agents that are not merely reactive but genuinely self-directing in their pursuit of continual improvement.

6.4 Key Strategy of Self-evolving Agent

An essential dimension of self-evolving agents is the set of learning strategies that transform raw interactions and observations into lasting improvements in policy, memory, tools, and workflow. These strategies determine both the types of signals and the agent processes, such as textual critique, scalar rewards, or environmental outcomes, and the methods by which these signals are utilized, including imitation, reinforcement, distillation, and population search. The literature identifies two primary, orthogonal families of strategies: imitation-based approaches, which depend on high-quality demonstrations as the main training signal, and interaction-based approaches, which derive learning signals from the agent's engagement with its environment.

6.4.1 *Imitation-Based Self-evolution*

Imitation-based self-evolution treats successful executions, whether produced by human experts, stronger “teacher” models, or the agent’s own past best attempts, as the canonical learning signal [36]. This paradigm is particularly powerful for bootstrapping competence in domains where unguided exploration is inefficient or dangerous, and it provides a sample-efficient route to learning multi-step strategies that are otherwise difficult to discover. The literature clusters several operational modalities under this umbrella: behavioral cloning, distillation, self-imitation [31], cross-agent demonstration sharing, and hybrid demonstration pipelines.

Behavioral cloning (or supervised imitation) frames successful trajectories as supervised examples: state/action (or prompt/output) pairs are aggregated into datasets [11, 12] used to fine-tune the agent’s policy or generate prompt templates. This approach is straightforward and effective when demonstrations are plentiful and representative, and it can be implemented either via full-parameter supervised fine-tuning or parameter-efficient adaptations such as LoRA for large models [23]. Behavioral cloning has been used as a baseline in many agent pipelines where high-quality solutions are available or can be synthesized.

Knowledge distillation elevates imitation by enabling an agent to query a more capable but expensive “teacher” model for labels, explanations, or reasoning traces, and then compress those capabilities into a cheaper student model. Distillation is a pragmatic route to continuous offline improvement: it yields models that approximate the teacher’s competence while retaining a tractable run-time footprint. In agent contexts, distillation can operate on different representational levels, final answers, chain-of-thoughts, and even tool-selection policies, allowing a student agent to internalize richer decision-making heuristics than simple action imitation [20].

Self-imitation learning plays a complementary role: by curating a “hall of fame” of the agent’s own best episodes, the system repeatedly reuses its most successful trajectories as high-quality training data. Self-imitation mitigates catastrophic forgetting of rare but valuable behaviors and stabilizes learning by reinforcing strategies that have empirically worked. Methods such as STaR [48] and successive self-training pipelines illustrate how an agent can iteratively bootstrap reasoning and tool-use abilities from its own validated outputs [31].

Cross-agent demonstration learning and hybrid schemes extend the imitation paradigm beyond single-agent self-bootstrapping. In multi-agent or multi-model ecosystems, agents share successful episodes, verifiers, and curated demonstrations to accelerate collective learning; hybrid pipelines combine self-generated exemplars with externally sourced expert data and confidence-guided selection mechanisms to maximize robustness and diversity of the training set [25]. These mechanisms are particularly valuable in domains where high-quality demonstrations are unevenly distributed across tasks or where different agents specialize in different skills.

Despite its strengths, imitation-based evolution is not a panacea: its efficacy critically depends on the quality, coverage, and diversity of example trajectories. When expert demonstrations are scarce, noisy, or biased, imitation can overfit to subopti-

mal patterns; recent work therefore emphasizes verifier-guided selection, confidence-based filtering, and adaptive sampling strategies to make imitation robust in realistic settings.

6.4.2 Interaction-Based Self-evolution

Interaction-based self-evolution encompasses strategies in which agents acquire knowledge and refine their behavior through engagement with their environment, particularly in settings where explicit demonstrations or labeled data are unavailable [22]. In this paradigm, the agent treats the external world or an internal model of it as a source of feedback, using the consequences of its actions to guide learning and policy refinement. Feedback signals can take diverse forms, including textual annotations, scalar rewards, or more implicit indicators derived from task outcomes or human interactions. The literature generally classifies these methods according to the origin and nature of the feedback, identifying three principal categories: intrinsic motivation, in which agents generate internal reward signals to drive exploration; environmental objectives, in which rewards are externally provided by the environment; and implicit rewards, which are embedded in the dynamics of the task or in the responses of human collaborators.

A. Internal Rewards Interaction

Internal-reward mechanisms represent a core approach to intrinsic motivation, endowing agents with self-generated objectives that guide exploration and skill acquisition independently of external task-specific incentives. Curiosity-driven learning exemplifies this approach, as agents receive intrinsic rewards for encountering novel states or for reducing prediction error within an internal world model. This mechanism encourages systematic exploration and the progressive discovery of capabilities that are broadly transferable across tasks. Unsupervised skill discovery frameworks further extend this paradigm by rewarding behavioral diversity, leading to the emergence of reusable primitives that can later be composed to address downstream objectives. These internally generated reward loops are particularly valuable because they reduce reliance on carefully engineered external rewards, enabling agents to pre-train and refine competencies in open-ended, unstructured environments [41]. Recent studies underscore the utility of internal confidence metrics, ensemble-based self-evaluation, and self-rewarding training cycles as scalable and robust sources of supervision, demonstrating that agents can achieve meaningful autonomous improvement even in the absence of explicit guidance.

By leveraging interaction-based self-evolution, agents are able to develop increasingly sophisticated behaviors that are grounded in their own experiential engagement with the environment. This paradigm not only facilitates the acquisition of task-relevant skills but also fosters generalization, adaptability, and the emergence

of higher-order reasoning capabilities, providing a foundation for agents that can continuously improve in complex and dynamic domains.

B. External Rewards Interaction

External rewards are derived from objective signals provided by the environment or by task evaluators, such as success and failure labels, numeric performance scores, or responses from external tools. These signals provide clear and measurable criteria for guiding agent behavior, enabling direct optimization toward well-defined objectives, including successful web navigation, accurate program execution, or task completion in interactive environments. Reinforcement learning (RL) and reward-conditioned fine-tuning thus constitute the primary methodological frameworks for leveraging external rewards in self-evolving agents. In practice, however, these methods must contend with issues such as sparse feedback, delayed credit assignment, and the instability of training with high-capacity models. Consequently, they are frequently augmented with procedural innovations such as hierarchical reward structures, process-based evaluation models, or the design of curricula that gradually expose agents to increasingly complex tasks. For instance, hierarchical formulations that combine episode-level outcome signals with step-level shaping rewards have been shown to improve credit assignment in long-horizon decision-making. Likewise, outcome verification mechanisms, which incorporate external evaluation into the training loop, help ensure that learned policies align with human-defined objectives and maintain reliability under distributional shift [1, 34].

Many contemporary agent systems therefore adopt hybrid strategies that integrate RL with supervised fine-tuning and external verification, allowing agents to balance the stability of imitation with the adaptability of reinforcement. By grounding evolution in explicit external signals, agents are able to align their adaptations with performance objectives that are both measurable and interpretable, thereby facilitating predictable, goal-directed improvement [9]. This framework is particularly effective in domains where clear success metrics exist, such as program synthesis, robotic control, or large-scale multi-agent simulations [19], and remains one of the most widely used paradigms for structuring agent self-evolution.

C. Implicit Rewards Interaction

In numerous real-world scenarios, explicit numeric rewards are either unavailable or prohibitively expensive to obtain, and agents must instead infer learning signals from indirect observations, task outcomes, or patterns of human interaction. Implicit rewards transform such observable results into informative training signals, enabling autonomous adaptation even in the absence of formally defined objectives. For example, outcome-based self-reflection converts task successes and failures into implicit-reward signals by retrospectively evaluating whether actions contributed to goal achievement and by extracting error patterns or corrective strategies, which can

subsequently be incorporated into the agent’s memory and training data [40]. Human feedback provides another rich source of implicit supervision, ranging from explicit corrections to subtler cues such as engagement levels, satisfaction, or expressed preferences. These signals can be modeled and distilled into learned reward functions that shape agent alignment, effectively allowing agents to learn from nuanced social interaction without relying on scalar task-specific rewards.

Recent research also demonstrates that large language models can exploit internal or endogenous signals, including scalar cues embedded in context, token-level likelihoods, or uncertainty estimates, as pseudo-rewards. Such mechanisms allow the model to bootstrap adaptation using their own predictive structure, creating an internal reinforcement signal even when no explicit external feedback is available. This perspective broadens the scope of agent evolution, enabling adaptation in open-ended and under-specified domains where formal reward engineering is impractical.

Nevertheless, implicit-reward strategies are not without challenges. They require sophisticated inference mechanisms to reliably transform indirect observations into actionable learning targets, and they must carefully balance exploration with overfitting to spurious signals. Despite these difficulties, implicit-reward interactions represent a powerful complement to explicit external reinforcement. By leveraging nuanced environmental and interactional cues, agents gain the capacity to flexibly self-improve in domains where success cannot be reduced to a simple numeric score. In this sense, implicit rewards expand the frontier of self-evolving systems, equipping them to operate in complex, dynamic, and poorly structured environments [45].

Imitation- and interaction-based strategies are complementary rather than mutually exclusive. In practice, high-performance self-evolving agents blend the two: imitation provides sample-efficient initialization and stable heuristics, while interaction-based learning enables continual adaptation and discovery of novel capabilities. The design choices of what to imitate, which internal drives to instantiate, how to construct external reward models, and how to infer implicit signals govern the agent’s sample efficiency, safety, and generalization. Emerging best practices emphasize hybrid pipelines, confidence-aware selection mechanisms, and modular architectures that allow different evolutionary signals to update appropriate components with controlled influence.

6.5 Case Study: Self-evolving Agent

As a concrete instantiation of a self-evolving embodied system, the Mixture of Disentangled Prototypes (MoDP) framework illustrates how modular representation, targeted architectural adaptation, and online prototype refinement can together produce agents that continuously reorganize their behavior space in response to new compositions of embodiment and environment [6]. Unlike conventional end-to-end reinforcement learning or imitation-based frameworks, which often entangle heterogeneous interaction factors into a monolithic parameterization [17, 37], MoDP explicitly disentangles the structure of the problem and introduces modularity at the

level of both representation and policy architecture. This disentanglement allows the system to acquire a reusable building blocks that can be incrementally refined and recombined to accommodate novel agent, environment compositions. In this way, MoDP not only improves compositional generalization but also lays a foundation for continual adaptation, which is the hallmark of self-evolution [42].

6.5.1 Framework Design

The framework operates by decomposing the observation-to-action pipeline into three distinct but interrelated modules, each of which plays a complementary role in disentangling, specializing, and recomposing the factors of interaction. The agent, environment prototype disentanglement (AEPD) module takes as input the trajectory embedding of a given agent, environment pair, and maps it into two separate latent spaces, one capturing agent-specific dynamics and the other encoding environment-specific constraints. By projecting these latent vectors onto compact prototype sets through similarity-weighted convex combinations, the module ensures that interaction factors are represented by a finite and reusable repertoire of primitives. This separation provides the structural bias required for compositional generalization, as the same agent prototypes can be recombined with environment prototypes from previously unseen contexts to yield plausible representations for novel configurations. Building upon these disentangled representations, the policy-prototype disentanglement (PPD) module extracts a library of functional subnetworks from a large pretrained backbone using connection-sensitive pruning. Each subnetwork, or policy-prototype, is masked out from the backbone according to the sensitivity of its parameters to specific behaviors, thereby preserving the most relevant functional components while eliminating redundancy. This process transforms the policy space from a monolithic parameterization into a modular collection of interpretable sub-policies, each specializing in a subset of behaviors aligned with different prototypes. The final stage, dynamic prototype routing (DPR), introduces a lightweight mechanism for policy composition. Given the prototype activations from the AEPD module, DPR computes a soft assignment over the available policy prototypes and routes the input through an appropriate mixture of subnetworks, effectively assembling a composite policy without requiring global retraining. In this way, the routing mechanism enables fast adaptation to unseen agent, environment pairs by interpolating between specialized sub-policies in a parameter-free manner. Taken together, these three modules operationalize the principle of modular self-evolution: the disentanglement of heterogeneous factors, the specialization of functional components, and the flexible recomposition of learned primitives. This design constrains the otherwise intractable interaction space into a structured latent manifold with explicit mechanisms for incremental adaptation, providing both stability and plasticity in long-term learning and thereby supporting the continual self-evolution of embodied agents (Fig. 6.2).

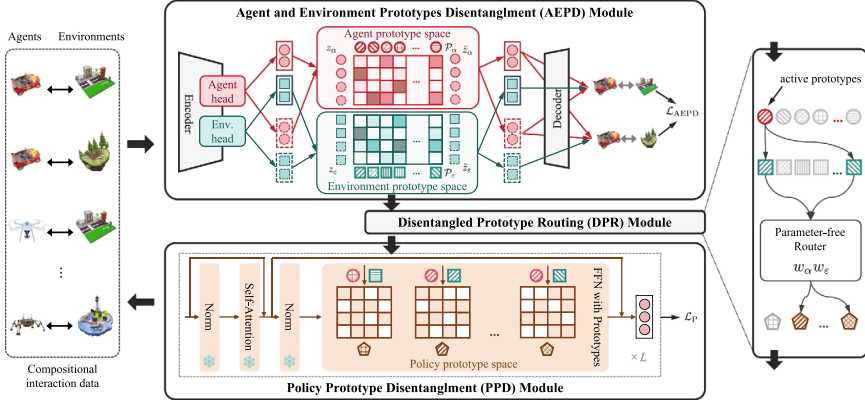


Fig. 6.2 The framework of the proposed MoDP

6.5.2 Learning Mechanism

An important strength of MoDP lies in its local and modular update rules, which allow continual refinement without catastrophic interference or the computational burden of global retraining. Rather than requiring parameter updates across the entire model, MoDP introduces structured mechanisms for targeted adaptation, thereby striking a balance between stability and plasticity [33]. The prototypes associated with agents and environments are optimized through a compositional reconstruction objective, reinforced with regularization terms that promote representativeness, uniqueness, and sparsity. This formulation ensures that prototypes remain both diverse and compact, preventing redundancy while still preserving sufficient coverage of the interaction space [16]. By maintaining this diversity, the framework enables new compositions of agents and environments to be effectively represented without necessitating a full reorganization of the learned structure.

Policy prototypes follow a complementary but distinct update mechanism. They are defined by connection sensitivity, a principle grounded in pruning and network compression literature, which identifies subnetworks of the backbone model according to their importance for specific behaviors [15]. These subnetworks are preserved through binary masks, which evolve as additional experiences accumulate. In this way, MoDP effectively treats each policy-prototype as a functional subnetwork that can be refined over time, enabling the emergence of specialized experts aligned with recurring patterns of interaction. Importantly, this process is not static; masks are dynamically refined as the distribution of tasks and environments changes, ensuring continual adaptation without discarding previously acquired competence.

All prototypes and their associated masks are further updated with a momentum-based aggregation scheme, which weights the influence of new experiences by prototype activation. This mechanism allows the system to balance short-term sensitivity to novel signals with long-term stability, thereby supporting incremental knowledge

accumulation without oscillations or catastrophic forgetting [24]. Taken together, these mechanisms illustrate a key principle of self-evolving systems: the ability to accumulate specialized primitives, refine them selectively, and adapt their composition to new circumstances in a way that is both robust and flexible.

6.5.3 *Evaluation*

The empirical evaluation of MoDP underscores the practical benefits of this learning mechanism. Across standard cross-embodiment benchmarks, MoDP consistently surpasses the strong transformer baseline HPT [29], achieving lower validation loss and reduced gradient magnitudes in the later stages of training. The reduction in gradient magnitudes provides evidence that prototype alignment reduces harmful interference between tasks, a persistent challenge in both continual and transfer learning [18]. This indicates that disentangled prototypes not only enhance efficiency but also mitigate the instabilities that typically accompany cross-domain adaptation.

More importantly, under a deliberately constructed compositional generalization regime involving 25 agent, environment pairs, MoDP exhibits substantial improvements across all levels of generalization difficulty. The framework achieves markedly higher success rates in domain-consistent cases and demonstrates superior robustness when tasked with generalizing to unseen agents, unseen environments, or entirely novel agent, environment compositions. The observed improvements are notable: from 0.59 to 0.73 in-domain, from 0.36 to 0.55 in cross-agent cases, from 0.39 to 0.54 in cross-environment cases, and from 0.22 to 0.38 in the most challenging cross-domain scenario. These results provide compelling evidence that disentangled prototypes and modular subnetwork specialization enable a more scalable form of knowledge transfer, transforming compositional novelty from a source of interference into an opportunity for structured reuse [6].

Beyond raw performance metrics, ablation studies provide critical evidence for disentangling the contributions of individual design choices within MoDP and thereby validate its self-evolving nature. One of the most striking observations is the role of the compositional reconstruction loss. When this objective is removed, performance drops significantly across compositional generalization benchmarks, confirming that explicitly modeling and enforcing disentanglement at the representation level is indispensable [16]. This result underscores the importance of constraining the representation space in a way that captures the agent- and environment-specific factors independently, rather than allowing them to collapse into entangled embeddings that impair transfer.

Similarly, the omission of the policy-prototype mechanism severely undermines performance, demonstrating that the improvements do not simply result from increased parameter capacity or incidental redundancy. Instead, they stem from a principled structuring of the adaptation process, in which modular subnetworks are explicitly aligned with recurring patterns of interaction [15]. By tying prototypes to subnetworks identified through connection sensitivity, MoDP ensures that each spe-

cialized policy expert maintains functional coherence, allowing them to be selectively reused or refined as the distribution of tasks evolves. This mechanism mirrors findings in continual learning research, where modularization and sparsity often reduce interference and enable more stable transfer [24, 33].

The analysis of prototype cardinality further illuminates the delicate balance between efficiency and expressivity. Results indicate that maintaining a compact set of prototypes achieves the best trade-off, with overly large prototype banks introducing redundancy and underutilization, while too few prototypes fail to capture the diversity of agent, environment interactions. This aligns with broader theories of representation learning, which emphasize the utility of compact, disentangled primitives as the foundation for compositional generalization. In this respect, MoDP supports the view that effective self-evolution requires both inductive bias in the form of disentanglement and architectural mechanisms for selective specialization [6]. Collectively, these ablation studies establish that MoDP’s benefits are not incidental but arise from its carefully designed mechanisms, which balance modularity, compactness, and targeted adaptability.

From a conceptual perspective, MoDP contributes valuable insights to the broader vision of self-evolving agents. Its factorization of interaction spaces into recombining, low-cardinality primitives offers a concrete strategy for addressing one of the most intractable challenges in artificial intelligence: combinatorial generalization. By reducing a high-dimensional, entangled problem space to a structured set of disentangled prototypes, MoDP transforms the challenge of extrapolating to unseen compositions into one of interpolating within a structured latent space, thereby making the problem tractable [18]. This insight resonates with a long tradition of cognitive science and machine learning that views modularity and compositionality as key enablers of systematic generalization.

Equally significant is MoDP’s coupling of representational disentanglement with architecture-level specialization. This dual mechanism ensures that adaptation is concentrated where it yields the greatest benefit for generalization while simultaneously reducing negative transfer between unrelated tasks [45]. By linking representational structure with functional modularity, MoDP illustrates how self-evolving agents can avoid the pitfalls of monolithic adaptation, where new learning often destabilizes previously acquired skills. Instead, it demonstrates a pathway toward continual, compositional improvement, where specialized components evolve in a coordinated yet decoupled fashion.

References

1. Bai, H., Zhou, Y., Cemri, M., Pan, J., Suhr, A., Levine, S., Kumar, A.: Digirl: training in-the-wild device-control agents with autonomous reinforcement learning (2024). <https://arxiv.org/abs/2406.11896>
2. Bommasani, R., Hudson, D.A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M.S., Bohg, J., et al.: On the opportunities and risks of foundation models. arXiv preprint [arXiv:2108.07258](https://arxiv.org/abs/2108.07258) (2021)

3. Borgeaud, S., Mensch, A., Hoffmann, J., Cai, T., Rutherford, E., Millican, K., Driessche, G.V.D., Lespiau, J.B., et al.: Improving language models by retrieving from trillions of tokens. In: International Conference on Machine Learning (ICML) (2022)
4. Cai, Y., Hao, Y., Zhou, J., Yan, H., Lei, Z., Zhen, R., Han, Z., Yang, Y., Li, J., Pan, Q., Huai, T., Chen, Q., Li, X., Chen, K., Zhang, B., Xipeng, Q., Liang, H.: Building self-evolving agents via experience-driven lifelong learning: a framework and benchmark. arXiv preprint [arXiv:2508.19005](https://arxiv.org/abs/2508.19005) (2025)
5. Chen, D., Shi, J., Wan, Y., Zhou, P., Gong, N.Z., Sun, L.: Self-cognition in large language models: an exploratory study. arXiv preprint [arXiv:2407.01505](https://arxiv.org/abs/2407.01505) (2024)
6. Chen, Y., et al.: Improving compositional generalization in cross-embodiment learning via mixture of disentangled prototypes. In: International Conference on Learning Representations (ICLR) (2025)
7. Christiano, P., Leike, J., Brown, T.B., Martic, M., Legg, S., Amodei, D.: Deep reinforcement learning from human preferences (2023). <https://arxiv.org/abs/1706.03741>
8. Fang, J., Peng, Y., Zhang, X., Wang, Y., Yi, X., Zhang, G., Xu, Y., Wu, B., Liu, S., Li, Z., et al.: A comprehensive survey of self-evolving AI agents: a new paradigm bridging foundation models and lifelong agentic systems. arXiv preprint [arXiv:2508.07407](https://arxiv.org/abs/2508.07407) (2025)
9. Feng, L., Xue, Z., Liu, T., An, B.: Group-in-group policy optimization for llm agent training (2025). <https://arxiv.org/abs/2505.10978>
10. Feng, T., Li, Q., Wang, X., Wang, M., Li, G., Zhu, W.: Multi-weather cross-view geolocalization using denoising diffusion models. In: Proceedings of the 2nd Workshop on UAVs in Multimedia: Capturing the World from a New Perspective, pp. 35–39 (2024)
11. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata: a large-scale cooperative perception dataset for swarm uavs autonomous flight. In: Proceedings of the 32nd ACM International Conference on Multimedia, pp. 7600–7608 (2024)
12. Feng, T., Wang, X., Han, F., Zhang, L., Zhu, W.: U2udata-2: a scalable swarm uavs autonomous flight dataset for long-horizon tasks. arXiv preprint [arXiv:2509.00055](https://arxiv.org/abs/2509.00055) (2025)
13. Feng, T., Wang, X., Jiang, Y.G., Zhu, W.: Embodied AI: From llms to world models. In: IEEE Circuits and Systems Magazine (2025)
14. Feng, T., Wang, X., Zhou, Z., Wang, R., Zhan, Y., Li, G., Li, Q., Zhu, W.: Evoagent: agent autonomous evolution with continual world model for long-horizon tasks. arXiv preprint [arXiv:2502.05907](https://arxiv.org/abs/2502.05907) (2025)
15. Frankle, J., Carbin, M.: The lottery ticket hypothesis: finding sparse, trainable neural networks. In: International Conference on Learning Representations (ICLR) (2019)
16. Ghosh, P., Sajjadi, M.S.M., Vergari, A., Black, M.J., Schölkopf, B.: From variational to deterministic autoencoders. In: Conference on Learning Representations (ICLR) (2020)
17. Glaese, A., McAleese, N., Firoiu, V., et al.: Improving alignment of dialogue agents via targeted human feedback: the sparrow dialogue agent. *Science* **382**(6671), 1432–1437 (2023)
18. Hadsell, R., Rao, D., Rusu, A.A., Pascanu, R.: Embracing change: continual learning in deep neural networks. *Trends Cogn. Sci.* **24**(12), 1028–1040 (2020)
19. Han, F., Zhang, L., Wang, X., Zhao, K.A., Zhong, Y., Su, Z., Feng, T., Zhu, W.: U2usim - a uav telepresence simulation platform with multi-agent sensing and dynamic environment. In: Proceedings of the 32nd ACM International Conference on Multimedia, pp. 11258–11260 (2024)
20. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. arXiv preprint [arXiv:1503.02531](https://arxiv.org/abs/1503.02531) (2015). <https://arxiv.org/abs/1503.02531>
21. Houlisby, N., Giurui, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., Gelly, S.: Parameter-efficient transfer learning for nlp. In: International Conference on Machine Learning (ICML) (2019)
22. Hu, B., Zhao, C., Zhang, P., Zhou, Z., Yang, Y., Xu, Z., Liu, B.: Enabling intelligent interactions between an agent and an llm: a reinforcement learning approach (2024). <https://arxiv.org/abs/2306.03604>
23. Hussein, A., Gaber, M.M., Elyan, E., Jayne, C.: Imitation learning: a survey of learning methods. *ACM Comput. Surv.* **50**(2), 21:1–21:35 (2017). <https://doi.org/10.1145/3054912>

24. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al.: Overcoming catastrophic forgetting in neural networks. *Proc. Natl. Acad. Sci. (PNAS)* **114**(13), 3521–3526 (2017)
25. Li, L., Chen, Z., Chen, G., Zhang, Y., Su, Y., Xing, E., Zhang, K.: Confidence matters: revisiting intrinsic self-correction capabilities of large language models (2024). <https://arxiv.org/abs/2402.12563>
26. Li, W., Li, D., Dong, K., Zhang, C., Zhang, H., Liu, W., Wang, Y., Tang, R., Liu, Y.: Adaptive tool use in large language models with meta-cognition trigger. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pp. 13346–13370 (2025)
27. Liang, X., He, Y., Xia, Y., Song, X., Wang, J., Tao, M., Sun, L., Yuan, X., Su, J., Li, K., Chen, J., Yang, J., Chen, S., Shi, T.: Self-evolving agents with reflective and memory-augmented abilities. *arXiv preprint arXiv:2409.00872* (2024)
28. Liu, H., Huang, X., Gu, J., Shi, J., He, N., Feng, T.: TCDformer-based momentum transfer model for long-term sports prediction. *Expert. Syst. Appl.* 128310 (2025)
29. Liu, Y., et al.: Hierarchical policy transformers for cross-domain generalization. In: *NeurIPS* (2023)
30. Luo, J., Zhang, W., Yuan, Y., Zhao, Y., Yang, J., Gu, Y., Wu, B., Chen, B., Qiao, Z., Long, Q., Tu, R., Luo, X., Ju, W., Xiao, Z., Wang, Y., Xiao, M., Liu, C., Yuan, J., Zhang, S., Jin, Y., Zhang, F., Wu, X., Zhao, H., Tao, D., Yu, P.S., Zhang, M.: Large language model agent: a survey on methodology, applications and challenges (2025). <https://arxiv.org/abs/2503.21460>
31. Oh, J., Guo, Y., Singh, S., Lee, H.: Self-imitation learning (2018). <https://arxiv.org/abs/1806.05635>
32. OpenAI: Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023)
33. Parisi, G.I., Kemker, R., Part, J.L., Kanan, C., Wermter, S.: Continual lifelong learning with neural networks: a review. *Neural Netw.* **113**, 54–71 (2019)
34. Qi, Z., Liu, X., Iong, I.L., Lai, H., Sun, X., Sun, J., Yang, X., Yang, Y., Yao, S., Xu, W., Tang, J., Dong, Y.: WebRL: training LLM web agents via self-evolving online curriculum reinforcement learning. In: *The Thirteenth International Conference on Learning Representations* (2025). <https://openreview.net/forum?id=oVKEAFjEqv>
35. Qian, H., Liu, Z., Zhang, P., Mao, K., Lian, D., Dou, Z., Huang, T.: Memorag: boosting long context processing with global memory-enhanced retrieval augmentation (2025). <https://arxiv.org/abs/2409.05591>
36. Reddy, S., Dragan, A.D., Levine, S.: Sqil: imitation learning via reinforcement learning with sparse rewards (2019). <https://arxiv.org/abs/1905.11108>
37. Reed, S., Zolna, K., Parisotto, E., et al.: A generalist agent. *arXiv preprint arXiv:2205.06175* (2022)
38. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., et al.: Toolformer: language models can teach themselves to use tools. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2023)
39. Shen, Y., Zhang, K., Zhou, Y., Huang, W., Zhang, C., Tang, J., Sun, M.: Hugginggpt: solving AI tasks with chatgpt and its friends in hugging face. *arXiv preprint arXiv:2303.17580* (2023)
40. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: language agents with verbal reinforcement learning (2023). <https://arxiv.org/abs/2303.11366>
41. Taubenfeld, A., Sheffer, T., Ofek, E., Feder, A., Goldstein, A., Gekhman, Z., Yona, G.: Confidence improves self-consistency in llms. In: *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 20090–20111. Association for Computational Linguistics (2025). <https://doi.org/10.18653/v1/2025.findings-acl.1030>
42. Wang, X., Zhang, J., Chen, Y., et al.: Self-evolving agents: a survey. *arXiv preprint arXiv:2401.12345* (2024)
43. Wei, J., Wang, X., Schuurmans, D., Bosma, M., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2022)
44. Xi, Y., Lin, J., Xiao, Y., Zhou, Z., Shan, R., Gao, T., Zhu, J., Liu, W., Yu, Y., Zhang, W.: A survey of llm-based deep search agents: paradigm, optimization, evaluation, and challenges. *arXiv preprint arXiv:2508.05668* (2025)

45. Yao, J., Sun, K., Li, M., et al.: Agents that learn to evolve: a survey of self-improving and self-evolving architectures. arXiv preprint [arXiv:2409.01234](https://arxiv.org/abs/2409.01234) (2024)
46. Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Liang, P., Narasimhan, K.: React: synergizing reasoning and acting in language models. In: International Conference on Learning Representations (ICLR) (2023)
47. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: synergizing reasoning and acting in language models (2023). <https://arxiv.org/abs/2210.03629>
48. Zelikman, E., Wu, Y., Mu, J., Goodman, N.D.: Star: bootstrapping reasoning with reasoning (2022). <https://arxiv.org/abs/2203.14465>
49. Zeng, Z., Xie, Y., Zhang, H., Chen, C., Wang, Z., Chen, B.: Meacap: memory-augmented zero-shot image captioning (2024). <https://arxiv.org/abs/2403.03715>

Index

A

Active perception, 25
Autonomous collaboration, 157
Autonomous communication, 160
Autonomous dynamic interaction, 108
Autonomous embodied AI, 89
Autonomous embodied cognition, 106
Autonomous environment learning, 98
Autonomous hardware embodiment, 109
Autonomous learning in humans, 91
Autonomous machine learning, 94
Autonomous robustness optimization, 163
Autonomous swarm embodied AI, 157

C

Case study, 260
Collaborative perception, 141
Communication issues, 154
Communication mechanism, 145
Continual learning, 11
Curriculum learning, 195

D

Datasets and Metrics, 58, 172
Drones, 54
Dynamic interaction, 37
Dynamics optimizer self-evolution, 230

E

Early collaboration, 143

Embodied AI with LLM/MLLM, 41
Embodied AI with world model, 45
Embodied Artificial Intelligence (Embodied AI), 3
Embodied cognition, 35

H

Hardware embodiment, 39

I

Imitation-based self-evolution, 257
Imitation learning, 15
Interaction-based self-evolution, 258
Intermediate collaboration, 146

J

Joint MLLM and World Model, 47
Joint model-memory self-evolution, 232, 254
Joint model-memory-tools self-evolution, 255
Joint model-tools self-evolution, 234, 254

L

Late collaboration, 152
Lifelong learning, 197
Localization error, 154
Loss function, 151

M

Mathematical formalization, [217](#)
Memory self-evolution, [226](#)
Meta learning, [192](#)
Model & Task Discrepancies, [155](#)
Model self-evolution, [218](#), [253](#)
Multimodal large language model, [18](#)

O

Operational robustness, [153](#)

P

Particle swarm optimization, [138](#)
Privacy & Security, [156](#)

R

Reinforcement learning, [6](#)
Robots, [52](#)

S

Self-evolving agent, [202](#), [247](#)
Self-evolving embodied AI, [189](#)
Self-evolving mechanism, [217](#)
Self-evolving world model, [201](#), [211](#)
Semantic reasoning, [41](#)
Simulators, [56](#), [169](#)
Single embodied agent limitations, [125](#)
Swarm embodied AI, [131](#)
Swarm intelligence, [131](#)

T

Task decomposition, [41](#)
Tools self-evolution, [228](#)

W

Why Need Formalization, [214](#)
World model, [21](#)